

Эта версия нужна для файла
Печать.tur

Всего страниц: 289

Билеты + вопросы в конце

Эскпортируйте, выключив
Tagged PDF, чтобы уменьшить
размер файла.

Нажмите Replace на старой
версии PDF, чтобы обновить
её новой

СОДЕРЖАНИЕ

1

Управление внешними устройствами: специальные файлы устройств, адресация внешних устройств и их идентификация в системе, тип `dev_t`. Система прерываний: типы прерываний и их особенности. Прерывания в последовательности ввода-вывода — обслуживание запроса процесса на ввод-вывод (диаграмма). Быстрые и медленные прерывания. Обработчики аппаратных прерываний: регистрация в системе — функция и ее параметры, примеры. Тасклеты — объявление, планирование (пример лаб. раб).

2

Управление внешними устройствами: специальные файлы устройств, идентификация внешних устройств в системе (тип `dev_t`). Система прерываний: типы прерываний и их особенности. Быстрые и медленные прерывания. Обработчики аппаратных прерываний: регистрация в системе. Линия IRQ (простейшая концептуальная схема — три шины). Верхние и нижние `softirq` — перечисления и функции ядра для выполнения `softirq`, тасклеты и очереди работ. К `softirq` — схема обслуживания входящих сетевых пакетов. `Softirq` и `tasklet`: объявление, инициализация, планирование тасклетов (примеры из лаб раб).

3

Сокеты как средство взаимодействия параллельных процессов. Сокеты семейства `AF_UNIX` и `AF_INET`. Сокеты в пространстве файловых имён. Сокеты `AF_INET`, сетевой стек. Классификация моделей ввода-вывода. Мультиплексирование при взаимодействии процессов в распределённых системах по модели клиент сервер. Ввод-вывод с мультиплексированием: мультиплексор `epoll` — особенности (типы и алгоритм работы). Пример из лабораторной работы — сервер производство-потребление с мультиплексированием.

4

Файловая подсистема `/proc` — назначение, особенности, файлы, поддиректории, ссылка `self`, информация об окружении, состоянии процесса, прерываниях. Структура `proc_dir_entry`: функции для работы с элементами `/proc`. Использование структуры `file_operations` для регистрации функций работы с файлами. Передача данных из адресного пространства пользователя в адресное пространство ядра и обратно (лаб. раб.). Обоснование необходимости использования специальных функций для передачи данных из пространства пользователя в ядро и из ядра в пространство пользователя.

5 Файловая подсистема ОС Linux: иерархическая структура ФС (уровни иерархии), поддержка большого числа файловых систем. Интерфейс VFS, именованное (дерево каталогов) и идентификация файлов. Супер_блок — назначение и поля `struct super_block`, обеспечивающие хранение и доступ к файлам, и структуры, на которые ссылается. Структура, описывающая тип файловой системы, и регистрация файловой системы в ОС Linux, монтирование ФС. Слаб-кэш: назначение, функции создания слаба и размещения в нём данных. Пример (лаб. раб. VFS)

6 Файловая система, задачи файловой системы и иерархическая организация ФС. Файловая подсистема LINUX: поддержка большого числа файловых систем и структура, описывающая файловую систему. VFS: четыре основные структуры файловой системы и связь между ними. Раздел жесткого диска и суперблок. Структура `struct super_operations`. Монтирование файловой системы, команда `mount` и функции монтирования и их параметры, точка монтирования — корневой каталог и `inode`. Пример (лаб. раб.)

7 Виртуальная файловая система `/proc`: структура `proc_dir_entry`, создание файла `proc`, директории и символические ссылки; точки входа и их регистрация, интерфейсы передачи данных из ядра в пространство пользователя — реализация и особенности. Примеры из лабораторных работ.

8 Средства взаимодействия процессов — сокеты Беркли. Создание сокета — семейство, тип, протокол. Системный вызов `sys_socket()` и `struct socket`. Состояния сокета. Адресация сокетов и ее особенности для разных типов сокетов. Модель клиент-сервер. Сетевые сокеты — сетевой стек, аппаратный и сетевой порядок байтов. Примеры реализации взаимодействия процессов по модели клиент-сервер с использованием сокетов и мультиплексированием (лаб. раб.).

Файловая система: процесс и файловые структуры связанные с процессом. Файлы и открытые файлы, связь структур, представляющих открытые файлы на разных уровнях. Системный вызов `open()` и библиотечная функция `foren()`: параметры и флаги, определенные на функции `open()`. Реализация системного вызова `open()` в ядре Linux. Пример: файл открывается два раза системным вызовом `open()` для записи и в него последовательно записывается строка «aaaaaaaaaaaa» по первому дескриптору и затем строка «vvvv» по второму дескриптору, затем файл закрывается два раза. Показать, что будет записано в файл и пояснить результат.

9

10

Файловая система ОС Linux: иерархическая структура каталогов, имена файлов и идентификация файлов. Inode и dentry: пример доступа к файлу `mbox`. Кэши inode и dentry. Назначение суперблока в интерфейсе VFS: монтирование файловой системы — суперблок и связанные с ним структуры; структура `struct super_block` и корневой каталог. Структура `struct super_operations`. Структуры `struct inode`, `struct inode_operations`. Функции `simple` и `generic`. Точка монтирования. Функции монтирования. Пример создания файловой системы, её регистрации и монтирование (лаб. раб.).

11

Аппаратные прерывания в Linux: запрос прерывания и линии IRQ. Простейшая схема аппаратной поддержки прерываний (концептуальная трёхшинная архитектура системы). Быстрые и медленные прерывания, флаги, пример быстрого прерывания. Нижняя и верхняя половины обработчиков прерывания, функция регистрации прерывания и её параметры. Нижняя половины: `softirq`, определение, массив `softirq_vec`, типы `soft_irq`, функции `open_softirq()`, `raise_softirq()`. Поток ядра `ksoftirqd`. Пример обработки приходящих пакетов `NET_RX_SOFTIRQ`.

12

Создание виртуальных файловых систем. Структура, описывающая файловую систему. Регистрация и прекращение регистрации файловой системы в ОС Linux. Монтирование файловой системы. Точка монтирования. Функции монтирования. Команда монтирования файловой системы. Суперблок, назначение суперблока. Inode и адресация очень больших файлов (ф.с. extX). Структура dentry. Пример доступа к файлу `mbox`. Кэширование в системе: dentry кэш и кэш inode. Кэши SLAB, функции для работы с кэшем SLAB. Функции, определённые в файлах и их регистрация (`struct file_operations`). Пример из лабораторной работы.

13

Файловая подсистема: особенности файловой подсистемы Unix/Linux: иерархическая структура файловой подсистемы. Виртуальная файловая система VFS в Linux. Четыре структуры VFS — `super_block`, `inode`, `dentry`, `file` — их назначение. Адресация файлов большого размера в файловой системе `extX` и пример, показывающий доступ к файлу `/usr/ast/mbox`. Кэширование `dentry` и `inode`. Slab кэш. Монтирование файловых систем. Команда `mount` и функции монтирования, пример из лаб раб.

14

Файловая подсистема `/proc` — назначение, особенности, файлы, поддиректории, ссылка `self`, информация об окружении, состоянии процесса, прерываниях. Структура `proc_dir_entry`: функции для работы с элементами `/proc`. Структура, перечисляющая функции, определенные на файлах. Использование структуры `file_operations` для регистрации собственных функций работы с файлами. Передача данных из пространства пользователя в пространство ядра и из ядра в пространство пользователя. Обоснование необходимости этих функций. Функция `printk()` — назначение и особенности. Пример программы «Фортунки» из лаб. работы.

15

Загружаемые модули ядра. Структура загружаемых модулей. Информация о процессах, доступная в ядре: `prio`, `priority`, `state`, `flags`. Пример вывода информации о запущенных процессах, символ `current` (лаб. раб.). Взаимодействие загружаемых модулей в ядре. Экспорт данных. Пример взаимодействия модулей (лаб. раб.). Функция `printk()` — назначение и особенности. Виртуальная файловая система `proc`: назначение, примеры директорий и файлов `/proc`, файлы `maps`, `pagemap`, `tasks`. Регистрация функций работы с файлами `proc`. Пример проинициализированных полей структуры `proc_ops`. Передача данных из пространства ядра в пространство пользователя. Примеры из лабораторный работ.

16

Открытые файлы: системный вызов `open()`: `int open(const char* pathname, int flags); int open(const char* pathname, int flags, mode_t mode)`; пояснить смысл параметров. Основные флаги. Флаг `CREATE`. Реализация системного вызова `open()` в системе — действия в ядре: `SYSCALL_DEFINE3(open,...) → ksys_open(filename, flags, mode) → do_sys_open() → do_sys_openat2() ...` найти наименьший файловый дескриптор... Действия, если флаг `O_CREATE` установлен?

1

1 Управление внешними устройствами: специальные файлы устройств, адресация внешних устройств и их идентификация в системе, тип `dev_t`. Система прерываний: типы прерываний и их особенности. Прерывания в последовательности ввода-вывода — обслуживание запроса процесса на ввод-вывод (диаграмма). Быстрые и медленные прерывания. Обработчики аппаратных прерываний: регистрация в системе — функция и ее параметры, примеры. Тасклеты — объявление, планирование (пример лаб. раб).

1.1 Управление внешними устройствами: специальные файлы устройств, адресация и идентификация, тип `dev_t`

Существует два типа внешних устройств:

- Байт-ориентированные (или символьные) устройства передают данные посимвольно, как непрерывный поток байтов (например, мышь, клавиатура).
- Блок-ориентированные (или блочные) устройства (например, жёсткий диск) передают данные блоками. Все запоминающие устройства относятся к блочным.

Чтобы управлять всем разнообразием внешних устройств, в Unix/Linux реализована концепция драйверов. Она заключается в том, что устройства должны быть зарегистрированы в системе и в системе существует набор драйверов, которые можно использовать для управления новыми устройствами. Драйвер — это часть кода ядра, которая предназначена для управления конкретным внешним устройством.

В Linux используется парадигма «всё — файл», поэтому устройство — тоже файл. Но в полном объёме невозможно взаимодействовать как с обычными файлами. Устройства в системе должны идентифицироваться. Для этого в системе определён специальный подход. Файлы внешних устройств находятся в директории `/dev`. В ней с помощью `ls -l` видим внешние устройства.

1	\$ ls -l /dev									
2	crw-rw-rw-	1	root	root	1,	3	Apr	11	2002	null
3	crw-----	1	root	root	10,	1	Apr	11	2002	psaux
4	crw-----	1	root	root	4,	1	Oct	28	03:04	tty1
5	crw-rw-rw-	1	root	tty	4,	64	Apr	11	2002	ttys0
6	crw-rw----	1	root	uucp	4,	65	Apr	11	2002	ttys1
7	crw--w----	1	vcsa	tty	7,	1	Apr	11	2002	vcs1
8	crw--w----	1	vcsa	tty	7,	129	Apr	11	2002	vcsa1
9	crw-rw-rw-	1	root	root	1,	5	Apr	11	2002	zero

Старший номер идентифицирует драйвер, связанный с устройством. Например, `/dev/null` и `/dev/zero` управляются драйвером 1, виртуальные терминалы — драйвером 4. **Младший номер** используется для различия конкретных устройств в рамках одного драйвера (например, жёсткий диск имеет один старший номер, а его разделы — разные младшие).

Стандарт POSIX-1 оговаривает существование типа `dev_t` — 32-разрядное число, в котором 12 бит отведены под старший номер, 20 — под младший. Код не должен делать предположений об организации номеров внутри числа. Для работы с `dev_t` используются макросы:

```
1 MAJOR(dev_t dev); // получить старший номер
2 MINOR(dev_t dev); // получить младший номер
```

Имея старший и младший номера, можно их объединить в значение типа `dev_t`.

```
1 MKDEV(int major, int minor) // объединить в dev_t
```

1.2 Выделение и освобождение номеров устройств

Одно из первых действий, которые должен сделать драйвер, когда устанавливает символьное устройство — он должен получить 1 или более номеров устройств, чтобы работать с ними, чтобы идентифицировать свой `char device`. Для этого вызывается функция

```
1 int register_chrdev_region(dev_t first, unsigned int count, char *name);
```

— `first` — начальный номер устройства доступного для выделения диапазона

Часто минорная часть значения `first` равна нулю. Но этот счёт нет никаких определённых требований.

— `count` — общее количество последовательных номеров устройств, которые могут быть запрошены. При этом значение `count` может превышать размер запрашиваемого диапазона. В этом случае будет переход на следующий старший номер, ошибка не возникает, если запрашиваемый диапазон доступен.

— `name` — имя устройства, которое связывается с диапазоном. Это можно увидеть или в `/proc/devices` или в `sysfs (/sys)`.

При успешном выделении функция возвращает 0.

Часто неизвестно значение старшего номера устройства, поэтому разработчики ядра всё время стремятся перейти на динамическое распределение номеров устройств. Для динамического выделения используется функция

```
1 int alloc_chrdev_region(dev_t *dev, unsigned int firstminor, unsigned int count,
    char *name);
```

— `dev`

Значение указателя при успешном завершении будет содержать первый номер в определённом диапазоне. Обычно указывают 0.

Остальные параметры аналогичны рассмотренной ранее функции.

Номера освобождаются функцией

```
1 void unregister_chrdev_region(dev_t first, unsigned int count);
```

Все эти функции вызываются в модуле в функции `init`, а последняя функция — в функции `exit`.

1.3 Система прерываний: типы прерываний и их особенности.

Три типа событий, переводящих систему в режим ядра:

- 1) Системные вызовы (программные прерывания) — процесс переключается в режим ядра и выполняет реентерабельный код ядра, после переключается в режим пользователя.
- 2) Исключения — синхронные. Два типа: исправимые (страничное прерывание, после обработки программа продолжает выполнение) и неисправимые (программа завершается аварийно).
- 3) Аппаратные прерывания — асинхронные. Три типа:
 - 3.1) от системного таймера (единственное периодическое прерывание в системе);
 - 3.2) от устройств ввода/вывода. Прерывание информирует процесс о завершении операции ввода/вывода. При этом, каждую такую операцию инициирует сам процессор.
 - 3.3) от действий оператора (использование горячих клавиш).

Синхронные — возникают в процессе выполнения программы; Асинхронные — не зависят ни от каких действий в системе.

1.4 Прерывания в последовательности ввода-вывода.

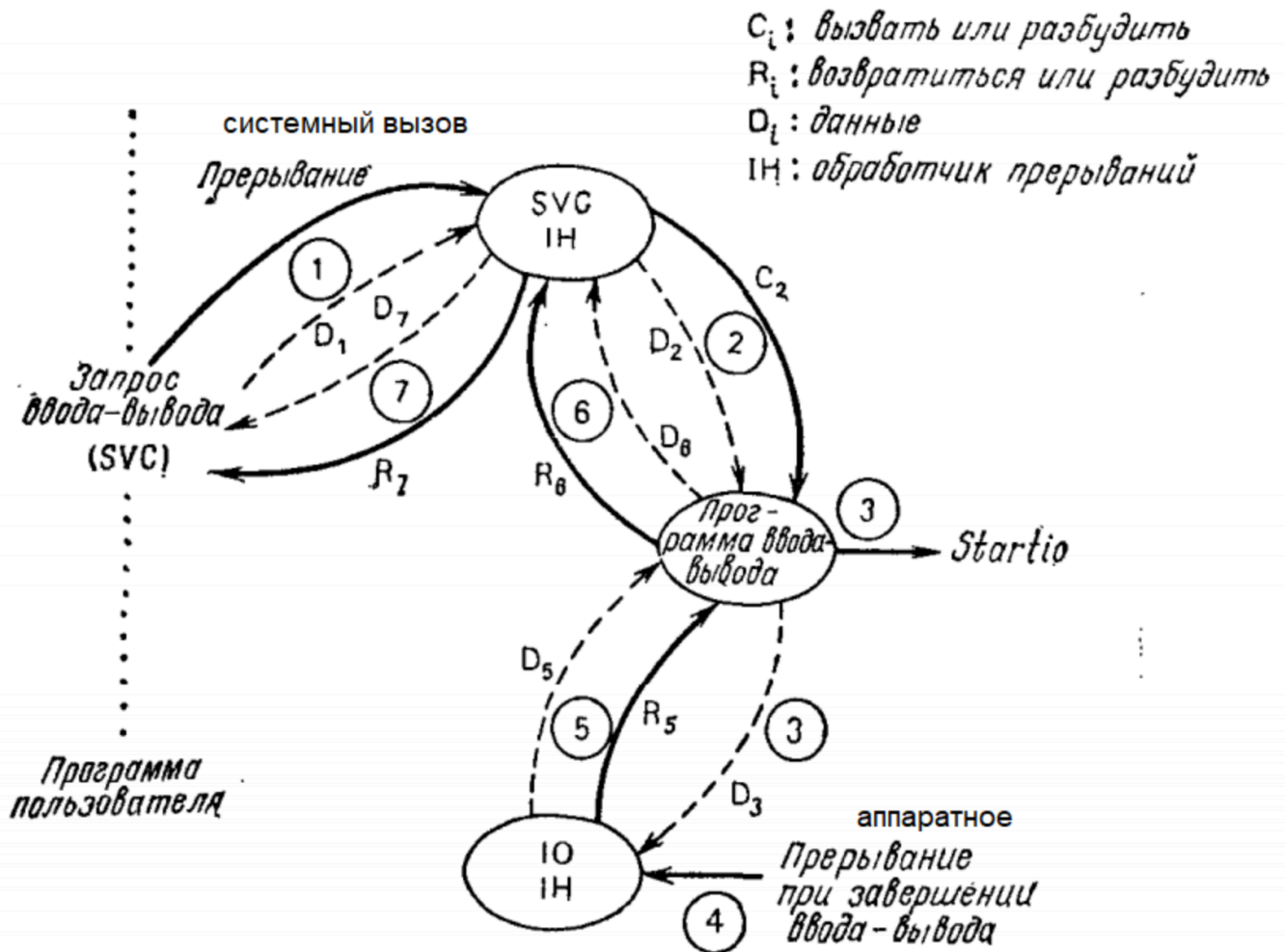


Рис. 4.5. Управление прерываниями в последовательности ввода-вывода.

Рис. 1. Блокирующий ввод/вывод

Схематически показано, что приложение запрашивает данные. Данные приложение может получить не сразу. Здесь включаются различные дополнительные действия, например, ввод данных с клавиатуры или с мыши (мышь тоже устройство ввода/вывода). Для однозначности рассуждений будем рассматривать клавиатуру.

Клавиатура — интерактивное внешнее устройство, у него есть контроллер. Клавиатура как большинство внешних устройств подключается к шине через порт клавиатуры, то есть взаимодействие с процессором выполняется по модели input/output mapping. При этом выполняется вся последовательность действий, характерных для работы с внешним устройством.

Есть драйвер. Когда приложение запрашивает ввод данных с клавиатуры, ядро системы выполняет целую последовательность действий.

Когда введенные пользователем данные поступят в буфер клавиатуры, контроллер клавиатуры отправит сигнал прерывания на контроллер прерывания. С контроллера этот сигнал по шине управления поступит на ножку процессора и процессор перейдет на выполнение обработчика прерывания. В результате процесс, запросивший ввод, будет разблокирован. Вся последовательность действий выполняется, когда процесс блокирован на запросе ввода данных. Это блокирующий ввод/вывод, в основном мы имеем дело именно с ним.

Поскольку схема рассматривает ситуацию в самом общем виде, на ней показан `supervisor call interrupt handler` — функция, которая вызывается в режиме ядра в ответ на системный вызов. В общем виде последовательность блокирующего ввода/вывода:

- 1) Запрос ввода/вывода — это системный вызов; при переходе в режим ядра он заблокирует приложение, то есть его состояние будет изменено на `D (uninterruptable sleep)`.
- 2) Начинает выполняться код ядра: будет вызвана точка входа драйвера устройства, которое идентифицируется запросом. В результате работы кода драйвера будет сформирована команда для устройства, которая по шине данных будет передана контроллеру устройства.
- 3) Отправив команду, процессор отключается от управления устройством — **это принцип распараллеливания функций**: процессор не управляет работой внешних устройств, ими управляют контроллеры устройств. Этот принцип привёл к необходимости включения в систему прерываний прерываний от внешних устройств: они информируют процессор о завершении операции ввода/вывода.
- 4) Когда данные готовы, будет вызван обработчик соответствующего аппаратного прерывания; его задача — считать данные из буфера устройства в буфер ядра и инициализировать следующие действия по обработке полученных данных.
- 5) Обработчик вызовет соответствующую `callback`-функцию драйвера, передаст полученные данные, и в результате этих действий приложение будет разблокировано: в буфер приложения (переменную, массив, поле структуры и так далее) будут записаны запрошенные данные.

Важно

В этой модели важно именно то, что процесс блокируется на запросе и разблокируется только тогда, когда данные могут быть процессом получены.

Дальше нужно скопировать, но это выполняет обработчик прерывания. Данные готовы = они поступили в буфер ядра.

Важно

Процессор не управляет внешним устройством.

Внизу справа — аппаратное прерывание, которое формируется, когда данные в буфере устройства. Из этого буфера данные пройдут несколько этапов прежде чем окажутся в буфере приложения, и приложение сможет продолжить своё выполнение.

Важно

Приложение не может продолжить своё выполнение, пока оно не получит запрошенные данные.

1.5 Быстрые и медленные прерывания

В ОС Linux различают быстрые и медленные прерывания.

1) Быстрые:

Обработка быстрых прерываний выполняется от начала до конца. В современных системах осталось единственное быстрое прерывание — прерывание от таймера, вытеснить его невозможно. При этом выполняется на процессоре 0 и *условно* этот процессор считается главным.

2) Медленные

Все остальные аппаратные прерывания, то есть прерывания от устройств ввода/вывода (внешних устройств) делятся на две части:

2.1) top half (верхняя половина):

2.1.1) interrupt (hardware — аппаратное)

2.2) bottom half (нижняя половина):

2.2.1) softirq

2.2.2) tasklet

2.2.3) workqueue

Главные задачи обработчика прерывания от таймера:

1) Инициализировать планировщик

Ставит в очередь на выполнение важные коды:

— Планировщик

— Диспетчер

— swapper

2) Инициализировать диспетчеризацию

Он не может выполняться долго. Если он выполняется долго, это приведет к снижению **отзывчивости** системы

1.6 Top half

Это аппаратные прерывания, выполняются на высоких уровнях приоритета. Системы, с которыми мы работаем не поддерживают вложенных прерываний, которые были написаны для IBM/360. Для выполнения аппаратных прерываний установлены следующие ограничения:

Если на процессоре выполняется обработчик аппаратного прерывания, то на этом процессоре запрещены все прерывания. На остальных процессорах запрещены прерывания по линии IRQ, на которой установлен выполняющийся обработчик прерывания. В результате если такой обработчик написан неправильно, выполнятся большой объем кода, это приведет к снижению отзывчивости системы.

Поэтому обработчик аппаратного прерывания должен выполнять минимальный объем действий. Например, если это обработчик прерывания от клавиатуры, то он должен записывать код нажатой клавиши, который поступает из буфера контроллера в буфер ядра, для последующей обработки должен установить задачу для последующего выполнения нижней половины, то есть отложенного действия.

1.7 Обработчики аппаратных прерываний: регистрация, функция и её параметры

Чтобы зарегистрировать в системе наш обработчик прерывания, в ядре имеется функция

```
1 static inline int __must_check request_irq(unsigned int irq, irq_handler_t handler,
      unsigned long flags, const char *name, void *dev)
2 {
3     return request_threaded_irq(irq, handler, NULL, flags | IRQF_COND_ONESHOT, name,
      dev);
4 }
```

Параметры:

- `irq`: номер прерывания. IRQ0 — прерывание от таймера, IRQ1 — от клавиатуры.
- `handler`: наш обработчик прерывания
- `flags`: флаги Interrupt ReQuest Flags (IRQF)
- `name`: имя, которое увидим в `/proc/interrupts`

Это функция обертка над `request_threaded_irq`, это базовая функция. При ее вызове в параметр `irq_handler_t thread_fn` равен `NULL`.

Флаги:

```

1 #define IRQF_SHARED          0x00000080
2 #define IRQF_PROBE_SHARED    0x00000100
3 #define __IRQF_TIMER          0x00000200
4 #define IRQF_PERCPU          0x00000400
5 #define IRQF_NOBALANCING     0x00000800
6 #define IRQF_IRQPOLL         0x00001000
7 #define IRQF_ONESHOT         0x00002000
8 #define IRQF_NO_SUSPEND      0x00004000
9 #define IRQF_FORCE_RESUME    0x00008000
10 #define IRQF_NO_THREAD       0x00010000
11 #define IRQF_EARLY_RESUME    0x00020000
12 #define IRQF_COND_SUSPEND    0x00040000
13 #define IRQF_NO_AUTOEN       0x00080000
14 #define IRQF_NO_DEBUG        0x00100000
15 #define IRQF_COND_ONESHOT    0x00200000
16
17 #define IRQF_TIMER            (__IRQF_TIMER | IRQF_NO_SUSPEND | IRQF_NO_THREAD)

```

- `IRQF_SHARED` — Этот флаг как раз и позволяет разделять линию прерывания несколькими обработчиками.
- `IRQF_PROBE_SHARED` — когда мы «не уверены»?
- `__IRQF_TIMER` — от системного таймера.
- `IRQF_NO_THREAD` — не может быть распараллелен.

Структура, описывающая обработчик:

```

1 struct irqaction {
2     irq_handler_t handler;
3     void *dev_id;
4     void __percpu *percpu_dev_id;
5     struct irqaction *next;
6     irq_handler_t thread_fn;
7     struct task_struct *thread;
8     struct irqaction *secondary;
9     unsigned int irq;
10    unsigned int flags;
11    unsigned long thread_flags;
12    unsigned long thread_mask;
13    const char *name;
14    struct proc_dir_entry *dir;
15 };

```

Поле `dir` заполняется автоматически на основе параметров, переданных `request_irq`.

dir — pointer to the proc/irq/NN/name entry.

Возвращаемые значения обработчика:

```
1 enum irqreturn {
2     IRQ_NONE          = (0 << 0), // прерывания не было или не обработано
3     IRQ_HANDLED       = (1 << 0), // прерывание обработано
4     IRQ_WAKE_THREAD   = (1 << 1), // запрос пробуждения потока хендлера
5 };
6
7 typedef enum irqreturn irqreturn_t;
```

Пример из ЛР:

```
1 static irqreturn_t irq_handler(int irq, void *dev_id)
2 {
3     int code;
4     int is_release;
5     int press_code;
6
7     if (irq == IRQ_NUM) {
8         code = inb(0x60);
9         is_release = (code & 0x80) ? 1 : 0;
10        press_code = code & 0x7F;
11
12        if (is_release && press_code < 84 && press_code != 28) {
13            irq_time = ktime_get();
14            tasklet->data = (unsigned long)code;
15            tasklet_schedule(tasklet);
16            return IRQ_HANDLED;
17        }
18    }
19    return IRQ_NONE;
20 }
```

Когда драйвер выгружается, необходимо отменить регистрацию соответствующего обработчика прерывания. Для этого существует функция:

```
1 void free_irq(unsigned int irq, void *dev_id);
```

Если указанная линия прерывания не является разделяемой, то функция `free_irq()` удаляет обработчик и отключает линию. Если линия прерывания разделяется, то обработчик, определённый `dev_id` удаляется, но линия прерывания будет отключена только при удалении последнего обработчика.

1.8 Тасклеты: объявление, планирование (пример из лаб. раб.)

Тасклеты — частный случай softirq. softirq одного типа может выполняться параллельно, поэтому требуются средства взаимного исключения. Тасклет одного типа может выполняться только на одном процессоре — средства взаимного исключения не требуются. Разные тасклеты могут выполняться параллельно. Тасклет — компромисс между производительностью и простотой использования.

softirq написаны таким образом, что они могут выполняться реально параллельно. Это требует, чтобы код был реентерабельный и в них реализовывалось взаимное исключение.

```
1 struct tasklet_struct {
2     struct tasklet_struct *next; // указатель на следующий тасклет в списке
3     unsigned long state;
4     atomic_t count;
5     bool use_callback;
6     union {
7         void (*func)(unsigned long data); // функция-обработчик тасклета
8         void (*callback)(struct tasklet_struct *t);
9     };
10    unsigned long data; // аргумент функции-обработчика
11 };
12
13 enum {
14     TASKLET_STATE_SCHED, /* Tasklet is scheduled for execution */
15     TASKLET_STATE_RUN    /* Tasklet is running (SMP only) */
16 };
```

Листинг 1. Структура tasklet_struct

Состояния: 0, TASKLET_STATE_SCHED, TASKLET_STATE_RUN. TASKLET_STATE_RUN имеет смысл только в SMP-архитектуре. Тасклет может выполняться только если count == 0.

Статическое объявление:

```
1 DECLARE_TASKLET(name, func, /* устаревшее, сейчас 3-го параметра нет */ data);
2
3 #define DECLARE_TASKLET(name, _callback) \
4 struct tasklet_struct name = {          \
5     .count = ATOMIC_INIT(0),            \
6     .callback = _callback,              \
7     .use_callback = true,               \
8 };
```

Листинг 2. Статическая инициализация tasklet

Динамическое объявление:

```
1 void tasklet_init(struct tasklet_struct *t, void (*func)(unsigned long),
2                 unsigned long data)
3 {
4     t->next = NULL;
5     t->state = 0;
6     atomic_set(&t->count, 0);
7     t->func = func;
8     t->data = data;
9 }
```

Листинг 3. Динамическая инициализация tasklet

Прежде чем инициализировать, нужно выделить память.

```
1 my_tasklet = (struct tasklet_struct *)kmalloc(sizeof(struct tasklet_struct),
2 GFP_KERNEL);
```

В exit нужно:

1) free irq

Нужно освободить линию прерывания от нашего обработчика

2) tasklet_kill

Передаем для данного действия my_tasklet

Планирование Для того чтобы запланировать выполнение тасклета, ядро предоставляет две функции:

```
1 extern void __tasklet_schedule(struct tasklet_struct *t);
2 static inline void tasklet_schedule(struct tasklet_struct *t)
3 {
4     if (!test_and_set_bit(TASKLET_STATE_SCHED, &t->state))
5         __tasklet_schedule(t);
6 }
7
8 extern void __tasklet_hi_schedule(struct tasklet_struct *t);
9 static inline void tasklet_hi_schedule(struct tasklet_struct *t)
10 {
11     if (!test_and_set_bit(TASKLET_STATE_SCHED, &t->state))
12         __tasklet_hi_schedule(t);
13 }
```

Пример из лабораторной работы (обработчик прерывания клавиатуры с тасклетом):

```
1 static struct tasklet_struct *tasklet = NULL;
```

```

2
3 static irqreturn_t interrupt_handler(int irq, void *dev_id)
4 {
5     if (irq == IRQ_NO) {
6         start_time = ktime_get();
7         tasklet_schedule(tasklet);
8         return IRQ_HANDLED;
9     }
10    return IRQ_NONE;
11 }
12
13 static int __init my_init(void)
14 {
15     int ret = request_irq(IRQ_NO, interrupt_handler, IRQF_SHARED,
16                          "interrupt_handler_tasklet", (void *) (interrupt_handler));
17     if (ret) return ret;
18
19     tasklet = kcalloc(sizeof(struct tasklet_struct), GFP_KERNEL);
20     if (!tasklet) {
21         free_irq(IRQ_NO, (void *) (interrupt_handler));
22         return -ENOMEM;
23     }
24     tasklet_init(tasklet, my_tasklet_fun, 0);
25     return 0;
26 }
27
28 static void __exit my_exit(void)
29 {
30     tasklet_kill(tasklet);
31     kfree(tasklet);
32     free_irq(IRQ_NO, (void *) (interrupt_handler));
33 }

```


2

2 Управление внешними устройствами: специальные файлы устройств, идентификация внешних устройств в системе (тип `dev_t`). Система прерываний: типы прерываний и их особенности. Быстрые и медленные прерывания. Обработчики аппаратных прерываний: регистрация в системе. Линия IRQ (простейшая концептуальная схема — три шины). Верхние и нижние `softirq` — перечисления и функции ядра для выполнения `softirq`, тасклеты и очереди работ. К `softirq` — схема обслуживания входящих сетевых пакетов. `Softirq` и `tasklet`: объявление, инициализация, планирование тасклетов (примеры из лаб раб).

Существует два типа внешних устройств:

- Байт-ориентированные (или символьные) устройства передают данные посимвольно, как непрерывный поток байтов (например, мышь, клавиатура).
- Блок-ориентированные (или блочные) устройства (например, жёсткий диск) передают данные блоками. Все запоминающие устройства относятся к блочным.

Чтобы управлять всем разнообразием внешних устройств, в Unix/Linux реализована концепция драйверов. Она заключается в том, что устройства должны быть зарегистрированы в системе и в системе существует набор драйверов, которые можно использовать для управления новыми устройствами. Драйвер — Это часть кода ядра, которая предназначена для управления конкретным внешним устройством.

1)

В Linux используется парадигма «всё файл», поэтому устройство — тоже файл. Но в полном объёме невозможно взаимодействовать как с обычными файлами. Устройства в системе должны идентифицироваться. Для этого в системе определён специальный подход. Файлы внешних устройств находятся в директории `/dev`. В ней с помощью `ls -l` видим внешние устройства.

1	\$ ls -l /dev									
2	crw-rw-rw-	1	root	root	1,	3	Apr	11	2002	null
3	crw-----	1	root	root	10,	1	Apr	11	2002	psaux
4	crw-----	1	root	root	4,	1	Oct	28	03:04	tty1
5	crw-rw-rw-	1	root	tty	4,	64	Apr	11	2002	ttys0
6	crw-rw----	1	root	uucp	4,	65	Apr	11	2002	ttyS1
7	crw--w----	1	vcsa	tty	7,	1	Apr	11	2002	vcs1
8	crw--w----	1	vcsa	tty	7,	129	Apr	11	2002	vcsa1
9	crw-rw-rw-	1	root	root	1,	5	Apr	11	2002	zero

Старший номер идентифицирует драйвер, связанный с устройством. Например, `/dev/null` и `/dev/zero` управляются драйвером 1, виртуальные терминалы — драйвером 4.

Младший номер используется для различия конкретных устройств в рамках одного драйвера (например, жёсткий диск имеет один старший номер, а его разделы — разные младшие).

Стандарт POSIX-1 оговаривает существование типа `dev_t` — 32-разрядное число, в котором 12 бит отведены под старший номер, 20 — под младший. Код не должен делать предположений об организации номеров внутри числа. Для работы с `dev_t` используются макросы:

```
1 MAJOR(dev_t dev); // получить старший номер
2 MINOR(dev_t dev); // получить младший номер
```

Имея старший и младший номера, можно их объединить в значение типа `dev_t`.

```
1 MKDEV(int major, int minor) // объединить в dev_t
```

2.1 Выделение и освобождение номеров устройств

Одно из первых действий, которые должен сделать драйвер, когда устанавливает символьное устройство, он должен получить 1 или более номеров устройств, чтобы работать с ними, чтобы идентифицировать свой `char device`. Для этого вызывается функция

```
1 int register_chrdev_region(dev_t first, unsigned int count, char *name);
```

— `first` — начальный номер устройства доступного для выделения диапазона

Часто минорная часть значения `first` равна нулю. Но этот счёт нет никаких определённых требований.

— `count` — общее количество последовательных номеров устройств, которые могут быть запрошены. При этом значение `count` может превышать размер запрашиваемого диапазона. В этом случае будет переход на следующий старший номер, ошибка не возникает, если запрашиваемый диапазон доступен.

— `*name` — имя устройства, которое связывается с диапазоном. Это можно увидеть или в `/proc/devices` или в `sysfs (/sys)`.

При успешном выделении функция возвращает 0.

Часто неизвестно значение старшего номера устройства, поэтому разработчики ядра всё время стремятся перейти на динамическое распределение номеров устройств. Для динамического выделения используется функция

```
1 int alloc_chrdev_region(dev_t *dev, unsigned int firstminor, unsigned int count,
    char *name);
```

— `*dev`

Значение указателя при успешном завершении будет содержать первый номер в определённом диапазоне. Обычно указывают 0

Остальные параметры аналогичны рассмотренной ранее функции.

Номера освобождаются функцией

```
1 void unregister_chrdev_region(dev_t first, unsigned int count);
```

Все эти функции вызываются в модуле в функции `init`, а последняя функция — в функции `exit`.

2.2 Система прерываний: типы прерываний и их особенности.

Три типа событий, переводящих систему в режим ядра:

- 1) Системные вызовы (программные прерывания) — процесс переключается в режим ядра и выполняет реентерабельный код ядра, после переключается в режим пользователя.
- 2) Исключения — синхронные. Два типа: исправимые (страничное прерывание, после обработки программа продолжает выполнение) и неисправимые (программа завершается аварийно).
- 3) Аппаратные прерывания — асинхронные. Три типа:
 - 3.1) от системного таймера (единственное периодическое прерывание в системе);
 - 3.2) от устройств ввода/вывода. Прерывание информирует процесс о завершении операции ввода/вывода. При этом, каждую такую операцию инициирует сам процессор.
 - 3.3) от действий оператора (использование горячих клавиш).

Синхронные — возникают в процессе выполнения программы; Асинхронные — не зависят ни от каких действий в системе.

2.3 Быстрые и медленные прерывания

В ОС Linux различают быстрые и медленные прерывания:

1) Быстрые:

Обработка быстрых прерываний выполняется от начала до конца. В современных системах осталось единственное быстрое прерывание — прерывание от таймера, вытеснить его невозможно. При этом выполняется на процессоре 0 и *условно* этот процессор считается главным.

2) Медленные:

Все остальные аппаратные прерывания, то есть прерывания от устройств ввода/вывода (внешних устройств), делятся на две части:

2.1) top half (верхняя половина):

2.1.1) interrupt (hardware — аппаратное)

2.2) bottom half (нижняя половина):

2.2.1) softirq

2.2.2) tasklet

2.2.3) workqueue

Пример быстрого прерывания — прерывание от системного таймера. Главные задачи его обработчика:

1) Инициализировать планировщик.

Ставит в очередь на выполнение очень важные коды:

- Планировщик
- Диспетчер
- swapper

2) Инициализировать диспетчеризацию.

Он не может выполняться долго. Если он выполняется долго, это приведёт к снижению **отзывчивости** системы.

2.4 Прерывания в последовательности ввода-вывода.

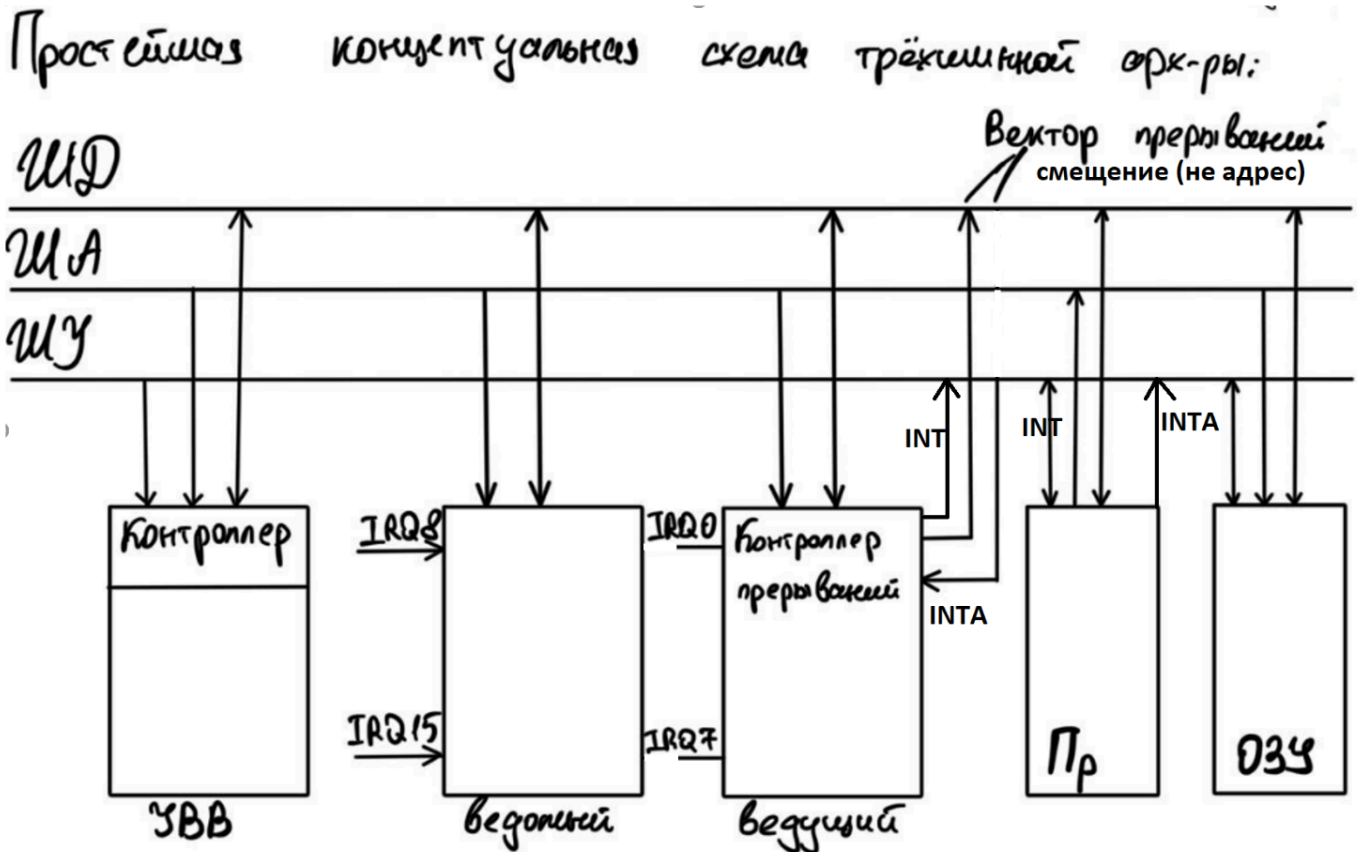


Рис. 2. Простейшая концептуальная схема

В компьютерах передаются сигналы трёх типов: данные (данные и команды), адреса, сигналы управления — отсюда три шины: шина данных, шина адреса, шина управления.

Обработка запроса прерывания по этой схеме:

1) На линию **IRQ** приходит сигнал от внешнего устройства.

- 2) Когда на контроллер прерываний приходит прерывание, он формирует сигнал INT, который по шине управления поступает на выделенную ножку процессора.
- 3) В конце цикла выполнения команды процессор проверяет наличие сигнала на выделенной ножке.
- 4) Если сигнал пришёл — процессор посылает контроллеру сигнал INTA (подтверждение).
- 5) Получив INTA, контроллер по шине данных выставляет вектор прерывания (базовый вектор + номер линии IRQ).
- 6) Процессор считывает вектор прерывания, переводит его смещение по таблице и считывает адрес обработчика.

2.5 Регистрация аппаратных прерываний в системе

2.6 Обработчики аппаратных прерываний: регистрация, функция и её параметры

Чтобы зарегистрировать в системе наш обработчик прерывания, в ядре имеется функция

```
1 static inline int __must_check request_irq(unsigned int irq, irq_handler_t handler,
2     unsigned long flags, const char *name, void *dev)
3 {
4     return request_threaded_irq(irq, handler, NULL, flags | IRQF_COND_ONESHOT, name,
5     dev);
6 }
```

Параметры:

- `irq`: номер прерывания. IRQ0 — прерывание от таймера, IRQ1 — от клавиатуры.
- `handler`: наш обработчик прерывания
- `flags`: флаги Interrupt ReQuest Flags (IRQF)
- `name`: имя, которое увидим в `/proc/interrupts`

Это функция обертка над `request_threaded_irq`, это базовая функция. При ее вызове в параметр `irq_handler_t thread_fn` равен NULL.

Флаги:

```
1 #define IRQF_SHARED          0x00000080
2 #define IRQF_PROBE_SHARED    0x00000100
3 #define __IRQF_TIMER          0x00000200
4 #define IRQF_PERCPU          0x00000400
5 #define IRQF_NOBALANCING      0x00000800
6 #define IRQF_IRQPOLL         0x00001000
7 #define IRQF_ONESHOT         0x00002000
8 #define IRQF_NO_SUSPEND      0x00004000
```

```

9 #define IRQF_FORCE_RESUME 0x00008000
10 #define IRQF_NO_THREAD    0x00010000
11 #define IRQF_EARLY_RESUME 0x00020000
12 #define IRQF_COND_SUSPEND 0x00040000
13 #define IRQF_NO_AUTOEN     0x00080000
14 #define IRQF_NO_DEBUG      0x00100000
15 #define IRQF_COND_ONESHOT  0x00200000
16
17 #define IRQF_TIMER          (__IRQF_TIMER | IRQF_NO_SUSPEND | IRQF_NO_THREAD)

```

- `IRQF_SHARED` — Этот флаг как раз и позволяет разделять линию прерывания несколькими обработчиками.
- `IRQF_PROBE_SHARED` — когда мы «не уверены»?
- `__IRQF_TIMER` — от системного таймера.
- `IRQF_NO_THREAD` — не может быть распараллелен.

Структура, описывающая обработчик:

```

1 struct irqaction {
2     irq_handler_t handler;
3     void *dev_id;
4     void __percpu *percpu_dev_id;
5     struct irqaction *next;
6     irq_handler_t thread_fn;
7     struct task_struct *thread;
8     struct irqaction *secondary;
9     unsigned int irq;
10    unsigned int flags;
11    unsigned long thread_flags;
12    unsigned long thread_mask;
13    const char *name;
14    struct proc_dir_entry *dir;
15 };

```

Поле `dir` заполняется автоматически на основе параметров, переданных `request_irq`.

`dir` — pointer to the `proc/irq/NN/name` entry.

Возвращаемые значения обработчика:

```

1 enum irqreturn {
2     IRQ_NONE          = (0 << 0), // прерывания не было или не обработано
3     IRQ_HANDLED       = (1 << 0), // прерывание обработано
4     IRQ_WAKE_THREAD   = (1 << 1), // запрос пробуждения потока хендлера

```

```

5 };
6
7 typedef enum irqreturn irqreturn_t;

```

Когда драйвер выгружается, необходимо отменить регистрацию соответствующего обработчика прерывания. Для этого существует функция:

```

1 void free_irq(unsigned int irq, void *dev_id);

```

Если указанная линия прерывания не является разделяемой, то функция `free_irq()` удаляет обработчик и отключает линию. Если линия прерывания разделяется, то обработчик, определённый `dev_id` удаляется, но линия прерывания будет отключена только при удалении последнего обработчика.

2.7 softirq (гибкие прерывания)

`softirq` (гибкие прерывания) — отложенные действия, определяются статически при компиляции. `softirq` инициализируется при загрузке системы. Если надо написать свой собственный `softirq`, то необходимо перекомпилировать ядро (изменить `enum`). Из версии ядра в версию ядра (как заклинание) разработчики переписывают: «пожалуйста, откажитесь от выделения нового `softirq`, если действительно не нужна высокая частота "threaded job scheduling"».

В настоящее время в системе определено 10 `softirq`.

```

1 enum
2 {
3     HI_SOFTIRQ = 0,
4     TIMER_SOFTIRQ,
5     NET_TX_SOFTIRQ,
6     NET_RX_SOFTIRQ,
7     BLOCK_SOFTIRQ,
8     IRQ_POLL_SOFTIRQ,
9     TASKLET_SOFTIRQ,
10    SCHED_SOFTIRQ,
11    HRTIMER_SOFTIRQ,
12    RCU_SOFTIRQ,      /* Preferable RCU should always be the last softirq */
13    NR_SOFTIRQS
14 };

```

Индекс	Приоритет	Назначение
HI_SOFTIRQ	0	Высокоприоритетные <code>softirq</code>
TIMER_SOFTIRQ	1	Таймер
NET_TX_SOFTIRQ	2	Отправка сетевых пакетов

Индекс	Приоритет	Назначение
NET_RX_SOFTIRQ	3	Приём сетевых пакетов
BLOCK_SOFTIRQ	4	Блочные устройства
IRQ_POLL_SOFTIRQ	5	Опрос блочного устройства
TASKLET_SOFTIRQ	6	Тасклет
SCHED_SOFTIRQ	7	Планировщик
HRTIMER_SOFTIRQ	8	Не используется
RCU_SOFTIRQ	9	Обязательно должен быть последним

Таблица 1. Типы softirq

```
1 const char * const softirq_to_name[NR_SOFTIRQS] = {
2   "HI", "TIMER", "NET_TX", "NET_RX", "BLOCK", "IRQ_POLL",
3   "TASKLET", "SCHED", "HRTIMER", "RCU"
4 };
```

Размер вектора 32, но реально определено 10. Меньшие номера имеют более высокий приоритет.

softirq NET_TX_SOFTIRQ и NET_RX_SOFTIRQ относятся к сетевой подсистеме. За выполнение softirq отвечает softirqd.

Когда возникает необходимость выполнения softirq — соответствующее аппаратное прерывание записывает в битовую строку бит pending. ksoftirqd проверяет эту маску и если в неё была записана соответствующая единица, запускает на выполнение соответствующий softirq (do_softirq).

softirq одного типа может выполняться параллельно на разных процессорах одновременно (сериализуются). Это сделано с целью ускорения обслуживания соответствующих аппаратных подсистем — ярким примером является сетевая подсистема. Поэтому в коде softirq должно быть реализовано взаимное исключение в полном объёме, код должен быть реентерабельным.

Важно

softirq никогда не вытесняет другой softirq. Единственное событие, которое может вытеснить softirq — это аппаратное прерывание.

Функции для работы с softirq:

```
1 void open_softirq(int nr, void (*action)(void)); // регистрация обработчика
2 void raise_softirq(unsigned int nr); // планирование softirq на текущем CPU
```

- `open_softirq()` выполняет регистрацию обработчика отложенного прерывания: ей передаются два параметра — индекс отложенного прерывания (индекс элемента массива `softirq_vec`) и функция-обработчик. Вызывается во время инициализации.
- Перед тем как зарегистрированное отложенное прерывание будет запущено, оно должно быть промаркировано — этот процесс называется генерацией отложенного прерывания (`raise_softirq`). `raise_softirq()` вызывается из обработчика аппаратного прерывания и помечает `softirq` с указанным номером как ожидающий (`pending`) на текущем процессоре.

Пример (обработчик аппаратного прерывания планирует `softirq`):

```

1 irq_return_t my_interrupt(int irq, void *dev_id) {
2     raise_softirq(MY_SOFT_IRQ); /* mark softirq pending */
3     ...
4     return IRQ_HANDLED;
5 }
6
7 void my_soft_handler(struct softirq_action *) { ... }
8
9 static int __init my_init(void) {
10     request_irq(IRQ_N0, my_interrupt, 0, "my_irq_handler", NULL);
11     open_softirq(MY_SOFT_IRQ, my_soft_handler);
12 }
```

2.8 Схема обслуживания входящих сетевых пакетов (NET_RX_SOFTIRQ)

NAPI — New API. Сети были описаны очень давно, никто не мог предполагать современное состояние сетей, когда передаются огромные потоки данных. В частности, протокол TCP стал проблемой сетей: возникли DDoS-атаки, они работают из-за особенностей протокола TCP. Возможна ситуация, когда на определённый хост поступает большое количество пакетов с большой частотой. Каждый приход пакета — обработка прерывания, что является затратным действием: выполняется на самом высоком уровне привилегий, и, обрабатывая пакеты, система может потерять работоспособность. Поэтому был написан NAPI — он переводит сетевую систему в **режим опроса** (`poll`). Основная цель NAPI — сократить число прерываний при приёме пакетов.

Схема обслуживания входящих сетевых пакетов (NET_RX_SOFTIRQ):

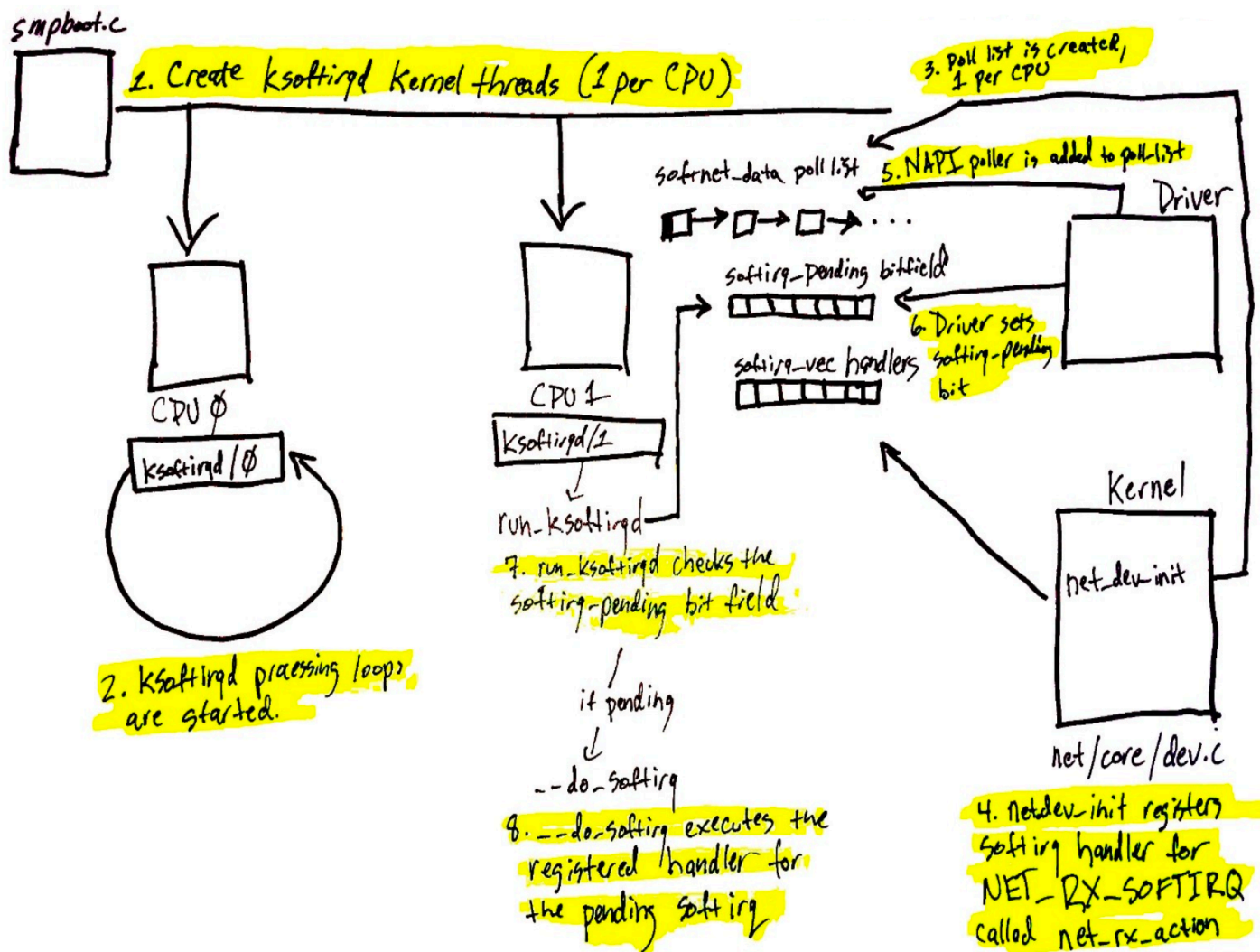


Рис. 3. `softirq` — `NET_RX_SOFTIRQ` и `ksoftirqd`

* Claude

Пояснение пунктов по картинке от Claude:

- 1) При загрузке системы (`smpboot.c`) создаётся по одному потоку ядра `ksoftirqd` на каждый CPU.
- 2) Запускаются processing loop'ы `ksoftirqd` — потоки входят в цикл обработки.
- 3) Создаётся `softnet_data` poll list — по одному на каждый CPU.
- 4) `net_dev_init` (`net/core/dev.c`) регистрирует обработчик `softirq` для `NET_RX_SOFTIRQ` — функцию `net_rx_action`.
- 5) Когда подключается сетевое устройство, его NAPI poller добавляется в poll list.
- 6) Приходит пакет → драйвер устанавливает бит в `softirq_pending` bitfield.
- 7) `run_ksoftirqd` проверяет `softirq_pending` bitfield — если бит выставлен (it pending), передаёт управление `__do_softirq`.
- 8) `__do_softirq` выполняет зарегистрированный обработчик для pending softirq — то есть вызывает `net_rx_action`, которая обходит poll list и опрашивает драйверы на наличие пакетов.

Пояснение от Рязановой: Для каждого процессора имеется свой softirqd. Чтобы он начал работу вызывается функция run softirqd. softirq pending («Дать пендаля»)

Если соответствующее битовое поле включено для соответствующего softirq, то вызывается функция do_softirq. Она выполняет зарегистрированный хендлер для добавленного softirq. Эту информацию задает драйвер.

2.9 Тасклеты

Тасклеты не сериализуются: один тип тасклета может выполняться только на одном процессоре одновременно, т.е. средства взаимного исключения не требуются. Разные тасклеты могут выполняться параллельно. Тасклет — компромисс между производительностью и простотой использования.

```
1 struct tasklet_struct {
2     struct tasklet_struct *next; // указатель на следующий таскет в списке
3     unsigned long state;
4     atomic_t count;
5     bool use_callback;
6     union {
7         void (*func)(unsigned long data); // функция-обработчик тасклета
8         void (*callback)(struct tasklet_struct *t);
9     };
10    unsigned long data; // аргумент функции-обработчика
11 };
12
13 enum {
14     TASKLET_STATE_SCHED, /* Tasklet is scheduled for execution */
15     TASKLET_STATE_RUN    /* Tasklet is running (SMP only) */
16 };
```

Листинг 5. Структура tasklet_struct

Состояния: 0, TASKLET_STATE_SCHED, TASKLET_STATE_RUN. TASKLET_STATE_RUN имеет смысл только в SMP-архитектуре. Таскет может выполняться только если count == 0.

Статическое объявление:

```
1 DECLARE_TASKLET(name, func, /* устаревшее, сейчас 3-го параметра нет */ data);
2
3 #define DECLARE_TASKLET(name, _callback) \
4 struct tasklet_struct name = { \
5     .count = ATOMIC_INIT(0), \
6     .callback = _callback, \
7     .use_callback = true, \
```

Листинг 6. Статическая инициализация tasklet

Динамическое объявление:

```

1 void tasklet_init(struct tasklet_struct *t, void (*func)(unsigned long),
2                 unsigned long data)
3 {
4     t->next = NULL;
5     t->state = 0;
6     atomic_set(&t->count, 0);
7     t->func = func;
8     t->data = data;
9 }
```

Листинг 7. Динамическая инициализация tasklet

Прежде чем инициализировать, нужно выделить память.

```

1 my_tasklet = (struct tasklet_struct *)kmalloc(sizeof(struct tasklet_struct),
        GFP_KERNEL);
```

В exit нужно:

1) free irq

Нужно освободить линию прерывания от нашего обработчика

2) tasklet_kill

Передаем для данного действия my_tasklet

Планирование Для того чтобы запланировать выполнение тасклета, ядро предоставляет две функции:

```

1 extern void __tasklet_schedule(struct tasklet_struct *t);
2
3 static inline void tasklet_schedule(struct tasklet_struct *t)
4 {
5     if (!test_and_set_bit(TASKLET_STATE_SCHED, &t->state))
6         __tasklet_schedule(t);
7 }
8
9 extern void __tasklet_hi_schedule(struct tasklet_struct *t);
10
11 static inline void tasklet_hi_schedule(struct tasklet_struct *t)
12 {
13     if (!test_and_set_bit(TASKLET_STATE_SCHED, &t->state))
14         __tasklet_hi_schedule(t);
15 }
```

2.10 Очереди работ

Альтернатива тасклетам. Тасклет выполняется в контексте прерывания, код тасклета не делим. Работа выполняется в контексте специального потока ядра и может быть заблокирована. Тасклет всегда выполняется на процессоре, на котором завершилась обработка прерывания. Работу можно выполнить на другом процессоре и отложить на некоторое время.

Основные понятия все между собой связаны. Везде чаще встречаются сокращения имён.

1) **work**

Для **work** определяется функция, которая будет выполнять соответствующие действия.

2) **workqueue**

work необходимо поставить в **очередь работ**, которую мы для **work** создаём.

3) **worker (work_thread)**

Поток ядра.

4) **worker_pool**

отношение **worker_pool** и **worker** — 1 к 1

5) **pool_workqueue (pwq)**

Устанавливает отношение между **workqueue** и **worker_pool**, причём

- **workqueue** и **pwq** — один ко многим
- **pwq** и **worker_pool** — один к одному

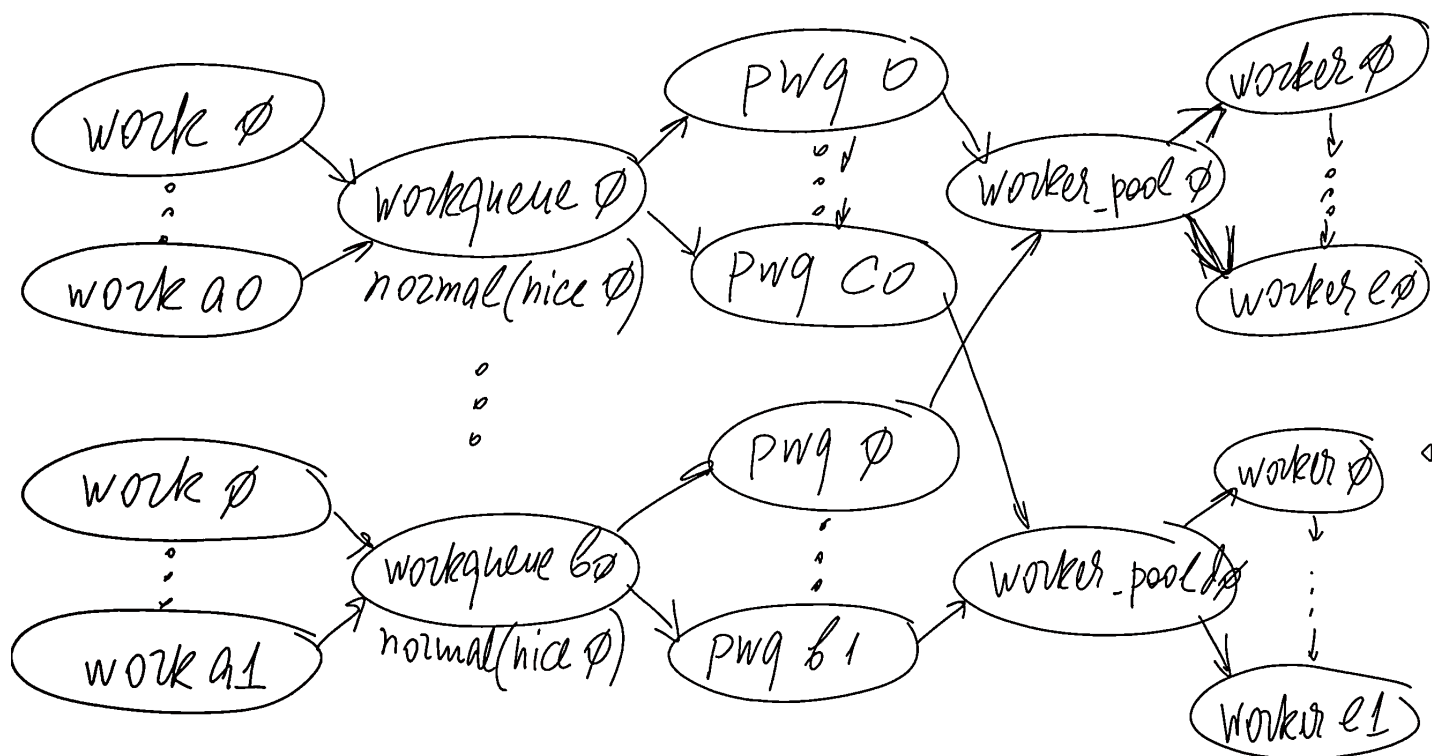


Рис. 4. Очереди работ

— work ставятся в workqueue

2.11 worker

worker — поток ядра.

Все задействованные worker'ы рано или поздно будут обслужены

Пулы worker'ов — per CPU. Мы это видели в первом модуле. Мы видели, что их определено много для каждого CPU. (Не писать, но может спросить, я думаю)

Смысл картинки: всё per CPU, поэтому определены эти пулы, сосредоточение worker'ов. Главное, что work ставится в workqueue, а он в итоге будет выполняться worker'ом, который принадлежит worker_pool

```

1 struct worker {
2     ...
3     struct work_struct *current_work; /* K: work being processed and its */
4     work_func_t current_func; /* K: function */
5     struct pool_workqueue *current_pwq; /* K: pwq */
6     ...
7     int sleeping; /* S: is worker sleeping? */
8     ...
9     struct list_head scheduled; /* L: scheduled works */
10    struct task_struct *task; /* I: worker task */
11    struct worker_pool *pool; /* A: the associated pool */
12    ...

```

```

13 unsigned int flags; /* L: flags */
14 int id; /* I: worker id */
15 ...
16 struct workqueue_struct *rescue_wq; /* I: the workqueue to rescue */
17 };

```

Листинг 9. Структура worker

- Работа, которая будет выполнена рабочим потоком. Смысл этого потока в том, что она планируется планировщиком ядра. Мы эту информацию получали, когда выводили task_struct

```

1 struct worker_pool {
2     ...
3     int    cpu; /* I: the associated cpu */
4     ...
5     int    id; /* I: pool ID */
6     unsigned int flags; /* L: flags */
7     ...
8     int    nr_running;
9     int    nr_workers; /* L: total number of workers */
10    int    nr_idle; /* L: currently idle workers */
11
12    struct list_head idle_list; /* L: list of idle workers */
13    struct timer_list idle_timer; /* L: worker idle timeout */
14    struct work_struct idle_cull_work; /* L: worker idle cleanup */
15
16    struct timer_list mayday_timer; /* L: SOS timer for workers */
17
18    /* a workers is either on busy_hash or idle_list, or the manager */
19    DECLARE_HASHTABLE(busy_hash, BUSY_WORKER_HASH_ORDER);
20    /* L: hash of busy workers */
21
22    struct worker *manager; /* L: purely informational */
23    struct list_head workers; /* A: attached workers */
24
25    struct ida worker_ida; /* worker IDs for task name */
26
27    struct workqueue_attrs *attrs; /* I: worker attributes */
28    struct hlist_node hash_node; /* PL: unbound_pool_hash node */
29    int refcnt; /* PL: refcnt for unbound pools */
30    #ifdef CONFIG_PREEMPT_RT
31    spinlock_t cb_lock; /* BH worker cancel lock */
32    #endif
33    /*

```

```

34  * Destruction of pool is RCU protected to allow dereferences
35  * from get_work_pool().
36  */
37  struct rcu_head  rcu;
38  };

```

Листинг 10. Структура worker_pool

— Раз это пул, значит, самое главное — списки.

— nr_workers, nr_idle

Нужны для работы планировщика. Планировщик должен предоставить всем процессам примерно одинаковое процессорное время, то есть дать возможность выполняться всем процессам.

```

1  // статическая инициализация
2  DECLARE_WORK(name, void (*func)(void *));
3
4  // динамическая инициализация
5  INIT_WORK(struct work_struct *work, void (*func)(void *), void *data);
6
7  // постановка в очередь
8  int queue_work(struct workqueue_struct *wq, struct work_struct *work);
9  int queue_work_on(int cpu, struct workqueue_struct *wq, struct work_struct *work);
10 int queue_delayed_work(struct workqueue_struct *wq, struct work_struct *work,
11                       unsigned long delay);
12
13 void destroy_workqueue(struct workqueue_struct *wq);

```

Критерий сравнения	tasklet	workqueue
Контекст выполнения	Контекст прерывания того обработчика, который поставил его на выполнение. Обычно tasklet выполняется сразу после первой половины.	Специальный поток ядра. Планирует его стандартный обработчик.
Возможность блокироваться	—	+
Процессор, на котором выполняется	Только на том, который и поставил tasklet на выполнения	По умолчанию как у tasklet, но это можно настроить с помощью флага WQ_UNBOUND

Таблица 2. Сравнение очередей работ и тасклетов

Пример из лабораторной работы (обработчик прерывания клавиатуры с тасклетом):

```
1 static struct tasklet_struct *tasklet = NULL;
2 static irqreturn_t interrupt_handler(int irq, void *dev_id) {
3     if (irq == IRQ_NO) {
4         start_time = ktime_get();
5         tasklet_schedule(tasklet);
6         return IRQ_HANDLED;
7     }
8     return IRQ_NONE;
9 }
10
11 static int __init my_init(void) {
12     int ret = request_irq(IRQ_NO, interrupt_handler, IRQF_SHARED,
13                          "interrupt_handler_tasklet", (void *) (interrupt_handler));
14     if (ret) return ret;
15     tasklet = kmalloc(sizeof(struct tasklet_struct), GFP_KERNEL);
16     if (!tasklet) {
17         free_irq(IRQ_NO, (void *) (interrupt_handler));
18         return -ENOMEM;
19     }
20     tasklet_init(tasklet, my_tasklet_fun, 0);
21     return 0;
22 }
23
24 static void __exit my_exit(void) {
25     tasklet_kill(tasklet);
26     kfree(tasklet);
27     free_irq(IRQ_NO, (void *) (interrupt_handler));
28 }
```

Пример из лабораторной работы (обработчик прерывания клавиатуры с workqueue):

```
1 static irqreturn_t irq_handler(int irq, void *dev_id) {
2     if (irq == IRQ_NO) {
3         start_time = ktime_get();
4         tasklet_schedule(tasklet);
5         return IRQ_HANDLED;
6     }
7     return IRQ_NONE;
8 }
9
10
```

```

11 static int __init my_init(void)
12 {
13     int ret;
14     wq = alloc_workqueue("irq_wq", WQ_MEM_RECLAIM, 2);
15     if (!wq)
16         return -ENOMEM;
17     work1 = kmalloc(sizeof(*work1), GFP_KERNEL);
18     if (!work1) {
19         destroy_workqueue(wq);
20         return -ENOMEM;
21     }
22     work2 = kmalloc(sizeof(*work2), GFP_KERNEL);
23     if (!work2) {
24         destroy_workqueue(wq);
25         kfree(work1);
26         return -ENOMEM;
27     }
28     INIT_WORK(&work1->work, work1_fn);
29     INIT_WORK(&work2->work, work2_fn);
30     ret = request_irq(IRQ_NUM, irq_handler, IRQF_SHARED, "irq_workqueue",
31                      irq_handler);
32     if (ret) {
33         kfree(work1);
34         kfree(work2);
35         destroy_workqueue(wq);
36         return ret;
37     }
38     if (!proc_create(PROC_NAME, 0444, NULL, &proc_fops)) {
39         free_irq(IRQ_NUM, irq_handler);
40         kfree(work1);
41         kfree(work2);
42         destroy_workqueue(wq);
43         return -ENOMEM;
44     }
45     printk(KERN_INFO "irq_workqueue: loaded\n");
46     return 0;
47 }
48
49 static void __exit my_exit(void) {
50     remove_proc_entry(PROC_NAME, NULL);
51     synchronize_irq(IRQ_NUM);
52     free_irq(IRQ_NUM, irq_handler);
53     flush_workqueue(wq);
54     destroy_workqueue(wq);
55     kfree(work1);

```

```
55     kfree(work2);  
56     printk(KERN_INFO "irq_workqueue: unloaded\n");  
57 }
```

3

3 Сокеты как средство взаимодействия параллельных процессов. Сокеты семейства AF_UNIX и AF_INET. Сокеты в пространстве файловых имён. Сокеты AF_INET, сетевой стек. Классификация моделей ввода-вывода. Мультиплексирование при взаимодействии процессов в распределённых системах по модели клиент сервер. Ввод-вывод с мультиплексированием: мультиплексор `epoll` — особенности (типы и алгоритм работы). Пример из лабораторной работы — сервер производство-потребление с мультиплексированием.

3.1 Сокеты как средство взаимодействия параллельных процессов

Сокет — абстракция точки соединения. Являются универсальным средством меж-процессного взаимодействия: могут служить как для взаимодействия процессов на отдельно стоящей машине (память, процессор — всё в одном месте.), так и для взаимодействия в распределённых системах (много машин, соединённых кабелями, и у каждой машины своя память). Были разработаны Unix BSD, поэтому часто называются сокетами Беркли или сокетами BSD.

Определение 3.1 (Сокет)

Сокет — абстракция точки соединения.

Абстракция от особенностей взаимодействия. Средство универсальное: на конкретной машине особенности определяются тем, что это процессор и память, а в распределённой системе — тем, что память распределённая.

В компьютере обращение к данным всегда выполняется по адресу, но на отдельно стоящей машине один способ обращения, а в распределённых системах — другой, поскольку нужно иметь адреса хостов.

Разработаны парные сокеты — отдельный системный вызов `socketpair()`. Парные сокеты сделаны **как альтернатива** `pipe`, но в отличие от последних, которые обеспечивают симплексную связь, сокеты имеют **дуплексную связь**.

Все остальные сокеты создаются системным вызовом `socket()`:

```
1 int socket(int family, int type, int protocol);
```

Системный вызов `socket()` создает точку соединения и возвращает файловый дескриптор, который ссылается на эту точку. При успешном вызове возвращает файловый дескриптор с наименьшим номером дескриптора файла, который в данный момент еще не открыт для процесса.

3.2 Сокеты семейства AF_UNIX и AF_INET

Параметры системного вызова `socket()`:

- 1) `family` — пространство (семейство) адресов.
 - 1.1) `AF_UNIX` — сокеты для взаимодействия параллельных процессов на отдельно стоящих машинах в пространстве файловых имён.
 - 1.2) `AF_INET` — сокеты семейства протоколов TCP/IP основанные основанные на взаимодействии процессов в сети интернет IPv4
 - 1.3) `AF_INET6` — IPv6
 - 1.4) `AF_IPX`
 - 1.5) `AF_UNSPEC`
- 2) `type` — тип сокета. В зависимости от того, где используется сокет, у него будет соответствующий тип.
 - 2.1) `SOCK_STREAM` — определяет ориентированное на потоки надёжное упорядоченное полнодуплексное логическое соединение двух сокетов (между двумя сокетами).
 - 2.2) `SOCK_DGRAM` — определяет ненадёжную службу дейтаграмм (datagram) без установления логического соединения, где пакеты могут передаваться без сохранения порядка. Это широковещательная передача данных.
 - 2.3) `SOCK_RAW` — прямой (сырой, низкоуровневый) сокет.
- 3) `protocol` — определяет конкретный протокол, который будет использоваться с сокетом. Обычно параметр устанавливается равным 0, тогда протокол задаётся по умолчанию. Протоколы обозначаются символьными константами с префиксами `IPPROTO_*`, например `IPPROTO_TCP`.
 - По умолчанию для сокетов `AF_INET` типа `SOCK_STREAM` будет установлен протокол TCP, для этих же сокетов типа `SOCK_DGRAM` — протокол UDP.

Таким образом, сокеты `AF_UNIX` используются для взаимодействия параллельных процессов на отдельно стоящей машине, а сокеты `AF_INET` — для взаимодействия процессов в распределённой системе (в сети).

3.3 Сокеты в пространстве файловых имён

Сокеты BSD поддерживают множество протоколов, поэтому на сокетах была определена общая структура `struct sockaddr`:

```
1 struct sockaddr
2 {
3     sa_family_t sa_family; // AF_XXX
4     char sa_data[14];
5 };
```

Листинг 11. Структура `sockaddr`

Ничего не конкретизировано, просто 14 байт. Структура определена в таком виде, потому что разработчики понимали, что в разных системах нужна разная адресация.

Для сокетов `AF_UNIX` в поле `sa_data` помещается имя файла, поскольку эти сокеты взаимодействуют через пространство файловых имён

В системе сокет определён как открытый файл — системный вызов `socket()` возвращает файловый дескриптор. `struct file` описывает открытый файл, в самой структуре есть указатель на `struct inode`, которая описывает физический файл. `struct inode` в Unix описывает большое количество разных типов файлов — в том числе сокеты в пространстве файловых имён, программные каналы и т.д.

Сокет сервера можно показать в файловой подсистеме: командой `ls -ial` выводится файл сокета с его `inode`, тип файла обозначается буквой `s`.

3.4 Сокеты `AF_INET`, сетевой стек

Для сетей адрес в виде структуры `struct sockaddr` не подходит, мы должны указать сетевой адрес и номер порта, поэтому объявлена структура `struct sockaddr_in` для IPv4. Для IPv6 также объявлена структура `struct sockaddr_in6`.

```
1 struct in_addr
2 {
3     _u32 s_addr;
4 };
5
6 struct sockaddr_in
7 {
8     sa_family_t sin_family;      // AF_INET
9     unsigned short int sin_port; // Номер порта
10    struct in_addr sin_addr;      // Сетевой адрес
11    unsigned char sin_zero[sizeof(struct sockaddr) - sizeof(sa_family_t) -
12                           sizeof(uint16_t) - sizeof(struct in_addr)];
12 };
```

Листинг 12. Структура `sockaddr_in`

В наших системах 2 адресных пространства: физическая память и порты ввода/вывода. Порт — это адрес, но он локальный.

Когда создавался интернет, был изменён порядок байтов. Аппаратный и сетевой порядок байтов:

- `big_endian` — network byte order (сетевой порядок);
- `little_endian` — host byte order (порядок хоста).

Функции для работы с порядком байтов:

1) htons() — host to network short:

```
1 #include <arpa/inet.h>
2 uint16_t htons(uint16_t hostint16);
3 uint32_t htonl(uint32_t hostint32);
```

2) ntohs() — network to host:

```
1 #include <arpa/inet.h>
2 uint16_t ntohs(uint16_t netint16);
3 uint32_t ntohl(uint32_t netint32);
```

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. Эти функции нужно вызывать в обязательном порядке, даже если порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, функция не выполняется.

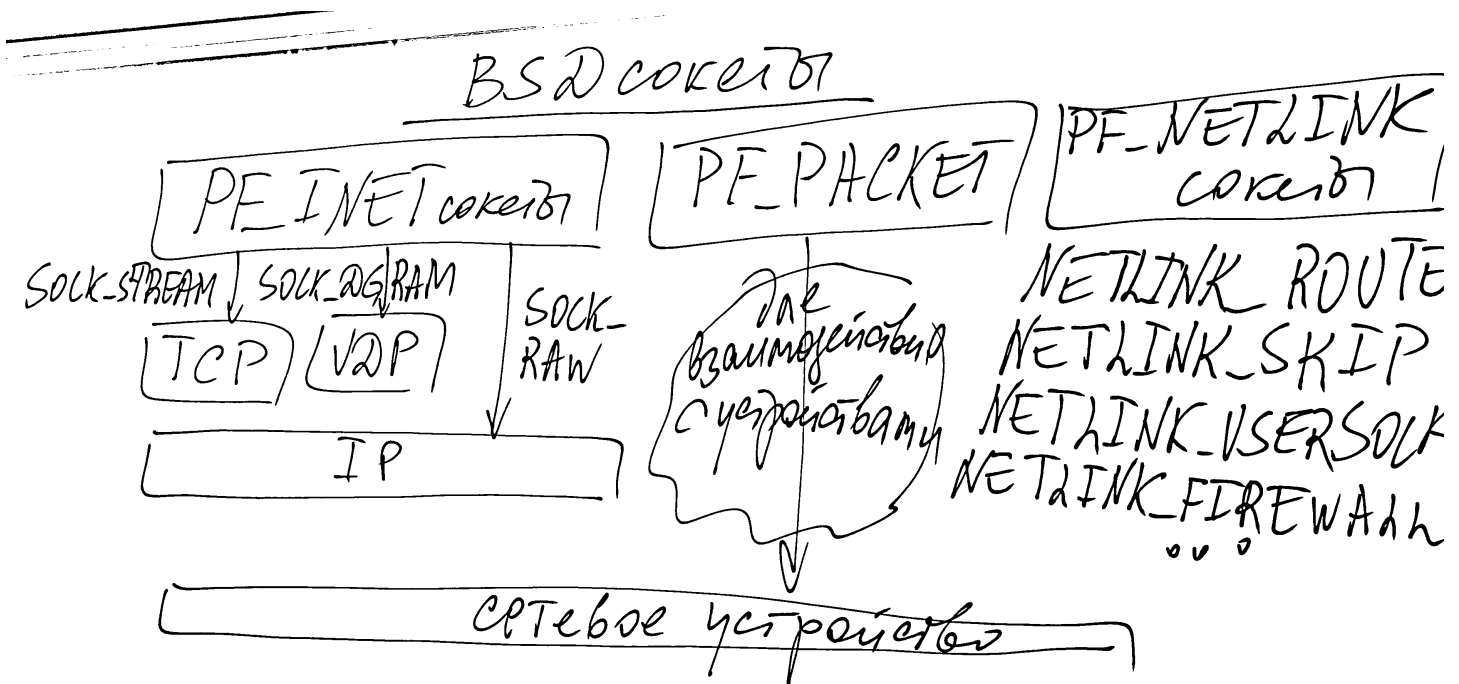


Рис. 5. BSD сокеты

Пакеты приходят на сетевую карту и с неё же уходят.

Для работы с сокетами ядро Linux предоставляет системный вызов `sys_socketcall`. Этот вызов является общей точкой входа для различных операций с сокетами.

```
1 asmlinkage long sys_socketcall(int call, unsigned long *args)
2 {
3     int err;
4     if (copy_from_user(a, args, args[call]))
5         return -EFAULT;
```

```

6
7  a0 = a[0];
8  a1 = a[1];
9
10 switch (call)
11 {
12     case SYS_SOCKET:
13         err = sys_socket(a0, a1, a[2]);
14         break;
15     case SYS_BIND:
16         err = sys_bind(a0, (struct sockaddr *) a1, a[2]);
17         break;
18     case SYS_CONNECT:
19         err = sys_connect(a0, (struct sockaddr *) a1, a[2])
20         break;
21     ...
22 }
23
24 return err;
25 }

```

Листинг 13. Системный вызов `sys_socketcall()`

В качестве одного из аргументов передаётся номер операции: создание сокета, привязка адреса, установление соединения, ожидание соединений, приём или передача данных. Внутри ядра по этому номеру выбирается соответствующая функция, например `socket`, `bind`, `connect`, `listen`, `accept`, `send`, `recv` и другие.

Перед выполнением операции ядро копирует аргументы из пользовательского пространства в пространство ядра. Напрямую обращаться к памяти процесса нельзя, поэтому используются специальные функции вроде `copy_from_user` и `copy_to_user`.

`sys_socketcall()` приводит к вызову `sys_socket()`:

```

1 asmlinkage long sys_socket(int family, int type, int protocol)
2 {
3     int retval;
4     struct socket *sock;
5     ...
6     retval = socket_create(family, type, protocol, &sock);
7     ...
8     return retval;
9 }

```

Листинг 14. Системный вызов `sys_socket()`

В ядре сокет описывает структура `struct socket`:

```
1 struct socket
2 {
3     socket_state state;
4     short type;
5     unsigned long flags;
6     const struct proto_ops *ops; // protocol operations
7     struct fasync_struct *fasync_list;
8     struct file *file;
9     struct sock *sk;
10    wait_queue_head_t wait;
11 };
```

Листинг 15. Структура `socket`

У сокета может быть 5 состояний (`state`):

- 1) `SS_FREE` — свободный сокет.
- 2) `SS_UNCONNECTED` — несоединённый сокет
- 3) `SS_CONNECTING` — сокет находится в процессе соединения
- 4) `SS_CONNECTED` — соединённый сокет
- 5) `SS_DISCONNECTING` — сокет в процессе разъединения

Взаимодействие по модели клиент-сервер на сетевых сокетах:

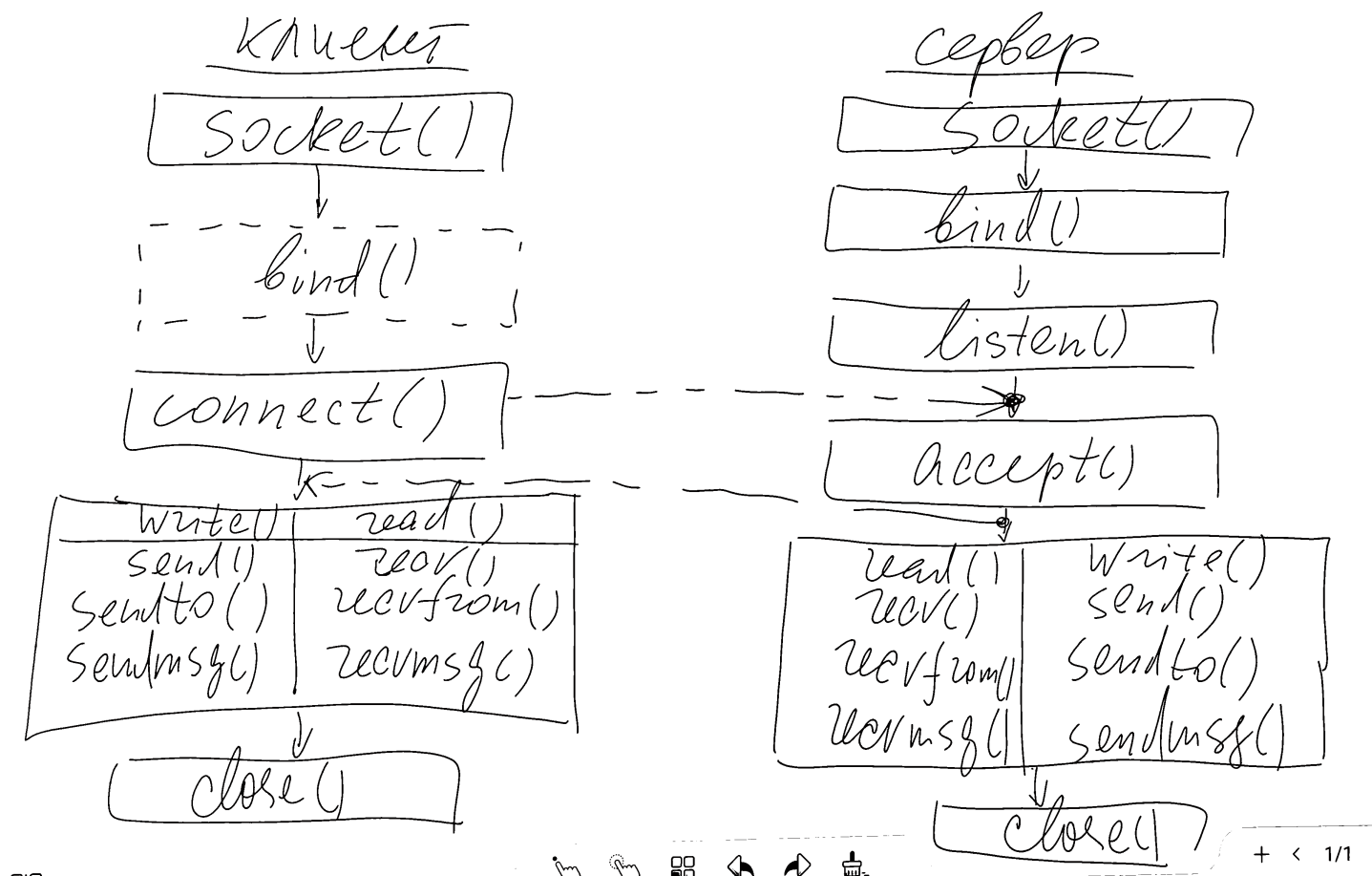


Рис. 6. Клиент-сервер на сокетах

Системный вызов `socket()` создаёт сокет и возвращает файловый дескриптор сокета. Затем на стороне сервера в обязательном порядке вызывается системный вызов `bind()`, который связывает файловый дескриптор сокета с адресом, то есть до `bind()` необходимо инициализировать поля структуры `struct sockaddr_in`. После этого на стороне сервера вызывается системный вызов `listen()`, который переводит серверный процесс в режим пассивного ожидания соединения. Когда соединение произошло, то есть клиент на своей стороне выполнил системный вызов `connect()`, ядро выполняет предварительную регистрацию соединения и выполняется системный вызов `accept()`. После этого начинается обмен данными.

```

1 #include <sys/socket.h>
2 int bind(int sockfd, struct sockaddr *addr, int addrlen);
3 int listen(int sockfd, int backlog);
4 int connect(int sockfd, struct sockaddr *serv_addr, int addrlen);
5 int accept(int sockfd, void *addr, int *addrlen);

```

Листинг 16. Объявления функций

Системный вызов `bind()`. Назначает сокету `sockfd` адрес, указанный `addr`; параметр `addrlen` — размер в байтах структуры адреса, на которую указывает `addr`. Для сетевых сокетов адрес состоит из IP-адреса сетевого интерфейса локальной

системы и номера порта. Клиенты могут не вызывать `bind()`, так как их адрес часто никакой роли не играет — в таком случае адрес назначается им автоматически.

Системный вызов `listen()`. На стороне сервера переводит сервер в режим пассивного прослушивания. Сервер вызывает `listen()` для того, чтобы проинформировать операционную систему, что на сокете нужно принять соединение. Это имеет смысл только для ориентированных на соединение протоколов (TCP). Параметр `backlog` определяет максимальную длину, до которой может вырасти очередь ожидающих соединений для `sockfd`.

Системный вызов `connect()`. Вызывается на стороне клиента, чтобы создать активное соединение. Клиент должен знать адрес сервера (`serv_addr` — адрес сервера, нужно обратить на это внимание). Для протокола TCP активное соединение устанавливается по указанному адресу. Для протокола UDP `connect()` может использоваться для указания адреса назначения для всех передаваемых впоследствии пакетов.

Системный вызов `accept()`. Выполняется на стороне сервера для принятия соединения при условии, что ранее поступил запрос на соединение. В противном случае `accept()` будет блокирован до поступления запроса на соединение.

Важно

Когда соединение принимается, то есть фактически выполняется системный вызов `accept()`, создаётся копия исходного сокета, при этом исходный сокет останется в состоянии `LISTEN`, а созданная копия будет находиться в состоянии `CONNECTED`.

Системный вызов `accept()` возвращает файловый дескриптор созданной копии сокета, то есть нового сокета. Такое дублирование сокетов в ходе принятия соединений даёт серверу возможность продолжать принимать новые соединения без необходимости предварительно закрывать предыдущие.

Сервер блокируется на системном вызове `accept()` до тех пор, пока не поступит запрос на соединение от клиента.

Для протокола TCP определено тройное рукопожатие (clnt: SYN, srv: SYN-ACK, clnt: ACK): сначала клиент обращается к серверу с запросом на установление соединения; получив первое ответное сообщение, клиент так же отправляет подтверждение получения, и только после этого соединение устанавливается. В настоящее время тройное рукопожатие стало огромной проблемой — DDOS-атаки.

3.5 Классификация моделей ввода–вывода

3.5.1 Блокирующий ввод/вывод (blocking I/O)

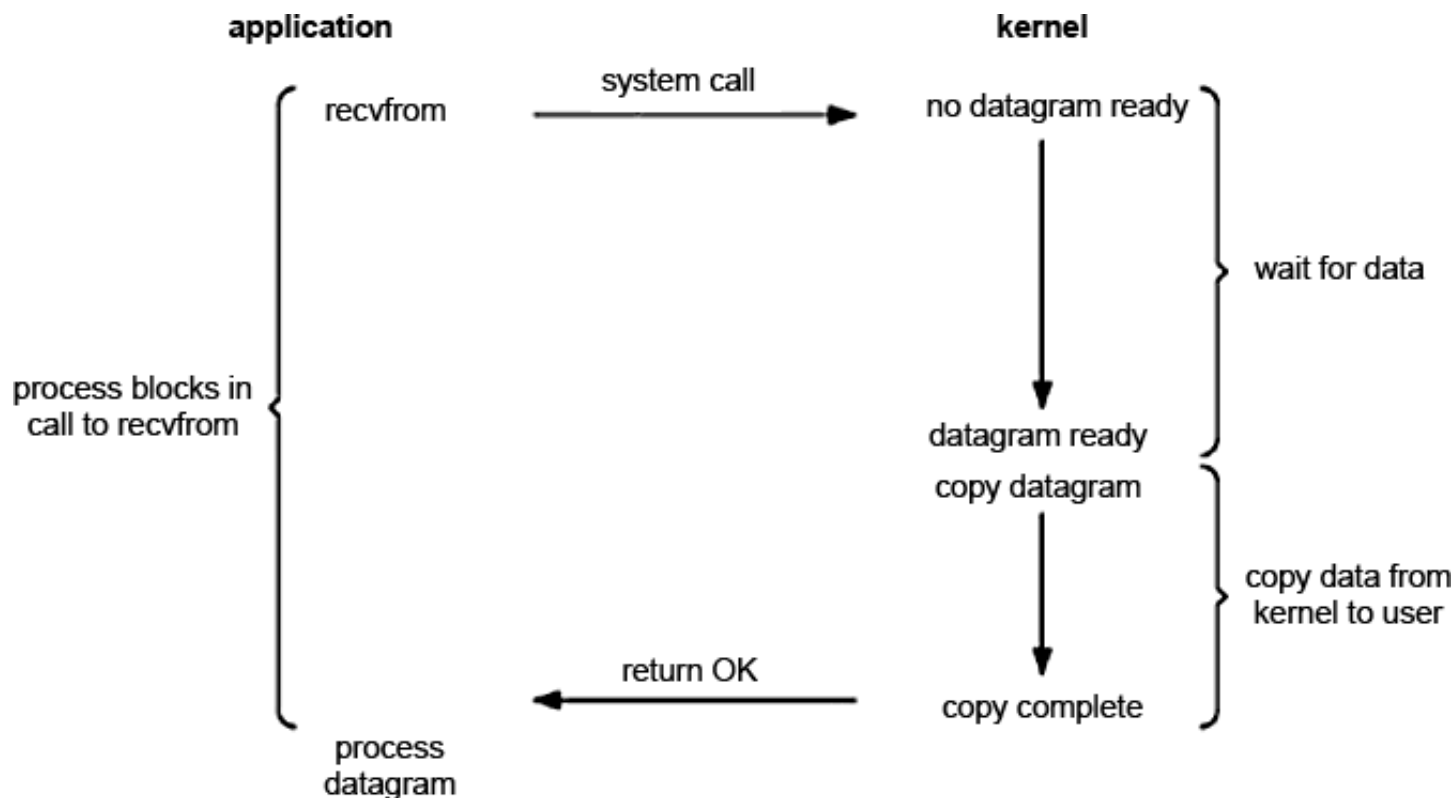


Рис. 7. Блокирующий ввод/вывод

Процесс, выполнивший системный вызов ввода/вывода, будет ожидать получения данных: система вернёт либо код ошибки либо значение, соответствующее успешному выполнению системного вызова.

Когда приложение запрашивает ввод данных с клавиатуры (в примере на картинке — `receive_from`), процесс блокируется до тех пор, пока не получит данные и ядро системы выполняет целую последовательность действий:

- 1) Когда введённые пользователем данные поступят в буфер клавиатуры, контроллер клавиатуры отправит сигнал прерывания на контроллер прерываний.
- 2) С контроллера этот сигнал по шине управления поступит на ножку процессора, и процессор перейдёт на выполнение обработчика прерывания.
- 3) В результате процесс, запросивший ввод, будет разблокирован.

Вся последовательность действий выполняется, пока процесс заблокирован на запросе ввода данных. Это блокирующий ввод/вывод, в основном мы имеем дело именно с ним.

Блокирующий вв/выв (blocking I/r)

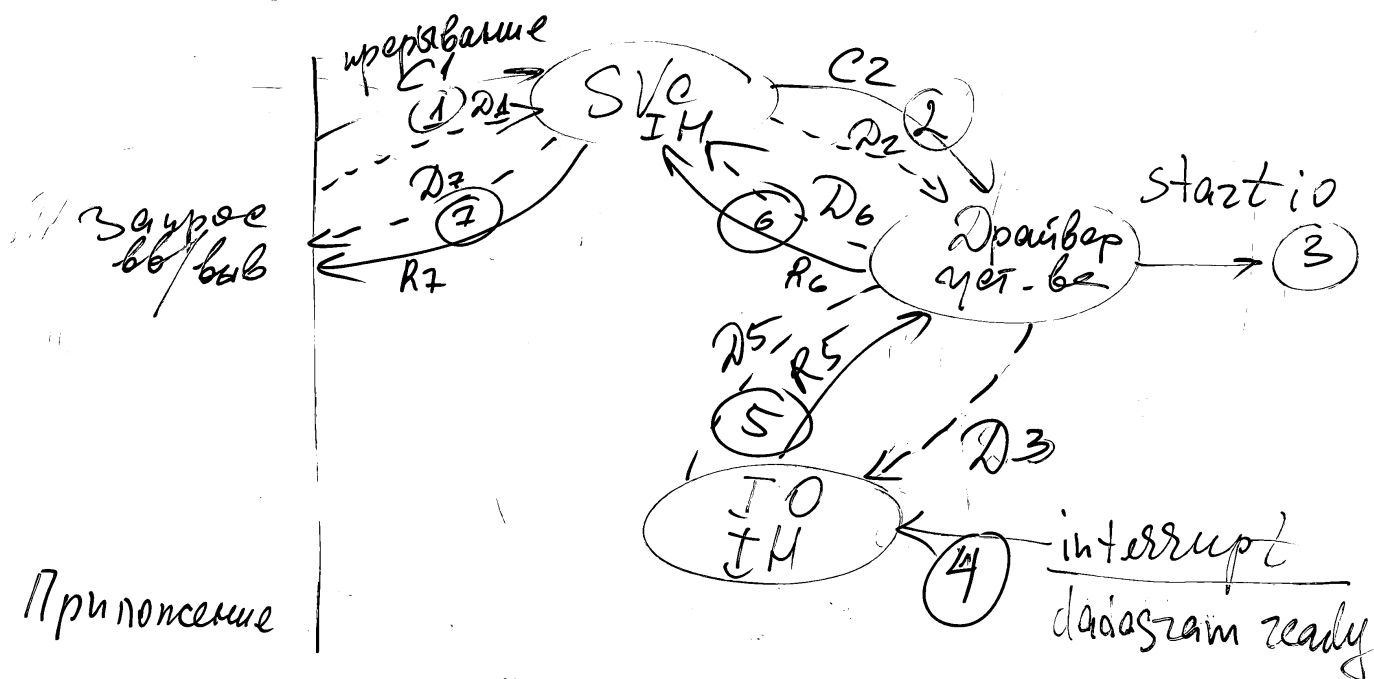


Рис. 8. Блокирующий ввод/вывод. По Шоу

Эта модель была декларирована намного раньше, чем та, которую мы берём из книги по сетям.

Важно

Разрыв справа на схеме говорит о том, что процессор не управляет внешним устройством.

Внизу справа — аппаратное прерывание, которое формируется, когда данные в буфере устройства. Из этого буфера данные пройдут несколько этапов, прежде чем окажутся в буфере приложения, и приложение сможет продолжить своё выполнение.

Важно

В этой модели важно именно то, что процесс блокируется на запросе и разблокируется только тогда, когда данные могут быть процессом получены.

Дальше нужно скопировать, но это выполняет обработчик прерывания. Данные готовы = они поступили в буфер ядра.

Процессор не управляет внешним устройством.

Внизу справа — аппаратное прерывание, которое формируется, когда данные в буфере устройства. Из этого буфера данные пройдут несколько этапов прежде чем окажутся в буфере приложения, и приложение сможет продолжить своё выполнение.

Поскольку схема рассматривает ситуацию в самом общем виде, на ней показан `supervisor call interrupt handler` — функция, которая вызывается в режиме ядра в ответ на системный вызов. В общем виде последовательность блокирующего ввода/вывода:

- 1) Запрос ввода/вывода — это системный вызов; при переходе в режим ядра он заблокирует приложение, то есть его состояние будет изменено на `D (uninterruptable sleep)`.
- 2) Начинает выполняться код ядра: будет вызвана точка входа драйвера устройства, которое идентифицируется запросом. В результате работы кода драйвера будет сформирована команда для устройства, которая по шине данных будет передана контроллеру устройства.
- 3) Отправив команду, процессор отключается от управления устройством — **это принцип распараллеливания функций**: процессор не управляет работой внешних устройств, ими управляют контроллеры устройств. Этот принцип привёл к необходимости включения в систему прерываний прерываний от внешних устройств: они информируют процессор о завершении операции ввода/вывода.
- 4) Когда данные готовы, будет вызван обработчик соответствующего аппаратного прерывания; его задача — считать данные из буфера устройства в буфер ядра и инициализировать следующие действия по обработке полученных данных.
- 5) Обработчик вызовет соответствующую `callback`-функцию драйвера, передаст полученные данные, и в результате этих действий приложение будет разблокировано: в буфер приложения (переменную, массив, поле структуры и так далее) будут записаны запрошенные данные.

3.5.2 Неблокирующий ввод/вывод (non-blocking I/O, polling (опрос))

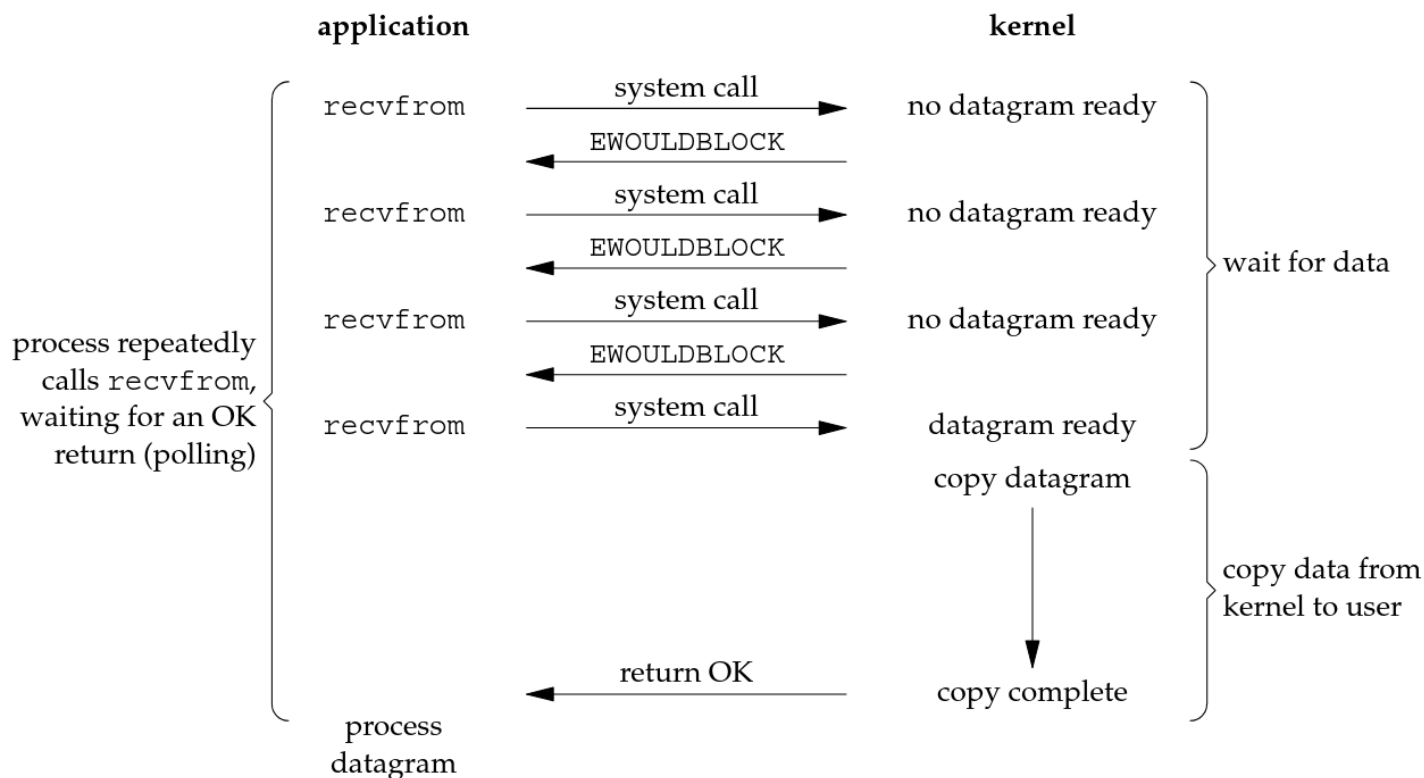


Рис. 9. Неблокирующий ввод/вывод

Опрос (polling) в системах был реализован раньше прерываний от устройств ввода/вывода: процессор опрашивал внешние устройства об их готовности (когда данные поступают в буфер устройства). При этом процесс не блокируется, но процессорное время тратится на опрос, никакой другой полезной работы процессор не выполняет (непроизводительные затраты). Это неэффективно, потому что быстродействующий процессор ждёт медленное устройство.

Сокеты бывают двух видов: блокирующие и неблокирующие. Благодаря многопоточности можно блокирующие сокеты использовать во всех случаях: и тогда, когда нужно обрабатывать несколько сокетов параллельно, и тогда, когда требуется возможность прервать операцию на сокете. Для установки неблокирующего сокета существует макрос `O_NONBLOCK`:

```
1 {
2     ...
3     sk_fd = socket(AF_INET, SOCK_STREAM, 0);
4     fcntl(sk_fd, F_SET, O_NONBLOCK); // Установка неблокирующего режима сокета
5     ...
6 }
```

Если таким образом установлен неблокирующий режим сокета, то после любого вызова функции `read` будет возвращено управление сразу, если на входе нет

данных для чтения. При этом функцией read будет возвращено значение EAGAIN (EWOULDBLOCK).

3.5.3 Мультиплексирование ввода/вывода (multiplexing I/O)

Мультиплексер (коммутатор) - устройство, которое объединяет информацию, поступающую по нескольким каналам ввода, и выдает ее по одному выходному каналу.

Мультиплексирование — процесс совмещения нескольких сообщений, передаваемых одновременно, в одной физической или логической среде.

Мультиплексирование является альтернативой многопоточности. Мультиплексирование выполняет сервер, так как сервер принимает большое количество соединений. Сервер блокируется на мультиплексоре (например, select) до тех пор, пока не будет готова хотя бы одна точка соединения, после чего получает и обрабатывает данные.

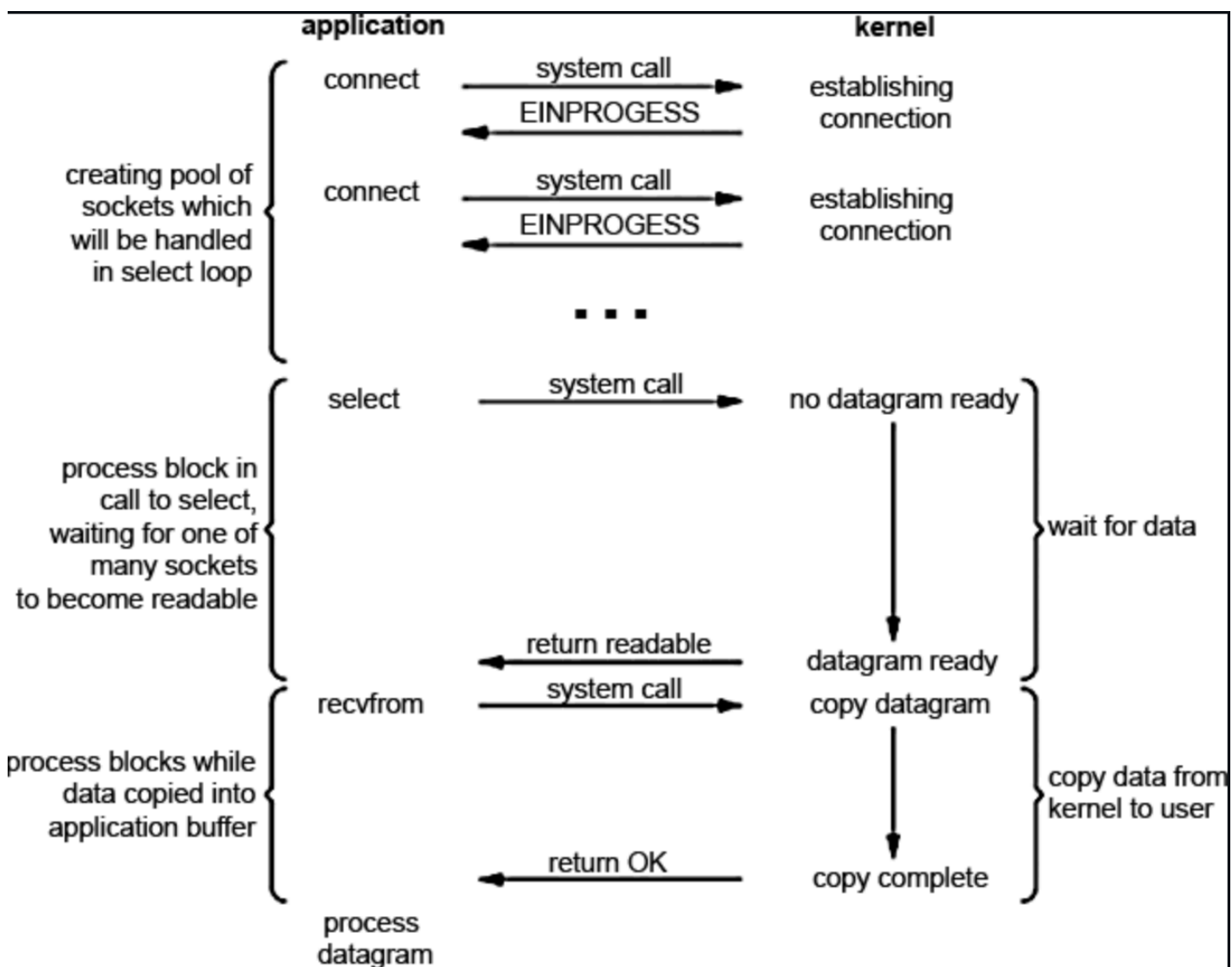


Рис. 10. Мультиплексирование

Приложение клиента вызывает системный вызов `connect`, чтобы установить соединение с сервером. Такое соединение устанавливается не мгновенно, поэтому на схеме есть `establishing connection`, при этом возможно, что соединение в данный момент не установлено — приложение повторяет попытки соединиться. В результате запросов клиентов на соединение на стороне сервера будет сформирован пул сокетов, который надо обслужить, обработать.

На стороне сервера работает селектор. Он опрашивает точки соединения, и будет обработано первое возникшее готовое соединение. Опрашивается пул точек соединения, чтобы определить первую готовую для соединения точку. Клиентов много, сервер обрабатывает большое количество клиентов, но не все клиенты одновременно готовы установить соединение. Клиенты устанавливают соединение по мере готовности и надобности, а сервер опрашивает эти точки, чтобы начать обслуживать первое готовое соединение.

При этом на `select` сервер блокируется до тех пор, пока не будет готова хотя бы одна точка соединения. Если такая точка готова, значит, данные могут быть получены сервером для последующей обработки. В результате сервер сможет выполнить получение данных (`receive from`) и копирование этих данных. Когда данные получены, они копируются в приложение сервера, и сервер начинает их обрабатывать.

Почему потоки являются альтернативной мультиплексору?

Потому что поток, когда выполнил системный вызов `read`, заблокировался, и не участвует в планировании. Когда данные готовы, он разблокируется и продолжает выполнение.

Бытовой пример: Переключатель режимов на стиральной машине, с помощью которого мы выбираем нужную точку.

3.5.4 Ввод/вывод, управляемый сигналами (Signal-Driven I/O)

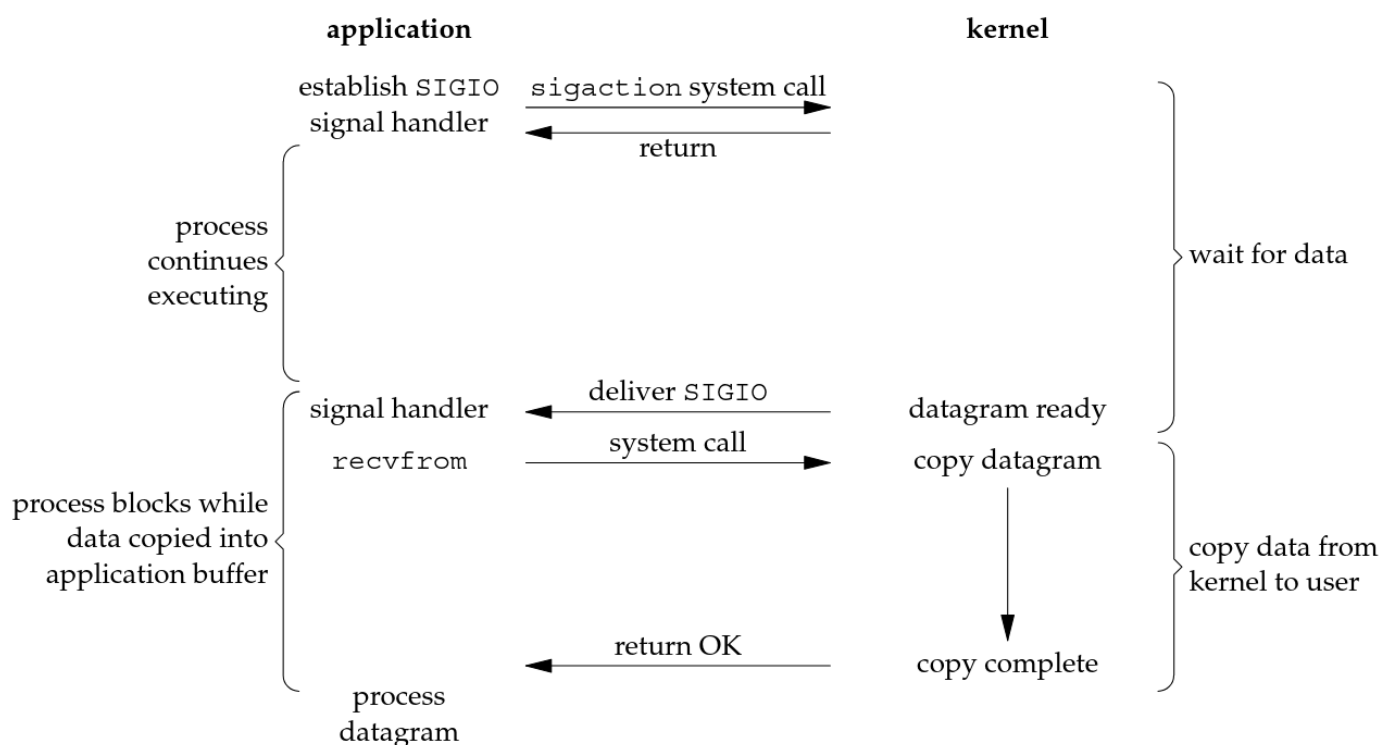


Рис. 11. Ввод/вывод, управляемый сигналами (Signal-Driven I/O)

Существует специальный сигнал `SIGIO`. Приложение должно установить обработчик сигнала — для этого вызывается системный вызов `int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);`. Этот обработчик сигнала будет информировать о том, что данные готовы (`datagram ready`). Установив обработчик сигнала, процесс не блокируется в ожидании завершения ввода и продолжает выполнение. Когда процесс получает сигнал, срабатывает обработчик, после этого процесс может получить данные: при вызове `recvfrom` выполняется копирование дейтаграммы из `kernel mode` в `user mode`. Процесс блокируется только на время, пока данные копируются в буфер приложения.

Важно

В данном способе взаимодействия с внешним устройством всю работу выполняет ядро операционной системы. Во-первых, ядро отслеживает, когда данные будут готовы. Во-вторых, после того как данные готовы, посылает сигнал `SIGIO`, в результате чего будет вызван обработчик сигнала, причём этот обработчик называется функцией обратного вызова (`callback`).

Дальше идёт `recvfrom` или `read`, но вызов можно выполнить или в обработчике сигнала, или в основном потоке программы. При этом ожидание может выполняться через `polling` (опрос).

Существует ограничение: сигнал типа SIGIO для каждого процесса может быть только 1, следовательно обработать можно только 1 файловый дескриптор, то есть взаимодействие можно выполнять только с одним открытым файлом. На время выполнения обработчика сигнала сигнал SIGIO блокируется. Это значит, что если во время блокировки сигнал поступит несколько раз, то эти сигналы будут потеряны.

У сигнала может быть установлена маска NULL. Это приведёт к тому, что другие сигналы блокироваться не будут: блокируется только такой же сигнал SIGIO.

3.5.5 Асинхронный ввод-вывод (Asynchronous I/O)

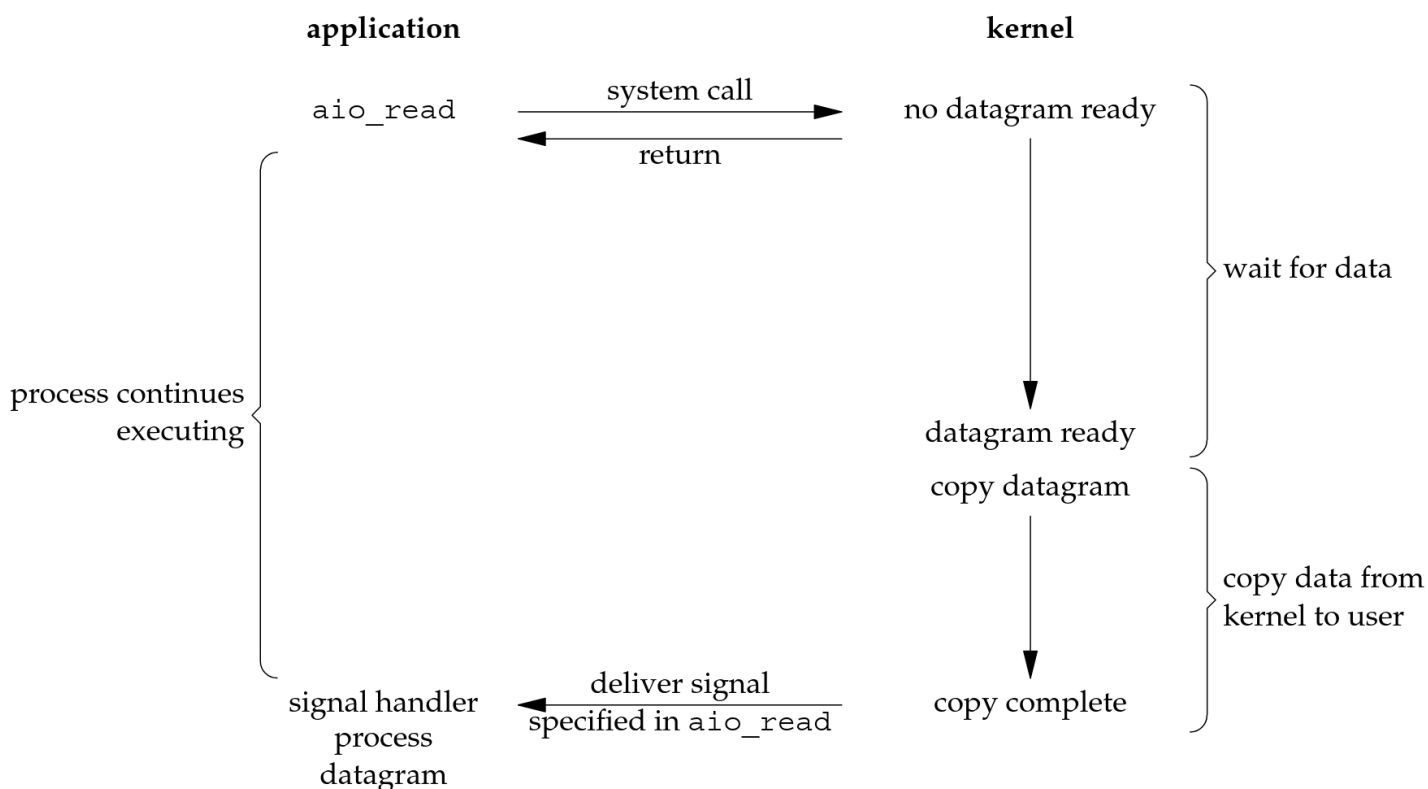


Рис. 12. Асинхронный ввод-вывод (Asynchronous I/O)

При асинхронном вводе-выводе приложение вызывает специальные функции, например `aio_read()`, передавая дескриптор, адрес буфера и его размер. После вызова управление сразу возвращается процессу, поэтому он не блокируется и может продолжать выполнение.

Основную работу берёт на себя ядро: оно контролирует выполнение операции и сообщает о её завершении, например с помощью связанного с АИО сигнала. Когда данные готовы, процесс получает результат и обрабатывает их.

Главная особенность этой модели в том, что программу нужно специально организовывать: после запроса ввода-вывода процесс должен выполнять другую работу,

а не ждать завершения операции. Это позволяет ускорить выполнение за счёт отсутствия блокировки.

Проблема асинхронного ввода-вывода состоит в том, что надо получать данные синхронно. В асинхронном вводе-выводе внимание сосредотачивается на:

- 1) возможности определить, что ввод-вывод можно выполнить быстро;
- 2) завершении операции ввода-вывода, а именно: при невозможности выполнить его сразу возвращается ошибка.

3.5.6 Классификация моделей

	Блокирующий Blocking	Неблокирующий Non-blocking	
Синхронный Synchronous	read / write ①	read / write O_NONBLOCK	polling
Асинхронный Asynchronous	IO Multiplexing (select/poll/epoll) ③	AIO ⑤	

Таблица 3. Классификация моделей ввода-вывода

Когда модель неблокирующая, запрос надо повторить. В любом случае это `polling`.

✱ Claude

В классификации Стивенса (UNIX Network Programming), из которой взяты сами модели, мультиплексирование относится к синхронному вводу-выводу: асинхронной считается только модель POSIX AIO, поскольку лишь в ней процесс не блокируется ни на ожидании готовности, ни на копировании данных. В таблице из конспекта мультиплексирование помещено в строку «асинхронный».

3.6 Ввод-вывод с мультиплексированием: мультиплексор `epoll` — особенности (типы и алгоритм работы)

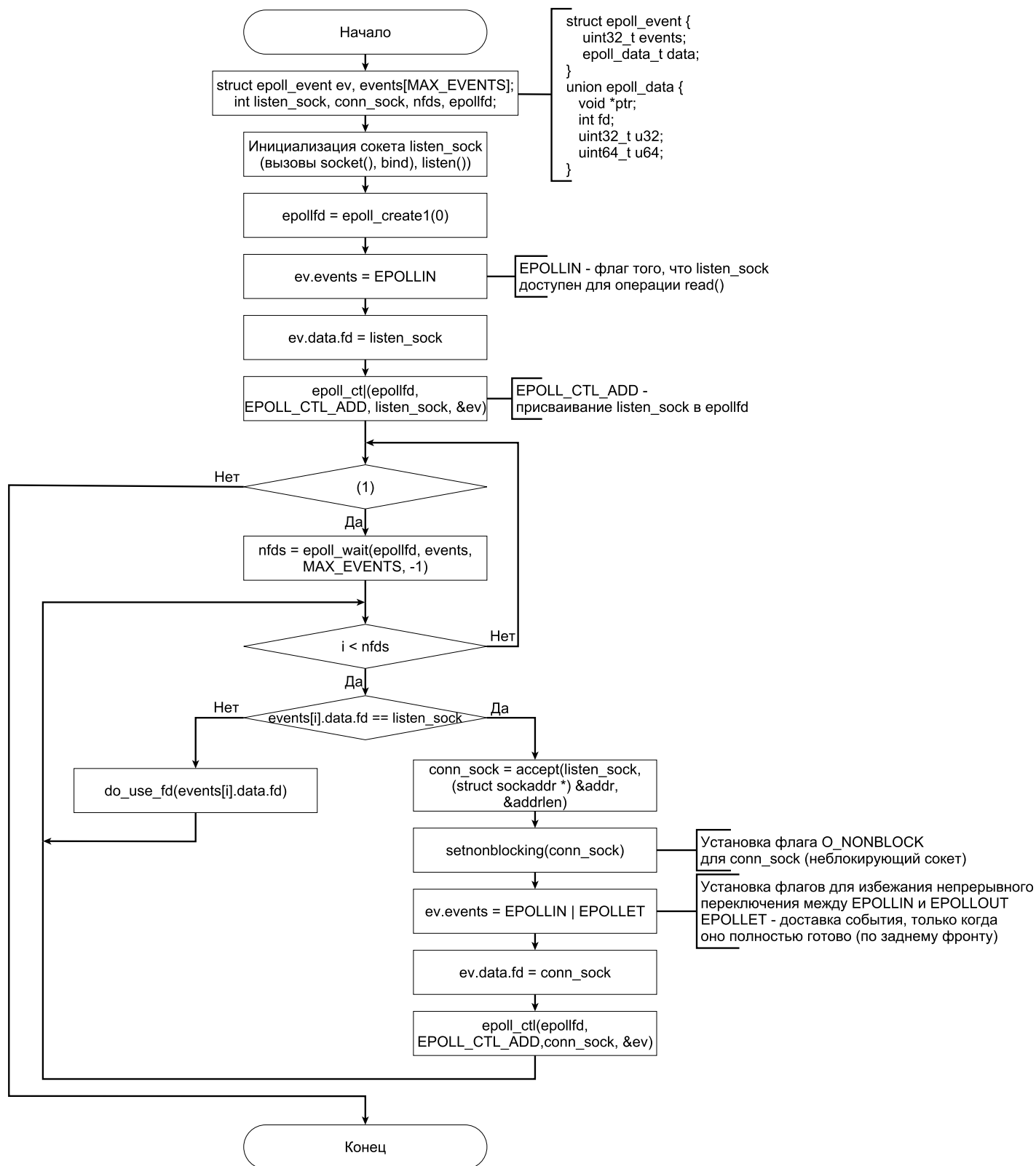


Рис. 13. Схема алгоритма `epoll`

Алгоритм работы (думаю можно не писать, но прочтите перед сдачей)

- Мультиплексор работает с массивом events: `struct epoll_event ev, events[MAX_EVENTS];`. В структуре `struct epoll_event` есть поле `data`, а в нём — поле `fd` (файловый дескриптор).
- Мультиплексор создаётся вызовом `epoll_create1(0)`, который возвращает файловый дескриптор `epollfd`.
- В структуре события указывается интересующее событие: `ev.events = EPOLLIN`; (готовность к чтению).
- Сокет нужно добавить в набор отслеживаемых: `epoll_ctl(epollfd, EPOLL_CTL_ADD, listen_sock, &ev)`.
- Вызовом `nfds = epoll_wait(epollfd, events, MAX_EVENTS, -1)`; создаётся массив готовых событий: сервер блокируется на `epoll_wait`, пока не будет готов хотя бы один дескриптор.
- Дальше цикл по соединениям, которые мы зафиксировали: просматриваются поля массива `events`.
 - Если это исходный (слушающий) сокет (`events[n].data.fd == listen_sock`), то вызывается `accept()`: создаётся копия исходного сокета, она переводится в неблокирующий режим (`setnonblocking`), и этот файловый дескриптор добавляется в массив дескрипторов мультиплексора с событиями `EPOLLIN | EPOLLET`.
 - Иначе обслуживается готовый дескриптор уже принятого соединения (`do_use_fd`).

Разница в последовательности действий по сравнению с обычным сервером: мультиплексор определяет готовые к соединению обращения клиентов, то есть клиент вызвал `connect`, выполнил соответствующие действия и ждёт соединения. Фактически в этом `if` анализируется: если это необработанное соединение, то `accept` (принять). Мультиплексор подготавливает информацию для принятия соединения.

Важно

В: Показывает на блок-решение на схеме алгоритма, для чего?

О: Если соединение не обработано то `accept()`

3.7 Пример из лабораторной работы — сервер производство—потребление с мультиплексированием

Реализуется классическая задача производство-потребление. У нас 2 типа клиентов — клиент-производитель и клиент-потребитель. Клиент посылает серверу

признак, кто он: P (производитель) или C (consumer, потребитель), то есть что он хочет сделать — положить данные в кольцевой буфер или считать данные из буфера.

Сервер с мультиплексированием однопоточный: создаётся массив events, дальше пробегаем по нему, обслуживаются соединения (по схеме из предыдущего подраздела). Когда мы переходим к обработке по else, для нас важно, производитель это или потребитель — какую функцию вызывать (практически как у потоков в многопоточной версии).

Синхронизация:

- Не нужно обеспечивать взаимоисключение (сервер однопоточный, блокировок нет), а синхронизация нужна: считающие семафоры выкинуть не получается, иначе буфер будет переполнен. Как определить, что буфер заполнен? — Всё равно вводить счётчик.
- В обеих версиях (многопоточной и с мультиплексированием) должны быть семафоры, но в мультиплексоре не должно быть бинарного.
- Считающие семафоры, чтобы исключить переход в режим ядра, можно было бы заменить счётчиками: переход в режим ядра — затратное действие. Но счётчики не позволят нам выполнить блокировку.
- Если буфер заполнен ($N = \text{max}$), производитель ничего положить в буфер не может — значит, мы можем послать ему, что он должен ждать. Если мы используем семафор, то сервер будет заблокирован на семафоре, значит, функция не отправит клиенту сообщение, что всё в порядке. Что должен получать производитель от сервера? — Он должен получить ОК, что его данные положены. Пока ответ не получен, клиент заблокирован на запросе receive from, и только после получения он сможет выполнить следующую итерацию.

✱ Claude

В однопоточном сервере с мультиплексированием блокировка на считавшем семафоре остановила бы весь сервер, включая обслуживание потребителей, которые могли бы освободить место в буфере — получился бы deadlock. Поэтому проверку семафора в такой версии нужно делать неблокирующей (например, sem_trywait) и при заполненном буфере отвечать клиенту, что нужно повторить попытку, о чём и говорится в конспекте («послать, что он должен ждать»).

```
1 #define _GNU_SOURCE
2 #include <errno.h>
3 #include <netinet/in.h>
4 #include <stdio.h>
```

```

5 #include <stdlib.h>
6 #include <sys/epoll.h>
7 #include <sys/ipc.h>
8 #include <sys/sem.h>
9 #include <sys/socket.h>
10 #include <sys/stat.h>
11 #include <time.h>
12 #include <unistd.h>
13
14 #define SERVER_PORT      9878
15 #define SERVER_BACKLOG  5
16 #define BUF_SIZE        52
17 #define MAX_EVENTS      64
18
19 #define OP_PRODUCER 'P'
20 #define OP_CONSUMER 'C'
21
22 #define SEM_KEY          200
23 #define SEM_STORAGE_EMPTY 0
24 #define SEM_STORAGE_FULL  1
25 #define SEM_LOCK         -1
26 #define SEM_UNLOCK       1
27
28 static char ring_buffer[BUF_SIZE];
29 static int write_idx = 0;
30 static int read_idx = 0;
31
32 static int semaphore_id;
33 static int listen_fd;
34 static int epoll_fd;
35 static int should_exit = 0;
36
37 struct sembuf stop_write = {SEM_STORAGE_EMPTY, SEM_LOCK, IPC_NOWAIT | SEM_UNDO};
38 struct sembuf start_read = {SEM_STORAGE_FULL, SEM_UNLOCK, SEM_UNDO};
39 struct sembuf stop_read = {SEM_STORAGE_FULL, SEM_LOCK, IPC_NOWAIT | SEM_UNDO};
40 struct sembuf start_write = {SEM_STORAGE_EMPTY, SEM_UNLOCK, SEM_UNDO};
41
42 void producer(int fd) {
43     char letter;
44     if (recv(fd, &letter, 1, 0) <= 0) {
45         return;
46     }
47     if (semop(semaphore_id, &stop_write, 1) == -1) {
48         send(fd, "WAIT", 5, 0);

```

```

49     printf("[P %d] FULL, rejected '%c'\n", fd, letter);
50     return;
51 }
52 ring_buffer[write_idx] = letter;
53 write_idx = (write_idx + 1) % BUF_SIZE;
54 semop(semaphore_id, &start_read, 1);
55 send(fd, "OK", 3, 0);
56 printf("[P %d] put '%c'\n", fd, letter);
57 }
58 void consumer(int fd) {
59     if (semop(semaphore_id, &stop_read, 1) == -1) {
60         send(fd, "WAIT", 5, 0);
61         printf("[C %d] EMPTY\n", fd);
62         return;
63     }
64     char resp[2];
65     resp[0] = ring_buffer[read_idx];
66     resp[1] = '\0';
67     read_idx = (read_idx + 1) % BUF_SIZE;
68     semop(semaphore_id, &start_write, 1);
69     send(fd, resp, 2, 0);
70     printf("[C %d] got '%c'\n", fd, resp[0]);
71 }
72
73 void client_service(int fd, struct timespec* time_start) {
74     char op;
75     if (recv(fd, &op, 1, 0) <= 0) {
76         close(fd);
77         return;
78     }
79     if (op == OP_PRODUCER) {
80         producer(fd);
81     } else if (op == OP_CONSUMER) {
82         consumer(fd);
83     }
84     close(fd);
85 }
86 int main(void) {
87     struct sockaddr_in server_addr = {0}, client_addr;
88     socklen_t client_len = sizeof(client_addr);
89     if ((listen_fd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
90         perror("socket");
91         exit(1);
92     }

```

```

93     server_addr.sin_family = AF_INET;
94     server_addr.sin_addr.s_addr = INADDR_ANY;
95     server_addr.sin_port = htons(SERVER_PORT);
96     if (bind(listen_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) ==
-1) {
97         perror("bind");
98         exit(1);
99     }
100    if (listen(listen_fd, SERVER_BACKLOG) == -1) {
101        perror("listen"); exit(1);
102    }
103    int permissions = S_IRWXU | S_IRWXG | S_IRWXO;
104    semaphore_id = semget(SEM_KEY, 2, IPC_CREAT | permissions);
105    if (semaphore_id == -1) {
106        perror("semget"); exit(1);
107    }
108    if (semctl(semaphore_id, SEM_STORAGE_EMPTY, SETVAL, BUF_SIZE) == -1) {
109        perror("semctl"); exit(1);
110    }
111    if (semctl(semaphore_id, SEM_STORAGE_FULL, SETVAL, 0) == -1) {
112        perror("semctl"); exit(1);
113    }
114    epoll_fd = epoll_create1(0);
115    if (epoll_fd == -1) {
116        perror("epoll_create1"); exit(1);
117    }
118    struct epoll_event ev;
119    ev.events = EPOLLIN;
120    ev.data.fd = listen_fd;
121    if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, listen_fd, &ev) == -1) {
122        perror("epoll_ctl"); exit(1);
123    }
124    struct epoll_event events[MAX_EVENTS];
125    while (!should_exit) {
126        int nfds = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
127        if (nfds == -1) {
128            perror("epoll_wait");
129            break;
130        }
131        for (int i = 0; i < nfds; i++) {
132            if (events[i].data.fd == listen_fd) {
133                int conn_fd = accept(listen_fd, (struct sockaddr*)&client_addr,
&client_len);
134                if (conn_fd == -1) {
135                    perror("accept");

```

```

136             continue;
137         }
138         ev.events = EPOLLIN;
139         ev.data.fd = conn_fd;
140         if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, conn_fd, &ev) == -1) {
141             perror("epoll_ctl");
142             close(conn_fd);
143             continue;
144         }
145     } else {
146         client_service(events[i].data.fd, &time_start);
147     }
148 }
149 }
150 close(epoll_fd);
151 if (semctl(semaphore_id, 0, IPC_RMID) == -1) {
152     perror("semctl");
153     exit(1);
154 }
155 return EXIT_SUCCESS;
156 }

```

Клиент в цикле подключается к серверу, отправляет признак P/C (производитель дополнительно шлёт символ данных) и обрабатывает ответ сервера: OK — запрос выполнен, WAIT — буфер полон/пуст, попытку нужно повторить.

```

1  #include <arpa/inet.h>
2  #include <netinet/in.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <sys/socket.h>
7  #include <unistd.h>
8
9  #define SERVER_IP   "127.0.0.1"
10 #define SERVER_PORT 9878
11 #define MSG_SIZE    8
12 #define OP_PRODUCER 'P'
13 #define OP_CONSUMER 'C'
14
15 static int loop_flag = 1;
16 int connect_to_server(void) {
17     int fd = socket(AF_INET, SOCK_STREAM, 0);
18     if (fd == -1) {

```

```

19     perror("socket");
20     return -1;
21 }
22 struct sockaddr_in server_addr;
23 memset(&server_addr, 0, sizeof(server_addr));
24 server_addr.sin_family = AF_INET;
25 server_addr.sin_port = htons(SERVER_PORT);
26 inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
27 if (connect(fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1) {
28     perror("connect");
29     close(fd);
30     return -1;
31 }
32 return fd;
33 }
34 int producer(int fd, char* letter) {
35     char response[MSG_SIZE] = {0};
36     if (recv(fd, response, MSG_SIZE, 0) <= 0) {
37         perror("recv"); return -1;
38     }
39     if (strcmp(response, "WAIT") == 0) {
40         printf("P: buffer full\n"); return 1;
41     }
42     printf("P: '%c' -> %s\n", *letter, response);
43     *letter = (*letter == 'z') ? 'a' : *letter + 1;
44     return 0;
45 }
46 int consumer(int fd) {
47     char response[MSG_SIZE] = {0};
48     if (recv(fd, response, MSG_SIZE, 0) <= 0) {
49         perror("recv"); return -1;
50     }
51     if (strcmp(response, "WAIT") == 0) {
52         printf("C: buffer empty\n"); return 1;
53     }
54     printf("C: got '%c'\n", response[0]);
55     return 0;
56 }
57 int main(void) {
58     char letter = 'a';
59     for (int i = 0; i < ITERATIONS && loop_flag; i++) {
60         char op = ((i % 3) == 2) ? OP_CONSUMER : OP_PRODUCER;
61         int fd = connect_to_server();
62         if (fd == -1) return EXIT_FAILURE;

```

```
63     int rc;
64     if (op == OP_PRODUCER) {
65         char msg[2] = {op, letter};
66         if (send(fd, msg, 2, 0) == -1) {
67             perror("send");
68             close(fd);
69             return EXIT_FAILURE;
70         }
71         rc = producer(fd, &letter);
72     } else {
73         if (send(fd, &op, 1, 0) == -1) {
74             perror("send");
75             close(fd);
76             return EXIT_FAILURE;
77         }
78         rc = consumer(fd);
79     }
80     close(fd);
81     if (rc == -1) return EXIT_FAILURE;
82 }
83 return EXIT_SUCCESS;
84 }
```


4

4 **Файловая подсистема /proc** — назначение, особенности, файлы, поддиректории, ссылка `self`, информация об окружении, состоянии процесса, прерываниях. Структура `proc_dir_entry`: функции для работы с элементами /proc. Использование структуры `file_operations` для регистрации функций работы с файлами. Передача данных из адресного пространства пользователя в адресное пространство ядра и обратно (лаб. раб.). Обоснование необходимости использования специальных функций для передачи данных из пространства пользователя в ядро и из ядра в пространство пользователя.

4.1 Файловая подсистема /proc

Виртуальная файловая подсистема /proc (process information pseudo-file-system) позволяет в режиме пользователя получить большой объем информации о ресурсах процесса. Виртуальная — то есть не существующая на самом деле (не хранящаяся во вторичной памяти), а создающаяся “на лету” по мере обращения к данным. По сути интерфейс /proc позволяет получить информацию из структур ядра, не обращаясь к ним напрямую, а потому работая в режиме пользователя. Большинство файлов доступны только для чтения, однако некоторые доступны для записи, что позволяет изменять переменные ядра.

файловая система /proc является немонтируемой. Под этим подразумевается, что она не связана ни с каким внешним носителем (она видна в файловых системах, подключенных к корневому каталогу, но не связана ни с каким внешним устройством. Поэтому она и называется псевдофайловой системой).

Для каждого процесса в /proc определена директория идентификатора процесса (/proc/<pid>/), она будет создана налету. В ней хранятся файлы, директории и символические ссылки, используя которые можно получить развёрнутую информацию о конкретном процессе. Вся информация доступна из `usermode`:

Элемент	Тип	Содержание
<code>cmdline</code>	Файл	holds the complete command line Содержит командную строку процесса
<code>cwd</code>	Символическая ссылка	Указывает на рабочую директорию процесса
<code>environ</code>	Файл	Содержит список переменных окружения процесса
<code>exe</code>	Символическая ссылка	Указывает на образ процесса (на его файл) Символическая ссылка, содержащая фактический путь к выполняемой команде.
<code>comm</code>	Файл	Имя команды, связанное с процессом. Это именно то имя файла, которое приведёт к выполнению нашего кода.

Элемент	Тип	Содержание
		Замечание 4.1 Разные потоки в одном и том же процессе могут иметь разные значения, доступные через <code>/proc/<pid>/task/<tid>/comm</code>. <code>task</code> — файл, в котором находятся поддиректории потоков.
stat	Файл	Содержит информацию о процессе (vsize, start_brk и др.)
fd	Директория	Содержит ссылки на открытые процессом файлы Замечание 4.2 В системе различаются файлы и открытые файлы. Открытые файлы описываются структурой <code>struct file</code>. Просто файлы представляются в системе деревом каталогов, а описываются физические файлы <code>struct inode</code>. Один и тот же файл может быть открыт много раз, причём разными процессами. В таблице открытых файлов на каждый такой открытый файл будет дескриптор <code>struct file</code>
root	soft link	Это символическая ссылка, которая указывает на корень файловой системы процесса. Замечание 4.3 Процессы, которые выполняются на нашем компьютере, могут принадлежать разным файловым системам. Пока мы имя исполняемого файла не указали в командной строке, пока он не начал выполняться, он лежит во вторичной памяти.
maps	Файл	Это адресное пространство того процесса, <code>pid</code> которого мы указали. В зависимости от <code>pid</code> информация будет отличаться, но есть неизменные составляющие:
pagemap	файл	Это 64-х разрядное значение для каждой строки процесса. При этом нас интересует фактически несколько бит: 1) 63 бит — present (in RAM) 2) 62 бит — swap space, область своппинга 3) 61 — shared anonymous page 4) 55 бит — dirty. Он указывается в дескрипторе страницы.

Элемент	Тип	Содержание
		С 54 по 0: если страница присутствует в физической памяти, то эти биты являются номером фрейма, то есть номером физической страницы.
task	Директория	Содержит поддиректории потоков
interrupts	Файл	Предоставляет таблицу о количестве прерываний на каждом из процессоров в следующем виде: 1) Первая колонка: номер прерывания 2) Колонки CPUx: счётчики прерываний на каждом из процессоров 3) Следующая колонка: вид прерывания: 3.1) IO-APIC-edge — прерывание по фронту на контроллер I/O APIC. 3.2) IO-APIC-fastestoi — прерывание по уровню на контроллер I/O APIC. 3.3) PCI-MSI-edge — MSI прерывание. 3.4) XT-PIC-XT-PIC — прерывание на PIC контроллер. 4) Последняя колонка: устройство, ассоциированное с данным прерыванием.

Таблица 4. Содержание директории /proc/<pid>

4.1.1 Ссылка self

/proc/self — символическая ссылка, существующая для каждого процесса и указывающая на его собственную директорию /proc/<pid>; позволяет процессу обращаться к своим данным, не вызывая getpid().

4.1.2 Информация об окружении

Окружение несёт файл environ (для текущего процесса — /proc/self/environ): в нём исходное окружение, установленное при запуске (execve()), переменные разделены нулевыми символами ('\0'), на строки файл не разбит. Изменения окружения после execve() (например, putenv()) в нём не отражаются.

4.1.3 Информация о состоянии процесса

Информацию о состоянии процесса несёт файл stat — важную, подобранную разработчиками информацию о процессе: его идентификатор pid, состояние state, имя команды comm, идентификатор родителя ppid, приоритет prio, размер виртуального адресного пространства vsize, адрес начала кучи start_brk и т.д.

4.1.4 Информация о прерываниях

Файл `/proc/interrupts` несёт информацию о прерываниях на каждом CPU с указанием их источника. Формат записи:

- номер прерывания;
- колонки CPUх — счётчики прерываний на каждом процессоре;
- вид прерывания (IO-APIC-edge/-level, PCI-MSI-edge, XT-PIC и т.д.);
- устройство, ассоциированное с прерыванием.

Кроме `/proc/interrupts`, информацию о системе в целом дают, например, `/proc/cpuinfo`, `/proc/vmstat`, `/proc/softirqs`.

4.2 Структура `proc_dir_entry`: функции для работы с элементами `proc`

Файлы и поддиректории файловой системы `/proc` описаны структурой `proc_dir_entry`.

```
1 struct proc_dir_entry {
2     atomic_t in_use;
3     // ...
4     struct list_head pde_openers;
5     // ...
6     const struct inode_operations *proc_iops;
7     union {
8         const struct proc_ops *proc_ops;
9         const struct file_operations *proc_dir_ops;
10    };
11
12    const struct dentry_operations *proc_dops;
13    union {
14        const struct seq_operations *seq_ops;
15        int (*single_show)(struct seq_file *, void *);
16    };
17
18    proc_write_t write;
19    void *data;
20    unsigned int state_size;
21    unsigned int low_ino;
22    nlink_t nlink;
23    // ...
24    loff_t size;
25    struct proc_dir_entry *parent;
26    // ...
27    char *name;
```

```

28 // ...
29 u8 namelen;
30 char inline_name[];
31 };

```

Листинг 17. Структура `struct proc_dir_entry`

- `atomic_t` — неделимая переменная, счётчик.
- `const struct inode_operations *proc_iops` — у файла `proc` должен быть `inode` и должны быть определены `inode_operations`.
- Первый `union` — интерфейс регистрации функций. До шестой версии ядра существовала только `struct file_operations *proc_dir_ops`. В качестве альтернативы определена структура `const struct proc_ops *proc_ops` — в ней практически такие же функции. Объясняется такое решение тем, что функции `struct file_operations` вызываются интенсивно: все драйверы вызывают функции, определённые в этой структуре. Чтобы не нагружать эти функции дополнительными задачами, для работы с VFS `proc`, которая является информационной (используется для получения разработчиками информации), введена отдельная структура.
- Второй `union` — здесь видна информация о двух `seq`-интерфейсах взаимодействия с `user-mode`:
 - `const struct seq_operations *seq_ops` — полный интерфейс, полный набор функций;
 - `int (*single_show)(struct seq_file *, void *)` — упрощённый интерфейс.
- `proc_write_t write` — в самых старых версиях структуры было определено 2 функции: `read` и `write`, сейчас оставлена только функция `write`. Передача данных из `user-mode` в `kernel-mode` — это `write`, обратно — `read`.
- `void *data` — приватные данные для работы с файлами `proc`.
- `unsigned int low_ino` — `inode` нижнего уровня (мы с ним не работаем). Поскольку `proc` — виртуальная файловая система, данные не хранятся во вторичной памяти, всё создаётся на лету в оперативной памяти, но `inode` у любого файла, в том числе и виртуального, всё равно имеется.
- `struct proc_dir_entry *parent` — указатель на родителя: поддерживается иерархия файловой системы `proc`.
- `loff_t size` — размер. О типе `loff_t`: введено огромное количество типов, детализированных по назначению (ярчайший пример — `pid_t`, хотя мы понимаем, что это `int`). Это сделано для дополнительного контроля соответствия типов при компиляции.

4.3 Создание файла proc, директории и символические ссылки

Для создания файла виртуальной файловой системы proc в ядре имеется функция `proc_create_data`, для создания каталогов, используется функция `proc_mkdir`, символические ссылки создаются — с помощью `proc_symlink` (третий параметр — путь, на который указывает ссылка). Удаляется элемент proc (файл, директория или ссылка) функцией `remove_proc_entry`:

```
1 // создание файла
2 extern struct proc_dir_entry *proc_create_data(
3     const char *name,
4     umode_t mode,
5     struct proc_dir_entry *parent,
6     const struct file_operations *proc_ops,
7     void *data);
8
9 struct proc_dir_entry *proc_create(
10     const char *name,
11     umode_t mode,
12     struct proc_dir_entry *parent,
13     const struct file_operations *proc_fops);
14
15 // создание директории
16 extern struct proc_dir_entry *proc_mkdir(
17     const char *name,
18     struct proc_dir_entry *parent);
19
20 // создание символической ссылки
21 extern struct proc_dir_entry *proc_symlink(
22     const char * name,
23     struct proc_dir_entry *parent,
24     const char *);
25
26 // удаление
27 remove_proc_entry(
28     const char *name,
29     struct proc_dir_entry *parent);
```

Параметры:

- `name` — имя файла в proc;
- `mode` — права доступа (read/write для user, group, others);
- `parent` — указатель на родительскую директорию, то есть поддерживается иерархия файловой системы proc: чтобы файл создавался в поддиректории proc, нужно до `proc_create_data` вызвать `proc_mkdir` и передать эту директорию; если

в параметре `parent` указать `NULL`, файл будет создаваться в корневом каталоге `proc`;

- `proc_ops` — указатель на проинициализированную структуру `proc_ops` с нашими функциями.

4.4 Использование структуры `file_operations` для регистрации функций

В загружаемом модуле ядра, работающем с файлом `proc`, 6 точек входа: `open`, `read`, `write`, `release`, `init`, `exit`.

Свои функции регистрируются инициализацией полей структуры `struct proc_ops` (для старых ядер — `struct file_operations`):

```
1 struct proc_ops {
2     ...
3     int (*proc_open)(struct inode *, struct file *);
4     ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
5     ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
6     ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
7     loff_t (*proc_lseek)(struct file *, loff_t, int);
8     int (*proc_release)(struct inode *, struct file *);
9     ...
10 };
```

Листинг 18. Структура `proc_ops`

Для стандарта C99 начиная с версии GCC 2.5 принят следующий синтаксис (назначенные инициализаторы):

```
1 static const struct proc_ops device_proc_ops = {
2     .proc_open = device_open,
3     .proc_read = device_read,
4     .proc_write = device_write,
5     .proc_release = device_release,
6 };
```

Регистрация точек входа происходит при инициализации полей структуры `proc_ops` (до регистрации эти функции должны быть объявлены и описаны), а затем проинициализированная структура передаётся функции создания виртуального файла `proc_create`.

4.5 Передача данных из пространства пользователя в ядро и обратно

`copy_to_user`, которая копирует данные из буфера ядра в буфер пользователя.

```
1 unsigned long __copy_to_user(void __user *to, const void *from, unsigned long n);
2 unsigned long __copy_from_user(void *to, const void __user *from, unsigned long n);
```

Параметры `copy_to_user`:

- `to` — буфер пользователя,
- `from` — буфер ядра,
- `n` — количество байт.

В случае успеха возвращает 0. Для `copy_from_user` — обратная ситуация.

4.6 Обоснование необходимости специальных функций

В своих функциях `read`, `write` мы должны вызывать специальные функции ядра для передачи данных из пространства пользователя в пространство ядра и обратно: `copy_to_user` копирует данные из буфера ядра в буфер пользователя, `copy_from_user` — из буфера пользователя в буфер ядра.

Специальные функции нужны, потому что ядро и загружаемый модуль ядра, становящийся частью ядра, оперирует физическими адресами: ядро загружено в физическую память резидентно, а процессы имеют только виртуальные адресные пространства. Процесс получает ресурс «физическая страница» только по мере надобности. При этом ядро, чтобы передать данные, должно обратиться к буферу пространства пользователя. Вероятна ситуация, когда буфер (страница), к которому обращается ядро, выгружен — в результате **возникнет страничное прерывание**, которое должно обработать ядро (должен быть выполнен код ядра). Ядро обратилось, стало источником прерывания, оно же и должно его обработать; если не предпринять специальных мер, возникнет паника ядра.

4.7 Пример из лабораторной работы «Фортунки»

```
1 #define COOKIE_POT_SIZE 4096
2 #define FILENAME "fortune_file"
3 #define DIRNAME "fortune_dir"
4 #define SYMLINK "fortune_link"
5
6 static struct proc_dir_entry *fortune_file;
7 static struct proc_dir_entry *fortune_dir;
8 static struct proc_dir_entry *fortune_link;
9
10 static char *cookie_pot = NULL;
11 char buffer[COOKIE_POT_SIZE];
12
13 static int fortune_open(struct inode *inode, struct file *file)
14 {
15     printk(KERN_INFO "+fortune: open\n");
```

```

16     return 0;
17 }
18 static int fortune_release(struct inode *inode, struct file *file)
19 {
20     printk(KERN_INFO "+fortune: release\n");
21     return 0;
22 }
23 static ssize_t fortune_write(struct file *file, const char __user *buf, size_t len,
    loff_t *fpos)
24 {
25     printk(KERN_INFO "+fortune: write\n");
26     ...
27     if (copy_from_user(&cookie_pot[write_index], buf, len) != 0) {
28         printk(KERN_ERR "+fortune: copy_from_user error\n");
29         return -EFAULT;
30     }
31     ...
32     return len;
33 }
34
35 static ssize_t fortune_read(struct file *file, char __user *buf, size_t len, loff_t
    *fpos)
36 {
37     printk(KERN_INFO "+fortune: read\n");
38     ...
39     read_len = snprintf(buffer, COOKIE_POT_SIZE, "%s\n", &cookie_pot[read_index]);
40     if (copy_to_user(buf, buffer, read_len) != 0) {
41         printk(KERN_ERR "+fortune: copy_to_user error\n");
42         return -EFAULT;
43     }
44     ...
45     return read_len;
46 }

```

```

1 static int __init md_init(void)
2 {
3     printk(KERN_INFO "+fortune: init\n");
4     if ((cookie_pot = vmalloc(COOKIE_POT_SIZE)) == NULL) {
5         printk(KERN_ERR "+fortune: memory error\n");
6         return -ENOMEM;
7     }
8     memset(cookie_pot, 0, COOKIE_POT_SIZE);
9
10    if ((fortune_file = proc_create(FILENAME, 0640, NULL, &fops)) == NULL) {
11        ...

```

```
12  }
13  if ((fortune_dir = proc_mkdir(DIRNAME, NULL)) == NULL) {
14      ...
15  }
16  if ((fortune_link = proc_symlink(SYMLINK, NULL, FILENAME)) == NULL) {
17      ...
18  }
19  printk(KERN_INFO "+fortune: loaded\n");
20  return 0;
21 }
22
23 static void __exit md_exit(void)
24 {
25     printk(KERN_INFO "+fortune: exit\n");
26     remove_proc_entry(FILENAME, NULL);
27     remove_proc_entry(DIRNAME, NULL);
28     remove_proc_entry(SYMLINK, NULL);
29     vfree(cookie_pot);
30 }
31
32 module_init(md_init);
33 module_exit(md_exit);
```


5

5 **Файловая подсистема ОС Linux: иерархическая структура ФС (уровни иерархии), поддержка большого числа файловых систем. Интерфейс VFS, именование (дерево каталогов) и идентификация файлов. Супер_блок — назначение и поля struct super_block, обеспечивающие хранение и доступ к файлам, и структуры, на которые ссылается. Структура, описывающая тип файловой системы, и регистрация файловой системы в ОС Linux, монтирование ФС. Слаб-кэш: назначение, функции создания слаба и размещения в нём данных. Пример (лаб. раб. VFS)**

5.1 Файловая подсистема ОС Linux: иерархическая структура ФС (уровни иерархии)

Файл — поименованная совокупность данных, может быть бессмысленная, хранящаяся во вторичной памяти. Понятие файла связано с долговременным хранением информации.

Файловая система — это порядок, определяющий способ организации хранения, именования и доступа к данным на вторичных носителях информации. Файловая система используется для управления хранением данных и доступа к ним. ФС должна обеспечить доступ к хранимой в системе на вторичных дисках информации.

Файловая система — важнейшая составляющая любой вычислительной системы: без долговременного хранения данные нельзя обработать повторно (содержимое ОЗУ при выключении пропадает), поэтому разработчики UNIX начали построение системы именно с неё. Внешние ЗУ — блочные устройства, на них же вытесняются страницы при свопинге (swapping) и пейджинге (paging).

Задачи файловой системы:

- 1) Долговременное хранение данных;
- 2) Обеспечение доступа к хранящимся данным.

5.1.1 Уровни иерархии ФС

Для обеспечения доступа к данным нужно предоставить пользователю достаточно удобный способ доступа. Уровни файловой системы:

- 1) Символьный;
- 2) Базовый;
- 3) Проверка прав доступа;
- 4) Логический;
- 5) Физический.

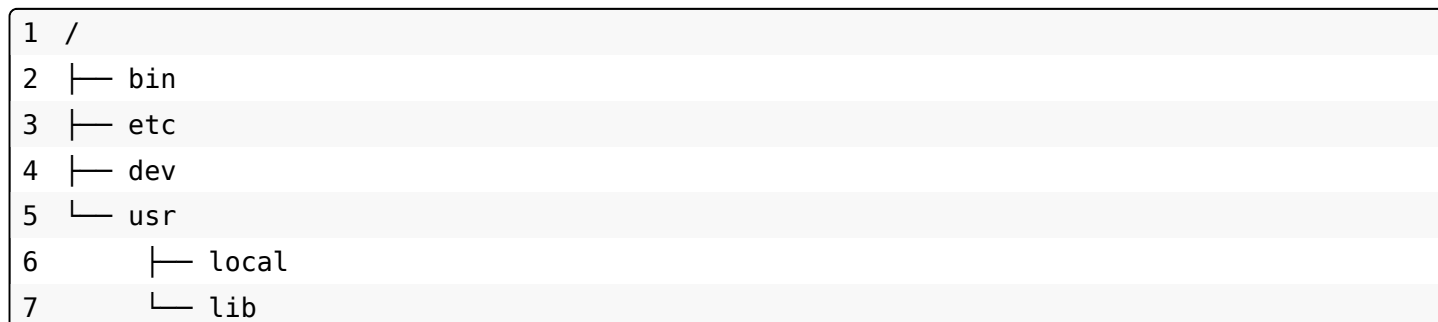
— **Символьный уровень** — именование файлов: у файлов имена (массив символов), собранные по каталогам; корневой каталог в UNIX/Linux — /. Имя не является идентификатором файла, а лишь позволяет удобно с ним работать;

идентификатором файла является inode; определены жёсткие (hard links) и символические (soft links) ссылки.

- **Базовый уровень** — идентификация файла: это inode; идентификатором является номер inode, который описывает физический файл.
- **Проверка прав доступа** — права отдельно для пользователя, группы и остальных (команда `chmod`).
- **Логический уровень** — процесс открывает файл и выполняет над ним операции: файл считает, что начинается с нулевого смещения (offset), каждая операция чтения/записи смещает указатель; файл байт-ориентированный, размера не имеет. Здесь определяется адресация данных, преобразуемая в физические адреса на диске.
- **Физический уровень** — уровень физической памяти; логический и физический уровни связывает драйвер дискового устройства, переводящий смещения в физические адреса.

5.1.2 Именованное: иерархическая структура каталогов (дерево каталогов)

Файловые системы знакомых нам ОС являются иерархическими — это негласное правило, впервые определённое именно в UNIX. Для Unix характерно существование корневого каталога /, от которого ветвится дерево каталогов:



5.2 Интерфейс VFS, поддержка большого числа файловых систем, идентификация файлов

Для Unix был предложен интерфейс VFS/vnode (virtual file system, virtual node). Linux отказался от vnode, оставив только VFS. VFS — интерфейс, через который ОС получает доступ к большому числу разных ФС, сократив до минимума количество системных вызовов: любая работа с файлом любой ФС идёт через один и тот же интерфейс. Основа такой работы — монтирование: прежде чем ФС станет доступна (станут видны её каталоги и файлы), она должна быть смонтирована.

В основе интерфейса VFS лежат 4 структуры (это ядерные структуры):

- 1) `struct super_block;`
- 2) `struct inode;`
- 3) `struct dentry;`

4) struct file.

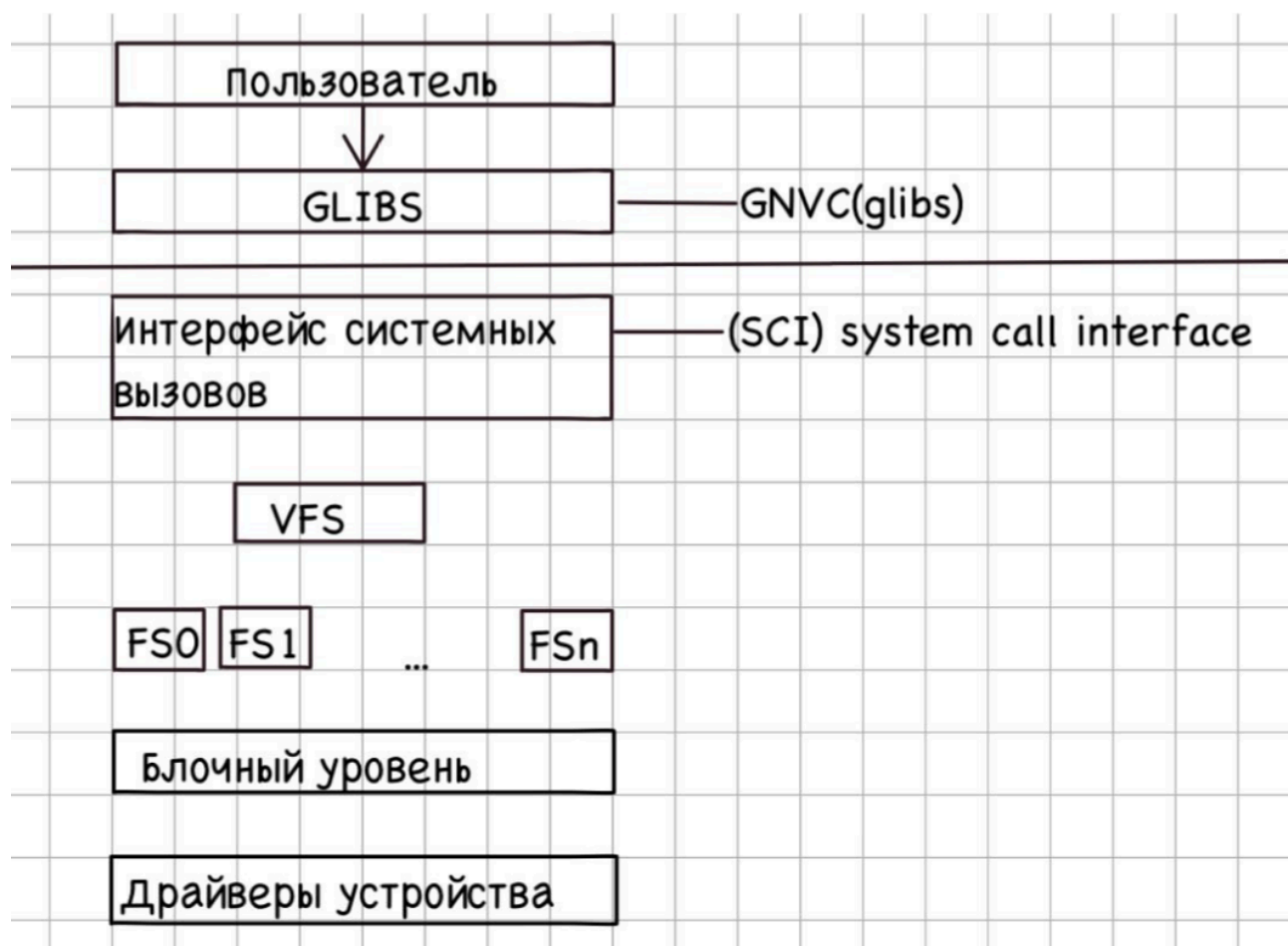


Рис. 14. Virtual File System

Все четыре структуры взаимодействуют между собой указателями, и для каждого файла создаются их экземпляры. Кратко их назначение: `super_block` описывает смонтированную ФС, `inode` — физический файл, `dentry` — элемент пути (имя), `file` — открытый файл.

Идентификатором файла является `inode`: дескриптор файла содержит множество полей, но идентификатором является номер `inode` (`i_ino`, метаданные); `inode` описывает физический файл — реально хранящий информацию.

5.3 Суперблок — назначение и поля `struct super_block`, обеспечивающие хранение и доступ к файлам, и структуры, на которые он ссылается

Адресное пространство диска делится на разделы, один раздел занимает одна ФС.

Суперблок представляет в VFS информацию о смонтированной ФС — структура с данными, необходимыми для монтирования и доступа к файлам ФС: метаданные для обращения к файлам, хранящимся «вразброс» (адресация соответствующих участков диска). В каждой ФС ровно один суперблок, он располагается в начале

раздела (есть резервная копия). При монтировании суперблок считывается в память ядра и находится там до размонтирования.

```
1 struct super_block {
2     struct list_head      s_list;
3     dev_t                 s_dev;
4     unsigned long         s_blocksize;
5     loff_t                s_maxbytes;    // Max file size
6     struct file_system_type *s_type;
7     const struct super_operations *s_op;
8     unsigned long         s_flags;
9     unsigned long         s_magic;
10    struct dentry          *s_root;      // точка монтирования
11    struct rw_semaphore    s_umount;
12    struct list_head       s_mounts;
13    struct block_device     *s_bdev;
14    struct list_head       s_inodes;     // all inodes
15 };
```

- `s_list` — список подмонтированных ФС (`list_head` — двусвязный список); `s_dev`, `s_bdev` — устройство и блочное устройство ФС.
- `s_blocksize`, `s_maxbytes` — размер блока и максимальный размер файла.
- `s_type`, `s_op` — тип ФС и функции на суперблоке (`super_operations`); `s_flags` — флаги монтирования; `s_magic` — магическое число ФС.
- `s_root` — точка монтирования (корневой каталог); `s_inodes` — список всех `inode`.
- `s_umount` — семафор «читатели-писатели» для монтирования; `s_mounts` — список монтирований данной ФС (одна и та же ФС может быть смонтирована много раз).

Осторожно

Вместо первого рисунка суперблока, который Рязанова не показывала (показывала), вероятно, хочет видеть второй.

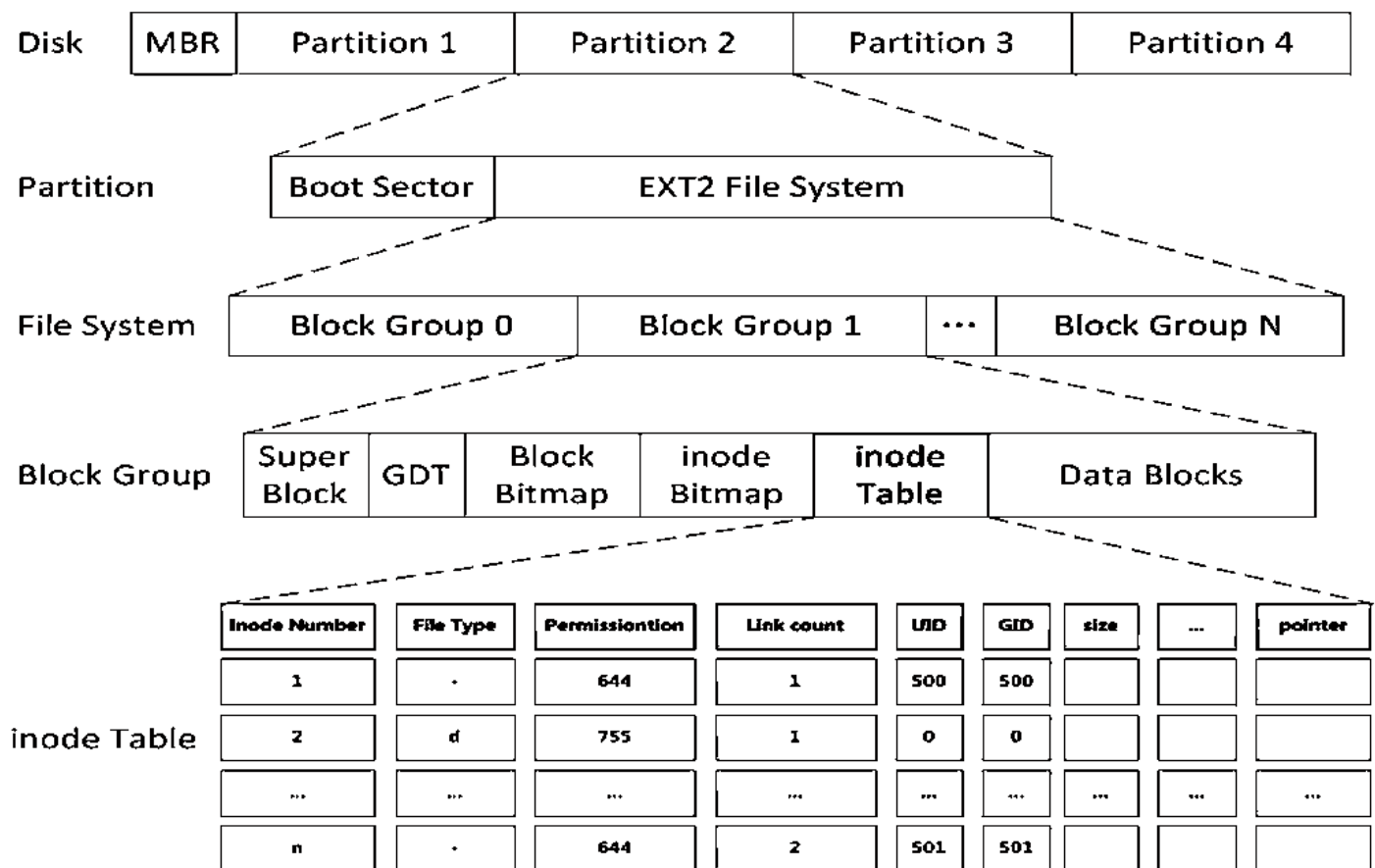


Рис. 15. Суперблок

5.4 Структура, описывающая тип файловой системы, и регистрация файловой системы в ОС Linux

Для описания возможностей и характеристик каждой поддерживаемой ФС в ядре определена структура `file_system_type`:

```

1 struct file_system_type {
2     const char *name;
3     int fs_flags;           // флаги типа ФС (FS_REQUIRES_DEV и др.)
4     struct dentry *(*mount) (struct file_system_type *, int, const char *, void *);
5     void (*kill_sb) (struct super_block *);
6     struct module *owner;
7     struct file_system_type *next;
8     struct hlist_head fs_supers;
9 };

```

- `name` — имя ФС.
- `fs_flags` — флаги типа ФС; для ФС, работающей с внешним ЗУ, нужен флаг `FS_REQUIRES_DEV` (от него зависят вызываемые функции монтирования). Флаг `FS_USERNS_MOUNT` означает, что необязательно всё выполнять в ядре.
- `mount`, `kill_sb` — монтирование и прекращение доступа к суперблоку.
- `owner` — модуль-владелец ФС; нужен для счётчика ссылок (модуль не выгружается, пока ФС примонтирована).

- `next` — связный список `file_system_type`.
- `fs_supers` — список суперблоков.

Для регистрации можно объявить `file_system_type` минимального содержания: достаточно полей `owner` и `name`, но чтобы смонтировать ФС в дерево каталогов, нужно зарегистрировать в этой структуре ещё свою функцию `mount` (и `kill_sb`). Для каждого типа файловой системы существует только одна структура `file_system_type`, независимо от того, сколько экземпляров этой ФС смонтировано: такая ФС может быть смонтирована в системе много раз, но зарегистрирована будет 1 раз.

Ядро предоставляет нам функции регистрации ФС и deregистрации:

```
1 int register_filesystem(struct file_system_type *);
2 int unregister_filesystem(struct file_system_type *);
```

В этих функциях нет выделения памяти: они лишь регистрируют проинициализированный тип ФС, поэтому регистрация не требует полного кода монтирования. `unregister_filesystem` — просто удаление типа из списка типов ФС. В загружаемом модуле ядра вызов `register_filesystem` выполняется в функции `init` модуля, а удаление из числа зарегистрированных — вызовом `unregister_filesystem` в функции `exit`. Увидеть зарегистрированную файловую систему в перечне файловых систем ядра можно через `cat /proc/filesystems`.

5.5 Монтирование файловой системы

Дерево файлов и каталогов в Linux формируется из ветвей, соответствующих разным носителям (разным ФС), но это одно большое дерево; некоторые каталоги могут быть точками монтирования.

Монтирование — система действий, делающая ФС (и все её файлы) доступной. Точка монтирования — каталог, к которому подмонтируется ФС: файлы и директории монтируемой ФС становятся файлами и поддиректориями точки монтирования, а её прежние файлы скрываются до размонтирования (поэтому лучше использовать пустые каталоги в качестве точки монтирования). Доступ к файлам ФС возможен только после монтирования. В Unix/Linux всегда есть корневая ФС (root fs); её монтирование — часть инициализации системы.

Когда ФС монтируется, создаётся структура `struct vfsmount`, которая представляет конкретный экземпляр смонтированной ФС:

```
1 struct vfsmount {
2     struct dentry      *mnt_root;      /* root of the mounted tree */
3     struct super_block *mnt_sb;        /* pointer to superblock */
4     int                mnt_flags;
```

```

5 struct mnt_idmap *mnt_idmap;
6 };

```

`mnt_root` — корень подмонтированного дерева, `mnt_sb` — указатель на суперблок, `mnt_flags` — флаги монтирования.

5.5.1 Функции монтирования

В ядре определено несколько функций `mount()`; все они имеют модификатор `extern` и возвращают значение типа `struct dentry*`. В версии ядра 6.17.13 определено только две:

```

1 extern struct dentry *mount_bdev(struct file_system_type *fs_type, int flags,
2                                 const char *dev_name, void *data, int (*fill_super)
3                                 (struct super_block *, void *, int));
4 extern struct dentry *mount_nodev(struct file_system_type *fs_type, int flags,
5                                 void *data, int (*fill_super)
6                                 (struct super_block *, void *, int));

```

Параметры: тип файловой системы (`fs_type`), флаги монтирования (`flags`), имя устройства (`dev_name`, только у `mount_bdev`), произвольные данные монтирования (`data`) и указатель на функцию `fill_super`. Внешнее отличие — в параметре `const char *dev_name`, которого нет у `mount_nodev`:

- `mount_bdev` — для монтирования ФС, находящейся на блочном устройстве;
- `mount_nodev` — для монтирования ФС, не связанной ни с каким устройством.

Функциям монтирования передаётся функция `fill_super` — она выполняет основные действия для монтирования: инициализирует поля `struct super_block` (размер блока, смещение, магическое число, `super_operations`). Функции монтирования возвращают объект `dentry`, которым должен быть `root`: для ФС нужно создать корневой каталог, а для этого — `inode`. В системе существует готовая функция `simple_fill_super`, которую находим в коде ядра и используем как основу для собственной `fill_super`.

Для создания `root` надо проинициализировать `inode`, выделив для него память функцией `new_inode()` (внутри неё виден аллокатор — `inode` аллоцируется в `slab`-кеше):

```

1 struct inode *new_inode(struct super_block *sb) {
2     struct inode *inode = new_inode_pseudo(sb); // -> alloc_inode(sb): выделение
        из slab
3     if (inode)
4         inode_sb_list_add(inode);
5     return inode;

```

Когда файловая система отмонтируется, всё должно быть удалено — этим занимается функция `kill_sb` из `file_system_type`. В ядре для этого определены функции:

```
1 void generic_shutdown_super(struct super_block *sb);
2 void kill_block_super(struct super_block *sb);
3 void kill_anon_super(struct super_block *sb);
4 void kill_litter_super(struct super_block *sb);
```

Общую работу по удалению суперблока выполняет `generic_shutdown_super`.

5.5.2 Команда монтирования файловой системы

Команда `mount` имеет следующий синтаксис:

```
1 mount <устройство> <точка монтирования>
```

Ключи: `-t` <тип ФС> `-o` <опции ФС>, то есть полная форма:

```
1 mount -t type device dir
```

Точка монтирования — любой каталог; в самой ФС нужно создать корневой каталог (`root`) — именно он монтируется. Ключ `-t` необязателен: система сама может определить тип ФС. После монтирования каталоги и файлы ФС доступны. Обычно монтирование требует прав суперпользователя. С помощью команды

```
1 mount [-l] [-t type]
```

можно получить список смонтированных файловых систем типа `type`. Размонтируется ФС командой `umount <точка монтирования>`.

5.6 Слаб-кэш: назначение, функции создания слаба и размещения в нём данных

Slab — «брусочек». Ядро оперирует ограниченным числом объектов (структур), для каждого нужно выделить память и проинициализировать поля, что занимает время. Если удалять объект после использования, для следующего такого же объекта придётся проделать ту же работу заново. Идея: не удалять объекты, а сохранять их в slab для повторного использования. Под разные структуры нужны разные кешы — это и реализует Slab-кеш. Каждый кеш хранит объекты одного размера (больше или меньше страницы); есть слабы и для часто используемых размеров буферов (16, 32, 64, ...). Все кешы видны через `cat /proc/slabinfo`.

Важно

Для каждого типа данных создается свой SLAB кеш. Не один на всех, а **для каждого**.

Побочный эффект слабов — сокращение фрагментации памяти. С версии ядра 2.6.23 SLAB переименован в SLUB (есть ещё SLOB). Это только код ядра: из userspace слаб недоступен — память выделяется страницами или слабами уже в ядре.

Имеются структуры `kmem_cache`; каждый кеш представляется такой структурой, а кешы связаны друг с другом в список `cache_chain` (цепочка кешей):

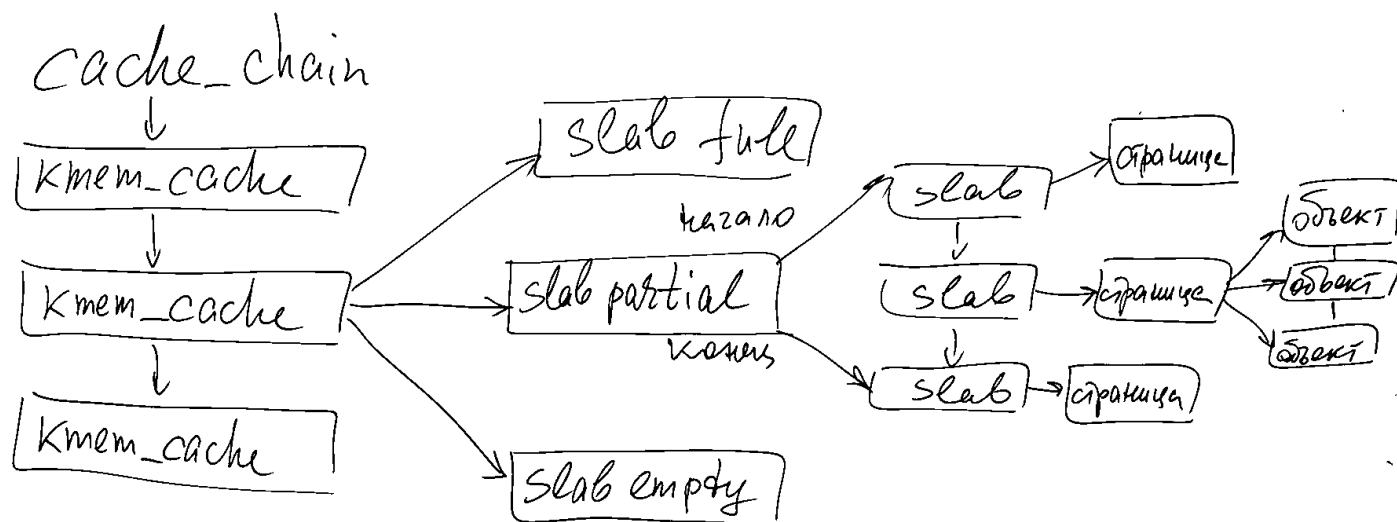


Рис. 16. cache_chain

Каждый slab может находиться в одном из трёх состояний:

- **slab full** — полностью распределённый: нет свободных объектов, все распределены и используются;
- **slab partial** — частично распределённый: есть как выделенные, так и свободные объекты;
- **slab empty** — пустой: нет ни одного распределённого объекта, все объекты свободны для последующего использования.

Выделение: новый объект берётся из частично распределённого слаба, иначе из пустого; если пустых нет, слабы создаются по мере необходимости. Побочный эффект — уменьшение фрагментации памяти.

```
1 /* Создание кэша */
2 struct kmem_cache *kmem_cache_create(const char *name, size_t size, size_t offset,
3                                     unsigned long flags, void (*constructor)
4                                     (void *));
```

```

5  /* Уничтожение кэша */
6  void kmem_cache_destroy(struct kmem_cache *cache);
7
8  /* Выделить объект из кэша */
9  void *kmem_cache_alloc(struct kmem_cache *cache, gfp_t flags);
10
11 /* Удалить объект из кэша */
12 void kmem_cache_free(struct kmem_cache *cache, void *obj);

```

Кеш SLAB напрямую используется подсистемой VFS: объекты `dentry` создаются и уничтожаются вызовами `kmem_cache_alloc()` и `kmem_cache_free()`, а внутри `new_inode` виден аллокатор — `inode` аллоцируется в `slab`-кеше.

5.7 Пример из лабораторной работы (ЛР VFS)

В ЛР создаётся собственная (виртуальная) ФС в загружаемом модуле ядра; по отношению к VFS выполняются два действия:

- 1) **Регистрация ФС:** объявляем свой `struct file_system_type` (минимально `owner`, `name`, плюс функции `mount` и `kill_sb`); в `init` вызываем `register_filesystem`, в `exit` — `unregister_filesystem`.
- 2) **Монтирование ФС:** в своей функции `mount` вызываем `mount_nodev` (ФС не связана с устройством), передавая ей свою `fill_super` (на основе `simple_fill_super`): она инициализирует поля `struct super_block`, через `new_inode` создаёт `inode` корневого каталога и создаёт `root` (`d_make_root`). Освобождение суперблока выполняет своя функция `kill_sb`.

После загрузки модуля ФС монтируется командой `mount` (не менее 2 раз — одна и та же ФС может быть смонтирована много раз) и показывается двумя способами: в дереве каталогов (`ls -ial`) и через `proc` (`cat /proc/mounts`); зарегистрированный тип ФС — через `cat /proc/filesystems`. Кроме того, в задании создаётся свой SLUB-кеш: объявляем свою структуру, создаём для неё кеш, через `alloc` получаем указатель на `slab` и выделяем объекты.

```

1  static int my_simple_fill_super(struct super_block *s, void *data, int silent)
2  {
3      struct inode *inode;
4      s->s_blocksize = PAGE_SIZE;
5      s->s_magic = MY_MAGIC_NUMBER;
6      s->s_op = &myfs_super_ops;
7
8      inode = new_inode(s);                // inode корневого каталога
9      inode->i_ino = 1;
10     inode->i_mode = S_IFDIR | 0755;

```

```

11     inode->i_op = &my_simple_dir_inode_operations;
12     inode->i_fop = &my_simple_dir_operations;      // операции на файле
13     set_nlink(inode, 2);
14     s->s_root = d_make_root(inode);                // создание root
15     return s->s_root ? 0 : -ENOMEM;
16 }

```

```

1 static struct dentry *myfs_mount(struct file_system_type *type, int flags,
2                                 char const *dev, void *data)
3 {
4     return mount_nodev(type, flags, data, my_simple_fill_super);
5 }
6
7 static struct file_system_type myfs_type = {
8     .owner    = THIS_MODULE,
9     .name     = "my_fs",
10    .mount     = myfs_mount,
11    .kill_sb   = my_kill_block_super,
12 };
13
14 static int __init my_init(void) { return register_filesystem(&myfs_type); }
15 static void __exit my_exit(void) { unregister_filesystem(&myfs_type); }
16 module_init(my_init);
17 module_exit(my_exit);

```

```

1 struct myfs_inode {
2     int i_mode;
3     unsigned long i_ino;
4 };
5
6 static struct kmem_cache* inode_cache = NULL;
7 static struct myfs_inode** inode_cached_array = NULL;
8
9 static struct dentry *my_fs_mount(struct file_system_type *type, int flags,
10                                  const char *dev, void *data)
11 {
12     ...
13     for (i = 0; i < OBJECTS_PER_MOUNT; i++)
14     {
15         struct myfs_inode *inode = kmem_cache_alloc(inode_cache, GFP_KERNEL);
16         if (!inode)
17         {
18             for (int j = 0; j < i; j++)
19                 kmem_cache_free(inode_cache, inode_cached_array[j]);
20             return ERR_PTR(-ENOMEM);

```

```
21     }
22     ...
23 }
24 ...
25 }
```

```
1 static int __init myfs_init(void) {
2     ...
3     // Создание slab-кэша в init
4     inode_cache = kmem_cache_create(SLAB_NAME, sizeof(struct myfs_inode), 0, 0,
5     NULL);
6     if (inode_cache == NULL)
7         return -ENOMEM;
8     ...
9 }
10 static void __exit myfs_exit(void) {
11     ...
12     // освобождение всех объектов
13     for (i = 0; i < cached_count; i++)
14         kmem_cache_free(inode_cache, inode_cached_array[i]);
15     // удаление slab-кэша
16     kmem_cache_destroy(inode_cache);
17     ...
18 }
```


6

6 **Файловая система, задачи файловой системы и иерархическая организация ФС.**
Файловая подсистема **LINUX**: поддержка большого числа файловых систем и структура, описывающая файловую систему. **VFS**: четыре основные структуры файловой системы и связь между ними. Раздел жесткого диска и суперблок. Структура **struct super_operations**. Монтирование файловой системы, команда **mount** и функции монтирования и их параметры, точка монтирования — корневой каталог и **inode**. Пример (лаб. раб.)

6.1 Файловая система

Файл — поименованная совокупность данных, может быть бессмысленная, хранящаяся во вторичной памяти. Понятие файла связано с долговременным хранением информации.

Файловая система — это порядок, определяющий способ организации хранения, именования и доступа к данным на вторичных носителях информации.

Файловая система используется для управления хранением данных и доступа к ним. ФС должна обеспечить доступ к хранимой в системе на вторичных дисках информации.

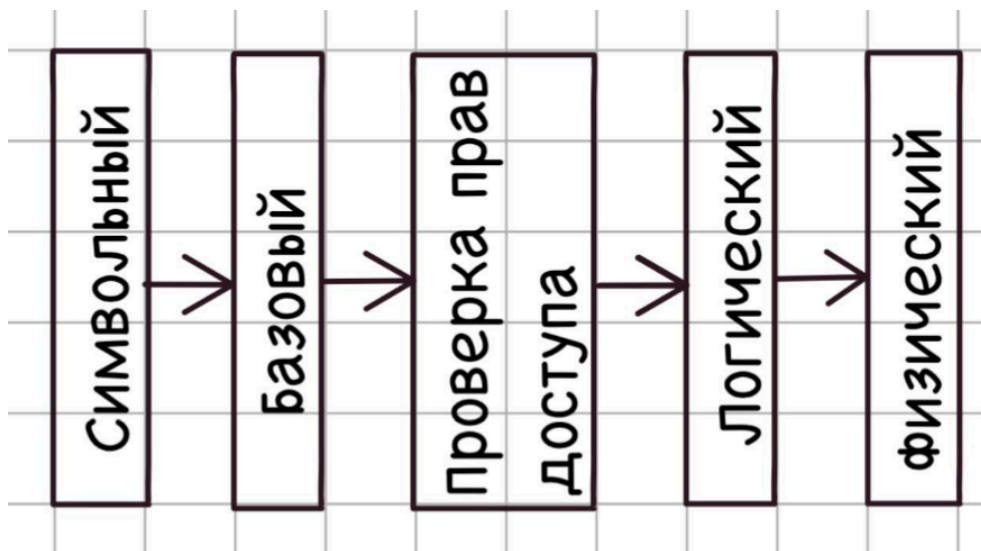
6.2 Задачи файловой системы

- 1) Долговременное хранение данных
- 2) Обеспечение доступа к хранящимся данным

6.3 Иерархическая организация ФС

Для обеспечения доступа к данным нужно предоставить пользователю достаточно удобный способ доступа. Уровни файловой системы:

- 1) Символьный
- 2) Базовый
- 3) Проверка прав доступа
- 4) Логический
- 5) Физический



- **Символьный** уровень — именованье файлов: у файлов **имена** (массив символов), собранные по каталогам; корневой каталог в UNIX/Linux — /. Имя не является идентификатором файла, а лишь позволяет удобно с ним работать; идентификатором файла является inode; определены жёсткие (hard links) и символические (soft links) ссылки.
- **Базовый** уровень — идентификация файла: это inode; идентификатором является номер inode, который описывает физический файл.
- **Проверка прав доступа** — права отдельно для пользователя, группы и остальных (команда chmod).
- **Логический** уровень — процесс открывает файл и выполняет над ним операции: файл считает, что начинается с нулевого смещения (offset), каждая операция чтения/записи смещает указатель; файл байт-ориентированный, размера не имеет. Здесь определяется адресация данных, преобразуемая в физические адреса на диске.
- **Физический** уровень — уровень внешнего ЗУ; логический и физический уровни связывает драйвер дискового устройства, переводящий смещения в физические адреса.

Файловые системы знакомых нам ОС являются иерархическими — это негласное правило, впервые определённое именно в UNIX. Для Unix характерно существование корневого каталога /, от которого ветвится дерево каталогов:

1	/
2	├─ bin
3	├─ etc
4	├─ dev
5	└─ usr
6	├─ local
7	└─ lib

6.4 Файловая подсистема LINUX: поддержка большого числа файловых систем и структура, описывающая файловую систему

6.4.1 Поддержка большого числа файловых систем

Для Unix был предложен интерфейс VFS/vnode (virtual file system, virtual node). Linux отказался от vnode, оставив только VFS. VFS — интерфейс, через который ОС получает доступ к большому числу разных ФС, сократив до минимума количество системных вызовов: любая работа с файлом любой ФС идёт через один и тот же интерфейс. Основа такой работы — монтирование: прежде чем ФС станет доступна (станут видны её каталоги и файлы), она должна быть смонтирована.

6.4.2 Структура, описывающая файловую систему

Для описания возможностей и характеристик каждой поддерживаемой ФС в ядре определена структура `file_system_type`. Это тип ФС, которая будет монтироваться; инициализируя её поля, мы определяем собственный тип ФС.

```
1 struct file_system_type {
2     const char *name;
3     int fs_flags;           // флаги типа ФС (FS_REQUIRES_DEV и др.)
4     struct dentry *(*mount) (struct file_system_type *, int, const char *, void *);
5     void (*kill_sb) (struct super_block *);
6     struct module *owner;
7     struct file_system_type *next;
8     struct hlist_head fs_supers;
9 };
```

Листинг 19. Структура `file_system_type` (сокр.)

- `name` — имя ФС.
- `fs_flags` — флаги типа ФС; для ФС, работающей с внешним ЗУ, нужен флаг `FS_REQUIRES_DEV` (от него зависят вызываемые функции монтирования). Флаг `FS_USERNS_MOUNT` означает, что необязательно всё выполнять в ядре.
- `mount`, `kill_sb` — монтирование и прекращение доступа к суперблоку.
- `owner` — модуль-владелец ФС; нужен для счётчика ссылок (модуль не выгружается, пока ФС примонтирована).
- `next` — связный список `file_system_type`.
- `fs_supers` — список суперблоков.

Для каждого типа файловой системы существует только одна структура `file_system_type`, независимо от того, сколько экземпляров этой ФС смонтировано: такая ФС может быть смонтирована в системе много раз, но зарегистрирована будет

1 раз. Для регистрации достаточно полей owner и name; зарегистрировать ФС можно вызовом register_filesystem, для deregистрации — unregister_filesystem.

6.5 VFS: четыре основные структуры файловой системы и связь между ними

В основе интерфейса VFS лежат 4 структуры (это ядерные структуры):

- 1) struct super_block;
- 2) struct inode;
- 3) struct dentry;
- 4) struct file.

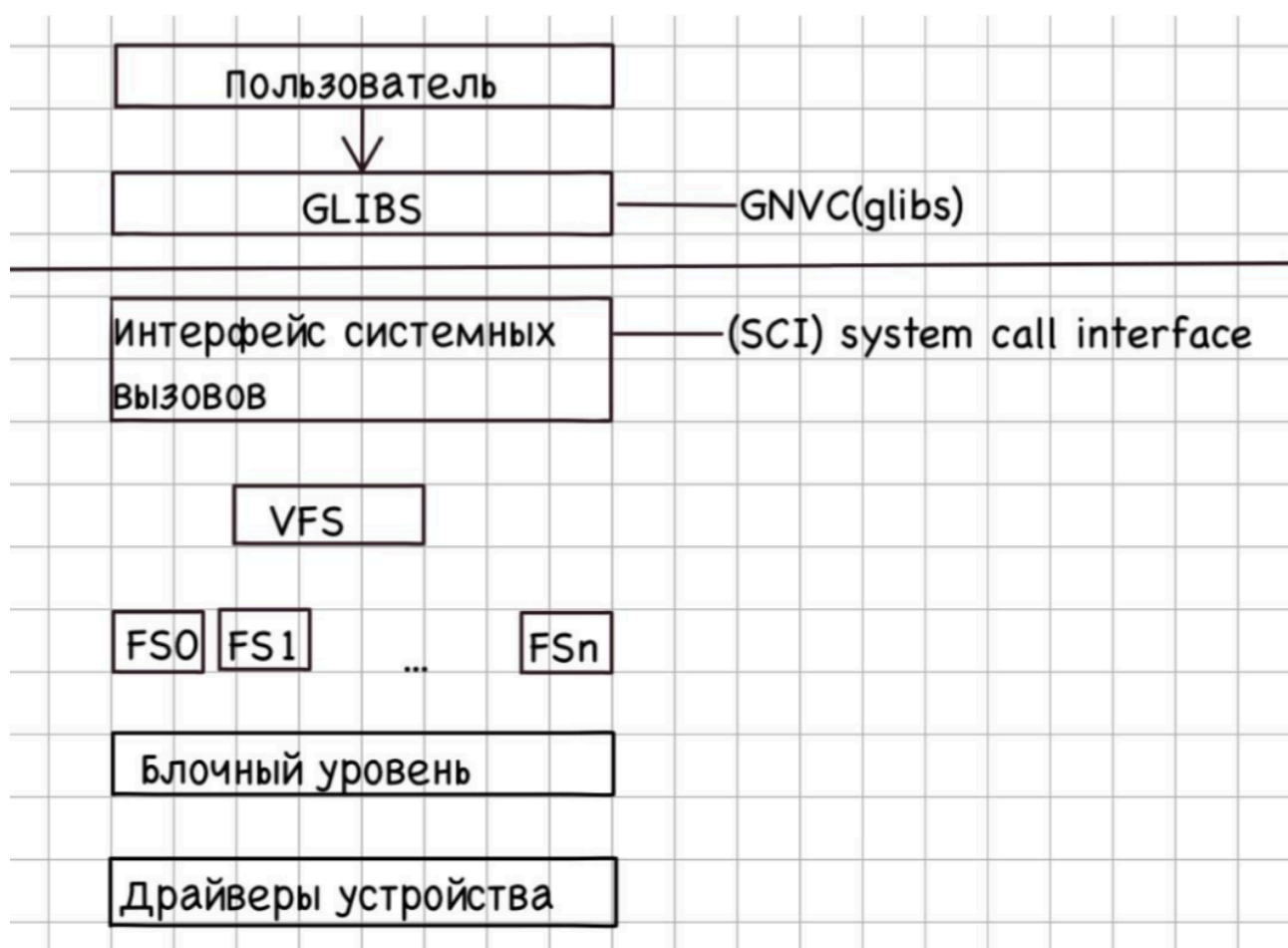


Рис. 17. Virtual File System

Все четыре структуры взаимодействуют между собой указателями, и для каждого файла создаются их экземпляры. Кратко их назначение:

- super_block — описывает смонтированную ФС (метаданные для обращения к файлам, хранящимся «вразброс»);
- inode — дескриптор физического файла (идентификатор файла — номер inode);
- dentry — элемент пути (имя); создаётся «на лету», на диске не хранится;
- file — дескриптор открытого файла, с которым работает процесс.

Связь структур строится относительно процесса (именно процессы открывают файлы): через `task_struct` по файловому дескриптору находится `struct file`; его поле `f_path/f_inode` ведёт к `dentry` и `inode`, а из `inode` (поле `i_sb`) и со стороны монтирования — к `super_block` смонтированной ФС.

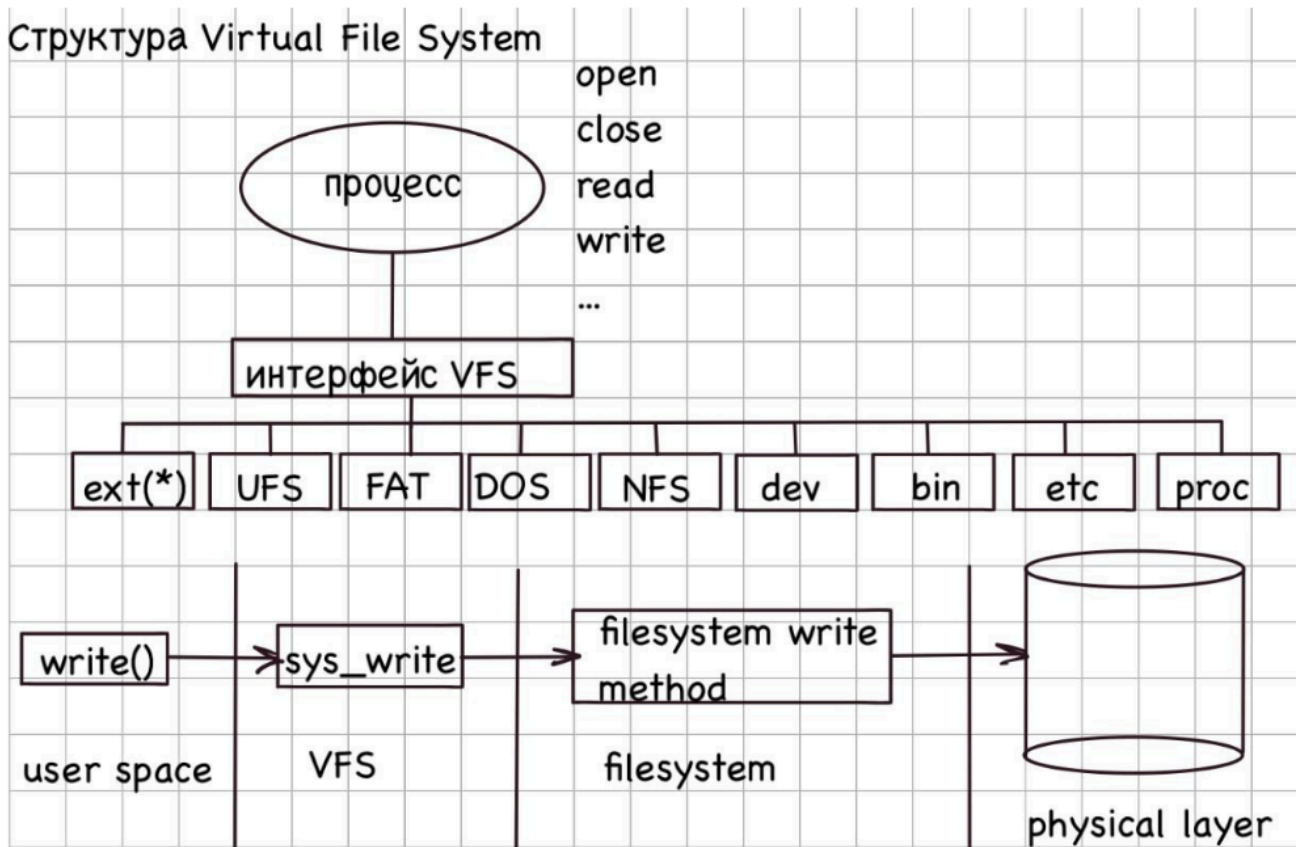


Рис. 18. Структура VFS

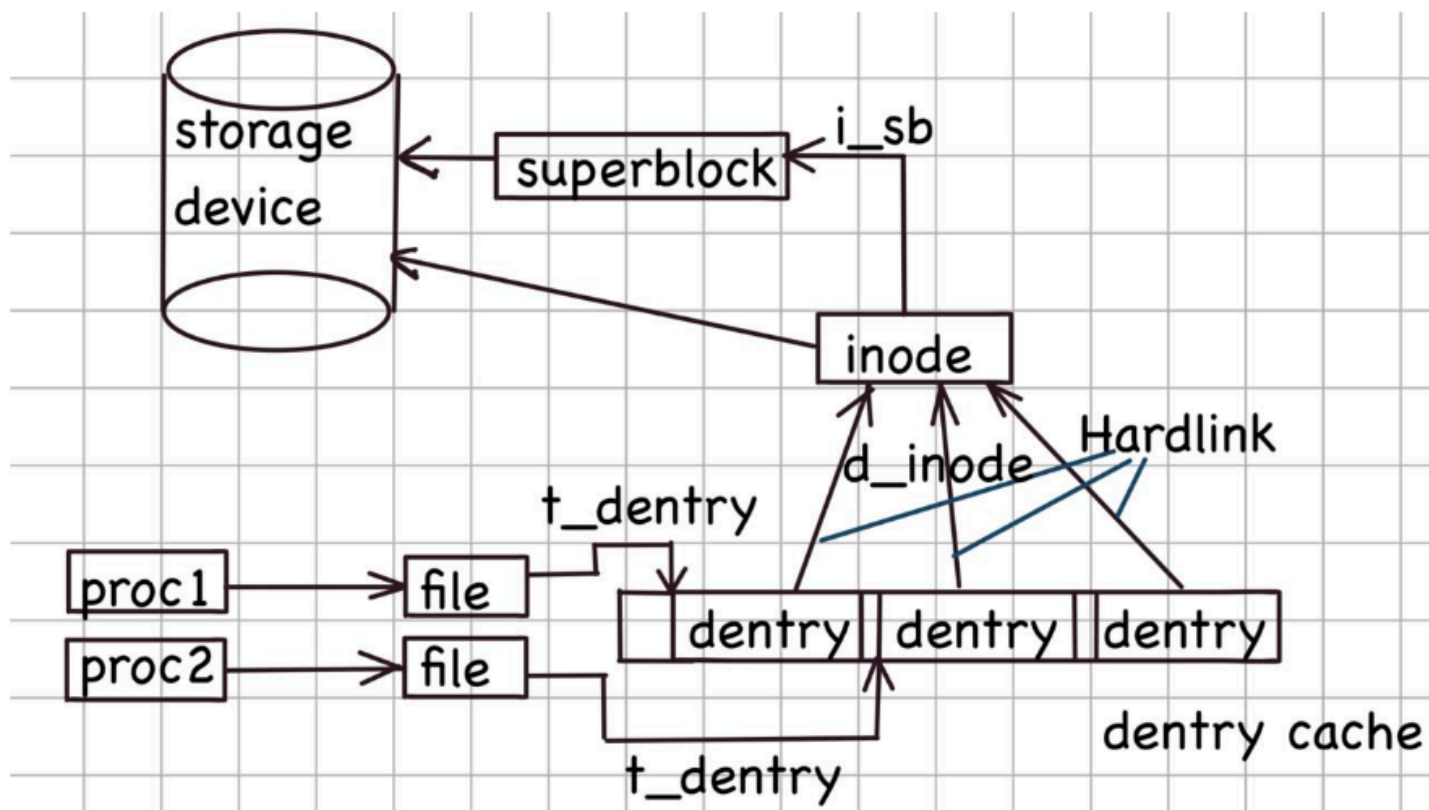


Рис. 19. Связи структур

6.6 Раздел жесткого диска и суперблок

Адресное пространство диска делится на разделы, один раздел занимает одна ФС.

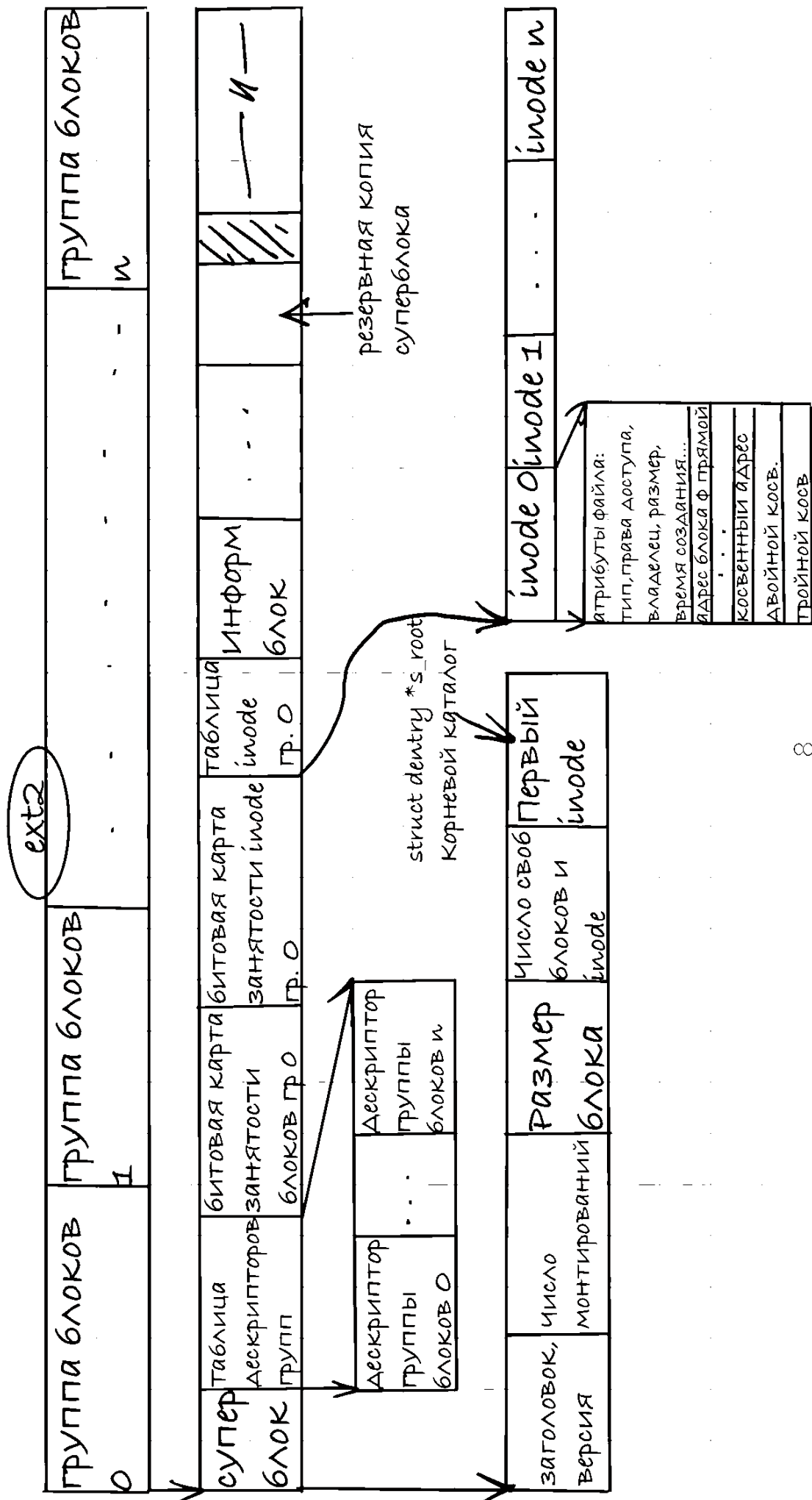


Рис. 20. Обобщённая структурная схема файловой системы ext2

Суперблок представляет в VFS информацию о смонтированной ФС — структура с данными, необходимыми для монтирования и доступа к файлам ФС: метаданные для обращения к файлам, хранящимся «вразброс» (адресация участков диска). В каждой ФС ровно один суперблок, он располагается в начале раздела (есть резервная копия). При монтировании суперблок считывается в память ядра и находится там до размонтирования.

```
1 struct super_block {
2     struct list_head      s_list;
3     dev_t                 s_dev;
4     unsigned long         s_blocksize;
5     loff_t                s_maxbytes;    // Max file size
6     struct file_system_type *s_type;
7     const struct super_operations *s_op;
8     unsigned long         s_flags;
9     unsigned long         s_magic;
10    struct dentry          *s_root;       // точка монтирования
11    struct rw_semaphore    s_umount;
12    struct list_head       s_mounts;
13    struct block_device     *s_bdev;
14    struct list_head       s_inodes;     // all inodes
15 };
```

Листинг 20. Структура `super_block` (сокр.)

- `s_list` — список подмонтированных ФС (`list_head` — двусвязный список); `s_dev`, `s_bdev` — устройство и блочное устройство ФС.
- `s_blocksize`, `s_maxbytes` — размер блока и максимальный размер файла.
- `s_type`, `s_op` — тип ФС и функции на суперблоке; `s_flags` — флаги монтирования; `s_magic` — магическое число ФС.
- `s_root` — точка монтирования (корневой каталог); `s_inodes` — список всех `inode`.
- `s_umount` — семафор «читатели-писатели» для монтирования; `s_mounts` — список монтирований данной ФС (одна и та же ФС может быть смонтирована много раз).

Осторожно

Вместо первого рисунка суперблока, который Рязанова не показывала (показывала), вероятно, хочет видеть второй.

Суперблок:

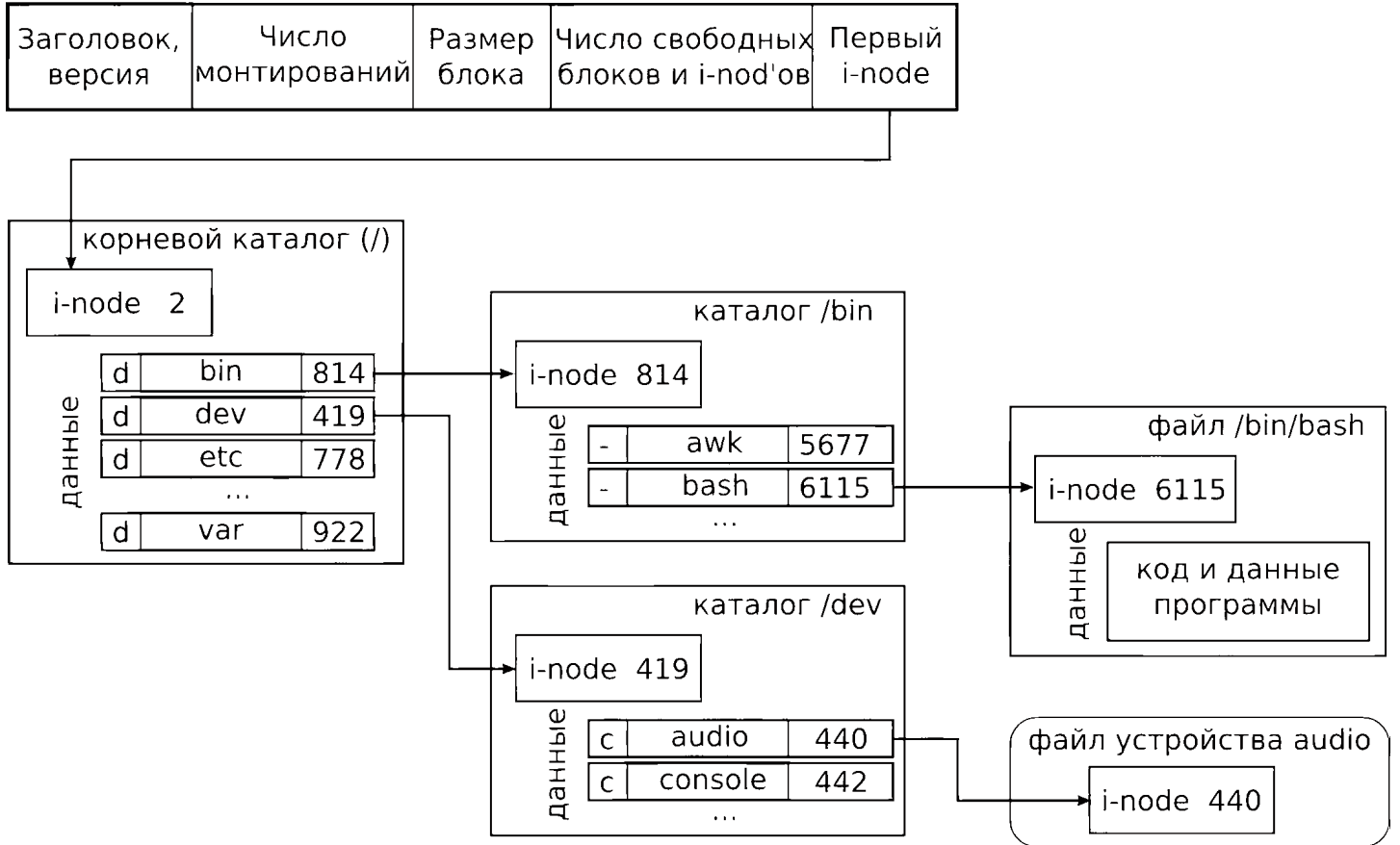


Рис. 21. Суперблок

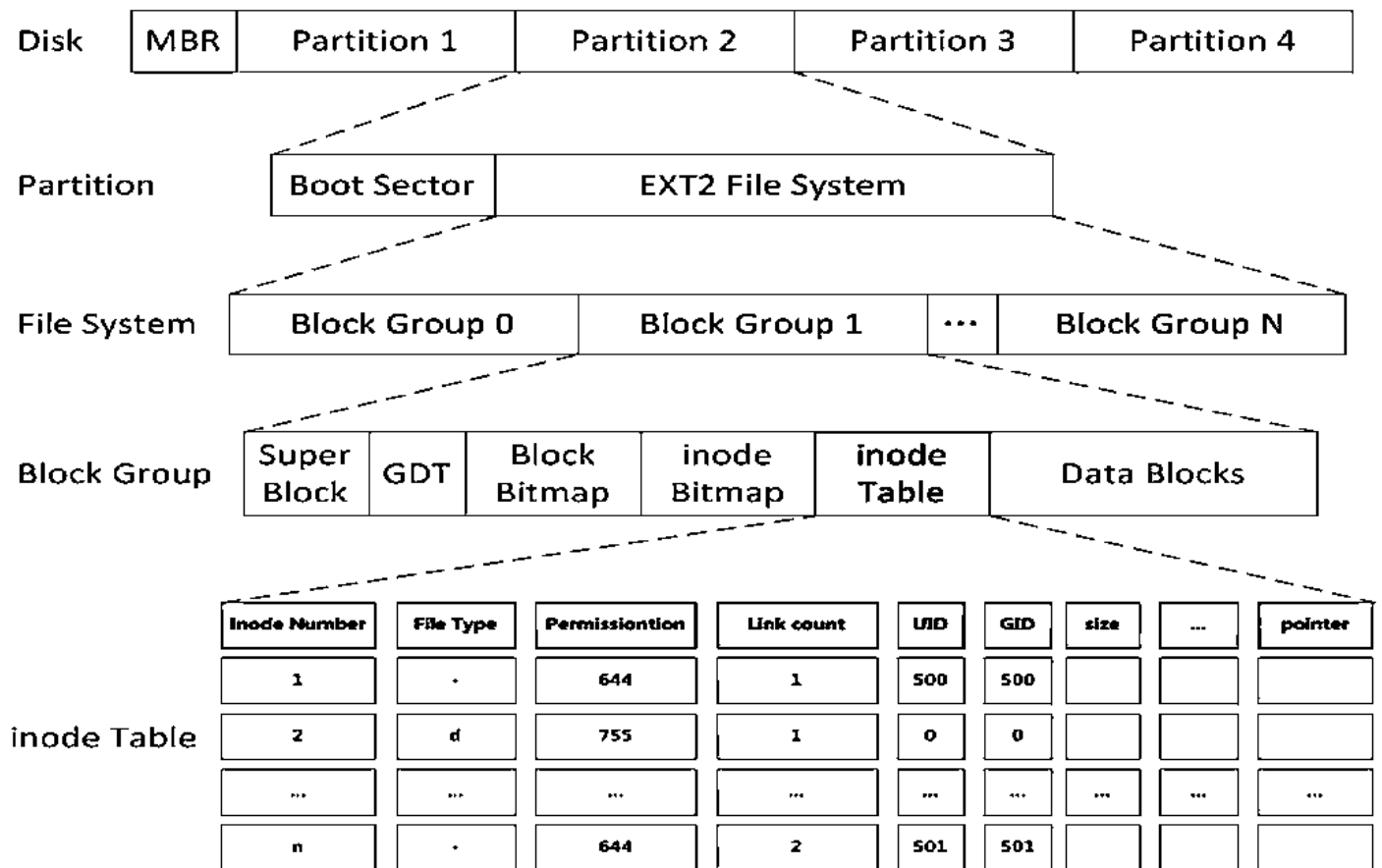


Рис. 22. Суперблок

6.7 Структура `struct super_operations`

Для каждой ФС определяется свой набор операций над объектами суперблока. Поскольку разработчики Linux ставили задачу обеспечить интерфейс для большого числа ФС, перечень операций на объектах суперблока широк:

```
1 struct super_operations {
2     struct inode *(*alloc_inode)(struct super_block *sb);
3     void (*destroy_inode)(struct inode *);
4     void (*free_inode)(struct inode *);
5     void (*dirty_inode) (struct inode *, int flags);
6     int (*write_inode) (struct inode *, struct writeback_control *wbc);
7     int (*drop_inode) (struct inode *);
8     ...
9     void (*put_super) (struct super_block *);
10    ...
11    int (*freeze_fs) (struct super_block *);
12    ...
13    int (*remount_fs) (struct super_block *, int *, char *);
14    void (*umount_begin) (struct super_block *);
15    int (*show_options)(struct seq_file *, struct dentry *);
16    ...
17    int (*show_path)(struct seq_file *, struct dentry *);
18    ...
19 };
```

Листинг 21. `struct super_operations`

Основное количество перечисляемых операций — это операции с `inode`:

- `alloc_inode` — создаёт и инициализирует новый объект `inode` для конкретной смонтированной ФС (выделяет адресное пространство);
- `destroy_inode` — удаляет `inode` из связного списка `inode`;
- `free_inode` — информация с метаданными (с номером `inode`) не удаляется, и данный `inode` может быть повторно использован;
- `dirty_inode` — вызывается, чтобы переместить `inode` в другой список, если в него были введены изменения;
- `write_inode` — записывает `inode` на диск (только `alloc` недостаточно);
- `drop_inode` — вызывается интерфейсом VFS, когда количество ссылок на `inode` равно 0;
- `put_super` — вызывается VFS при размонтировании ФС; данный суперблок изымается из связного списка монтирований;
- `remount_fs` — вызывается VFS для монтирования суперблока с другими параметрами;

- `umount_begin` — вызывается VFS для прерывания монтирования; используется сетевыми ФС, например NFS (Network File System).

В перечне `super_operations` нет функции создания суперблока: сам суперблок создаёт отдельная функция `alloc_super` (не входит в `struct super_operations`), которая выделяет и инициализирует новую структуру `super_block` и возвращает указатель на неё.

6.8 Монтирование файловой системы, команда `mount` и функции монтирования и их параметры, точка монтирования — корневой каталог и `inode`

6.8.1 Монтирование файловой системы. Точка монтирования

Дерево файлов и каталогов в Linux формируется из ветвей, соответствующих разным носителям (разным ФС), но это одно большое дерево; некоторые каталоги могут быть точками монтирования.

Монтирование — система действий, делающая ФС (и все её файлы) доступной. Точка монтирования — каталог, к которому подмонтируется ФС: файлы и директории монтируемой ФС становятся файлами и поддиректориями точки монтирования, а её прежние файлы скрываются до размонтирования (поэтому лучше использовать пустые каталоги в качестве точки монтирования). Доступ к файлам ФС возможен только после монтирования. В Unix/Linux всегда есть корневая ФС (root fs); её монтирование — часть инициализации системы.

Когда ФС монтируется, создаётся структура `struct vfsmount`, которая представляет конкретный экземпляр смонтированной ФС:

```
1 struct vfsmount {
2     struct dentry      *mnt_root;  /* root of the mounted tree */
3     struct super_block *mnt_sb;    /* pointer to superblock */
4     int                 mnt_flags;
5     struct mnt_idmap   *mnt_idmap;
6 };
```

Листинг 22. Структура `vfsmount`

`mnt_root` — корень подмонтированного дерева, `mnt_sb` — указатель на суперблок, `mnt_flags` — флаги монтирования.

6.8.2 Команда `mount`

Команда `mount` имеет следующий синтаксис:

```
1 mount <устройство> <точка монтирования>
```

Ключи: `-t <тип ФС> -o <опции ФС>`, то есть полная форма:

```
1 mount -t type device dir
```

Точка монтирования — любой каталог; в самой ФС нужно создать корневой каталог (`root`) — именно он монтируется. Ключ `-t` необязателен: система сама может определить тип ФС. После монтирования каталоги и файлы ФС доступны. Обычно монтирование требует прав суперпользователя.

С помощью команды `mount [-l] [-t type]` можно получить список смонтированных файловых систем типа `type`. Размонтируется ФС командой `umount <точка монтирования>`.

6.8.3 Функции монтирования и их параметры

В ядре определено несколько функций `mount()`; все они имеют модификатор `extern` и возвращают значение типа `struct dentry*`. В версии ядра 6.17.13, в отличие от предыдущих подверсий, определено только две:

```
1 extern struct dentry *mount_bdev(struct file_system_type *fs_type, int flags,  
2                               const char *dev_name, void *data, int (*fill_super)  
3                               (struct super_block *, void *, int));  
4 extern struct dentry *mount_nodev(struct file_system_type *fs_type, int flags,  
5                                  void *data, int (*fill_super)  
6                                  (struct super_block *, void *, int));
```

Параметры функций монтирования:

- `fs_type` — тип файловой системы;
- `flags` — флаги монтирования;
- `dev_name` — имя устройства (только у `mount_bdev`; внешнее отличие двух функций именно в этом параметре);
- `data` — произвольные данные монтирования;
- `fill_super` — указатель на функцию заполнения суперблока.

Различаются функции существенно кодом и областью применения:

- `mount_bdev` — для монтирования ФС, находящейся на блочном устройстве;
- `mount_nodev` — для монтирования ФС, не связанной ни с каким устройством.

Функциям монтирования передаётся функция `fill_super` — она выполняет основные действия для монтирования: инициализирует поля `struct super_block` (размер блока, смещение, магическое число, `super_operations`). В системе существует готовая функция `simple_fill_super`, которую находим в коде ядра и используем как основу для собственной `fill_super`.

Когда файловая система отмонтируется, всё должно быть удалено — этим занимается функция `kill_sb` из `file_system_type`. В ядре для этого определены функции:

```
1 void generic_shutdown_super(struct super_block *sb);
2 void kill_block_super(struct super_block *sb);
3 void kill_anon_super(struct super_block *sb);
4 void kill_litter_super(struct super_block *sb);
```

Общую работу по удалению суперблока выполняет `generic_shutdown_super`.

6.8.4 Точка монтирования — корневой каталог и `inode`

Функции монтирования возвращают объект `dentry`, которым должен быть `root`: точкой монтирования является корневой каталог ФС. Поэтому, чтобы смонтировать ФС, в ней нужно создать корневой каталог (`root`) — в `struct super_block` это поле `s_root`.

Чтобы создать `root`, надо проинициализировать `inode`, выделив для него память функцией `new_inode()` (внутри неё виден аллокатор — `inode` аллоцируется в `slab`-кеше):

```
1 struct inode *new_inode(struct super_block *sb) {
2     struct inode *inode = new_inode_pseudo(sb); // -> alloc_inode(sb): выделение
    из slab
3     if (inode)
4         inode_sb_list_add(inode);
5     return inode;
6 }
```

Листинг 23. Функция `new_inode` (сокр.)

Из созданного `inode` функцией `d_make_root` делается корневой `dentry`, который записывается в поле `s_root` суперблока. Так точка монтирования (корневой каталог) оказывается связана с корневым `inode`.

6.9 Пример (лаб. раб.)

В лабораторной работе создаётся собственная (виртуальная) файловая система в загружаемом модуле ядра; по отношению к VFS выполняются два действия:

- 1) **Регистрация ФС:** объявляем свой `struct file_system_type` (в коде — `myfs_type`; минимально `owner`, `name`, плюс поля `mount` и `kill_sb`); в `init(myfs_init)` вызываем `register_filesystem`, в `exit(myfs_exit)` — `unregister_filesystem`.
- 2) **Монтирование ФС:** в своей функции `mount(myfs_mount)` вызываем `mount_nodev` (ФС не связана с устройством), передавая ей свою `fill_super` (в коде `myfs_fill_sb`, написана на основе `simple_fill_super`). Именно `myfs_fill_sb` инициализирует поля `struct super_block` (размер блока, магическое число,

свой набор `super_operations`), через `new_inode` создаёт `inode` корневого каталога (с готовыми «simple»-операциями каталога) и функцией `d_make_root` делает из него `root`, записывая его в `s_root`. Освобождение суперблока поручено готовой функции ядра `kill_litter_super` (поле `kill_sb`); чтобы при размонтировании в `syslog` попадало сообщение, в своём наборе `super_operations` задаётся собственный `put_super` (`myfs_put_super`).

После загрузки модуля ФС монтируется командой `mount` и становится видна в дереве каталогов (может быть смонтирована несколько раз). Результат показывают `ls -ial`, `cat /proc/mounts` (смонтированные ФС) и `cat /proc/filesystems` (зарегистрированный тип ФС).

```
1 #include <linux/module.h>
2 #include <linux/kernel.h>
3 #include <linux/init.h>
4 #include <linux/fs.h>
5 #include <asm/atomic.h>
6 #include <asm/uaccess.h>
7
8 MODULE_LICENSE("GPL");
9 MODULE_AUTHOR("Zhizhin Nikita");
10
11 #define MYFS_MAGIC_NUMBER 0x13131313
12 static void myfs_put_super(struct super_block *sb) {
13     printk("myfs: put_super\n");
14 }
15 static struct super_operations const myfs_super_ops = {
16     .put_super = myfs_put_super,
17     .statfs    = simple_statfs,
18     .drop_inode = generic_delete_inode
19 };
20 static int myfs_fill_sb(struct super_block *sb, void *data, int silent) {
21     struct inode *inode;
22     struct dentry *root;
23     sb->s_blocksize = PAGE_SIZE;
24     sb->s_blocksize_bits = PAGE_SHIFT;
25     sb->s_magic = MYFS_MAGIC_NUMBER;
26     sb->s_op = &myfs_super_ops;
27     sb->s_time_gran = 1;
28     inode = new_inode(sb);
29     if (!inode) {
30         printk(KERN_ERR "myfs: new_inode error\n");
31         return -ENOMEM;
32     }
```

```

33     inode->i_ino = 1;
34     inode->i_mode = S_IFDIR | 0755;
35     simple_inode_init_ts(inode);
36
37     inode->i_op  = &simple_dir_inode_operations;
38     inode->i_fop = &simple_dir_operations;
39
40     root = d_make_root(inode);
41     printk("myfs: d_make_root\n");
42     if (!root) {
43         printk(KERN_ERR "myfs: d_make_root error\n");
44         iput(inode);
45         return -ENOMEM;
46     }
47     sb->s_root = root;
48     printk(KERN_INFO "myfs: VFS root created\n");
49     printk(KERN_INFO "myfs: super_block init");
50     return 0;
51 }
52 static struct dentry *myfs_mount(struct file_system_type *type, int flags, const
    char *dev, void *data) {
53     struct dentry *const entry = mount_nodev(type, flags, data, myfs_fill_sb);
54     if (IS_ERR(entry))
55         printk(KERN_ERR "myfs: mount error\n");
56     else
57         printk(KERN_INFO "myfs: mount\n");
58     return entry;
59 }
60 static struct file_system_type myfs_type = {
61     .owner    = THIS_MODULE,
62     .name     = "myfs",
63     .mount    = myfs_mount,
64     .kill_sb  = kill_litter_super
65 };
66 static int __init myfs_init(void) {
67     int ret;
68
69     printk(KERN_INFO "myfs: register_filesystem call\n");
70     ret = register_filesystem(&myfs_type);
71     if (ret) {
72         printk(KERN_ERR "myfs: register_filesystem error\n");
73         return ret;
74     }
75     printk(KERN_INFO "myfs: loaded\n");
76     return 0;

```

```
77 }
78 static void __exit myfs_exit(void) {
79     int ret;
80     ret = unregister_filesystem(&myfs_type);
81     if (ret)
82         printk(KERN_ERR "myfs: unregister_filesystem error\n");
83     else
84         printk("myfs: unregister_filesystem\n");
85     printk(KERN_INFO "myfs: unloaded\n");
86 }
87 module_init(myfs_init);
88 module_exit(myfs_exit);
```


7

7 Виртуальная файловая система /proc: структура proc_dir_entry, создание файла proc, директории и символические ссылки; точки входа и их регистрация, интерфейсы передачи данных из ядра в пространство пользователя — реализация и особенности. Примеры из лабораторных работ.

7.1 Виртуальная файловая система /proc

Виртуальная файловая подсистема /proc (process information pseudo-file-system) позволяет в режиме пользователя получить большой объем информации о ресурсах процесса. Виртуальная — то есть не существующая на самом деле (не хранящаяся во вторичной памяти), а создающаяся “на лету” по мере обращения к данным. По сути интерфейс /proc позволяет получить информацию из структур ядра, не обращаясь к ним напрямую, а потому работая в режиме пользователя. Большинство файлов доступны только для чтения, однако некоторые доступны для записи, что позволяет изменять переменные ядра.

ФС /proc является немонтируемой. Под этим подразумевается, что она не связана ни с каким внешним носителем (она видна в файловых системах, подключенных к корневому каталогу, но не связана ни с каким внешним устройством. Поэтому она и называется псевдофайловой системой).

Для каждого процесса в /proc определена директория идентификатора процесса (/proc/<pid>/), она будет создана налету. В ней хранятся файлы, директории и символические ссылки, используя которые можно получить развёрнутую информацию о конкретном процессе. Вся информация доступна из usermode:

Элемент	Тип	Содержание
cmdline	Файл	holds the complete command line Содержит командную строку процесса
cwd	Символическая ссылка	Указывает на рабочую директорию процесса
environ	Файл	Содержит список переменных окружения процесса
exe	Символическая ссылка	Указывает на образ процесса (на его файл) Символическая ссылка, содержащая фактический путь к выполняемой команде.
comm	Файл	Имя команды, связанное с процессом. Это именно то имя файла, которое приведёт к выполнению нашего кода.

Элемент	Тип	Содержание
		Замечание 7.1 Разные потоки в одном и том же процессе могут иметь разные значения, доступные через <code>/proc/<pid>/task/<tid>/comm</code>. <code>task</code> — файл, в котором находятся поддиректории потоков.
stat	Файл	Содержит информацию о процессе (vsize, start_brk и др.)
fd	Директория	Содержит ссылки на открытые процессом файлы Замечание 7.2 В системе различаются файлы и открытые файлы. Открытые файлы описываются структурой <code>struct file</code>. Просто файлы представляются в системе деревом каталогов, а описываются физические файлы <code>struct inode</code>. Один и тот же файл может быть открыт много раз, причём разными процессами. В таблице открытых файлов на каждый такой открытый файл будет дескриптор <code>struct file</code>
root	soft link	Это символическая ссылка, которая указывает на корень файловой системы процесса. Замечание 7.3 Процессы, которые выполняются на нашем компьютере, могут принадлежать разным файловым системам. Пока мы имя исполняемого файла не указали в командной строке, пока он не начал выполняться, он лежит во вторичной памяти.
maps	Файл	Это адресное пространство того процесса, <code>pid</code> которого мы указали. В зависимости от <code>pid</code> информация будет отличаться, но есть неизменные составляющие:
pagemap	файл	Это 64-х разрядное значение для каждой строки процесса. При этом нас интересует фактически несколько бит: 1) 63 бит — present (in RAM) 2) 62 бит — swap space, область своппинга 3) 61 — shared anonymous page 4) 55 бит — dirty. Он указывается в дескрипторе страницы.

Элемент	Тип	Содержание
		С 54 по 0: если страница присутствует в физической памяти, то эти биты являются номером фрейма, то есть номером физической страницы.
task	Директория	Содержит поддиректории потоков
interrupts	Файл	Предоставляет таблицу о количестве прерываний на каждом из процессоров в следующем виде: 1) Первая колонка: номер прерывания 2) Колонки CPUx: счётчики прерываний на каждом из процессоров 3) Следующая колонка: вид прерывания: 3.1) IO-APIC-edge — прерывание по фронту на контроллер I/O APIC. 3.2) IO-APIC-fastestoi — прерывание по уровню на контроллер I/O APIC. 3.3) PCI-MSI-edge — MSI прерывание. 3.4) XT-PIC-XT-PIC — прерывание на PIC контроллер. 4) Последняя колонка: устройство, ассоциированное с данным прерыванием.

Таблица 5. Содержание директории /proc/<pid>

7.2 Структура proc_dir_entry: функции для работы с элементами proc

Файлы и поддиректории файловой системы /proc описаны структурой proc_dir_entry.

1	struct proc_dir_entry {
2	atomic_t in_use;
3	// ...
4	struct list_head pde_openers;
5	// ...
6	const struct inode_operations *proc_iops;
7	union {
8	const struct proc_ops *proc_ops;
9	const struct file_operations *proc_dir_ops;
10	};
11	
12	const struct dentry_operations *proc_dops;
13	union {
14	const struct seq_operations *seq_ops;

```

15     int (*single_show)(struct seq_file *, void *);
16 };
17
18     proc_write_t write;
19     void *data;
20     unsigned int state_size;
21     unsigned int low_ino;
22     nlink_t nlink;
23     // ...
24     loff_t size;
25     struct proc_dir_entry *parent;
26     // ...
27     char *name;
28     // ...
29     u8 namelen;
30     char inline_name[];
31 };

```

Листинг 24. Структура struct proc_dir_entry

- atomic_t — неделимая переменная, счётчик.
- const struct inode_operations *proc_iops — у файла proc должен быть inode и должны быть определены inode_operations.
- Первый union — интерфейс регистрации функций. До шестой версии ядра существовала только struct file_operations *proc_dir_ops. В качестве альтернативы определена структура const struct proc_ops *proc_ops — в ней практически такие же функции. Объясняется такое решение тем, что функции struct file_operations вызываются интенсивно: все драйверы вызывают функции, определённые в этой структуре. Чтобы не нагружать эти функции дополнительными задачами, для работы с VFS proc, которая является информационной (используется для получения разработчиками информации), введена отдельная структура.
- Второй union — здесь видна информация о двух seq-интерфейсах взаимодействия с user-mode:
 - const struct seq_operations *seq_ops — полный интерфейс, полный набор функций;
 - int (*single_show)(struct seq_file *, void *) — упрощённый интерфейс.
- proc_write_t write — в самых старых версиях структуры было определено 2 функции: read и write, сейчас оставлена только функция write. Передача данных из user-mode в kernel-mode — это write, обратно — read.

- `void *data` — приватные данные для работы с файлами `proc`.
- `unsigned int low_ino` — `inode` нижнего уровня (мы с ним не работаем). Поскольку `proc` — виртуальная файловая система, данные не хранятся во вторичной памяти, всё создаётся на лету в оперативной памяти, но `inode` у любого файла, в том числе и виртуального, всё равно имеется.
- `struct proc_dir_entry *parent` — указатель на родителя: поддерживается иерархия файловой системы `proc`.
- `loff_t size` — размер. О типе `loff_t`: введено огромное количество типов, детализированных по назначению (ярчайший пример — `pid_t`, хотя мы понимаем, что это `int`). Это сделано для дополнительного контроля соответствия типов при компиляции.

7.3 Создание файла `proc`, директории и символические ссылки

Для создания файла виртуальной файловой системы `proc` в ядре имеется функция `proc_create_data`, для создания каталогов, используется функция `proc_mkdir`, символические ссылки создаются — с помощью `proc_symlink` (третий параметр — путь, на который указывает ссылка). Удаляется элемент `proc` (файл, директория или ссылка) функцией `remove_proc_entry`:

```

1 // создание файла
2 extern struct proc_dir_entry *proc_create_data(
3     const char *name,
4     umode_t mode,
5     struct proc_dir_entry *parent,
6     const struct file_operations *proc_ops,
7     void *data);
8
9 struct proc_dir_entry *proc_create(
10     const char *name,
11     umode_t mode,
12     struct proc_dir_entry *parent,
13     const struct file_operations *proc_fops);
14
15 // создание директории
16 extern struct proc_dir_entry *proc_mkdir(
17     const char *name,
18     struct proc_dir_entry *parent);
19
20 // создание символической ссылки
21 extern struct proc_dir_entry *proc_symlink(
22     const char * name,
```

```

23 struct proc_dir_entry *parent,
24 const char *);
25
26 // удаление
27 remove_proc_entry(
28 const char *name,
29 struct proc_dir_entry *parent);

```

Параметры:

- name — имя файла в proc;
- mode — права доступа (read/write для user, group, others);
- parent — указатель на родительскую директорию, то есть поддерживается иерархия файловой системы proc: чтобы файл создавался в поддиректории proc, нужно до proc_create_data вызвать proc_mkdir и передать эту директорию; если в параметре parent указать NULL, файл будет создаваться в корневом каталоге proc;
- proc_ops — указатель на проинициализированную структуру proc_ops с нашими функциями.

7.4 Точки входа и их регистрация

В загружаемом модуле ядра, работающем с файлом proc, 6 точек входа: open, read, write, release, init, exit.

Свои функции регистрируются инициализацией полей структуры struct proc_ops (для старых ядер — struct file_operations):

```

1 struct proc_ops {
2     ...
3     int (*proc_open)(struct inode *, struct file *);
4     ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
5     ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
6     ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
7     loff_t (*proc_lseek)(struct file *, loff_t, int);
8     int (*proc_release)(struct inode *, struct file *);
9     ...
10 };

```

Листинг 25. Структура proc_ops

Для стандарта C99 начиная с версии GCC 2.5 принят следующий синтаксис (назначенные инициализаторы):

```

1 static const struct proc_ops device_proc_ops = {
2     .proc_open = device_open,

```

```

3     .proc_read = device_read,
4     .proc_write = device_write,
5     .proc_release = device_release,
6 };

```

Регистрация точек входа происходит при инициализации полей структуры `proc_ops` (до регистрации эти функции должны быть объявлены и описаны), а затем проинициализированная структура передаётся функции создания виртуального файла `proc_create`.

7.5 Интерфейсы передачи данных из ядра в пространство пользователя — реализация и особенности

В своих функциях `read`, `write` мы должны вызывать специальные функции ядра для передачи данных из пространства пользователя в пространство ядра и обратно: `copy_to_user` копирует данные из буфера ядра в буфер пользователя, `copy_from_user` — из буфера пользователя в буфер ядра.

Специальные функции нужны, потому что ядро и загружаемый модуль ядра, становящийся частью ядра, оперирует физическими адресами: ядро загружено в физическую память резидентно, а процессы имеют только виртуальные адресные пространства. Процесс получает ресурс «физическая страница» только по мере надобности. При этом ядро, чтобы передать данные, должно обратиться к буферу пространства пользователя. Вероятна ситуация, когда буфер (страница), к которому обращается ядро, выгружен — в результате **возникнет страничное прерывание**, которое должно обработать ядро (должен быть выполнен код ядра). Ядро обратилось, стало источником прерывания, оно же и должно его обработать; если не предпринять специальных мер, возникнет паника ядра.

```

1  #ifdef INLINE_COPY_TO_USER
2
3  static inline __must_check unsigned long
4  _copy_to_user(void __user *to, const void *from, unsigned long n)
5  {
6      might_fault();
7      if (should_fail_usercopy())
8          return n;
9      if (access_ok(to, n)) {
10         instrument_copy_to_user(to, from, n);
11         n = raw_copy_to_user(to, from, n);
12     }
13     return n;
14 }

```

```

15
16 #else
17
18 extern __must_check unsigned long
19 _copy_to_user(void __user *to, const void *from, unsigned long n);
20
21 #endif

```

Листинг 26. Определение функции `copy_to_user`

```

1 static inline __must_check unsigned long
2 _copy_from_user(void *to, const void __user *from, unsigned long n)
3 {
4     ...
5 }

```

Листинг 27. Определение функции `copy_from_user`

Для `copy_to_user` указатель `to` — это указатель на буфер пользователя, `from` — указатель на буфер ядра, `n` — количество байт; для `copy_from_user` — обратная ситуация. Обе функции возвращают количество байт, которые не могут быть скопированы; в случае успешного выполнения будет возвращён 0.

7.6 Примеры из лабораторных работ

Этой теме посвящены 2 лабораторные работы, обе связаны с файлами VFS `proc`; работа ведётся в ядре, то есть пишется загружаемый модуль ядра.

7.6.1 ЛР «Фортуны»

Лабораторная относится к теме передачи данных из пространства пользователя в пространство ядра и наоборот. В модуле 6 точек входа: `open`, `read`, `write`, `release`, `init`, `exit`. Функции регистрируются на виртуальном файле и вызываются автоматически процессами `echo` (запись строк-фортунок) и `cat` (чтение); каждая функция пишет в `syslog` через `printk`, что она прочитала или записала. В `init` создаются файл, директория и символическая ссылка (в `proc_create_data` передаётся созданная директория), а перед этим выделяется память для буфера, чтобы было куда записывать данные. В `read/write` данные передаются через `copy_to_user/copy_from_user`.

```

1 #define COOKIE_POT_SIZE 4096
2 #define FILENAME "fortune_file"
3 #define DIRNAME "fortune_dir"
4 #define SYMLINK "fortune_link"
5
6 static struct proc_dir_entry *fortune_file;
7 static struct proc_dir_entry *fortune_dir;
8 static struct proc_dir_entry *fortune_link;

```

```

9
10 static char *cookie_pot = NULL;
11 char buffer[COOKIE_POT_SIZE];
12
13 static int fortune_open(struct inode *inode, struct file *file)
14 {
15     printk(KERN_INFO "+fortune: open\n");
16     return 0;
17 }
18
19 static int fortune_release(struct inode *inode, struct file *file)
20 {
21     printk(KERN_INFO "+fortune: release\n");
22     return 0;
23 }
24
25 static ssize_t fortune_write(struct file *file, const char __user *buf, size_t len,
    loff_t *fpos)
26 {
27     printk(KERN_INFO "+fortune: write\n");
28     ...
29     if (copy_from_user(&cookie_pot[write_index], buf, len) != 0)
30     {
31         printk(KERN_ERR "+fortune: copy_from_user error\n");
32         return -EFAULT;
33     }
34     ...
35     return len;
36 }
37
38 static ssize_t fortune_read(struct file *file, char __user *buf, size_t len, loff_t
    *fpos)
39 {
40     printk(KERN_INFO "+fortune: read\n");
41     ...
42     read_len = snprintf(buffer, COOKIE_POT_SIZE, "%s\n", &cookie_pot[read_index]);
43     if (copy_to_user(buf, buffer, read_len) != 0)
44     {
45         printk(KERN_ERR "+fortune: copy_to_user error\n");
46         return -EFAULT;
47     }
48     ...
49     return read_len;
50 }

```

Листинг 28. Точки входа open, release, write, read модуля «фортунки»

```

1 static int __init md_init(void) {
2     printk(KERN_INFO "+fortune: init\n");
3     if ((cookie_pot = vmalloc(COOKIE_POT_SIZE)) == NULL) {
4         printk(KERN_ERR "+fortune: memory error\n");
5         return -ENOMEM;
6     }
7     memset(cookie_pot, 0, COOKIE_POT_SIZE);
8     if ((fortune_file = proc_create(FILENAME, 0640, NULL, &fops)) == NULL) {
9         ...
10    }
11    if ((fortune_dir = proc_mkdir(DIRNAME, NULL)) == NULL) {
12        ...
13    }
14    if ((fortune_link = proc_symlink(SYMLINK, NULL, FILENAME)) == NULL) {
15        ...
16    }
17    printk(KERN_INFO "+fortune: loaded\n");
18    return 0;
19 }
20
21 static void __exit md_exit(void) {
22     printk(KERN_INFO "+fortune: exit\n");
23     remove_proc_entry(FILENAME, NULL);
24     remove_proc_entry(DIRNAME, NULL);
25     remove_proc_entry(SYMLINK, NULL);
26     vfree(cookie_pot);
27 }
28
29 module_init(md_init);
30 module_exit(md_exit);

```

Листинг 29. Создание файла, директории и символической ссылки в `init` и их удаление в `exit`

8

8 Средства взаимодействия процессов — сокеты Беркли. Создание сокета — семейство, тип, протокол. Системный вызов `sys_socket()` и `struct socket`. Состояния сокета. Адресация сокетов и ее особенности для разных типов сокетов. Модель клиент-сервер. Сетевые сокеты — сетевой стек, аппаратный и сетевой порядок байтов. Примеры реализации взаимодействия процессов по модели клиент-сервер с использованием сокетов и мультиплексированием (лаб. раб.).

8.1 Сокеты

Сокет — абстракция точки соединения. Являются универсальным средством меж-процессного взаимодействия: могут служить как для взаимодействия процессов на отдельно стоящей машине (память, процессор — всё в одном месте.), так и для взаимодействия в распределённых системах (много машин, соединённых кабелями, и у каждой машины своя память). Были разработаны Unix BSD, поэтому часто называются сокетами Беркли или сокетами BSD.

Определение 8.1 (Сокет)

Сокет — абстракция точки соединения.

Абстракция от особенностей взаимодействия. Средство универсальное: на конкретной машине особенности определяются тем, что это процессор и память, а в распределённой системе — тем, что память распределённая.

В компьютере обращение к данным всегда выполняется по адресу, но на отдельно стоящей машине один способ обращения, а в распределённых системах — другой, поскольку нужно иметь адреса хостов.

Разработаны парные сокеты — отдельный системный вызов `socketpair()`. Парные сокеты сделаны как альтернатива `pipe`, но в отличие от последних, которые обеспечивают симплексную связь, сокеты имеют дуплексную связь.

8.2 Создание сокета — семейство, тип, протокол

```
1 int socket(int family, int type, int protocol);
```

Системный вызов `socket()` создаёт конечную точку взаимодействия и возвращает файловый дескриптор, который ссылается на эту конечную точку: при успешном вызове возвращается дескриптор с наименьшим номером, который в данный момент ещё не открыт для процесса.

Параметры системного вызова `socket()`:

- 1) `family` — пространство (семейство) адресов. В компьютере всё адресуется. Раньше чаще использовалось ключевое слово `domain`, сейчас — `family`.

- 1.1) `AF_UNIX` — сокет для взаимодействия параллельных процессов на отдельно стоящих машинах. Для него используется тип `SOCK_DGRAM`.
 - 1.2) `AF_INET` — имеет тип `SOCK_DGRAM`.
 - 1.3) `AF_INET6`
 - 1.4) `AF_IPX`
 - 1.5) `AF_UNSPEC`
- 2) `type` — тип сокета. В зависимости от того, где используется сокет, у него будет соответствующий тип.
- 2.1) `SOCK_STREAM` — определяет ориентированное на потоки надёжное упорядоченное полнодуплексное логическое соединение двух сокетов (между двумя сокетами).
 - 2.2) `SOCK_DGRAM` — определяет ненадёжную службу дейтаграмм (datagram) без установления логического соединения, где пакеты могут передаваться без сохранения порядка. Это широковещательная передача данных.
 - 2.3) `SOCK_RAW` — прямой (сырой, низкоуровневый) сокет.
- 3) `protocol` — выбирается для определённого типа сокета. Обычно параметр устанавливается равным 0, тогда протокол задаётся по умолчанию. Если возможно несколько протоколов, то это нужно указывать специальным образом. Протоколы обозначаются символьными константами с префиксами `IPPROTO_*`, например `IPPROTO_TCP`.
- По умолчанию для сокетов `AF_INET` типа `SOCK_STREAM` будет установлен протокол TCP, для этих же сокетов типа `SOCK_DGRAM` — протокол UDP.

Таким образом, сокеты `AF_UNIX` используются для взаимодействия параллельных процессов на отдельно стоящей машине, а сокеты `AF_INET` — для взаимодействия процессов в распределённой системе (в сети).

8.3 Системный вызов `sys_socket()` и `struct socket`

Сокеты описываются как открытые файлы: они не хранятся во вторичной памяти, это файлы специального типа.

Открытый файл описывает структура `struct file`, а в самой структуре есть указатель на структуру `struct inode`, которая описывает физический файл в любом варианте, то есть хранящийся во вторичной или оперативной памяти.

Для работы с сокетами ядро Linux предоставляет системный вызов `sys_socketcall`. Этот вызов является общей точкой входа для различных операций с сокетами.

В качестве одного из аргументов передаётся номер операции: создание сокета, привязка адреса, установление соединения, ожидание соединений, приём или

передача данных. Внутри ядра по этому номеру выбирается соответствующая функция, например `socket`, `bind`, `connect`, `listen`, `accept`, `send`, `recv` и другие.

Перед выполнением операции ядро копирует аргументы из пользовательского пространства в пространство ядра. Напрямую обращаться к памяти процесса нельзя, поэтому используются специальные функции вроде `copy_from_user` и `copy_to_user`.

`sys_socketcall()` приводит к вызову `sys_socket()`:

```
1 asmlinkage long sys_socket(int family, int type, int protocol)
2 {
3     int retval;
4     struct socket *sock;
5     ...
6     retval = socket_create(family, type, protocol, &sock);
7     ...
8     return retval;
9 }
```

Листинг 30. Системный вызов `sys_socket()`

В ядре сокет описывает структура `struct socket`:

```
1 struct socket
2 {
3     socket_state state;
4     short type;
5     unsigned long flags;
6     const struct proto_ops *ops; // protocol operations
7     struct fasync_struct *fasync_list;
8     struct file *file;
9     struct sock *sk;
10    wait_queue_head_t wait;
11 };
```

Листинг 31. Структура `socket`

У сокета может быть 5 состояний (`state`):

- 1) `SS_FREE` — свободный сокет.
- 2) `SS_UNCONNECTED` — несоединённый сокет
- 3) `SS_CONNECTING` — сокет находится в процессе соединения
- 4) `SS_CONNECTED` — соединённый сокет
- 5) `SS_DISCONNECTING` — сокет в процессе разъединения

8.4 Адресация сокетов и ее особенности для разных типов сокетов

Сокеты BSD поддерживают множество протоколов. Так как сокеты адресуются и обращение к ним тоже выполняется по адресу, на сокетах была определена общая структура `struct sockaddr`:

```
1 struct sockaddr
2 {
3     sa_family_t sa_family; // AF_XXX
4     char sa_data[14];
5 };
```

Листинг 32. Структура `sockaddr`

Ничего не конкретизировано, просто 14 байт. Структура определена в таком виде, потому что разработчики понимали, что в разных системах нужна разная адресация. Поле `sa_family` определяет семейство адресов, но точный формат адреса не определён.

Особенности для сокетов AF_UNIX. Для сокетов `AF_UNIX` в поле `sa_data` помещается имя файла, поскольку эти сокеты взаимодействуют через пространство файловых имён. Адрес сокета `AF_UNIX` — имя файла. Сокет сервера можно показать в файловой подсистеме: командой `ls -ial` выводится файл сокета с его `inode`, при этом тип файла обозначается буквой `s`.

Особенности для сокетов AF_INET. Для сетей адрес в виде структуры `struct sockaddr` не подходит, мы должны указать сетевой адрес и номер порта, поэтому объявлена структура `struct sockaddr_in` для IP версии 4. Для IP версии 6 также объявлена структура `struct sockaddr_in6`.

```
1 struct in_addr
2 {
3     _u32 s_addr;
4 };
5
6 struct sockaddr_in
7 {
8     sa_family_t sin_family;      // AF_INET
9     unsigned short int sin_port; // Номер порта
10    struct in_addr sin_addr;      // Сетевой адрес
11    unsigned char sin_zero[sizeof(struct sockaddr) - sizeof(sa_family_t) -
12                           sizeof(uint16_t) - sizeof(struct in_addr)];
12 };
```

Листинг 33. Структура `sockaddr_in`

В наших системах 2 адресных пространства: физической памяти и портов ввода/вывода. Порт — это адрес, но он локальный.

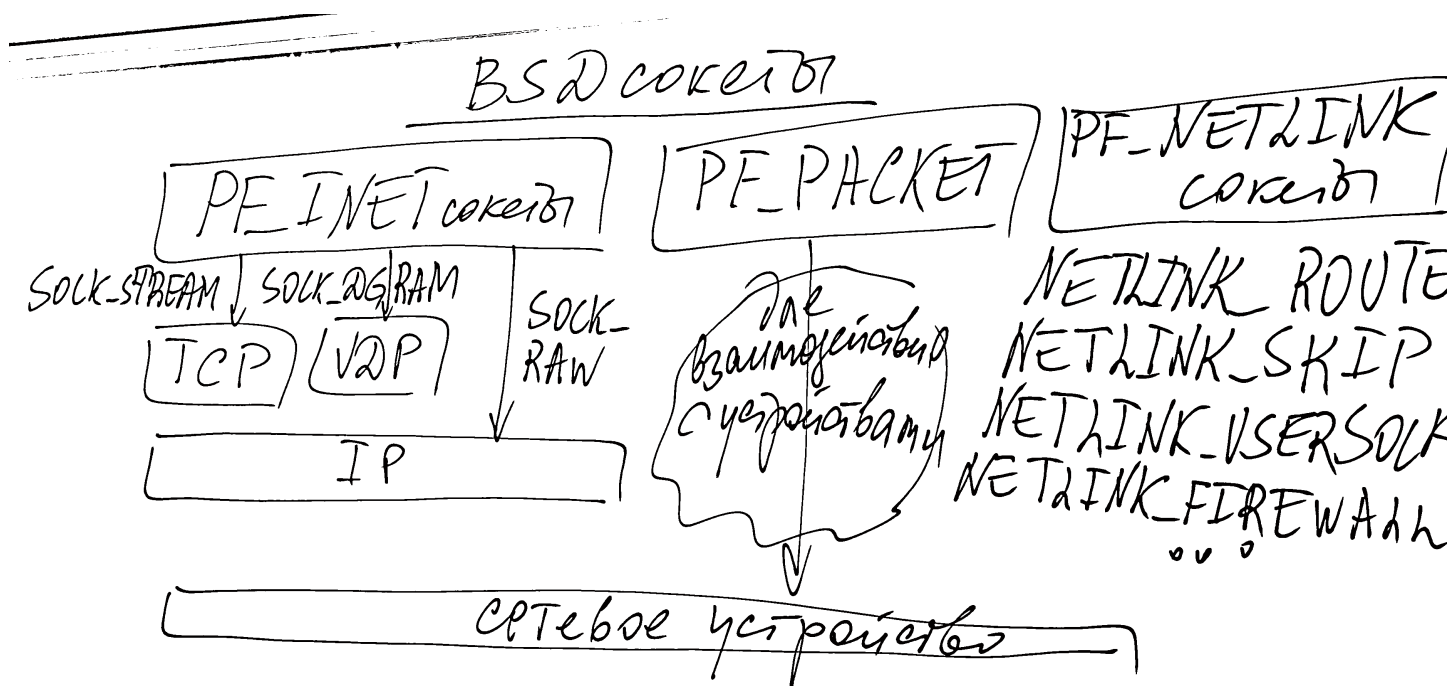


Рис. 23. BSD сокетс

8.5 Модель клиент-сервер

Когда клиент запрашивает соединение с сервером, сервер может либо принять, либо отклонить это соединение. Если соединение принято, то сервер устанавливает и поддерживает соединение с клиентом по определённому протоколу. Клиент не предоставляет общий доступ ни к одному из своих ресурсов, но запрашивает данные или службу у сервера.

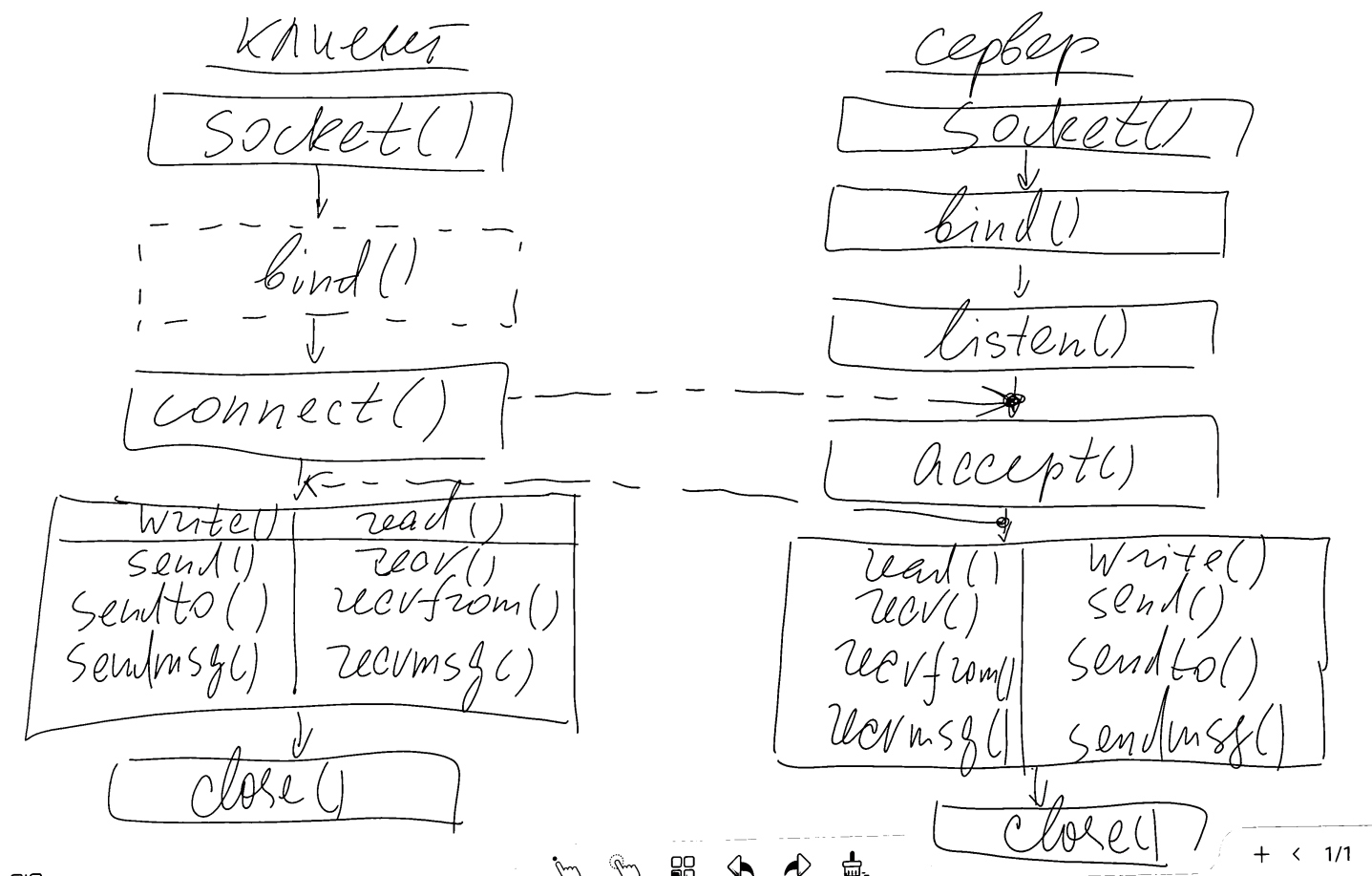


Рис. 24. Клиент-сервер на сокетах

Системный вызов `socket()` создаёт сокет и возвращает файловый дескриптор сокета. Затем на стороне сервера в обязательном порядке вызывается системный вызов `bind()`, который связывает файловый дескриптор сокета с адресом, то есть до `bind()` необходимо инициализировать поля структуры `struct sockaddr_in`. После этого на стороне сервера вызывается системный вызов `listen()`, который переводит серверный процесс в режим пассивного ожидания соединения. Когда соединение произошло, то есть клиент на своей стороне выполнил системный вызов `connect()`, ядро выполняет предварительную регистрацию соединения и выполняется системный вызов `accept()`. После этого начинается обмен данными.

Системный вызов `bind()`. Назначает сокету `sockfd` адрес, указанный `addr`; параметр `addrlen` — размер в байтах структуры адреса, на которую указывает `addr`. Для сетевых сокетов адрес состоит из IP-адреса сетевого интерфейса локальной системы и номера порта. Клиенты могут не вызывать `bind()`, так как их адрес часто никакой роли не играет — в таком случае адрес назначается им автоматически.

```
1 #include <sys/socket.h>
2 int bind(int sockfd, struct sockaddr *addr, int addrlen);
```

Листинг 34. Объявление функции `bind()`

Системный вызов `listen()`. На стороне сервера переводит сервер в режим пассивного прослушивания. Сервер вызывает `listen()` для того, чтобы проинформировать операционную систему, что на сокете нужно принять соединение. Это имеет смысл только для ориентированных на соединение протоколов (TCP). Параметр `backlog` определяет максимальную длину, до которой может вырасти очередь ожидающих соединений для `sockfd`.

```
1 #include <sys/socket.h>
2 int listen(int sockfd, int backlog);
```

Листинг 35. Объявление функции `listen()`

Системный вызов `connect()`. Вызывается на стороне клиента, чтобы создать активное соединение. Клиент должен знать адрес сервера (`serv_addr` — адрес сервера, нужно обратить на это внимание). Для протокола TCP активное соединение устанавливается по указанному адресу. Для протокола UDP `connect()` может использоваться для указания адреса назначения для всех передаваемых впоследствии пакетов.

```
1 #include <sys/socket.h>
2 int connect(int sockfd, struct sockaddr *serv_addr, int addrlen);
```

Листинг 36. Объявление функции `connect()`

Системный вызов `accept()`. Выполняется на стороне сервера для принятия соединения при условии, что ранее поступил запрос на соединение. В противном случае `accept()` будет блокирован до поступления запроса на соединение.

```
1 #include <sys/socket.h>
2 int accept(int sockfd, void *addr, int *addrlen);
```

Листинг 37. Объявление функции `accept()`

Важно

Когда соединение принимается, то есть фактически выполняется системный вызов `accept()`, создаётся копия исходного сокета, при этом исходный сокет останется в состоянии `LISTEN`, а созданная копия будет находиться в состоянии `CONNECTED`.

Системный вызов `accept()` возвращает файловый дескриптор созданной копии сокета, то есть нового сокета. Такое дублирование сокетов в ходе принятия соединений даёт серверу возможность продолжать принимать новые соединения без необходимости предварительно закрывать предыдущие.

Сервер блокируется на системном вызове `accept()` до тех пор, пока не поступит запрос на соединение от клиента.

Для протокола TCP определено тройное рукопожатие (clnt: SYN, srv: SYN-ACK, clnt: ACK): сначала клиент обращается к серверу с запросом на установление соединения; получив первое ответное сообщение, клиент так же отправляет подтверждение получения, и только после этого соединение устанавливается. В настоящее время тройное рукопожатие стало огромной проблемой — DDOS-атаки.

8.6 Сетевые сокеты — сетевой стек, аппаратный и сетевой порядок байтов

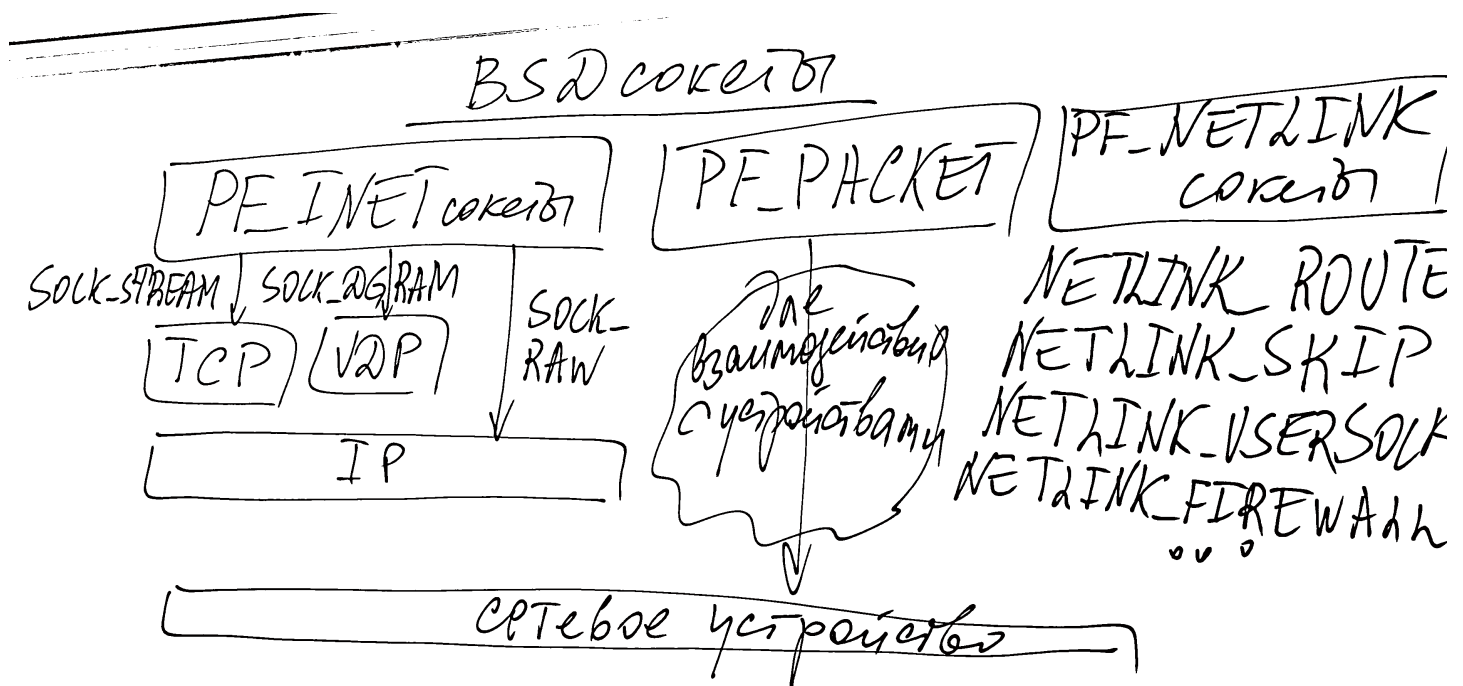


Рис. 25. BSD сокеты

Когда создавался интернет, был изменён порядок байтов. Порядок следования байтов на разных архитектурах различен, различают аппаратный и сетевой порядок байтов:

- `big_endian` — network byte order (сетевой порядок): старший значащий байт целого числа имеет наименьший адрес. В таком формате передаются данные по сети.
- `little_endian` — host byte order (порядок хоста): соответственно наоборот. В таком формате данные хранятся, например, на процессорах Intel.

Функции для работы с порядком байтов:

- 1) `htons()` — host to network short:

```
1 #include <arpa/inet.h>
```

```
2 uint16_t htons(uint16_t hostint16);  
3 uint32_t htonl(uint32_t hostint32);
```

2) ntohs() — network to host:

```
1 #include <arpa/inet.h>  
2 uint16_t ntohs(uint16_t netint16);  
3 uint32_t ntohl(uint32_t netint32);
```

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. В результате никакая работа будет невозможна. Эти функции нужно вызывать в обязательном порядке, даже в том случае, если мы уверены, что порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, то функция не выполняется.

8.7 Примеры реализации взаимодействия процессов по модели клиент-сервер с использованием сокетов и мультиплексированием (лаб. раб.)

8.7.1 Сервер производство-потребление из лабораторной работы

Реализуется классическая задача производство-потребление. У нас 2 типа клиентов — клиент-производитель и клиент-потребитель. Клиент посылает серверу признак, кто он: P (производитель) или C (consumer, потребитель), то есть что он хочет сделать — положить данные в кольцевой буфер или считать данные из буфера.

Сервер с мультиплексированием однопоточный: создаётся массив events, дальше пробегаем по нему, обслуживаются соединения (по схеме из предыдущего подраздела). Когда мы переходим к обработке по else, для нас важно, производитель это или потребитель — какую функцию вызывать (практически как у потоков в многопоточной версии).

Синхронизация:

- Не нужно обеспечивать взаимоисключение (сервер однопоточный, блокировок нет), а синхронизация нужна: считающие семафоры выкинуть не получается, иначе буфер будет переполнен. Как определить, что буфер заполнен? — Всё равно вводить счётчик.
- В обеих версиях (многопоточной и с мультиплексированием) должны быть семафоры, но в мультиплексоре не должно быть бинарного.

- Считающие семафоры, чтобы исключить переход в режим ядра, можно было бы заменить счётчиками: переход в режим ядра — затратное действие. Но счётчики не позволят нам выполнить блокировку.
- Если буфер заполнен ($N = \text{max}$), производитель ничего положить в буфер не может — значит, мы можем послать ему, что он должен ждать. Если мы используем семафор, то сервер будет заблокирован на семафоре, значит, функция не отправит клиенту сообщение, что всё в порядке. Что должен получать производитель от сервера? — Он должен получить ОК, что его данные положены. Пока ответ не получен, клиент заблокирован на запросе `receive from`, и только после получения он сможет выполнить следующую итерацию.

✱ Claude

В однопоточном сервере с мультиплексированием блокировка на считавшем семафоре остановила бы весь сервер, включая обслуживание потребителей, которые могли бы освободить место в буфере — получился бы deadlock. Поэтому проверку семафора в такой версии нужно делать неблокирующей (например, `sem_trywait`) и при заполненном буфере отвечать клиенту, что нужно повторить попытку, о чём и говорится в конспекте («послать, что он должен ждать»).

```
1 #define _GNU_SOURCE
2 #include <errno.h>
3 #include <netinet/in.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <sys/epoll.h>
7 #include <sys/ipc.h>
8 #include <sys/sem.h>
9 #include <sys/socket.h>
10 #include <sys/stat.h>
11 #include <time.h>
12 #include <unistd.h>
13
14 #define SERVER_PORT      9878
15 #define SERVER_BACKLOG   5
16 #define BUF_SIZE         52
17 #define MAX_EVENTS       64
18
19 #define OP_PRODUCER 'P'
20 #define OP_CONSUMER 'C'
21
22 #define SEM_KEY          200
```

```

23 #define SEM_STORAGE_EMPTY 0
24 #define SEM_STORAGE_FULL 1
25 #define SEM_LOCK -1
26 #define SEM_UNLOCK 1
27
28 static char ring_buffer[BUF_SIZE];
29 static int write_idx = 0;
30 static int read_idx = 0;
31
32 static int semaphore_id;
33 static int listen_fd;
34 static int epoll_fd;
35 static int should_exit = 0;
36
37 struct sembuf stop_write = {SEM_STORAGE_EMPTY, SEM_LOCK, IPC_NOWAIT | SEM_UNDO};
38 struct sembuf start_read = {SEM_STORAGE_FULL, SEM_UNLOCK, SEM_UNDO};
39 struct sembuf stop_read = {SEM_STORAGE_FULL, SEM_LOCK, IPC_NOWAIT | SEM_UNDO};
40 struct sembuf start_write = {SEM_STORAGE_EMPTY, SEM_UNLOCK, SEM_UNDO};
41
42 void producer(int fd) {
43     char letter;
44     if (recv(fd, &letter, 1, 0) <= 0) {
45         return;
46     }
47     if (semop(semaphore_id, &stop_write, 1) == -1) {
48         send(fd, "WAIT", 5, 0);
49         printf("[P %d] FULL, rejected '%c'\n", fd, letter);
50         return;
51     }
52     ring_buffer[write_idx] = letter;
53     write_idx = (write_idx + 1) % BUF_SIZE;
54     semop(semaphore_id, &start_read, 1);
55     send(fd, "OK", 3, 0);
56     printf("[P %d] put '%c'\n", fd, letter);
57 }
58 void consumer(int fd) {
59     if (semop(semaphore_id, &stop_read, 1) == -1) {
60         send(fd, "WAIT", 5, 0);
61         printf("[C %d] EMPTY\n", fd);
62         return;
63     }
64     char resp[2];
65     resp[0] = ring_buffer[read_idx];
66     resp[1] = '\0';

```

```

67     read_idx = (read_idx + 1) % BUF_SIZE;
68     semop(semaphore_id, &start_write, 1);
69     send(fd, resp, 2, 0);
70     printf("[C %d] got '%c'\n", fd, resp[0]);
71 }
72
73 void client_service(int fd, struct timespec* time_start) {
74     char op;
75     if (recv(fd, &op, 1, 0) <= 0) {
76         close(fd);
77         return;
78     }
79     if (op == OP_PRODUCER) {
80         producer(fd);
81     } else if (op == OP_CONSUMER) {
82         consumer(fd);
83     }
84     close(fd);
85 }
86 int main(void) {
87     struct sockaddr_in server_addr = {0}, client_addr;
88     socklen_t client_len = sizeof(client_addr);
89     if ((listen_fd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
90         perror("socket");
91         exit(1);
92     }
93     server_addr.sin_family = AF_INET;
94     server_addr.sin_addr.s_addr = INADDR_ANY;
95     server_addr.sin_port = htons(SERVER_PORT);
96     if (bind(listen_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) ==
97         -1) {
98         perror("bind");
99         exit(1);
100     }
101     if (listen(listen_fd, SERVER_BACKLOG) == -1) {
102         perror("listen");
103         exit(1);
104     }
105     int permissions = S_IRWXU | S_IRWXG | S_IRWXO;
106     semaphore_id = semget(SEM_KEY, 2, IPC_CREAT | permissions);
107     if (semaphore_id == -1) {
108         perror("semget");
109         exit(1);
110     }
111     if (semctl(semaphore_id, SEM_STORAGE_EMPTY, SETVAL, BUF_SIZE) == -1) {

```

```

111     perror("semctl");
112     exit(1);
113 }
114 if (semctl(semaphore_id, SEM_STORAGE_FULL, SETVAL, 0) == -1) {
115     perror("semctl");
116     exit(1);
117 }
118 epoll_fd = epoll_create1(0);
119 if (epoll_fd == -1) {
120     perror("epoll_create1");
121     exit(1);
122 }
123 struct epoll_event ev;
124 ev.events = EPOLLIN;
125 ev.data.fd = listen_fd;
126 if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, listen_fd, &ev) == -1) {
127     perror("epoll_ctl");
128     exit(1);
129 }
130 struct epoll_event events[MAX_EVENTS];
131 while (!should_exit) {
132     int nfds = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
133     if (nfds == -1) {
134         perror("epoll_wait");
135         break;
136     }
137     for (int i = 0; i < nfds; i++) {
138         if (events[i].data.fd == listen_fd) {
139             int conn_fd = accept(listen_fd, (struct sockaddr*)&client_addr,
140                                 &client_len);
141             if (conn_fd == -1) {
142                 perror("accept");
143                 continue;
144             }
145             ev.events = EPOLLIN;
146             ev.data.fd = conn_fd;
147             if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, conn_fd, &ev) == -1) {
148                 perror("epoll_ctl");
149                 close(conn_fd);
150                 continue;
151             }
152         } else {
153             client_service(events[i].data.fd, &time_start);
154         }
155     }
156 }

```

```

155     }
156     close(epoll_fd);
157     if (semctl(semaphore_id, 0, IPC_RMID) == -1) {
158         perror("semctl");
159         exit(1);
160     }
161     return EXIT_SUCCESS;
162 }

```

Клиент в цикле подключается к серверу, отправляет признак Р/С (производитель дополнительно шлёт символ данных) и обрабатывает ответ сервера: ОК — запрос выполнен, WAIT — буфер полон/пуст, попытку нужно повторить.

```

1  #include <arpa/inet.h>
2  #include <netinet/in.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <sys/socket.h>
7  #include <unistd.h>
8
9  #define SERVER_IP   "127.0.0.1"
10 #define SERVER_PORT 9878
11 #define MSG_SIZE    8
12 #define OP_PRODUCER 'P'
13 #define OP_CONSUMER 'C'
14
15 static int loop_flag = 1;
16 int connect_to_server(void) {
17     int fd = socket(AF_INET, SOCK_STREAM, 0);
18     if (fd == -1) {
19         perror("socket");
20         return -1;
21     }
22     struct sockaddr_in server_addr;
23     memset(&server_addr, 0, sizeof(server_addr));
24     server_addr.sin_family = AF_INET;
25     server_addr.sin_port = htons(SERVER_PORT);
26     inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
27     if (connect(fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1) {
28         perror("connect");
29         close(fd);
30         return -1;
31     }

```

```

32     return fd;
33 }
34 int producer(int fd, char* letter) {
35     char response[MSG_SIZE] = {0};
36     if (recv(fd, response, MSG_SIZE, 0) <= 0) {
37         perror("recv"); return -1;
38     }
39     if (strcmp(response, "WAIT") == 0) {
40         printf("P: buffer full\n"); return 1;
41     }
42     printf("P: '%c' -> %s\n", *letter, response);
43     *letter = (*letter == 'z') ? 'a' : *letter + 1;
44     return 0;
45 }
46 int consumer(int fd) {
47     char response[MSG_SIZE] = {0};
48     if (recv(fd, response, MSG_SIZE, 0) <= 0) {
49         perror("recv"); return -1;
50     }
51     if (strcmp(response, "WAIT") == 0) {
52         printf("C: buffer empty\n"); return 1;
53     }
54     printf("C: got '%c'\n", response[0]);
55     return 0;
56 }
57 int main(void) {
58     char letter = 'a';
59     for (int i = 0; i < ITERATIONS && loop_flag; i++) {
60         char op = ((i % 3) == 2) ? OP_CONSUMER : OP_PRODUCER;
61         int fd = connect_to_server();
62         if (fd == -1) return EXIT_FAILURE;
63         int rc;
64         if (op == OP_PRODUCER) {
65             char msg[2] = {op, letter};
66             if (send(fd, msg, 2, 0) == -1) {
67                 perror("send");
68                 close(fd);
69                 return EXIT_FAILURE;
70             }
71             rc = producer(fd, &letter);
72         } else {
73             if (send(fd, &op, 1, 0) == -1) {
74                 perror("send");
75                 close(fd);

```

```
76         return EXIT_FAILURE;
77     }
78     rc = consumer(fd);
79 }
80 close(fd);
81 if (rc == -1) return EXIT_FAILURE;
82 }
83 return EXIT_SUCCESS;
84 }
```


9

9 **Файловая система: процесс и файловые структуры связанные с процессом.** Файлы и открытые файлы, связь структур, представляющих открытые файлы на разных уровнях. Системный вызов `open()` и библиотечная функция `fopen()`: параметры и флаги, определенные на функции `open()`. Реализация системного вызова `open()` в ядре Linux. Пример: файл открывается два раза системным вызовом `open()` для записи и в него последовательно записывается строка «aaaaaaaaaaaa» по первому дескриптору и затем строка «vvvv» по второму дескриптору, затем файл закрывается два раза. Показать, что будет записано в файл и пояснить результат.

9.1 Файловая система: процесс и файловые структуры, связанные с процессом

Файл — поименованная совокупность данных, может быть бессмысленная, хранящаяся во вторичной памяти. Понятие файла связано с долговременным хранением информации.

Файловая система — это порядок, определяющий способ организации хранения, именования и доступа к данным на вторичных носителях информации.

Файловая система используется для управления хранением данных и доступа к ним. ФС должна обеспечить доступ к хранимой в системе на вторичных дисках информации.

Процесс — программа в стадии выполнения. Каждый процесс в системе описывается структурой `struct task_struct`. С каждым процессом в системе связаны: корневая файловая система, текущий рабочий каталог, список открытых им файлов. Для этого в `task_struct` есть два поля:

```
1 struct task_struct
2 {
3     ...
4     struct fs_struct *fs;
5     struct files_struct *files;
6     ...
7 }
```

Листинг 38. Структура `task_struct`

- `struct fs_struct` — это файловая система, которой принадлежит файл. Процесс — программа в стадии выполнения: до того, как она была запущена на выполнение, она была файлом, а файл принадлежит какой-то ФС. Процесс содержит информацию об этой файловой системе.
- `struct files_struct` — структура, инициализируемая для каждого процесса. В ней есть поле `struct file *fd_array[]` — таблица открытых файлов процесса. Каждый процесс имеет собственную таблицу.

```

1 struct fs_struct {
2     ...
3     atomic_t count;
4     ...
5     int umask;
6     struct path;
7     ...
8 }

```

Листинг 39. Структура `fs_struct`

```

1 struct files_struct
2 {
3     ...
4     struct
5         file *fd_array[NR_OPEN_DEFAULT];
6 }

```

Листинг 40. Структура `files_struct`

При порождении процесса, когда мы запустили своё или чужое приложение на выполнение, для любого процесса автоматически выделяются 3 дескриптора:

- 1) 0, `stdin`;
- 2) 1, `stdout`;
- 3) 2, `stderr`.

Если мы в своей программе открываем один файл, он получит индекс 3, следующий — 4. Причём прежде чем выдать индекс, система выполняет поиск наименьшего свободного дескриптора.

9.2 Файлы и открытые файлы, связь структур, представляющих открытые файлы на разных уровнях

В системе различаются файлы и открытые файлы. Просто файлы представляются в системе деревом каталогов, а физические файлы описываются структурой `struct inode`. Открытый файл — файл, с которым работает процесс; открытые файлы описываются структурой `struct file`.

Один и тот же файл может быть открыт много раз, причём разными процессами, и сколько раз открыт файл, столько будет в таблице открытых файлов дескрипторов открытых файлов.

В системе имеется одна глобальная таблица открытых файлов, каждая строка которой — проинициализированная структура `struct file`. Кроме того, каждый процесс имеет собственную таблицу открытых файлов (`fd_array`): файловый дескриптор `fd` — индекс в таблице открытых файлов конкретного процесса, и он ссылается на дескриптор открытого файла в системной таблице.

`struct file` — дескриптор открытого файла.

```

1 struct file {
2     ...
3     spinlock_t f_lock;
4     fmode_t f_mode; // режим доступа

```

```

5  atomic_long_t f_count; // счётчик ссылок на файл
6  struct mutex f_pos_lock;
7  loff_t f_pos;
8  unsigned int f_flags;
9  struct fown_struct f_owner;
10 ...
11 struct path f_path;
12 struct inode *f_inode; /* cached value */
13 const struct file_operations *f_op;
14 ...
15 void *private_data;
16 ...
17 }

```

Листинг 41. Структура file

- `f_lock` — нужен для обеспечения монопольного доступа к файлу.
- `f_mode` — режим доступа.
- `f_count` — счётчик ссылок на файл.
- `f_pos` — поле отражает логическую позицию в файле: любой файл считает, что он начинается с нулевого смещения, и каждая операция чтения/записи изменяет значение этого поля на соответствующее количество байт.

Поле `f_path` связывает открытый файл с элементом пути и со смонтированной файловой системой:

```

1 struct path {
2     struct vfsmount *mnt;
3     struct dentry *dentry;
4 };

```

Листинг 42. Структура path

```

1 struct vfsmount {
2     struct dentry *mnt_root; /* root of the mounted tree */
3     struct super_block *mnt_sb; /* pointer to superblock */
4     int mnt_flags;
5     struct mnt_idmap *mnt_idmap;
6 };

```

Листинг 43. Структура vfsmount

Поле `f_op` указывает на структуру `file_operations`, в которой перечислены функции, определённые на открытом файле. Это функции ядра: разработчики называют их аналогично функциям стандартных библиотек, но, например, в функции `read` присутствует указатель `__user *` на буфер пользователя.

```

1 struct file_operations {
2     struct module *owner;
3     ...
4     loff_t  (*llseek) (struct file *, loff_t, int);
5     ssize_t (*read)   (struct file *, char __user *, size_t, loff_t *);
6     ssize_t (*write)  (struct file *, const char __user *, size_t, loff_t *);
7     ...
8     int (*open)       (struct inode *, struct file *);
9     int (*flush)      (struct file *, fl_owner_t id);
10    int (*release)    (struct inode *, struct file *);
11    ...
12 };

```

Листинг 44. Структура `file_operations`

Все обсуждаемые структуры между собой связаны указателями. Связь строится относительно процесса, потому что именно процессы открывают файлы:

- 1) уровень процесса: `task_struct` через поле `files` указывает на `files_struct`, в которой по файловому дескриптору `fd` (индексу в `fd_array`) находится указатель на `struct file`;
- 2) уровень открытых файлов (системный): `struct file` — дескриптор открытого файла с собственным смещением `f_pos`, флагами `f_flags` и операциями `f_op`;
- 3) уровень пути: поле `f_path` содержит указатель на `dentry` — элемент пути к файлу. Структуры `dentry` создаются на лету и хранятся в `dentry`-кеше: когда необходимо обратиться к файлу, система проходит по элементам пути и ищет их в кеше, и только если `dentry` не найден, обращение идёт к диску;
- 4) уровень физического файла: `dentry` (поле `d_inode`) и поле `f_inode` указывают на `inode`, который описывает физический файл;
- 5) уровень файловой системы: любой физический файл доступен, только если ФС смонтирована, поэтому в `inode` есть указатель на `super_block` (поле `i_sb`), а `f_path.mnt` через `mnt_sb` также ведёт к суперблоку смонтированной ФС.

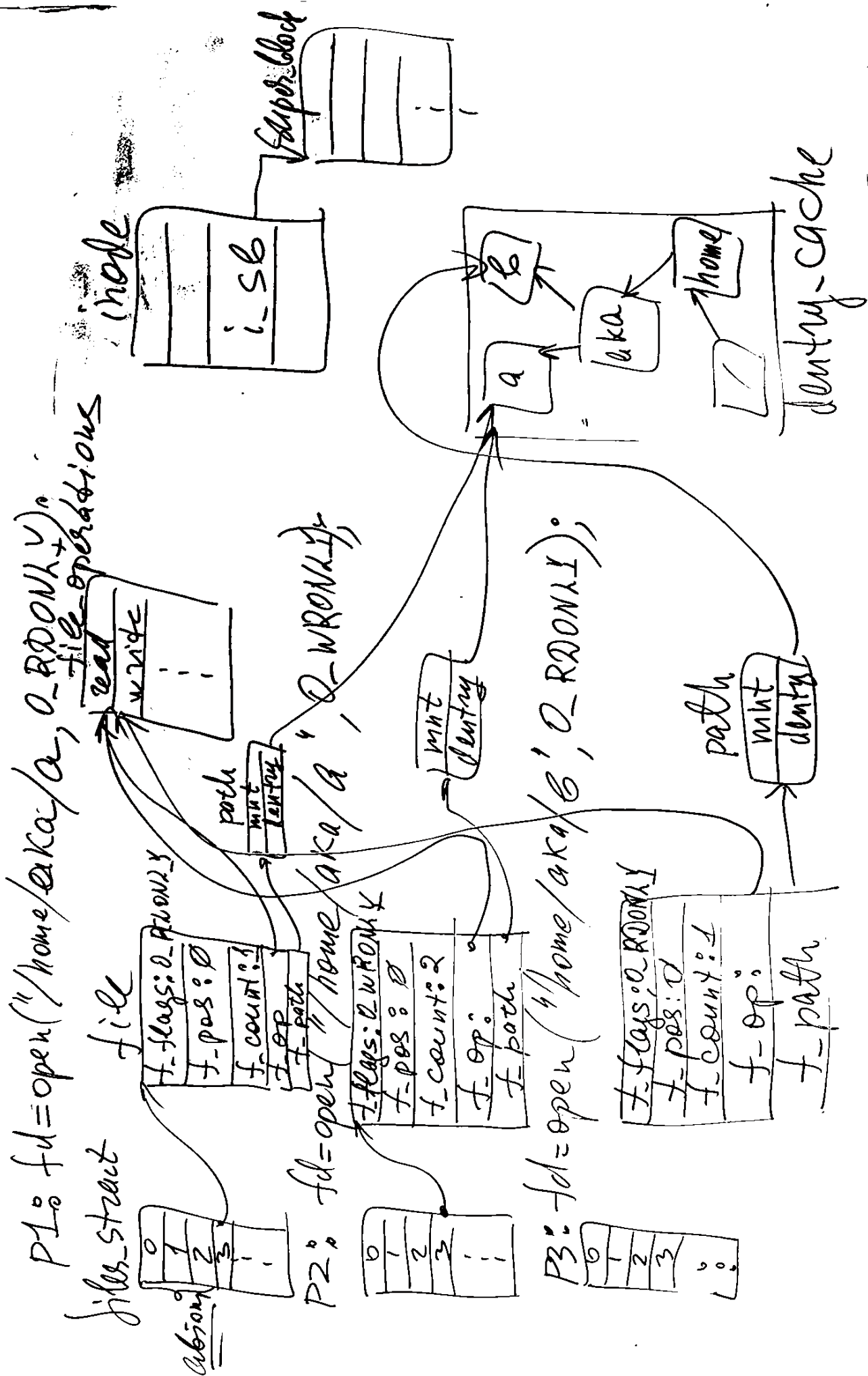


Рис. 26. Связь структур, представляющих открытые файлы, для трёх процессов

9.3 Системный вызов `open()` и библиотечная функция `foren()`: параметры и флаги, определённые на функции `open()`

9.3.1 Системный вызов `open()`

В userspace есть две функции `open`:

```
1 int open(const char *pathname, int flags);  
2 int open(const char *pathname, int flags, mode_t mode);
```

- `pathname` — имя открываемого файла;
- `flags` — флаги, определяющие режим доступа и поведение при открытии;
- `mode` — права доступа создаваемого файла.

Третий параметр `mode` необходим, когда пользователь вызывает `open` для создания нового файла: тогда нужно указать права доступа для трёх групп пользователей. Если же мы вызываем `open` для открытия существующего файла, `mode` не нужен. Когда создаём новый файл — создаём новый `inode`, то есть инициализируется структура `inode`; поскольку созданный файл открывается, для него инициализируются поля `struct file`.

Функция возвращает файловый дескриптор — целое неотрицательное число, которое можно использовать в последующих системных вызовах. Это будет наименьший дескриптор из таблицы открытых процессом файлов, который ещё не занят. Смещение в открытом файле устанавливается в начало файла. Системный вызов `open()` создаёт новый файловый дескриптор для открытого файла — запись в общесистемной таблице открытых файлов; эта запись регистрирует смещение в файле и флаги состояния файла.

Флаги доступа (один из них указывается обязательно):

- `O_RDONLY` — открыть файл только на чтение;
- `O_WRONLY` — открыть файл только на запись;
- `O_RDWR` — открыть файл на чтение и запись.

Некоторые флаги создания и поведения:

- `O_CREAT` — создаёт файл, если его не существует; если файл существует, флаг игнорируется;
- `O_APPEND` — дозапись в существующий файл: перед каждым вызовом `write()` смещение файла переносится на его конец, поэтому ранее записанная информация не теряется;
- `O_TRUNC` — если файл существует и открыт для записи, усекает его до нулевой длины;
- `O_NONBLOCK` — устанавливает режим неблокирования. Эта опция обычно полезна для специальных файлов типа FIFO и устройств типа терминалов: обычно

- для этих файлов `open` блокируется, пока файл не «готов», а если `O_NONBLOCK` установлен, `open` возвращается немедленно. Бит также воздействует на чтение и запись: он разрешает им возвращаться немедленно с состоянием ошибки, если не имеется никакого доступного ввода или если вывод не может быть записан;
- `O_CLOEXEC` — установить флаг `close-on-exec` на новом файловом дескрипторе;
 - `O_PATH` — получить лишь файловый дескриптор (сам файл не будет открыт);
 - `O_TMPFILE` — создать неименованный временный обычный файл.

Параметр `mode` задаёт права доступа, которые используются в случае создания нового файла: он всегда должен быть указан при использовании флага `O_CREAT`, во всех остальных случаях игнорируется. Для режима предусмотрены символические константы (для пользователя/группы):

- `S_IRWXU` / `S_IRWXG` / `S_IRWXO` — права на чтение, запись и выполнение для владельца / группы / остальных.
- `S_IRUSR` / `S_IRGRP` / `S_IROTH` — права на чтение для владельца / группы / остальных.;
- `S_IWUSR` / `S_IWGRP` / `S_IWOTH` — права на запись для владельца / группы / остальных.;
- `S_IXUSR` / `S_IXGRP` / `S_IXOTH` — права на выполнение для владельца / группы / остальных..

Пример:

```
1 open("file.txt", O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR); // права -rw-----
```

Чтение и запись по дескриптору, который возвращает `open()`, **не буферизуются**.

9.3.2 Библиотечная функция `fopen()`

При работе с вводом-выводом через библиотеку `stdio.h` используются функции `fopen()`, `fscanf()`, `fclose()`, ... Стандартная библиотека Си — буферизованный ввод/вывод.

Для открытия файла используется функция `fopen()`:

```
1 FILE *fopen(const char *fname, const char *mode);
```

Функция `fopen()` открывает файл, имя которого указано аргументом `fname`, и возвращает связанный с ним указатель на структуру `FILE`, которая называется файловым потоком. Тип операций, разрешённых над файлом, определяется аргументом `mode`. Ввод-вывод с использованием `stdio.h` **буферизуется**.

`FILE` — это синоним структуры `_IO_FILE`, в которой определяются буферы:

```
1 struct _IO_FILE
```

```

2 {
3     int _flags;
4     ...
5     char *_IO_buf_base; /* Start of reserve area. */
6     char *_IO_buf_end; /* End of reserve area. */
7     ...
8     int _fileno;
9     ...
10 };
11
12 typedef struct _IO_FILE FILE;

```

Листинг 45.

Поле `_fileno` — индекс открытого файла в таблице открытых файлов процесса. Так библиотечный уровень (`FILE`) связывается с уровнем системных вызовов (файловым дескриптором).

Особенность буферизованного ввода/вывода заключается в том, что сначала информация записывается в буфер, а потом выбирается из него:

- если мы читаем из файла (`fscanf`), то при первом вызове функции информация из файла будет записана в буфер, а затем считывается из него;
- если мы пишем в файл, то сначала информация записывается в буфер, и существует 3 причины записи содержимого буфера в файл:
 - 1) буфер заполнен, и следующая операция записи приведёт к тому, что содержимое буфера будет переписано в файл;
 - 2) принудительная запись в файл — вызов `fflush()`;
 - 3) закрытие файла — вызов `fclose()`.

9.4 Пример: файл открывается два раза системным вызовом `open()` для записи

Полная формулировка: файл открывается два раза системным вызовом `open()` для записи, и в него последовательно записывается строка «aaaaaaaaaaaa» по первому дескриптору и затем строка «vvvv» по второму дескриптору, затем файл закрывается два раза. Показать, что будет записано в файл, и пояснить результат.

```

1 int main() {
2     int fd1 = open("test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
3     int fd2 = open("test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
4     const char *str1 = "aaaaaaaaaaaa";
5     write(fd1, str1, strlen(str1));
6     const char *str2 = "vvvv";
7     write(fd2, str2, strlen(str2));

```

```
8   close(fd1);
9   close(fd2);
10  return 0;
11 }
```

Листинг 46. Пример: запись в файл, открытый два раза

Результат: в файле будет записано вvvvaaaaaaa.

Пояснение:

- 1) Системный вызов `open` вызывается два раза, при этом создаются два экземпляра структуры `struct file` (две записи в системной таблице открытых файлов), которые описывают один и тот же физический файл (один `inode`).
- 2) Каждый дескриптор имеет собственное поле `f_pos`, поэтому запись в файл и смещение `f_pos` по каждому дескриптору происходят независимо; у обеих структур поле инициализируется значением `f_pos = 0`.
- 3) `write(fd1, ...)` записывает строку из 12 символов «а» начиная с нулевой позиции; `f_pos` первого дескриптора смещается к концу записанной строки.
- 4) `write(fd2, ...)` использует `f_pos` второго дескриптора, который остался равным 0, поэтому запись по второму дескриптору начинается с начала файла: первые 4 символа «а» перезаписываются символами «в».
- 5) После двух `close()` в файле остаётся `vvvvaaaaaaaa` — 4 символа «в» и 8 оставшихся символов «а».

10

10 **Файловая система ОС Linux: иерархическая структура каталогов, имена файлов и идентификация файлов. Inode и dentry: пример доступа к файлу mbox. Кэши inode и dentry. Назначение суперблока в интерфейсе VFS: монтирование файловой системы — суперблок и связанные с ним структуры; структура struct_super_block и корневой каталог. Структура struct super_operations. Структуры struct inode, struct inode_operations. Функции simple и generic. Точка монтирования. Функции монтирования. Пример создания файловой системы, её регистрации и монтирование (лаб. раб.).**

10.1 Файловая система

Файл — поименованная совокупность данных, может быть бессмысленная, хранящаяся во вторичной памяти. Понятие файла связано с долговременным хранением информации.

Файловая система — это порядок, определяющий способ организации хранения, именования и доступа к данным на вторичных носителях информации.

Файловая система используется для управления хранением данных и доступа к ним. ФС должна обеспечить доступ к хранимой в системе на вторичных дисках информации.

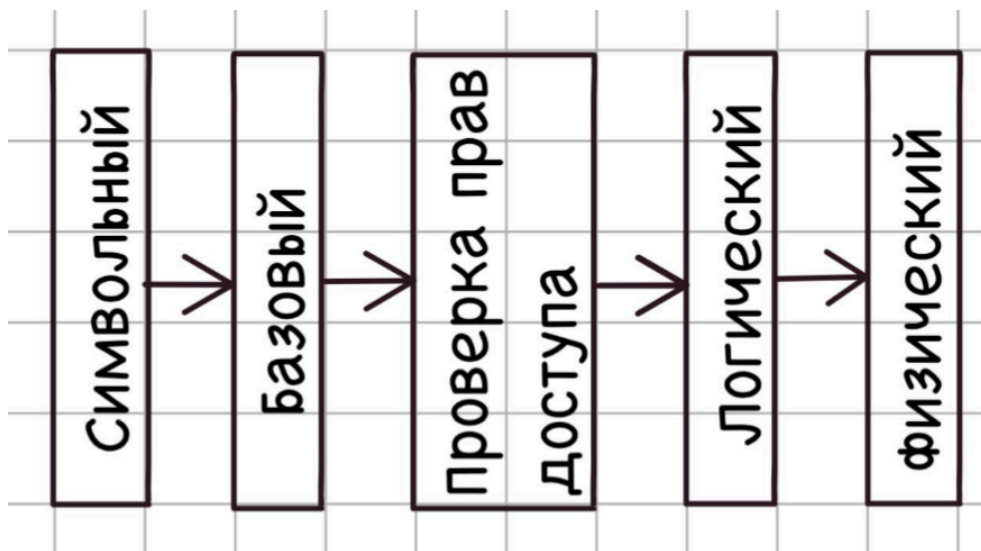
10.2 Задачи файловой системы

- 1) Долговременное хранение данных
- 2) Обеспечение доступа к хранящимся данным

10.3 Уровни иерархии ФС

Для обеспечения доступа к данным нужно предоставить пользователю достаточно удобный способ доступа. Уровни файловой системы:

- 1) Символьный
- 2) Базовый
- 3) Проверка прав доступа
- 4) Логический
- 5) Физический



- **Символьный** уровень — именованье файлов: у файлов **имена** (массив символов), собранные по каталогам; корневой каталог в UNIX/Linux — /. Имя не является идентификатором файла, а лишь позволяет удобно с ним работать; идентификатором файла является inode; определены жёсткие (hard links) и символические (soft links) ссылки.
- **Базовый** уровень — идентификация файла: это inode; идентификатором является номер inode, который описывает физический файл.
- **Проверка прав доступа** — права отдельно для пользователя, группы и остальных (команда chmod).
- **Логический** уровень — процесс открывает файл и выполняет над ним операции: файл считает, что начинается с нулевого смещения (offset), каждая операция чтения/записи смещает указатель; файл байт-ориентированный, размера не имеет. Здесь определяется адресация данных, преобразуемая в физические адреса на диске.
- **Физический** уровень — уровень физической памяти; логический и физический уровни связывает драйвер дискового устройства, переводящий смещения в физические адреса.

Для UNIX характерно существование корневого каталога /, от которого строится дерево каталогов:

1	/
2	├─ bin
3	├─ etc
4	├─ dev
5	└─ usr
6	├─ local
7	└─ lib

Для Unix был предложен интерфейс VFS/vnode (virtual file system, virtual node). Linux отказался от vnode, оставив только VFS. VFS — интерфейс, через который ОС получает доступ к большому числу разных ФС, сократив до минимума количество системных вызовов: любая работа с файлом любой ФС идёт через один и тот же интерфейс. Основа такой работы — монтирование: прежде чем ФС станет доступна (станут видны её каталоги и файлы), она должна быть смонтирована.

В основе интерфейса VFS лежат 4 структуры (это ядерные структуры):

- 1) struct super_block;
- 2) struct inode;
- 3) struct dentry;
- 4) struct file.

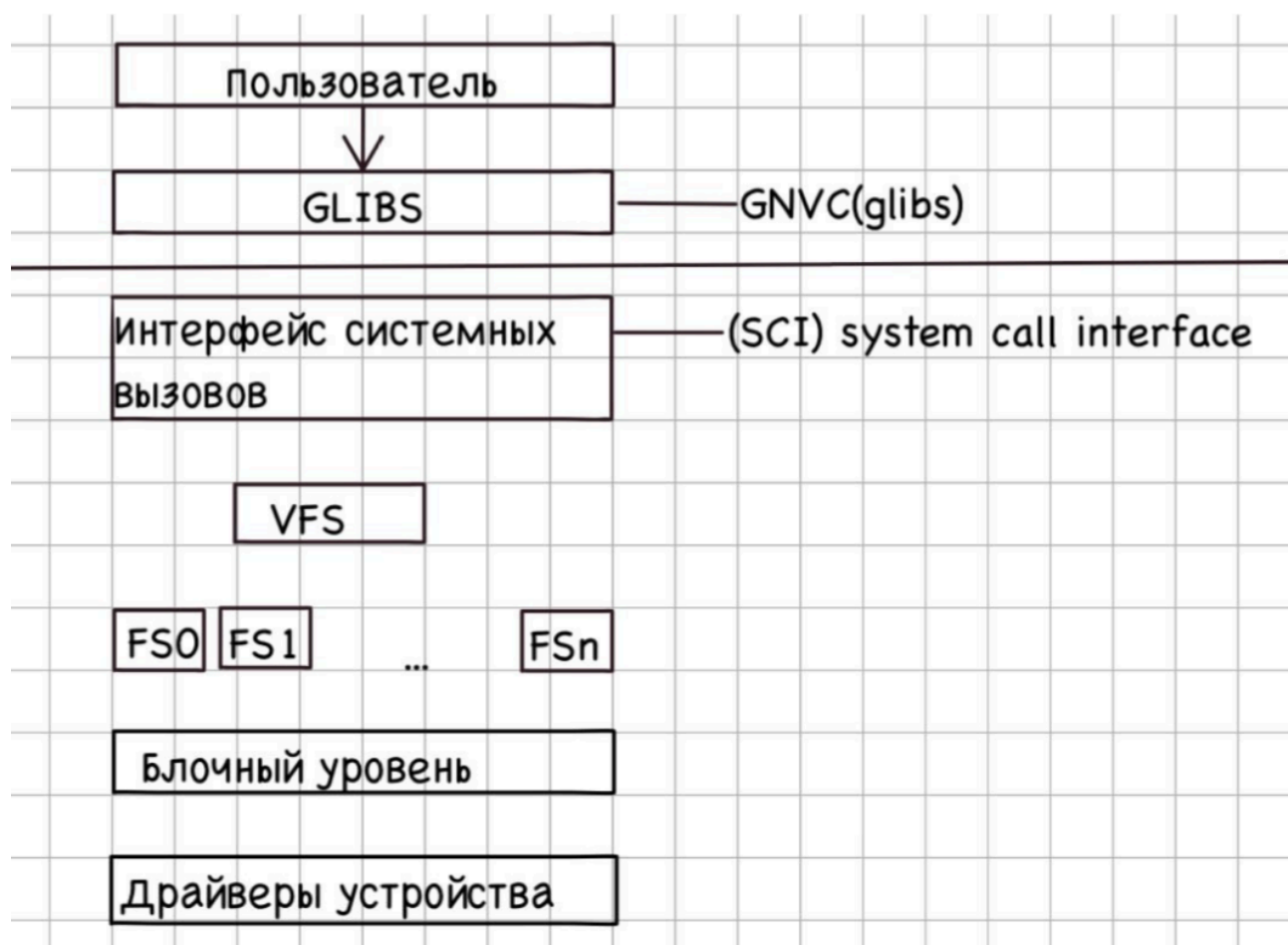


Рис. 27. Virtual File System

10.4 Inode и dentry: пример доступа к файлу mbox

struct inode — дескриптор физического файла. Два вида:

- 1) **дисковый** inode — описывает физический файл, содержит адресную информацию, то есть адреса блоков, в которых хранится информация, записанная в данный файл (на диске адреса и на диске информация);

2) **inode** в ядре содержит информацию о файле для ускорения доступа (копия в памяти, дополнительные поля: логический номер устройства, номер индекса, ссылки для хеш-очереди).

Каталоги в UNIX/Linux обрабатываются как файлы. Участок пути от / до следующего / — элемент пути, для него в VFS введена концепция dentry (directory entry). Структуре struct dentry не соответствует информация из вторичной памяти: объекты dentry создаются на лету при операциях с каталогами. Объектами dentry являются все элементы пути, включая обычные файлы (в пути /bin/vi — /, bin, vi).

Структура inode директории состоит из имён, сопоставленных с номерами inode:

inode number 3470036	
.	3470036
..	3470017
folder1	3470031
file1	3470043
file2	3470023
folder2	3470024
file3	3470065

Таблица 6. Структура inode каталога

Используется слово number, а не index, потому что index — это всегда смещение.

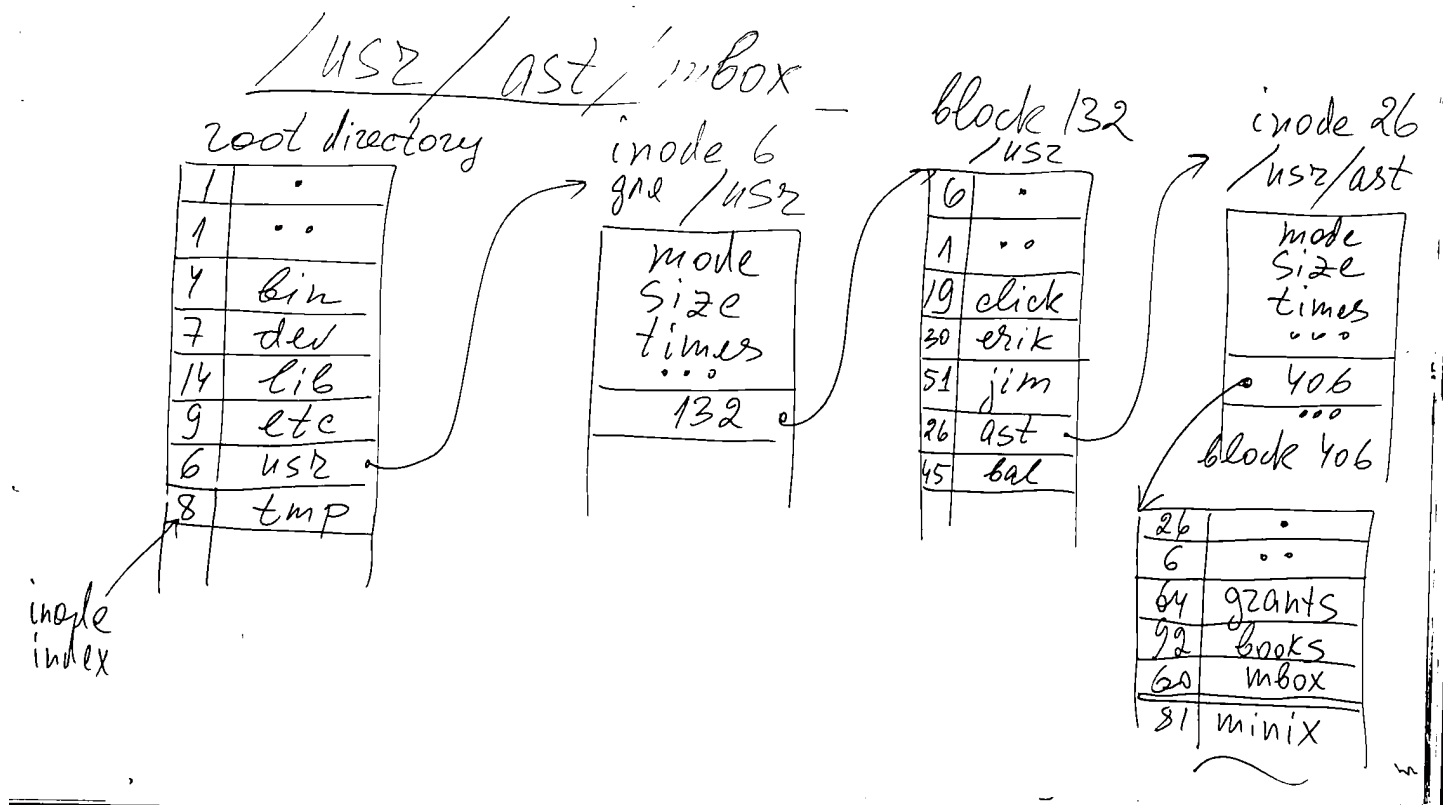


Рис. 28. Шаги доступа к файлу `/usr/ast/mbox`

Поиск файла всегда начинается с корневого каталога (в случае текущей рабочей директории часть пути уже пройдена). Доступ к файлу `/usr/ast/mbox` выполняется по шагам:

- 1) корневой индексный дескриптор уже находится в кэше (он закеширован во время монтирования). В таблице корневого каталога / имени `usr` сопоставлен номер `inode` 6;
- 2) обращаемся к `inode` 6 (это `/usr`): из его полей (`mode`, `size`, `times`, ...) и адреса блока получаем номер блока с содержимым каталога — блок 132;
- 3) в блоке 132 (содержимое `/usr`) имени `ast` сопоставлен номер `inode` 26;
- 4) обращаемся к `inode` 26 (это `/usr/ast`): из него получаем адрес блока с содержимым каталога — блок 406;
- 5) в блоке 406 (содержимое `/usr/ast`) имени `mbox` сопоставлен номер `inode` 60 — это искомый файл.

Содержимое индексного дескриптора представляет собой список, на основе которого можно перейти к соответствующему блоку, по информации блока — к соответствующему индексному дескриптору, и так далее. Сама структура `dentry` номера индексного дескриптора не содержит — он загружается отдельно.

10.5 Кэши `inode` и `dentry`

Проход по элементам пути требует много времени (имена сравниваются как строки), поэтому элементы пути кешируются. Действительный объект `dentry` может находиться в одном из трёх состояний:

- 1) **used** — используемый объект `dentry`; у него есть связанный `inode` (поле `d_inode`), а счётчик `d_count` больше 0. Такой объект не может быть просто удалён;
- 2) **unused** — неиспользуемый объект; ему соответствует допустимый `inode`, но `d_count == 0`. Такие объекты образуют двусвязный список **LRU** (least recently used), отсортированный по времени последнего обращения;
- 3) **negative** — объект, связанный с несуществующим номером `inode`. Такие элементы пути сохраняются в расчёте на повторное обращение и не удаляются сразу, что ускоряет обработку.

Соответственно, кэш `dentry` (`dcache`) состоит из трёх частей:

- 1) список используемых объектов `dentry` (`used`);
- 2) список неиспользуемых объектов `dentry` (`unused`, `LRU`);
- 3) хеш-таблица и хеш-функция, позволяющие быстро преобразовать заданный путь в объект `dentry`.

Кеширование каталогов и файловых индексов сокращает время обращения: после первого обращения к файлу высока вероятность, что находящиеся в кэше элементы

каталога и связанные с ними inode будут там в течение некоторого времени. Например, если выполнить `ls file` для несуществующего файла, повторный вызов не потребует повторного обращения к диску — для этого и существует состояние `negative`.

Кеш inode представляет собой:

- 1) глобальный хеш-массив `inode_hashtable`, в котором каждый inode хешируется по значению указателя на суперблок и по номеру inode;
- 2) глобальный список `inode_in_use` — содержит допустимые inode, у которых `i_count > 0` и `i_nlink > 0` (функция `new_inode` создаёт inode, добавляемый в этот список);
- 3) глобальный список `inode_unused` — содержит допустимые inode с `i_count == 0`;
- 4) для каждого смонтированного суперблока список `sb->s_dirty` — содержит модифицированные («грязные») inode с `i_count > 0` и `i_state == I_DIRTY`. Поддержка такого списка уменьшает накладные расходы на синхронизацию: изменённые inode должны быть записаны и на диск;
- 5) slab-кеш для объектов inode.

10.6 Назначение суперблока в интерфейсе VFS: монтирование файловой системы — суперблок и связанные с ним структуры; структура `struct super_block` и корневой каталог

Суперблок представляет в VFS информацию о смонтированной ФС — структура с данными, необходимыми для монтирования и доступа к файлам ФС: метаданные для обращения к файлам, хранящимся «вразброс» (адресация соответствующих участков диска). В каждой ФС ровно один суперблок, он располагается в начале раздела (есть резервная копия). При монтировании суперблок считывается в память ядра и находится там до размонтирования.

1	<code>struct super_block {</code>	
2	<code>struct list_head</code>	<code>s_list;</code>
3	<code>dev_t</code>	<code>s_dev;</code>
4	<code>unsigned long</code>	<code>s_blocksize;</code>
5	<code>loff_t</code>	<code>s_maxbytes; // Max file size</code>
6	<code>struct file_system_type</code>	<code>*s_type;</code>
7	<code>const struct super_operations</code>	<code>*s_op;</code>
8	<code>unsigned long</code>	<code>s_flags;</code>
9	<code>unsigned long</code>	<code>s_magic;</code>
10	<code>struct dentry</code>	<code>*s_root; // точка монтирования</code>
11	<code>struct rw_semaphore</code>	<code>s_umount;</code>
12	<code>struct list_head</code>	<code>s_mounts;</code>
13	<code>struct block_device</code>	<code>*s_bdev;</code>

```

14 struct list_head          s_inodes;    // all inodes
15 };

```

Листинг 47. Структура super_block (сокр.)

- `list_head` — двусвязные списки в Linux;
- `s_type` — указатель на тип ФС (`file_system_type`);
- `s_op` — указатель на операции суперблока (`super_operations`);
- `s_magic` — магическое число ФС;
- `s_root` — **точка монтирования**, корневой каталог смонтированной ФС;
- `s_umount` — семафор читателя-писатели для монтирования;
- `s_mounts` — список монтирований данной ФС (одна и та же ФС может быть смонтирована много раз).
- `s_bdev` — связанное блочное устройство;
- `s_inodes` — список всех inode данной ФС.

⚠ Осторожно

Вместо первого рисунка суперблока, который Рязанова не показывала (показывала), вероятно, хочет видеть второй.

Суперблок:

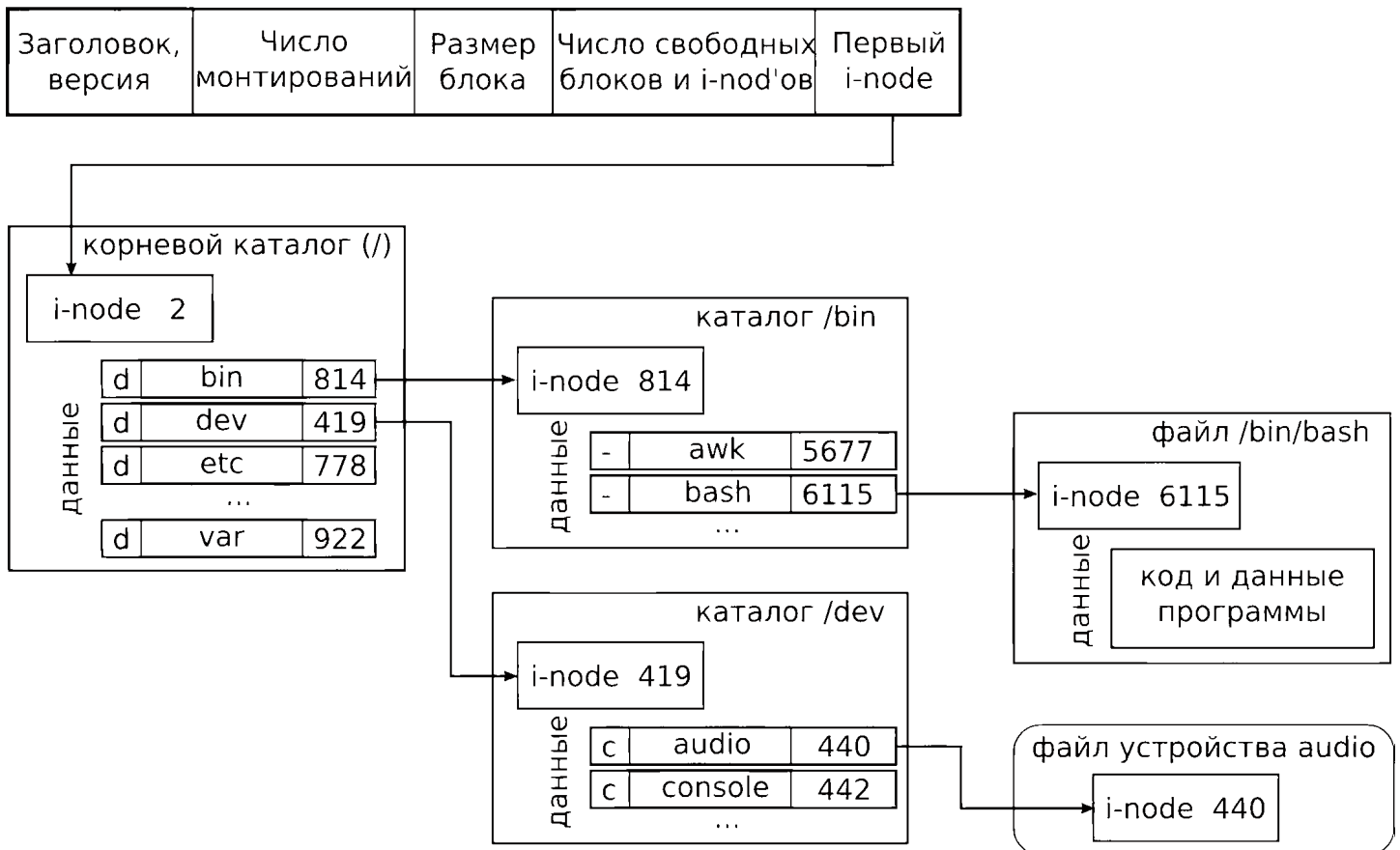


Рис. 29. Суперблок

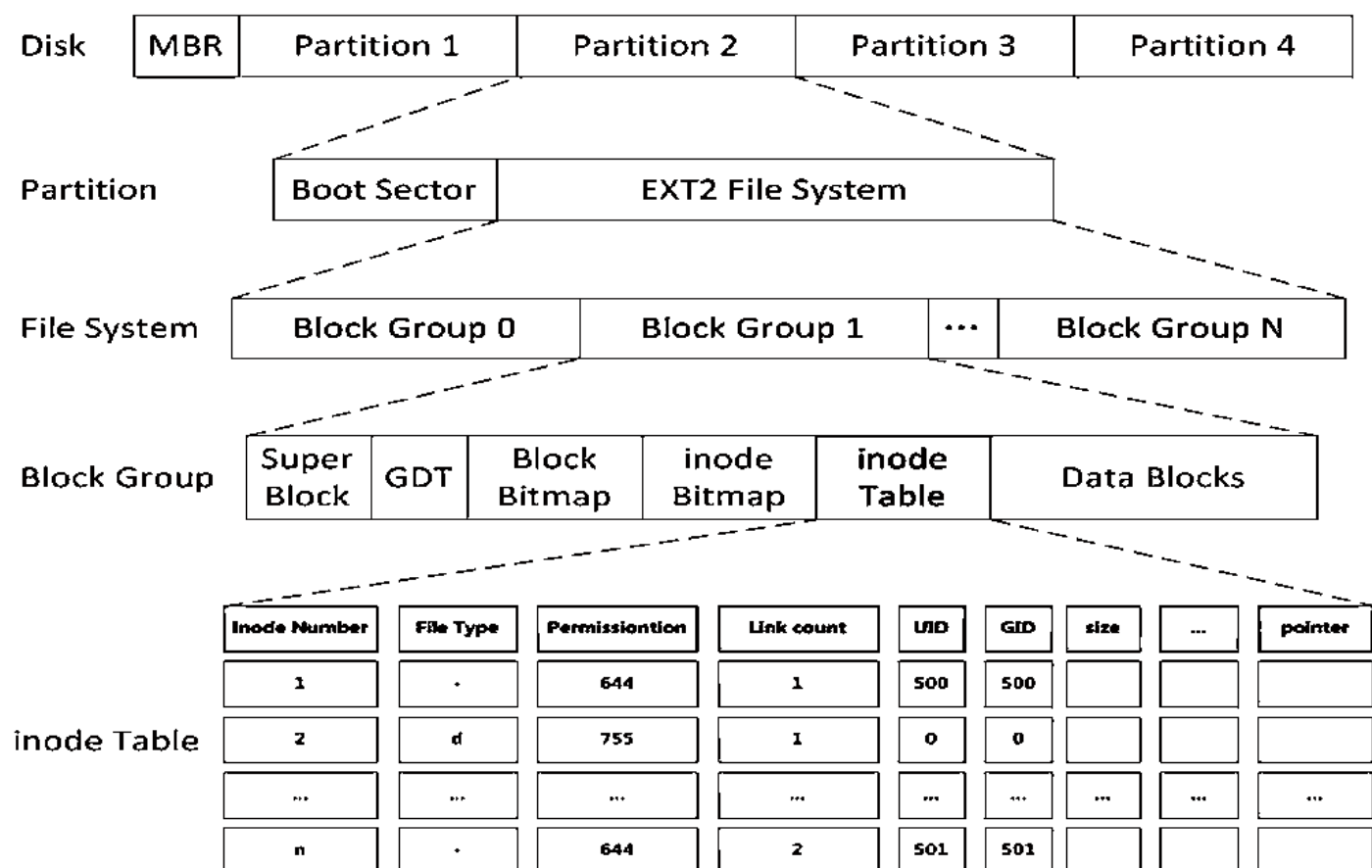


Рис. 30. Суперблок

Таким образом, суперблок связан указателями с остальными структурами интерфейса VFS: с типом ФС (`s_type`), с набором операций суперблока (`s_op`), с операциями над `dentry` (`s_d_op`), со списком всех `inode` (`s_inodes`) и с корневым каталогом ФС (`s_root`). Любой физический файл доступен, только если ФС смонтирована, поэтому в `inode` есть указатель на суперблок.

10.7 Структура `struct super_operations`

Для каждой ФС определяется свой набор операций над объектами суперблока. Поскольку разработчики Linux ставили задачу обеспечить интерфейс для большого числа ФС, перечень операций широк:

```

1 struct super_operations {
2     struct inode *(*alloc_inode)(struct super_block *sb);
3     void (*destroy_inode)(struct inode *);
4     void (*free_inode)(struct inode *);
5     void (*dirty_inode) (struct inode *, int flags);
6     int (*write_inode) (struct inode *, struct writeback_control *wbc);
7     int (*drop_inode) (struct inode *);
8     ...
9     void (*put_super) (struct super_block *);
10    ...
11    int (*freeze_fs) (struct super_block *);

```

```

12  ...
13  int (*remount_fs) (struct super_block *, int *, char *);
14  void (*umount_begin) (struct super_block *);
15  int (*show_options)(struct seq_file *, struct dentry *);
16  ...
17  int (*show_path)(struct seq_file *, struct dentry *);
18  ...
19  };

```

Листинг 48. struct super_operations

Основное количество перечисляемых операций — это операции с inode:

- alloc_inode — создаёт и инициализирует новый объект inode для конкретной смонтированной ФС (выделяет адресное пространство);
- destroy_inode — удаляет inode из связного списка inode;
- free_inode — информация с метаданными (с номером inode) не удаляется, и данный inode может быть повторно использован;
- dirty_inode — вызывается, чтобы переместить inode в другой список, если в него были введены изменения;
- write_inode — записывает inode на диск (только alloc недостаточно);
- drop_inode — вызывается интерфейсом VFS, когда количество ссылок на inode равно 0;
- put_super — вызывается VFS при размонтировании ФС; данный суперблок изымается из связного списка монтирований;
- remount_fs — вызывается VFS для монтирования суперблока с другими параметрами;
- umount_begin — вызывается VFS для прерывания монтирования; используется сетевыми ФС, например NFS (Network File System).

10.8 Структуры struct inode, struct inode_operations

Номер inode называется метаданными.

```

1  struct inode {
2      umode_t   i_mode;
3      ...
4      unsigned int i_flags;
5      ...
6      const struct inode_operations *i_op;
7      struct super_block *i_sb;
8      struct address_space *i_mapping;
9      ...
10     void      *i_security;
11     unsigned long i_ino;

```

```

12  unsigned int i_nlink; // Количество жёстких ссылок
13  loff_t      i_size;
14  ...
15  u8      i_blkbits;
16  ...
17  struct rw_semaphore i_rwsem;
18  ...
19  struct hlist_node i_hash;
20  ...
21  struct list_head i_lru; /* inode LRU list */
22  struct list_head i_sb_list;
23  struct list_head i_wb_list; /* backing dev writeback list */
24  union {
25      struct hlist_head i_dentry;
26      ...
27  };
28  ...
29  atomic_t i_readcount; /* struct files open R0 */
30  union {
31      const struct file_operations *i_fop; /* former ->i_op->default_file_ops */
32      ...
33  };
34  ...
35  union {
36      struct pipe_inode_info *i_pipe;
37      struct cdev *i_cdev;
38      char *i_link;
39  };
40  ...
41  };

```

Листинг 49. struct inode

- `i_mode`, `i_flags` — права доступа и флаги; флаги играют роль ограничений, указывая, что можно делать, а что нельзя.
- `i_op` — указатель на таблицу операций с индексом (`inode_operations`).
- `i_sb` — указатель на связанный суперблок: файловая система монтируется на конкретное физическое устройство, и любой файл относится к конкретной ФС.
- `i_ino` — номер `inode`.
- `i_nlink` — количество жёстких ссылок.
- `i_size` — размер файла; поскольку файлы связаны с блочными устройствами, в `inode` есть информация о размере блока (`i_blkbits`).
- `i_rwsem` — read-write семафор.

- списки (`i_lru`, `i_hash`, `i_sb_list`, ...) — обеспечивают быстрый доступ к файлам, так как обращение к внешнему устройству — медленное действие с накладными расходами.
- `i_readcount` — счётчик количества обращений к файлу для чтения (через `struct file`).

Все обсуждаемые структуры между собой связаны указателями, причём связь строится относительно процесса, потому что процессы открывают файлы (открытие файла выполняет, например, текстовый редактор, являющийся процессом).

В VFS каталоги обрабатываются как файлы, но VFS часто приходится выполнять специфичные для каталога операции — прежде всего поиск элемента пути по имени (`lookup`), включающий преобразование каждого элемента пути, проверку корректности и переход к следующему элементу:

```

1 struct inode_operations {
2     struct dentry * (*lookup) (struct inode *,struct dentry *, unsigned int);
3     ...
4     int (*permission) (struct mnt_idmap *, struct inode *, int);
5     ...
6     int (*readlink) (struct dentry *, char __user *,int);
7     int (*create) (struct mnt_idmap *, struct inode *,struct dentry *,umode_t, bool);
8     int (*link) (struct dentry *,struct inode *,struct dentry *);
9     int (*unlink) (struct inode *,struct dentry *);
10    int (*symlink) (struct mnt_idmap *, struct inode *,struct dentry *,const char *);
11    struct dentry *(*mkdir) (struct mnt_idmap *, struct inode *,struct dentry *,
        umode_t);
12    int (*rmdir) (struct inode *,struct dentry *);
13    int (*mknod) (struct mnt_idmap *, struct inode *,struct dentry *,umode_t,dev_t);
14    int (*rename) (struct mnt_idmap *, struct inode *, struct dentry *,struct inode *,
        struct dentry *, unsigned int);
15    ...
16    int (*atomic_open)(struct inode *, struct dentry *,struct file *, unsigned
        open_flag,umode_t create_mode);
17    ...
18 };

```

Листинг 50. `struct inode_operations`

10.9 Функции `simple` и `generic`

При инициализации файловой системы можно воспользоваться предопределёнными функциями ядра. Они делятся на две группы.

Функции `generic_xxx` выполняют всю не зависящую от конкретной ФС работу. Например, `generic_shutdown_super` — common helper для `kill_sb`, удаляющий

суперблок (проходит по объектам `struct super_block`). На его основе построены `kill_anon_super` и `kill_litter_super`:

```
1 void kill_litter_super(struct super_block *sb)
2 {
3     if (sb->s_root)
4         d_genocide(sb->s_root);
5     kill_anon_super(sb);
6 }
7
8 void kill_anon_super(struct super_block *sb)
9 {
10     dev_t dev = sb->s_dev;
11     generic_shutdown_super(sb);
12     kill_super_notify(sb);
13     free_anon_bdev(dev);
14 }
```

Листинг 51. Функции `kill_litter_super` и `kill_anon_super`

Функции **`simple_xxx`** дают готовую минимальную реализацию для простой (в частности, виртуальной, не связанной с диском) ФС:

- `simple_fill_super` — инициализирует поля суперблока (размер блока, магическое число, операции суперблока), создаёт корневой `inode` функцией `new_inode`, делает из него корневой каталог через `d_make_root` и заполняет `s_root`;
- `simple_dir_operations`, `simple_dir_inode_operations` — готовые наборы операций для работы с каталогами, чтобы не реализовывать их вручную.

10.10 Точка монтирования

В файловой системе UNIX/Linux определена важнейшая операция монтирования. Монтирование — это последовательность действий, в результате которой файловая система (и все её файлы) становится доступной. Только смонтированные ФС предоставляют доступ к своим каталогам и файлам.

Точкой монтирования является каталог, к которому подмонтируется ФС (это может быть любой каталог). Когда ФС монтируется в существующее дерево каталогов, все файлы и директории монтируемой ФС становятся файлами и поддиректориями точки монтирования. Если в точке монтирования было какое-то содержимое, оно становится невидимым до тех пор, пока ФС не будет размонтирована, поэтому в качестве точки монтирования лучше использовать пустые каталоги.

Чтобы смонтировать ФС, в ней нужно создать корневой каталог (`root`) — именно он монтируется; в `struct super_block` это поле `s_root`. Объекты `dentry` могут включать точки монтирования (в пути `/mnt/cdrom/foo` точка монтирования — `cdrom`).

Команда `mount` имеет следующий синтаксис:

```
1 mount -t type device dir
```

где `dir` — точка монтирования. Флаг `-t` необязателен. Команда `mount [-l] [-t type]` выводит смонтированные ФС типа `type`. Монтировать ФС обычно можно только с правами суперпользователя.

10.11 Функции монтирования

Файловую систему описывает структура `struct file_system_type`. Это тип ФС, которая будет монтироваться; данная структура объявлена в ядре, и, инициализируя её поля, мы определяем собственный тип ФС. Для каждого типа ФС существует только одна структура `file_system_type`, независимо от того, сколько экземпляров этой ФС смонтировано.

```
1 struct file_system_type {
2     const char* name;          /* Название файловой системы */
3     int fs_flags;              /* Флаги типа файловой системы */
4     #define FS_REQUIRES_DEV 1
5     #define FS_USERSNS_MOUNT 8
6     struct dentry* (*mount)(struct file_system_type*, int, const char *, void *);
7     void (*kill_sb)(struct super_block*); // прекращение доступа к суперблоку
8     struct module* owner;      // Модуль, владеющий файловой системой
9     struct file_system_type* next; // Следующая файловая система
10    struct hlist_head fs_supers; // Список объектов типа суперблок
11    ...
12 };
```

Листинг 52. Структура `file_system_type`

Минимальный набор полей, которые нужно проинициализировать для **регистрации** ФС, — это `owner` и `name`. Зарегистрировать ФС можно системным вызовом `register_filesystem`, для deregистрации используется `unregister_filesystem`:

```
1 extern int register_filesystem(struct file_system_type *);
2 extern int unregister_filesystem(struct file_system_type *);
```

Для **монтирования** этого недостаточно: нужно написать собственную функцию `mount` и зарегистрировать её в поле `mount` структуры `file_system_type`. В этой функции вызывается одна из четырёх функций монтирования ядра (все имеют модификатор `extern` и возвращают `struct dentry`):

— `mount_bdev` — для монтирования ФС, находящейся на блочном устройстве (требуется флаг `FS_REQUIRES_DEV`);

- `mount_single` — для монтирования ФС, точки монтирования которой разделяют один-единственный экземпляр ФС;
- `mount_nodev` — для монтирования ФС, не связанной ни с каким устройством;
- `mount_subtree`.

```
1 extern struct dentry *mount_nodev(struct file_system_type *fs_type, int flags,
2     void *data, int (*fill_super)(struct super_block *, void *, int));
```

При монтировании создаётся структура `struct vfsmount`, представляющая конкретный экземпляр ФС:

```
1 struct vfsmount {
2     struct dentry      *mnt_root; /* root of the mounted tree */
3     struct super_block *mnt_sb;   /* pointer to superblock */
4     int                 mnt_flags;
5     struct mnt_idmap    *mnt_idmap;
6 };
```

Листинг 53. Структура `vfsmount`

10.12 Пример создания файловой системы, её регистрации и монтирование (лаб. раб.)

В лабораторной работе создаётся собственная (виртуальная) файловая система в загружаемом модуле ядра; по отношению к VFS выполняются два действия:

- 1) **Регистрация ФС:** объявляем свой `struct file_system_type` (в коде — `myfs_type`; минимально `owner`, `name`, плюс поля `mount` и `kill_sb`); в `init(myfs_init)` вызываем `register_filesystem`, в `exit(myfs_exit)` — `unregister_filesystem`.
- 2) **Монтирование ФС:** в своей функции `mount(myfs_mount)` вызываем `mount_nodev` (ФС не связана с устройством), передавая ей свою `fill_super` (в коде `myfs_fill_sb`, написана на основе `simple_fill_super`). Именно `myfs_fill_sb` инициализирует поля `struct super_block` (размер блока, магическое число, свой набор `super_operations`), через `new_inode` создаёт `inode` корневого каталога (с готовыми «simple»-операциями каталога) и функцией `d_make_root` делает из него `root`, записывая его в `s_root`. Освобождение суперблока поручено готовой функции ядра `kill_litter_super` (поле `kill_sb`); чтобы при размонтировании в `syslog` попадало сообщение, в своём наборе `super_operations` задаётся собственный `put_super(myfs_put_super)`.

После загрузки модуля ФС монтируется командой `mount` и становится видна в дереве каталогов (может быть смонтирована несколько раз). Результат показывают `ls -ial`, `cat /proc/mounts` (смонтированные ФС) и `cat /proc/filesystems` (зарегистрированный тип ФС).

```

1 #include <linux/module.h>
2 #include <linux/kernel.h>
3 #include <linux/init.h>
4 #include <linux/fs.h>
5 #include <asm/atomic.h>
6 #include <asm/uaccess.h>
7
8 MODULE_LICENSE("GPL");
9 MODULE_AUTHOR("Zhizhin Nikita");
10
11 #define MYFS_MAGIC_NUMBER 0x13131313
12 static void myfs_put_super(struct super_block *sb) {
13     printk("myfs: put_super\n");
14 }
15 static struct super_operations const myfs_super_ops = {
16     .put_super    = myfs_put_super,
17     .statfs       = simple_statfs,
18     .drop_inode   = generic_delete_inode
19 };
20 static int myfs_fill_sb(struct super_block *sb, void *data, int silent) {
21     struct inode *inode;
22     struct dentry *root;
23     sb->s_blocksize = PAGE_SIZE;
24     sb->s_blocksize_bits = PAGE_SHIFT;
25     sb->s_magic = MYFS_MAGIC_NUMBER;
26     sb->s_op = &myfs_super_ops;
27     sb->s_time_gran = 1;
28     inode = new_inode(sb);
29     if (!inode) {
30         printk(KERN_ERR "myfs: new_inode error\n");
31         return -ENOMEM;
32     }
33     inode->i_ino = 1;
34     inode->i_mode = S_IFDIR | 0755;
35     simple_inode_init_ts(inode);
36
37     inode->i_op = &simple_dir_inode_operations;
38     inode->i_fop = &simple_dir_operations;
39
40     root = d_make_root(inode);
41     printk("myfs: d_make_root\n");
42     if (!root) {
43         printk(KERN_ERR "myfs: d_make_root error\n");
44         iput(inode);

```

```

45     return -ENOMEM;
46 }
47 sb->s_root = root;
48 printk(KERN_INFO "myfs: VFS root created\n");
49 printk(KERN_INFO "myfs: super_block init");
50 return 0;
51 }
52 static struct dentry *myfs_mount(struct file_system_type *type, int flags, const
    char *dev, void *data) {
53     struct dentry *const entry = mount_nodev(type, flags, data, myfs_fill_sb);
54     if (IS_ERR(entry))
55         printk(KERN_ERR "myfs: mount error\n");
56     else
57         printk(KERN_INFO "myfs: mount\n");
58     return entry;
59 }
60 static struct file_system_type myfs_type = {
61     .owner    = THIS_MODULE,
62     .name     = "myfs",
63     .mount    = myfs_mount,
64     .kill_sb  = kill_litter_super
65 };
66 static int __init myfs_init(void) {
67     int ret;
68
69     printk(KERN_INFO "myfs: register_filesystem call\n");
70     ret = register_filesystem(&myfs_type);
71     if (ret) {
72         printk(KERN_ERR "myfs: register_filesystem error\n");
73         return ret;
74     }
75     printk(KERN_INFO "myfs: loaded\n");
76     return 0;
77 }
78 static void __exit myfs_exit(void) {
79     int ret;
80     ret = unregister_filesystem(&myfs_type);
81     if (ret)
82         printk(KERN_ERR "myfs: unregister_filesystem error\n");
83     else
84         printk("myfs: unregister_filesystem\n");
85     printk(KERN_INFO "myfs: unloaded\n");
86 }
87 module_init(myfs_init);
88 module_exit(myfs_exit);

```


11

11 Аппаратные прерывания в Linux: запрос прерывания и линии IRQ. Простейшая схема аппаратной поддержки прерываний (концептуальная трёхшинная архитектура системы). Быстрые и медленные прерывания, флаги, пример быстрого прерывания. Нижняя и верхняя половины обработчиков прерывания, функция регистрации прерывания и её параметры. Нижняя половины: `softirq`, определение, массив `softirq_vec`, типы `soft_itq`, функции `open_softirq()`, `raise_softirq()`. Поток ядра `ksoftirqd`. Пример обработки приходящих пакетов `NET_RX_SOFTIRQ`.

11.1 Аппаратные прерывания в Linux: запрос прерывания и линии IRQ

Взаимодействие компьютера с внешними устройствами выполняется с помощью аппаратных прерываний — это концепция распараллеливания действий, когда управление внешними устройствами берут на себя специальные устройства — контроллеры. Как правило, контроллеры входят в состав внешних устройств, а адаптеры располагаются на материнской плате.

Так как внешними устройствами управляет специальный программируемый контроллер, возникла необходимость в информировании процессора о завершении операции ввода/вывода:

- 1) контроллер устройства генерирует сигнал прерывания — запрос прерывания (Interrupt Request);
- 2) в простейшей схеме этот сигнал поступает на вход контроллера прерываний PIC;
- 3) входы контроллера прерываний — это линии IRQ.

Примеры линий: IRQ0 — системный таймер, IRQ1 — прерывание от клавиатуры, IRQ12 — прерывание от мыши.

Сетевая карта при поступлении пакета формирует аппаратное прерывание — оно должно обрабатываться. Драйвер внешнего устройства **всегда** имеет в качестве одной из точек входа **обработчик прерывания**. Он вызывается, когда происходит прерывание от данного устройства.

11.2 Простейшая схема аппаратной поддержки прерываний (концептуальная трёхшинная архитектура системы)

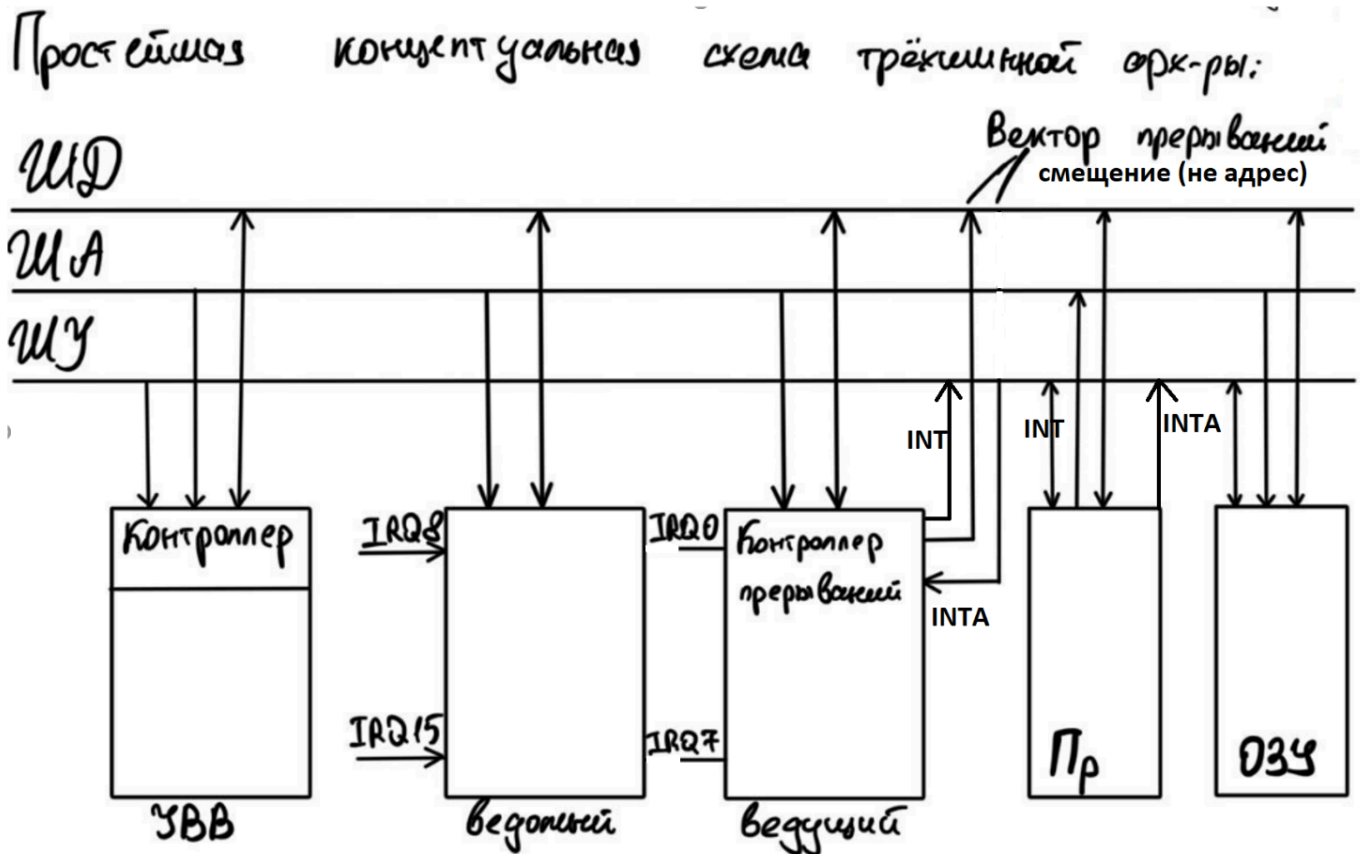


Рис. 31. Простейшая концептуальная схема

В компьютерах передаются сигналы трёх типов: данные (данные и команды), адреса, сигналы управления — отсюда три шины: шина данных, шина адреса, шина управления.

Обработка запроса прерывания по этой схеме:

- 1) На линию IRQ приходит сигнал от внешнего устройства.
- 2) Когда на контроллер прерываний приходит прерывание, он формирует сигнал INT, который по шине управления поступает на выделенную ножку процессора.
- 3) В конце цикла выполнения команды процессор проверяет наличие сигнала на выделенной ножке.
- 4) Если сигнал пришёл — процессор посылает контроллеру сигнал INTA (подтверждение).
- 5) Получив INTA, контроллер по шине данных выставляет вектор прерывания (базовый вектор + номер линии IRQ).
- 6) Процессор считывает вектор прерывания, переводит его смещение по таблице и считывает адрес обработчика.

11.3 Быстрые и медленные прерывания, пример быстрого прерывания

В ОС Linux различают быстрые и медленные прерывания:

1) Быстрые:

Обработка быстрых прерываний выполняется от начала до конца. В современных системах осталось единственное быстрое прерывание — прерывание от таймера, вытеснить его невозможно. При этом выполняется на процессоре 0 и *условно* этот процессор считается главным.

2) Медленные:

Все остальные аппаратные прерывания, то есть прерывания от устройств ввода/вывода (внешних устройств), делятся на две части:

2.1) top half (верхняя половина):

2.1.1) interrupt (hardware — аппаратное)

2.2) bottom half (нижняя половина):

2.2.1) softirq

2.2.2) tasklet

2.2.3) workqueue

Пример быстрого прерывания — прерывание от системного таймера. Главные задачи его обработчика:

1) Инициализировать планировщик.

Ставит в очередь на выполнение очень важные коды:

- Планировщик
- Диспетчер
- swapper

2) Инициализировать диспетчеризацию.

Он не может выполняться долго. Если он выполняется долго, это приведёт к снижению **отзывчивости** системы.

11.4 Нижняя и верхняя половины обработчиков прерывания, функция регистрации прерывания и её параметры

Аппаратные прерывания (top half) выполняются на высоких уровнях приоритета, причём системы, с которыми мы работаем, не поддерживают вложенных прерываний, которые были написаны для IBM/360. Для выполнения аппаратных прерываний установлены следующие ограничения:

- если на процессоре выполняется обработчик аппаратного прерывания, то на этом процессоре запрещены все прерывания;
- на остальных процессорах запрещены прерывания по линии IRQ, на которой установлен выполняющийся обработчик прерывания.

В результате, если такой обработчик написан неправильно и выполняет большой объём кода, это приведёт к снижению отзывчивости системы.

Поэтому, чтобы сократить время выполнения обработчиков, обработчик медленного аппаратного прерывания делится на две части:

- **Верхняя половина** обрабатывает аппаратное прерывание: выполняется минимально необходимый набор действий, в том числе для инициирования отложенного выполнения нижней половины.
- **Нижняя половина** выполняет остальные действия по обработке прерывания, является отложенным действием. Три типа нижней половины: `softirq`, `tasklet`, `work queue`.

Например, если это обработчик прерывания от клавиатуры, то он должен записать код нажатой клавиши, который поступает из буфера контроллера, в буфер ядра, а для последующей обработки должен установить задачу для последующего выполнения нижней половины, то есть отложенного действия.

Чтобы зарегистрировать в системе наш обработчик прерывания, в ядре имеется функция

```
1 static inline int __must_check request_irq(unsigned int irq, irq_handler_t handler,
2 unsigned long flags, const char *name, void *dev)
3 {
4     return request_threaded_irq(irq, handler, NULL, flags | IRQF_COND_ONESHOT, name,
5 dev);
6 }
```

Параметры:

- `irq` — номер прерывания. `IRQ0` — прерывание от таймера, `IRQ1` — от клавиатуры.
- `handler` — наш обработчик прерывания.
- `flags` — флаги Interrupt ReQuest Flags (IRQF).
- `name` — имя, которое увидим в `/proc/interrupts`.
- `dev` — указатель на некоторый объект.

Это функция-обёртка над `request_threaded_irq`, это базовая функция. При её вызове параметр `irq_handler_t thread_fn` равен `NULL`.

Флаги:

1	#define	IRQF_SHARED	0x00000080
2	#define	IRQF_PROBE_SHARED	0x00000100
3	#define	__IRQF_TIMER	0x00000200
4	#define	IRQF_PERCPU	0x00000400
5	#define	IRQF_NOBALANCING	0x00000800
6	#define	IRQF_IRQPOLL	0x00001000

```

7 #define IRQF_ONESHOT      0x00002000
8 #define IRQF_NO_SUSPEND   0x00004000
9 #define IRQF_FORCE_RESUME 0x00008000
10 #define IRQF_NO_THREAD    0x00010000
11 #define IRQF_EARLY_RESUME 0x00020000
12 #define IRQF_COND_SUSPEND 0x00040000
13 #define IRQF_NO_AUTOEN    0x00080000
14 #define IRQF_NO_DEBUG     0x00100000
15 #define IRQF_COND_ONESHOT 0x00200000
16
17 #define IRQF_TIMER         (__IRQF_TIMER | IRQF_NO_SUSPEND | IRQF_NO_THREAD)

```

- `IRQF_SHARED` — очень важный флаг: позволяет разделять линию прерывания несколькими обработчиками. Например, мы пишем обработчик прерывания клавиатуры, а в системе уже есть установленный стандартный обработчик прерывания клавиатуры — мы «вешаем» свой на ту же линию.
- `__IRQF_TIMER` — от системного таймера.
- `IRQF_NO_THREAD` — не может быть распараллелен.

В ядре определена структура, описывающая обработчик:

```

1 struct irqaction {
2     irq_handler_t handler;
3     void *dev_id;
4     void __percpu *percpu_dev_id;
5     struct irqaction *next;
6     irq_handler_t thread_fn;
7     struct task_struct *thread;
8     struct irqaction *secondary;
9     unsigned int irq;
10    unsigned int flags;
11    unsigned long thread_flags;
12    unsigned long thread_mask;
13    const char *name;
14    struct proc_dir_entry *dir;
15 };

```

Данная структура нигде в кодах не проявляется, она заполняется автоматически на основе параметров, которые передаём функции `request_irq`. Наиболее интересная строка — `struct proc_dir_entry *dir`: pointer to the `/proc/irq/NN/name` entry.

Значения, которые может вернуть наш обработчик:

```

1 enum irqreturn {
2     IRQ_NONE          = (0 << 0), // прерывания не было или оно не обработано

```

```

3   IRQ_HANDLED      = (1 << 0), // прерывание обработано
4   IRQ_WAKE_THREAD  = (1 << 1), // запрос пробуждения потока хендлера
5 };
6
7 typedef enum irqreturn irqreturn_t;

```

Когда драйвер выгружается, необходимо отменить регистрацию соответствующего обработчика прерывания. Для этого существует функция:

```

1 void free_irq(unsigned int irq, void *dev_id);

```

Если указанная линия прерывания не является разделяемой, то функция `free_irq()` удаляет обработчик и отключает линию. Если линия прерывания разделяется, то обработчик, определённый `dev_id` удаляется, но линия прерывания будет отключена только при удалении последнего обработчика.

11.5 Нижняя половины: `softirq`, определение, массив `softirq_vec`, типы `soft_irq`, функции `open_softirq()`, `raise_softirq()`

`softirq` (гибкие прерывания) — отложенные действия, определяются статически при компиляции. `softirq` инициализируется при загрузке системы. Если надо написать свой собственный `softirq`, то необходимо перекомпилировать ядро (изменить `enum`). Из версии ядра в версию ядра (как заклинание) разработчики переписывают: «пожалуйста, откажитесь от выделения нового `softirq`, если действительно не нужна высокая частота "threaded job scheduling"». Для большинства потребностей вполне достаточно `tasklet`’ов.

Обработчики `softirq` представлены в массиве `softirq_vec` — массиве экземпляров структуры `struct softirq_action` (`<linux/interrupt.h>`, массив определён в `kernel/softirq.c`):

```

1 struct softirq_action
2 {
3     void (*action)(struct softirq_action *); /* функция, которая должна выполняться */
4 };
5
6 static struct softirq_action softirq_vec[NR_SOFTIRQS];

```

Размер вектора 32, но реально определено 10.

В настоящее время в системе определено 10 `softirq`:

```

1 enum
2 {
3     HI_SOFTIRQ = 0,

```

```

4  TIMER_SOFTIRQ,
5  NET_TX_SOFTIRQ,
6  NET_RX_SOFTIRQ,
7  BLOCK_SOFTIRQ,
8  IRQ_POLL_SOFTIRQ,
9  TASKLET_SOFTIRQ,
10 SCHED_SOFTIRQ,
11 HRTIMER_SOFTIRQ,
12 RCU_SOFTIRQ,      /* Preferable RCU should always be the last softirq */
13 NR_SOFTIRQS
14 };

```

```

1 const char * const softirq_to_name[NR_SOFTIRQS] = {
2  "HI", "TIMER", "NET_TX", "NET_RX", "BLOCK", "IRQ_POLL",
3  "TASKLET", "SCHED", "HRTIMER", "RCU"
4  };

```

Меньшие номера имеют более высокий приоритет.

softirq NET_TX_SOFTIRQ и NET_RX_SOFTIRQ относятся к сетевой подсистеме. Работающие softirq можно увидеть через `cat /proc/softirqs`.

softirq одного типа может выполняться параллельно на разных процессорах одновременно (сериализуются). Это сделано с целью ускорения обслуживания соответствующих аппаратных подсистем — ярким примером является сетевая подсистема. Поэтому в коде softirq должно быть реализовано взаимное исключение в полном объёме, код должен быть реентерабельным.

Важно

softirq никогда не вытесняет другой softirq. Единственное событие, которое может вытеснить softirq — это аппаратное прерывание.

Функции для работы с softirq:

```

1 void raise_softirq(unsigned int nr); // Schedules softirq nr for the local CPU. It
   disables interrupts while updating the pending bit to ensure atomicity.
2
3 void open_softirq(int nr, void (*action)(void)); // Used during kernel initialization
   to register a handler function for a specific softirq index.

```

- `open_softirq()` выполняет регистрацию обработчика отложенного прерывания: ей передаются два параметра — индекс отложенного прерывания (индекс элемента массива `softirq_vec`) и функция-обработчик. Вызывается во время инициализации.

- Перед тем как зарегистрированное отложенное прерывание будет запущено, оно должно быть промаркировано — этот процесс называется генерацией отложенного прерывания (`raise softirq`). `raise_softirq()` вызывается из обработчика аппаратного прерывания и помечает `softirq` с указанным номером как ожидающий (`pending`) на текущем процессоре.

Пример (обработчик аппаратного прерывания планирует `softirq`):

```
1 irq_return_t my_interrupt(int irq, void *dev_id) {
2     raise_softirq(MY_SOFT_IRQ); /* mark softirq pending */
3     ...
4     return IRQ_HANDLED;
5 }
6
7 void my_soft_handler(struct softirq_action *) { ... }
8
9 static int __init my_init(void) {
10     request_irq(IRQ_NO, my_interrupt, 0, "my_irq_handler", NULL);
11     open_softirq(MY_SOFT_IRQ, my_soft_handler);
12 }
```

11.6 Поток ядра `ksoftirqd`

За выполнение `softirq` отвечает поток ядра `ksoftirqd`. Для каждого процессора имеется свой `ksoftirqd`:

- 1) Когда возникает необходимость выполнения `softirq`, соответствующее аппаратное прерывание записывает в битовую строку бит `pending`.
- 2) Чтобы `ksoftirqd` начал работу, вызывается функция `run_ksoftirqd` — она проверяет эту битовую маску.
- 3) Если в соответствующее битовое поле записана единица для соответствующего `softirq`, то вызывается функция `do_softirq`. Она выполняет зарегистрированный хендлер для добавленного `softirq`. Эту информацию задаёт драйвер.

SMP-архитектура (`symmetric multiprocessing`) предполагает, что в ней все процессоры равноправны: все процессоры могут равноправно принимать сетевые пакеты. При этом в SMP-архитектуре выделяется **один** процессор, который получает прерывания от системного таймера (процессор 0, условно главный).

Потоки `ksoftirqd` создаются при загрузке системы — по одному на каждый процессор (имя потока — `"ksoftirqd/%u\"`, где `%u` — номер процессора):

```
1 static struct smp_hotplug_thread softirq_threads = {
2     .store                = &ksoftirqd,
3     .thread_should_run    = ksoftirqd_should_run,
```

```

4  .thread_fn      = run_ksoftirqd,
5  .thread_comm    = "ksoftirqd/%u",
6  };
7
8  static __init int spawn_ksoftirqd(void)
9  {
10     cpuhp_setup_state_nocalls(CPUHP_SOFTIRQ_DEAD, "softirq:dead", NULL,
11                               takeover_tasklets);
12     BUG_ON(smpboot_register_percpu_thread(&softirq_threads));
13     return 0;
14 }
15 early_initcall(spawn_ksoftirqd);

```

Листинг 54. Создание потоков ksoftirqd

```

1  static void run_ksoftirqd(unsigned int cpu)
2  {
3     ksoftirqd_run_begin();
4     if (local_softirq_pending())
5     {
6         handle_softirqs(true);
7         ksoftirqd_run_end();
8         cond_resched();
9         return;
10    }
11    ksoftirqd_run_end();
12 }

```

Листинг 55. Функция run_ksoftirqd

11.7 Пример обработки приходящих пакетов NET_RX_SOFTIRQ

NAPI — New API. Сети были описаны очень давно, никто не мог предполагать современное состояние сетей, когда передаются огромные потоки данных. В частности, протокол TCP стал проблемой сетей: возникли DDoS-атаки, они работают из-за особенностей протокола TCP. Возможна ситуация, когда на определённый хост поступает большое количество пакетов с большой частотой. Каждый приход пакета — обработка прерывания, что является затратным действием: выполняется на самом высоком уровне привилегий, и, обрабатывая пакеты, система может потерять работоспособность. Поэтому был написан NAPI — он переводит сетевую систему в **режим опроса** (poll). Основная цель NAPI — сократить число прерываний при приёме пакетов.

RX — Receive, TX — Transmit. На рисунке показана SMP-архитектура, то есть многопроцессорная система:

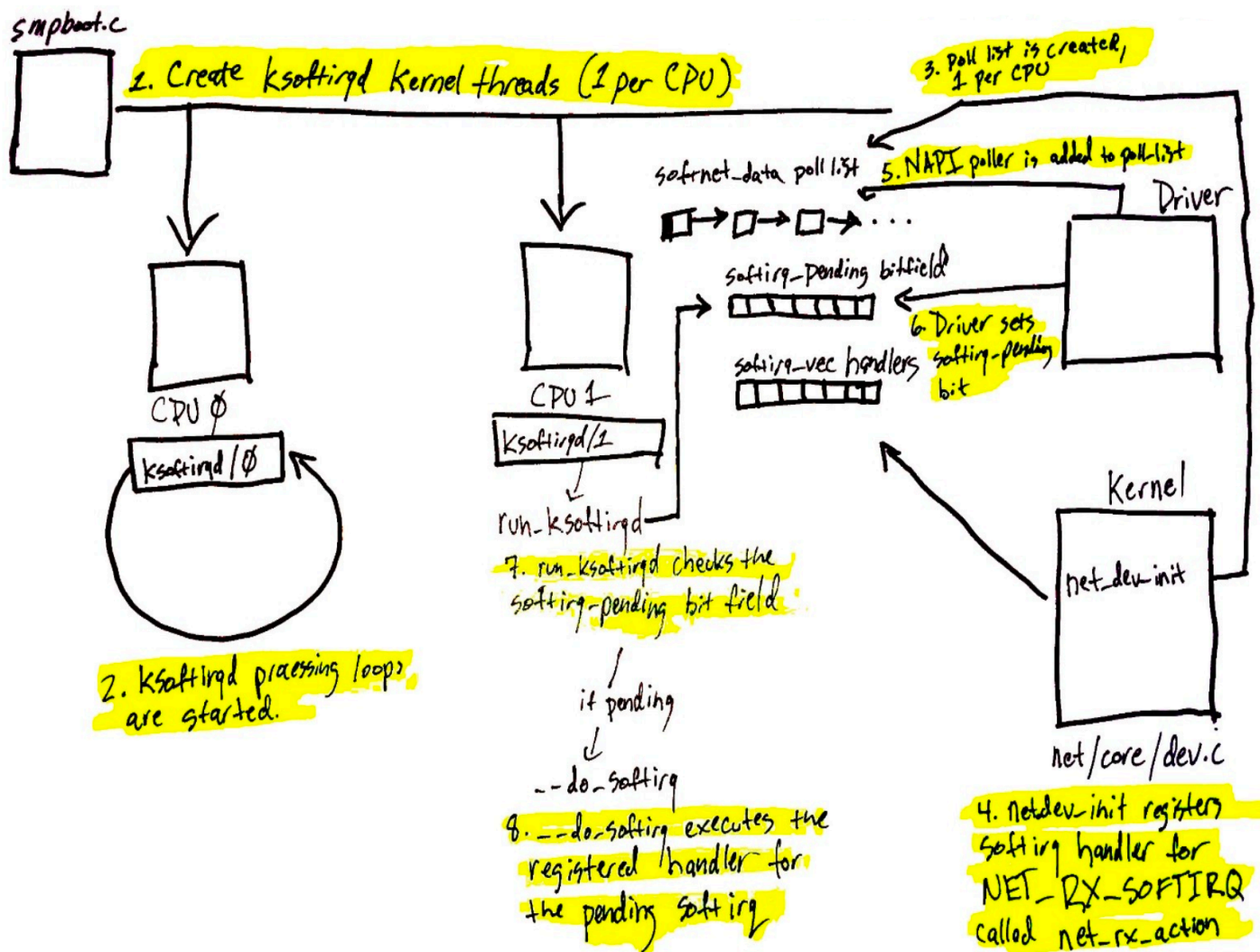


Рис. 32. `softirq` — `NET_пропустил` `RX_SOFTIRQ` и `ksoftirqd`

* Claude

Пояснение пунктов по картинке от Claude:

- 1) При загрузке системы (`smpboot.c`) создаётся по одному потоку ядра `ksoftirqd` на каждый CPU.
- 2) Запускаются processing loop'ы `ksoftirqd` — потоки входят в цикл обработки.
- 3) Создаётся `softnet_data poll list` — по одному на каждый CPU.
- 4) `net_dev_init` (`net/core/dev.c`) регистрирует обработчик `softirq` для `NET_RX_SOFTIRQ` — функцию `net_rx_action`.
- 5) Когда подключается сетевое устройство, его NAPI poller добавляется в poll list.
- 6) Приходит пакет → драйвер устанавливает бит в `softirq_pending bitfield`.
- 7) `run_ksoftirqd` проверяет `softirq_pending bitfield` — если бит выставлен (it pending), передаёт управление `--do_softirq`.
- 8) `--do_softirq` выполняет зарегистрированный обработчик для pending `softirq` — то есть вызывает `net_rx_action`, которая обходит poll list и опрашивает драйверы на наличие пакетов.

Пояснение от Рязановой: Для каждого процессора имеется свой softirqd. Чтобы он начал работу вызывается функция run softirqd. softirq pending («Дать пендаля»)

Если соответствующее битовое поле включено для соответствующего softirq, то вызывается функция do_softirq. Она выполняет зарегистрированный хендлер для добавленного softirq. Эту информацию задает драйвер.

Фрагмент регистрации сетевых softirq в net_dev_init (net/core/dev.c):

```
1 static int __init net_dev_init(void)
2 {
3     ...
4     open_softirq(NET_TX_SOFTIRQ, net_tx_action);
5     open_softirq(NET_RX_SOFTIRQ, net_rx_action);
6     ...
7 }
```

12

12 Создание виртуальных файловых систем. Структура, описывающая файловую систему. Регистрация и прекращение регистрации файловой системы в ОС Linux. Монтирование файловой системы. Точка монтирования. Функции монтирования. Команда монтирования файловой системы. Суперблок, назначение суперблока. Inode и адресация очень больших файлов (ф.с. extX). Структура dentry. Пример доступа к файлу mbox. Кэширование в системе: dentry кэш и кэш inode. Кэши SLAB, функции для работы с кэшем SLAB. Функции, определённые в файлах и их регистрация (struct file_operations). Пример из лабораторной работы.

12.1 Создание виртуальных файловых систем

Для Unix предложен интерфейс VFS/vnode; Linux оставил только VFS, отказавшись от vnode. VFS — интерфейс, через который ОС получает доступ к большому числу разных ФС, сократив до минимума количество системных вызовов: любая работа с файлом любой ФС идёт через один и тот же интерфейс. Основа работы — монтирование: прежде чем ФС станет доступна, она должна быть смонтирована.

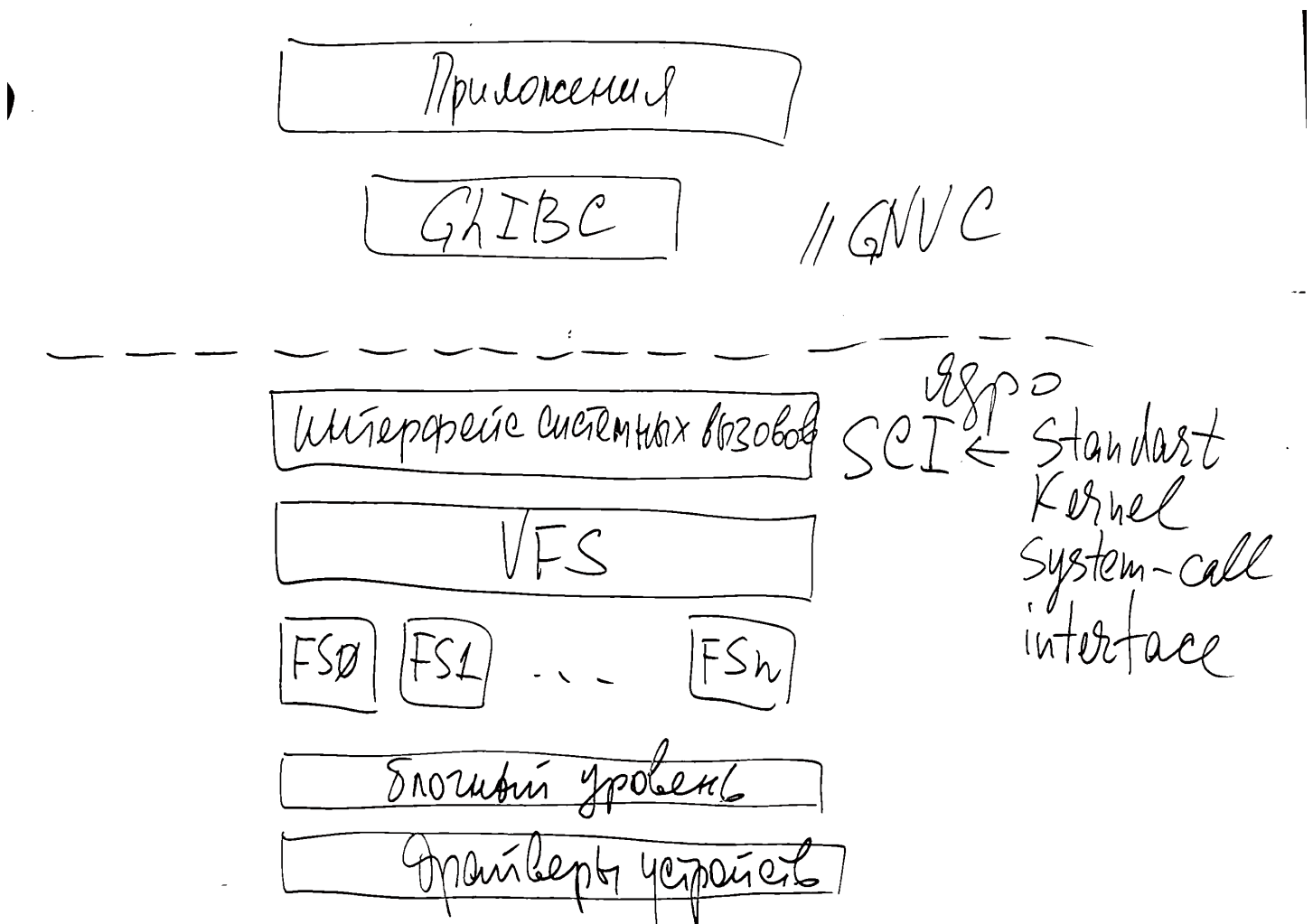


Рис. 33. Virtual File System

Виртуальная ФС — ФС, не монтируемая на физическое устройство. Пример — /proc: её файлов и каталогов реально не существует, они создаются на лету на

основе информации из системных таблиц. При этом используется стандартный интерфейс файловой системы и системных вызовов.

VFS позволяет создать собственную (виртуальную) файловую систему в загружаемом модуле ядра. Для этого необходимо:

- 1) определить тип своей ФС — проинициализировать поля структуры `struct file_system_type`;
- 2) зарегистрировать ФС в ядре (`register_filesystem`);
- 3) написать код, который позволит смонтировать ФС: зарегистрировать в `file_system_type` свою функцию `mount` (в ней вызывается функция ядра `mount_nodev`, так как виртуальная ФС не связана с устройством) и функцию `kill_sb` для освобождения суперблока;
- 4) в функции `fill_super` проинициализировать поля `struct super_block` и создать корневой каталог (`root`) — для этого проинициализировать `inode`, выделив для него память функцией `new_inode()`.

12.2 Структура, описывающая файловую систему

Для описания возможностей и характеристик каждой поддерживаемой ФС в ядре определена структура `file_system_type`:

```
1 struct file_system_type {
2     const char *name;
3     int fs_flags;           // флаги типа ФС (FS_REQUIRES_DEV и др.)
4     struct dentry *(*mount) (struct file_system_type *, int, const char *, void *);
5     void (*kill_sb) (struct super_block *);
6     struct module *owner;
7     struct file_system_type *next;
8     struct hlist_head fs_supers;
9 };
```

Листинг 56. Структура `file_system_type` (сокр.)

- `name` — имя ФС.
- `fs_flags` — флаги типа ФС; для ФС, работающей с внешним ЗУ, нужен флаг `FS_REQUIRES_DEV` (от него зависят вызываемые функции монтирования). Флаг `FS_USERNS_MOUNT` означает, что необязательно всё выполнять в ядре.
- `mount`, `kill_sb` — монтирование и прекращение доступа к суперблоку.
- `owner` — модуль-владелец ФС; нужен для счётчика ссылок (модуль не выгружается, пока ФС примонтирована).
- `next` — связный список `file_system_type`.
- `fs_supers` — список суперблоков.

Для регистрации можно объявить `file_system_type` минимального содержания: достаточно полей `owner` и `name`, но чтобы смонтировать ФС в дерево каталогов, нужно зарегистрировать в этой структуре ещё свою функцию `mount` (и `kill_sb`).

Для каждого типа файловой системы существует только одна структура `file_system_type`, независимо от того, сколько экземпляров этой ФС смонтировано: такая ФС может быть смонтирована в системе много раз, но зарегистрирована будет 1 раз.

12.3 Регистрация и прекращение регистрации файловой системы в ОС Linux

Ядро предоставляет нам функции регистрации ФС и deregистрации:

```
1 int register_filesystem(struct file_system_type *);
2 int unregister_filesystem(struct file_system_type *);
```

В этих функциях нет выделения памяти: они лишь регистрируют проинициализированный тип ФС, поэтому регистрация не требует полного кода монтирования. `unregister_filesystem` — просто удаление типа из списка типов ФС.

В загружаемом модуле ядра вызов `register_filesystem` выполняется в функции `init` модуля, а удаление из числа зарегистрированных — вызовом `unregister_filesystem` в функции `exit`. Увидеть зарегистрированную файловую систему в перечне файловых систем ядра можно через `cat /proc/filesystems`.

12.4 Монтирование файловой системы. Точка монтирования

Дерево файлов и каталогов в Linux формируется из ветвей, соответствующих разным носителям (разным ФС), но это одно большое дерево; некоторые каталоги могут быть точками монтирования.

Монтирование — система действий, делающая ФС (и все её файлы) доступной. Точка монтирования — каталог, к которому подмонтируется ФС: файлы и директории монтируемой ФС становятся файлами и поддиректориями точки монтирования, а её прежние файлы скрываются до размонтирования (поэтому лучше использовать пустые каталоги в качестве точки монтирования). Доступ к файлам ФС возможен только после монтирования. В Unix/Linux всегда есть корневая ФС (root fs); её монтирование — часть инициализации системы.

Когда ФС монтируется, создаётся структура `struct vfsmount`, которая представляет конкретный экземпляр смонтированной ФС:

```
1 struct vfsmount {
2     struct dentry      *mnt_root; /* root of the mounted tree */
```

```

3 struct super_block *mnt_sb;    /* pointer to superblock */
4 int                    mnt_flags;
5 struct mnt_idmap      *mnt_idmap;
6 };

```

Листинг 57. Структура `vfsmount`

`mnt_root` — корень подмонтированного дерева, `mnt_sb` — указатель на суперблок, `mnt_flags` — флаги монтирования.

12.5 Функции монтирования

В ядре определено несколько функций `mount()`; все они имеют модификатор `extern` и возвращают значение типа `struct dentry*`. В версии ядра 6.17.13, в отличие от предыдущих подверсий, определено только две:

```

1 extern struct dentry *mount_bdev(struct file_system_type *fs_type, int flags,
2                                const char *dev_name, void *data, int (*fill_super)
3                                (struct super_block *, void *, int));
4 extern struct dentry *mount_nodev(struct file_system_type *fs_type, int flags,
5                                void *data, int (*fill_super)
6                                (struct super_block *, void *, int));

```

Параметры: тип файловой системы (`fs_type`), флаги монтирования (`flags`), имя устройства (`dev_name`, только у `mount_bdev`), произвольные данные монтирования (`data`) и указатель на функцию `fill_super`. Эти функции существенно различаются кодом; внешнее отличие — в параметре `const char *dev_name`, которого нет у `mount_nodev`:

- `mount_bdev` — для монтирования ФС, находящейся на блочном устройстве;
- `mount_nodev` — для монтирования ФС, не связанной ни с каким устройством.

Функциям монтирования передаётся функция `fill_super` — она выполняет основные действия для монтирования: инициализирует поля `struct super_block` (размер блока, смещение, магическое число, `super_operations`). Функции монтирования возвращают объект `dentry`, которым должен быть `root`: для ФС нужно создать корневой каталог, а для этого — `inode`.

В системе существует готовая функция `simple_fill_super`, которую находим в коде ядра и используем как основу для функции `fill_super` своей файловой системы.

Для создания `root` надо проинициализировать `inode`, выделив для него память функцией `new_inode()` (внутри неё виден аллокатор — `inode` аллоцируется в `slab`-кеше):

```

1 struct inode *new_inode(struct super_block *sb) {

```

```

2 struct inode *inode = new_inode_pseudo(sb); // -> alloc_inode(sb): выделение
  из slab
3 if (inode)
4     inode_sb_list_add(inode);
5 return inode;
6 }

```

Листинг 58. Функция `new_inode` (сокр.)

Когда файловая система отмонтируется, всё должно быть удалено — этим занимается функция `kill_sb` из `file_system_type`. В ядре для этого определены функции:

```

1 void generic_shutdown_super(struct super_block *sb);
2 void kill_block_super(struct super_block *sb);
3 void kill_anon_super(struct super_block *sb);
4 void kill_litter_super(struct super_block *sb);

```

Общую работу по удалению суперблока выполняет `generic_shutdown_super`.

12.6 Команда монтирования файловой системы

Команда `mount` имеет следующий синтаксис:

```
1 mount <устройство> <точка монтирования>
```

Ключи: `-t <тип ФС>` `-o <опции ФС>`, то есть полная форма:

```
1 mount -t type device dir
```

Точка монтирования — любой каталог; в самой ФС нужно создать корневой каталог (`root`) — именно он монтируется. Ключ `-t` необязателен: система сама может определить тип ФС. После монтирования каталоги и файлы ФС доступны. Обычно монтирование требует прав суперпользователя.

С помощью команды

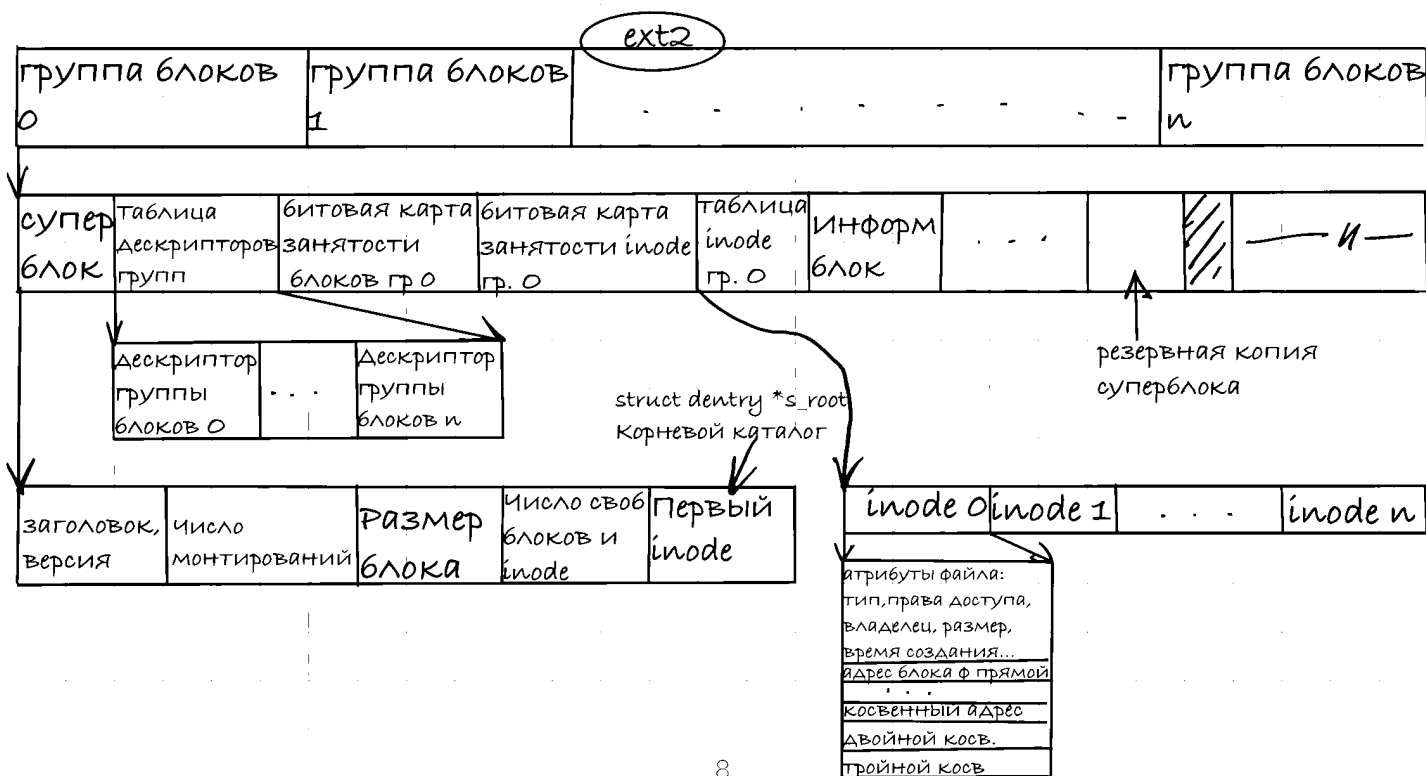
```
1 mount [-l] [-t type]
```

можно получить список смонтированных файловых систем типа `type`.

Размонтируется ФС командой `umount <точка монтирования>`.

12.7 Суперблок, назначение суперблока

Адресное пространство диска делится на разделы, один раздел занимает одна ФС.



8

Рис. 34. Обобщённая структурная схема файловой системы ext2

Суперблок представляет в VFS информацию о смонтированной ФС — структура с данными, необходимыми для монтирования и доступа к файлам ФС: метаданные для обращения к файлам, хранящимся «вразброс» (адресация участков диска). В каждой ФС ровно один суперблок, он располагается в начале раздела (есть резервная копия). При монтировании суперблок считывается в память ядра и находится там до размонтирования.

```

1 struct super_block {
2     struct list_head      s_list;
3     dev_t                 s_dev;
4     unsigned long         s_blocksize;
5     loff_t                s_maxbytes;    // Max file size
6     struct file_system_type *s_type;
7     const struct super_operations *s_op;
8     unsigned long         s_flags;
9     unsigned long         s_magic;
10    struct dentry          *s_root;      // точка монтирования
11    struct rw_semaphore    s_umount;
12    struct list_head       s_mounts;
13    struct block_device     *s_bdev;
14    struct list_head       s_inodes;    // all inodes
15 };

```

Листинг 59. Структура super_block (сокр.)

- `s_list` — список подмонтированных ФС (`list_head` — двусвязный список); `s_dev`, `s_bdev` — устройство и блочное устройство ФС.
- `s_blocksize`, `s_maxbytes` — размер блока и максимальный размер файла.
- `s_type`, `s_op` — тип ФС и функции на суперблоке; `s_flags` — флаги монтирования; `s_magic` — магическое число.
- `s_root` — точка монтирования (корневой каталог); `s_inodes` — список всех inode.
- `s_umount` — семафор «читатели-писатели» для монтирования; `s_mounts` — список монтирований данной ФС.

⚠ Осторожно

Вместо первого рисунка суперблока, который Рязанова не показывала (показывала), вероятно, хочет видеть второй.

Суперблок:

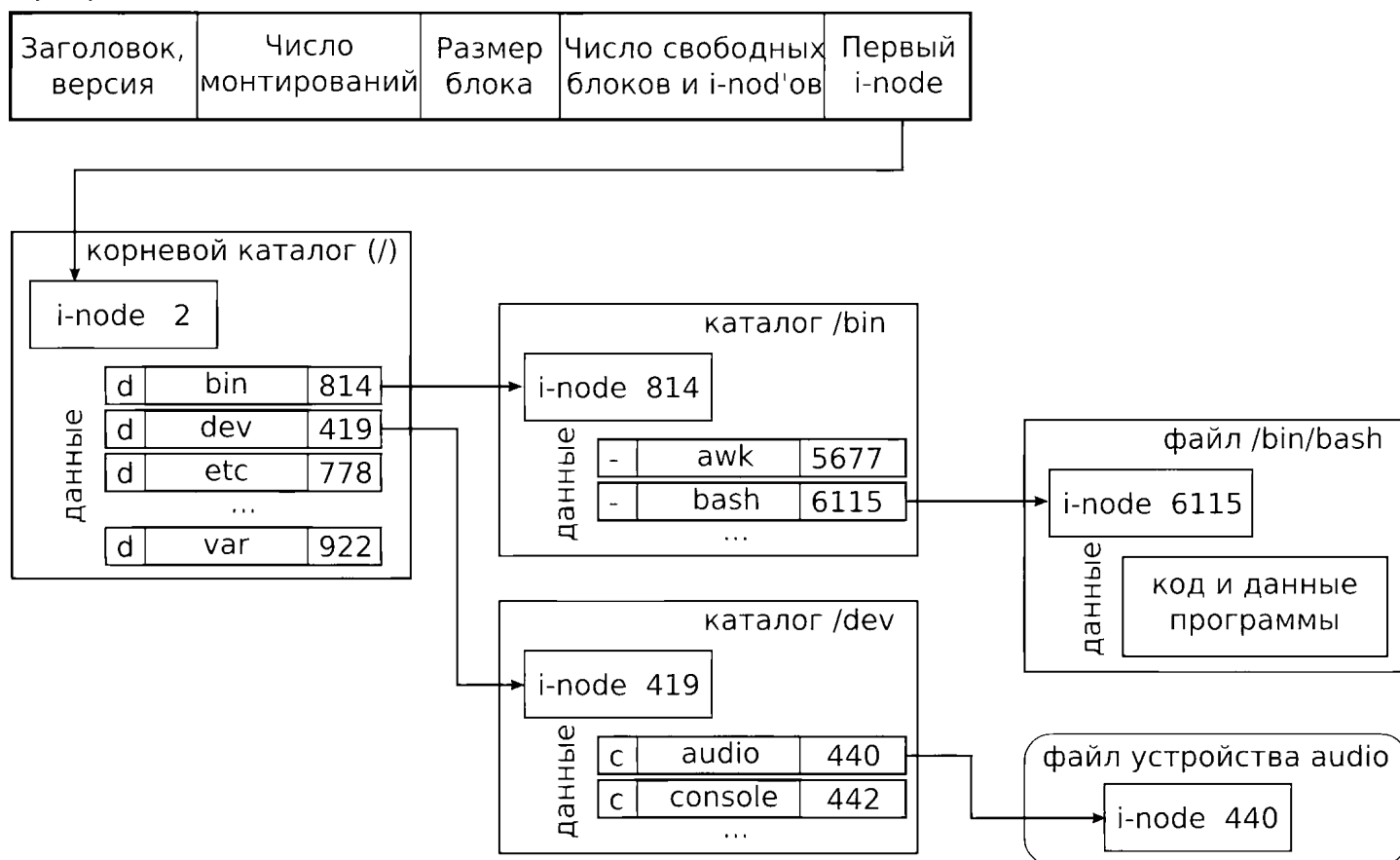


Рис. 35. Суперблок

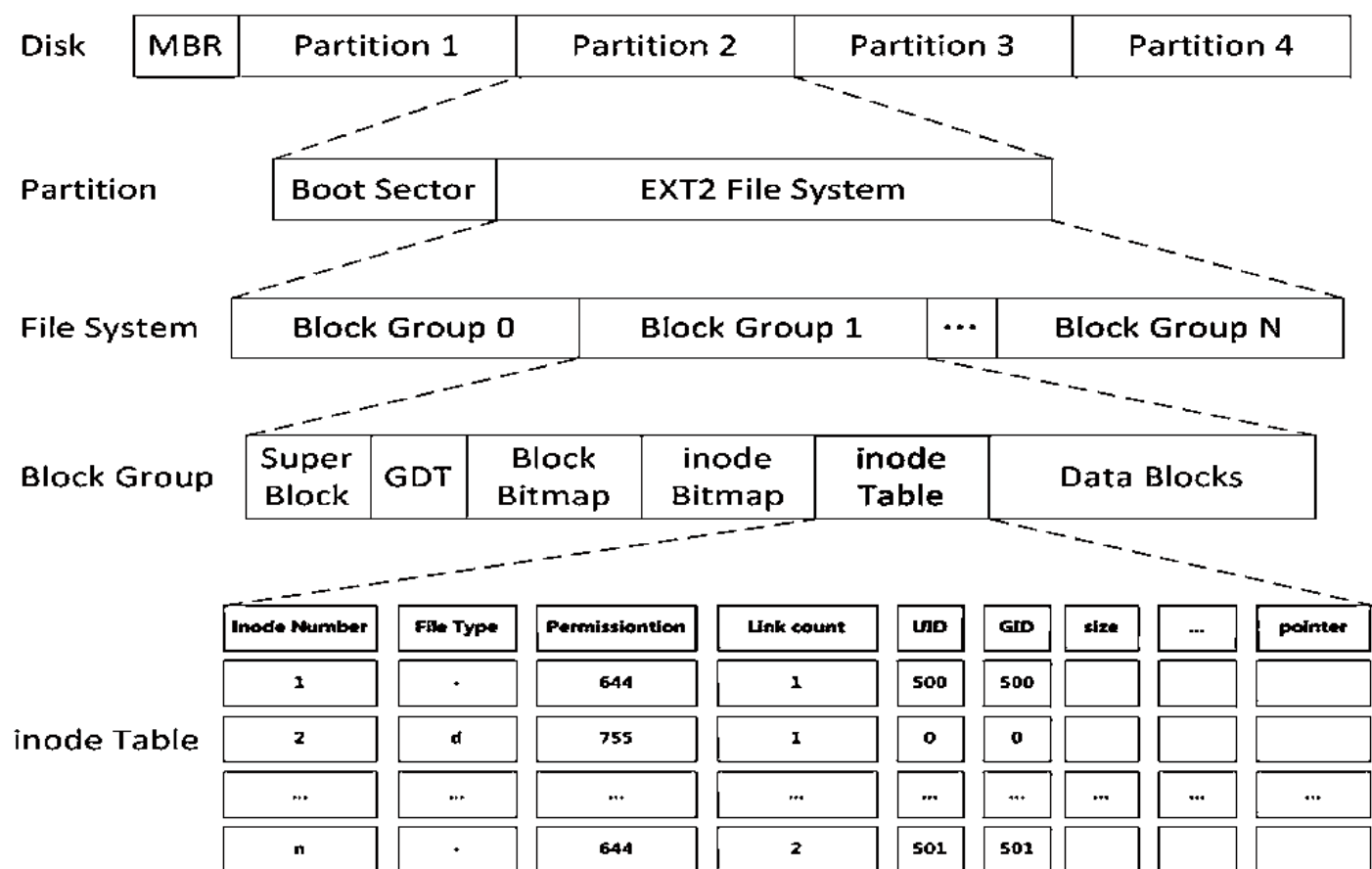


Рис. 36. Суперблок

12.8 Inode и адресация очень больших файлов (ф.с. extX)

inode — уровень идентификации файла (базовый уровень иерархии ФС). Дескриптор файла содержит множество полей, но идентификатором является номер inode (метаданные). inode описывает физический файл — реально хранящий информацию. Есть 2 вида struct inode:

- 1) **дисковый inode** — описывает физический файл, содержит адресную информацию, то есть адреса блоков, в которых хранится информация, записанная в данный файл (на диске адреса и на диске информация);
- 2) **inode в ядре** («ядерный» inode) — структура, которая находится в памяти ядра; позволяет дополнительно контролировать доступ к файлу и обеспечивает быстрый доступ к информации. Она же используется для описания виртуальных файлов.

```

1 struct inode {
2     umode_t    i_mode;
3     unsigned int i_flags;
4     const struct inode_operations *i_op;
5     struct super_block *i_sb;
6     unsigned long i_ino;
7     unsigned int i_nlink;    // Количество жёстких ссылок
8     loff_t     i_size;

```

```

9   u8   i_blkbits;
10  struct rw_semaphore i_rwsem;
11  ...           // списки/хеш: i_lru, i_sb_list, i_hash, i_dentry, ...
12  const struct file_operations *i_fop;
13  union { struct cdev *i_cdev; char *i_link; ... }; // для спецфайлов
14 };

```

Листинг 60. struct inode (сокр.)

- Флаги — ограничения: указывают допустимые операции над файлом и над самим inode.
- Файлы связаны с блочными устройствами, поэтому в inode есть размер блока и размер файла `i_size`.
- Есть read-write семафор `i_rwsem` и разные списки для быстрого доступа (обращение к внешнему устройству медленное).

Логический и физический уровни ФС связывает драйвер дискового устройства, переводящий смещения в логическом представлении файла в физические адреса; `struct inode` содержит адреса блоков с данными файла. Адресуемая единица файла — блок. Как адресуется файл на жёстком диске?

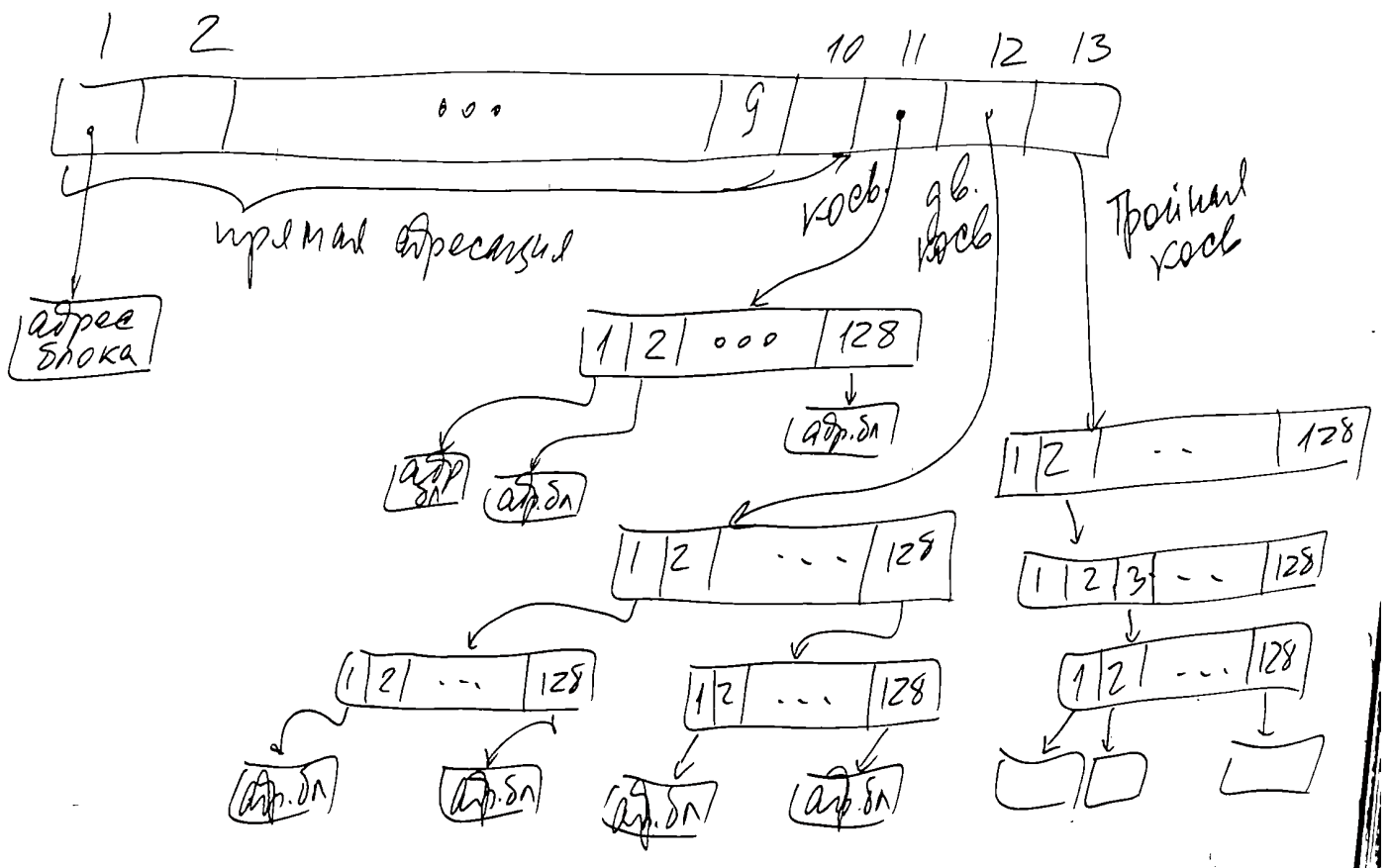


Рис. 37. Адресация файла на жёстком диске

Каждый блок по 512 байт. С помощью третьего уровня (тройной косвенной адресации) появляется возможность обращаться к $128^3 = 2097152$ блокам. Таким образом, в файловую систему Unix была заложена возможность создавать и работать с очень большими файлами.

Файловая система ext2 (Second Extended File System) была написана так, чтобы обеспечить доступ к хранению очень больших файлов:

- в дисковом inode 12 блоков адресуются прямо — inode непосредственно содержит адреса 12 блоков данных;
- далее используются косвенная, двойная косвенная и тройная косвенная адресация: блок, содержащий адреса блоков информации, имеет размер 4 Кб (страница), и при размере указателя 4 байта в нём помещается 1024 адреса ($1024 \cdot 4 \text{ байта} = 4 \text{ Кб}$);
- чтобы обратиться к блоку данных через косвенную адресацию, нужно многократно обращаться к диску за получением информации; это не делается аппаратно, а выполняется драйвером, поэтому время кратно увеличивается.

12.9 Структура dentry

В VFS каталоги обрабатываются как файлы и хранятся во вторичной памяти. Элемент пути (от / до следующего /) — это dentry. Например, в /bin/vi объекты dentry — /, bin, vi (первые два — каталоги, последний — файл). Объектами dentry являются все элементы пути, включая файлы и точки монтирования; VFS создаёт их «на лету» при операциях с каталогами.

Концепция dentry введена потому, что проход по элементам пути — долгое действие, связанное со сравнением строк, а VFS часто выполняет специфичные для каталога операции (поиск элемента пути по имени с проверкой и переходом к следующему элементу).

struct dentry не соответствует никакой информации из вторичной памяти: она нигде не хранится и создаётся «на лету» на основе данных директорий на диске. Поэтому у dentry нет флагов изменённости.

```
1 struct dentry {
2     /* RCU lookup touched fields */
3     unsigned int d_flags;           /* protected by d_lock */
4     ...
5     struct hlist_bl_node d_hash;    /* lookup hash list */
6     struct dentry *d_parent;        /* parent directory */
7     struct qstr d_name;
8     struct inode *d_inode;          /* Where the name belongs to - NULL is
   negative */
9     ...
```

```

10  const struct dentry_operations *d_op;
11  struct super_block *d_sb;           /* The root of the dentry tree */
12  ...
13  struct list_head d_lru;             /* LRU list */
14  ...
15  struct list_head d_subdirs;         /* our children */
16  ...
17  };

```

Листинг 61. struct dentry

- RCU (read, copy, update) lookup touched fields — механизм синхронизации: речь идёт о динамическом изменении данных, все вносимые изменения должны отражаться в соответствующих структурах, а структура находится в кэше.
- d_parent — объект dentry родительского каталога, d_name — имя, d_inode — связанный inode (если он NULL, элемент пути — negative).
- d_op — для каждого элемента пути перечисляются допустимые действия.
- d_sb — только для смонтированной файловой системы возможен доступ к её каталогам и файлам: любой каталог и файл относятся к конкретной ФС.

12.10 Пример доступа к файлу mbox

Каталоги хранятся на диске аналогично файлам. Содержимое каталога — таблица, сопоставляющая имена файлов и подкаталогов с номерами их inode:

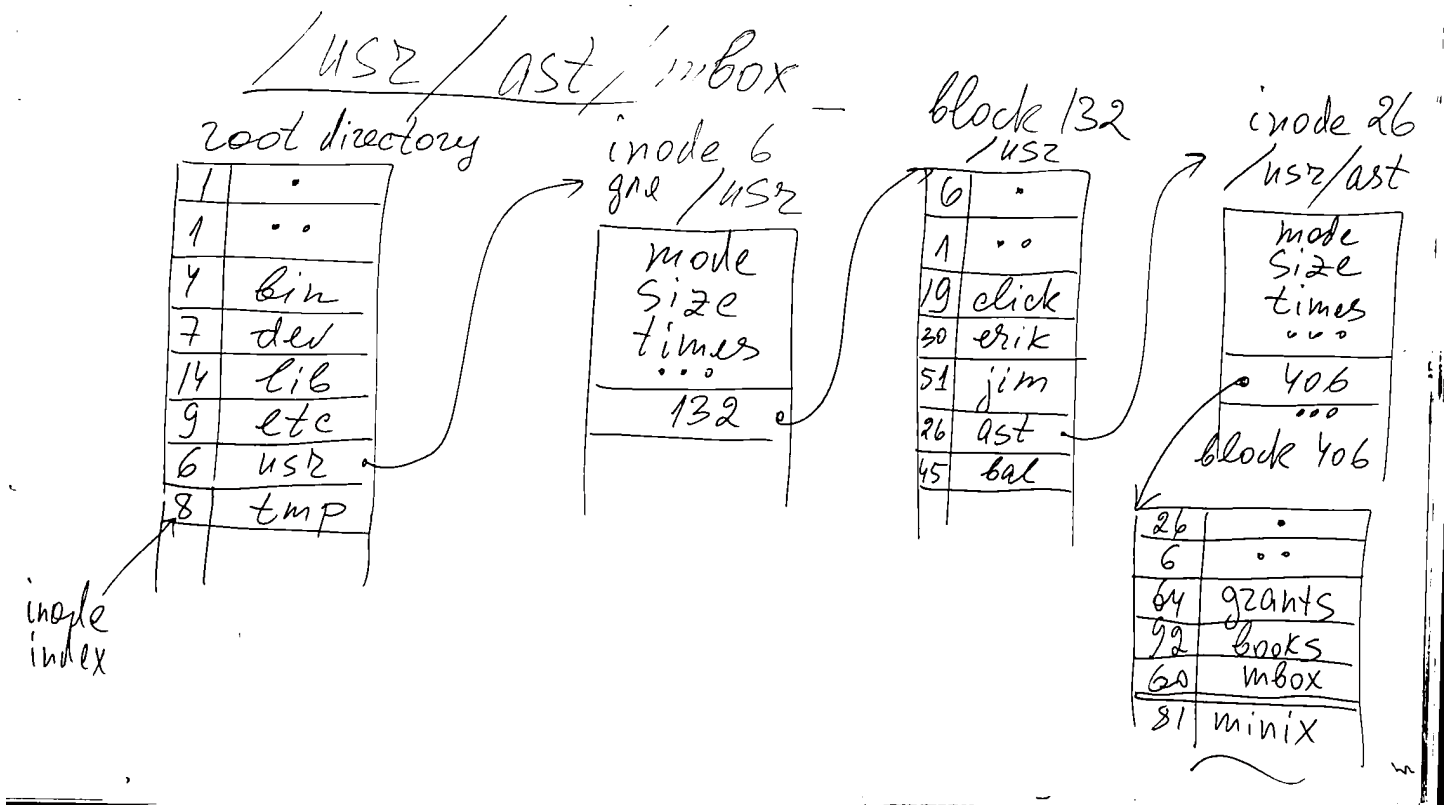
- В таблице видим имена файлов и папок (folder), видим номера inode.
- number — более правильное слово, потому что index — это всегда смещение.
- Структура inode директории состоит из имён, сопоставленных с inode.

inode number 3470036	
.	3470036
..	3470017
folder1	3470031
file1	3470043

Таблица 7. Структура inode каталога

Поиск файла всегда начинается с корневого каталога (для текущей директории её часть пути уже пройдена).

Пример: шаги для доступа к файлу mbox



Доступ к файлу /usr/ast/mbox (в пути 4 объекта dentry: /, usr, ast, mbox):

- 1) Путь начинается с корневого каталога (root directory); его содержимое известно системе.
- 2) В корневом каталоге ищется элемент пути `usr` — ему сопоставлен inode с номером 6.
- 3) Поля struct inode для данного каталога (mode, size, times, ...) позволяют идти дальше: нас интересует адресная часть — в inode 6 указано, что информация каталога находится в блоке 132. Обращаемся к диску: блок 132 — это содержимое директории /usr.
- 4) В блоке 132 ищется элемент `ast` — ему сопоставлен inode с номером 26 (это структура для /usr/ast). В этой структуре поля одни и те же, они не меняются в зависимости от того, чей это inode.
- 5) Из inode 26 берётся адрес блока 406 — это содержимое директории /usr/ast.
- 6) В блоке 406 ищется `mbox` — ему сопоставлен inode с номером 60; из этого inode берутся адреса блоков, в которых хранятся данные файла `mbox`.

Пример показывает, как много действий (обращений к диску) нужно выполнить для обращения к конкретному файлу, поэтому кэширование каталогов и файловых индексов позволяет сократить это время обращения.

12.11 Кэширование в системе: dentry кэш и кэш inode

Структуры dentry хранятся в кеше. Когда происходит первое обращение к элементу пути, создаётся объект dentry и он помещается в кеш. Когда необходимо обратиться

к файлу, система проходит по элементам пути и ищет их в dentry-кеше; если в нём dentry не найден, то обращение идёт к диску (см. пример с mbox выше).

12.11.1 Состояния dentry и dentry-кеш

Действительный объект dentry может быть в одном из трёх состояний (used, unused, negative); им соответствуют три части dentry-кеша:

- 1) **Список используемых** (used): у dentry есть inode (d_inode), счётчик d_count > 0; объект не удаляется, пока d_count не 0.
- 2) **Список неиспользуемых** (unused) — двусвязный LRU-список (dentry_lru), сортируется по времени последнего обращения; при нехватке памяти удаляются «хвостовые» объекты.

Состояние negative — элемент пути с несуществующим inode: кеширует факт отсутствия файла и не удаляется сразу, чтобы повторное обращение (например, повторный ls несуществующего файла) не шло к диску.

- 3) **Хеш-таблица** (dentry_hashtable) и **функция хеширования** быстро преобразуют путь в dentry; хеширование — d_hash(), поиск — d_lookup().

Доступ к файлам выполняется «в пространстве»: программы обращаются к разным файлам одного каталога, поэтому кеширование всех элементов пути одного файла сокращает время доступа и к соседним файлам этого каталога.

В примере с mbox получение корневого индексного дескриптора упрощается: он кеширован уже при монтировании корневого каталога. Сама dentry не содержит номера индексного дескриптора — он загружается отдельно.

12.11.2 Кэш inode

Кэш inode:

- 1) **Хеш-массив** inode_hashtable — inode хешируется по указателю на суперблок и номеру inode.
- 2) **Список** inode_in_use — допустимые inode с i_count > 0 и i_nlink > 0 (туда new_inode добавляет созданный inode).
- 3) **Список** inode_unused — допустимые inode с i_count == 0.
- 4) **Для каждого суперблока — список** sb->s_dirty — модифицированные («грязные», I_DIRTY) inode с i_count > 0; уменьшает накладные расходы на синхронизацию с диском.
- 5) inode_cached — slab-кеш.

Кеши взаимодействуют: inode, на которые ссылаются неиспользуемые объекты dentry, не удаляются, пока на них ссылается dentry-кеш.

12.12 Кэши SLAB, функции для работы с кэшем SLAB

Slab — «брусочек». Ядро оперирует ограниченным числом объектов (структур), для каждого нужно выделить память и проинициализировать поля, что занимает время. Если удалять объект после использования, для следующего такого же объекта придётся проделать ту же работу заново.

Идея: не удалять объекты, а сохранять их в slab для повторного использования. Под разные структуры нужны разные кэши — это и реализует Slab-кеш. Каждый кеш хранит объекты одного размера (больше или меньше страницы); есть слабы и для часто используемых размеров буферов (16, 32, 64, ...). Все кэши видны через `cat /proc/slabinfo`.

Важно

Для каждого типа данных создается свой SLAB кеш. Не один на всех, а **для каждого**.

Побочный эффект слабов — сокращение фрагментации памяти.

С версии ядра 2.6.23 SLAB переименован в SLUB (есть ещё SLOB). Это только код ядра: из `userspace` слаб недоступен — память выделяется страницами или слабами уже в ядре.

Имеются структуры `kmem_cache`; каждый кеш представляется такой структурой, а кэши связаны друг с другом в список `cache_chain` (цепочка кешей):

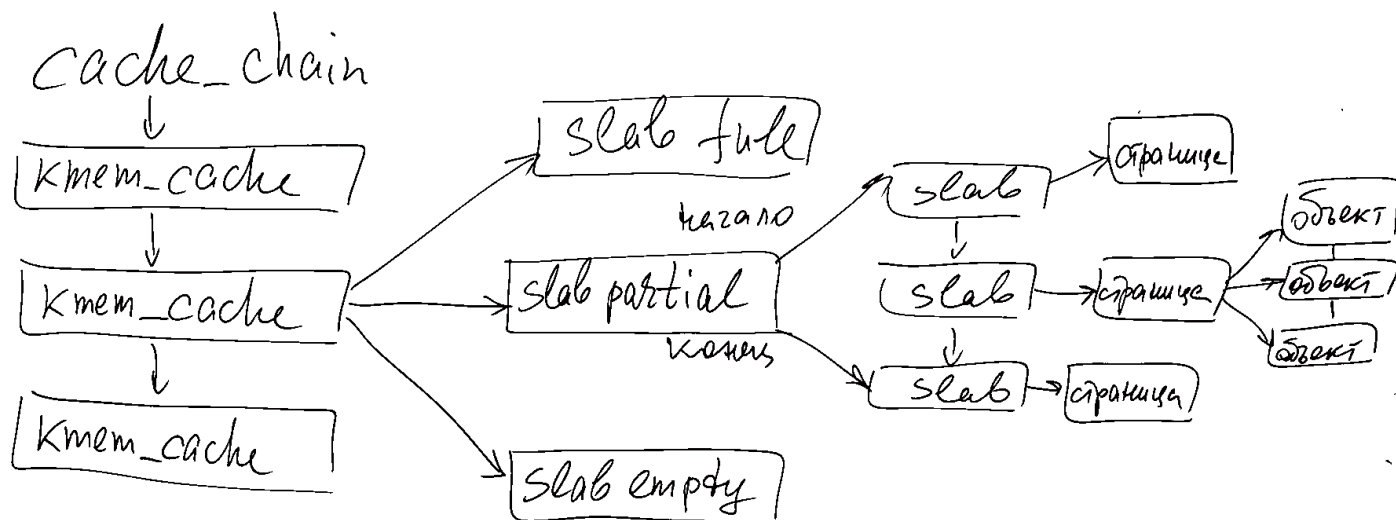


Рис. 39. `cache_chain`

Каждый slab может находиться в одном из трёх состояний:

- **slab full** — полностью распределённый: в таком слабе нет свободных объектов, все они распределены и используются;

- **slab partial** — частично распределённый: есть как выделенные, так и свободные объекты;
- **slab empty** — пустой: нет ни одного распределённого объекта, все объекты свободны для последующего использования.

Выделение: новый объект берётся из частично распределённого слаба, иначе из пустого; если пустых нет, слабы создаются по мере необходимости. Побочный эффект — уменьшение фрагментации памяти.

12.12.1 Функции для работы с кэшем SLAB

```

1  /* Создание кэша */
2  struct kmem_cache *kmem_cache_create(const char *name, size_t size, size_t offset,
3                                     unsigned long flags, void (*constructor)
4                                     (void *));
5
6  /* Уничтожение кэша */
7  void kmem_cache_destroy(struct kmem_cache *cache);
8
9  /* Выделить объект из кэша */
10 void *kmem_cache_alloc(struct kmem_cache *cache, gfp_t flags);
11
12 /* Удалить объект из кэша */
13 void kmem_cache_free(struct kmem_cache *cache, void *obj);

```

Листинг 62. Функции для работы с кешем SLAB

Кеш SLAB напрямую используется подсистемой VFS: объекты dentry создаются и уничтожаются вызовами `kmem_cache_alloc()` и `kmem_cache_free()`, а внутри `new_inode` виден аллокатор — inode аллоцируется в slab-кеше.

12.13 Функции, определённые в файлах и их регистрация (**struct file_operations**)

В `struct file` (дескрипторе открытого файла) поле `f_op` указывает на структуру `file_operations`, в которой перечислены функции, определённые на открытом файле:

```

1  struct file_operations {
2      struct module *owner;
3      ...
4      loff_t (*llseek) (struct file *, loff_t, int);
5      ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
6      ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
7      ...
8      int (*open) (struct inode *, struct file *);

```

```

9   int (*flush)    (struct file *, fl_owner_t id);
10  int (*release) (struct inode *, struct file *);
11  ...
12 };

```

Листинг 63. Структура `file_operations`

Важно

Здесь перечисляются функции ядра. `struct file` — структура ядра. Разработчики называют функции ядра аналогично функциям стандартных библиотек, но это **функция ядра**. Поэтому, например, в функции `read` присутствует указатель `__user *` на буфер пользователя.

- Для `struct file_operations` существует поле `owner` (владелец).
- В перечне функций ядра `close` отсутствует — закрытию соответствует `release`.

Регистрация функций выполняется инициализацией структуры.

```

1 struct file_operations fops = {
2   .read    = device_read,
3   .write   = device_write,
4   .open    = device_open,
5   .release = device_release
6 };

```

Проинициализированная структура привязывается к файлу через поле `i_fop` его `inode` (например, в `fill_super` для корневого каталога: `inode->i_fop = &simple_dir_operations`); у дескриптора открытого файла (`struct file`) на эти операции указывает поле `f_op`.

Драйверы работают со `struct file_operations`, переписать её нельзя. Чтобы разгрузить нагруженную функциями `struct file_operations`, для работы с ФС `proc` введена структура `struct proc_ops`, функции которой повторяют функции `file_operations`:

```

1 struct proc_ops {
2   ...
3   int (*proc_open)(struct inode *, struct file *);
4   ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
5   ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
6   ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
7   loff_t (*proc_lseek)(struct file *, loff_t, int);
8   int (*proc_release)(struct inode *, struct file *);

```

```
9    ...
10 };
```

Листинг 64. Структура proc_ops

Свои функции на файле proc регистрируют, инициализируя struct proc_ops и передавая её функции создания виртуального файла proc_create_data (или обёртке proc_create, передающей NULL в поле приватных данных):

```
1 struct proc_dir_entry *proc_create_data(const char *name, umode_t mode,
2     struct proc_dir_entry *parent, const struct proc_ops *proc_ops, void *data);
3 struct proc_dir_entry *proc_create(const char *name, umode_t mode,
4     struct proc_dir_entry *parent, const struct proc_ops *proc_ops);
```

- proc_ops — указатель на проинициализированную структуру; proc_dir_entry — дескриптор файла proc.
- parent задаёт иерархию: для файла в поддиректории её создают через proc_mkdir; при parent == NULL файл создаётся в корне proc.

12.14 Пример из лабораторной работы

12.14.1 ЛР VFS: собственная файловая система и SLAB-кеш

В ЛР создаётся собственная ФС; по отношению к VFS выполняются два действия:

- 1) **Регистрация ФС:** объявляем свой struct file_system_type (минимально owner, name, плюс функции mount и kill_sb); в init вызываем register_filesystem, в exit — unregister_filesystem.
- 2) **Монтирование ФС:** в своей функции mount вызываем mount_nodev (ФС не связана с устройством), передавая ей свою fill_super (на основе simple_fill_super): она инициализирует поля struct super_block, через new_inode создаёт inode корневого каталога и создаёт root (d_make_root). Освобождение суперблока выполняет своя функция kill_sb.

После загрузки модуля ФС монтируется командой mount (не менее 2 раз — одна и та же ФС может быть смонтирована много раз) и показывается двумя способами: в дереве каталогов (ls -ial) и через proc (cat /proc/mounts); зарегистрированный тип ФС — через cat /proc/filesystems.

Кроме того, в задании создаётся свой SLUB-кеш: объявляем свою структуру, создаём для неё кеш, через alloc получаем указатель на slab и выделяем объекты.

```
1 static int my_simple_fill_super(struct super_block *s, void *data, int silent)
2 {
3     struct inode *inode;
4     s->s_blocksize = PAGE_SIZE;
```

```

5  s->s_magic = MY_MAGIC_NUMBER;
6  s->s_op = &myfs_super_ops;
7
8  inode = new_inode(s);                // inode корневого каталога
9  inode->i_ino = 1;
10 inode->i_mode = S_IFDIR | 0755;
11 inode->i_op = &my_simple_dir_inode_operations;
12 inode->i_fop = &my_simple_dir_operations; // операции на файле
13 set_nlink(inode, 2);
14 s->s_root = d_make_root(inode);      // создание root
15 return s->s_root ? 0 : -ENOMEM;
16 }

```

Листинг 65. Функция fill_super собственной ФС (сокр.)

```

1  static struct dentry *myfs_mount(struct file_system_type *type, int flags,
2                                  char const *dev, void *data)
3  {
4      return mount_nodev(type, flags, data, my_simple_fill_super);
5  }
6
7  static struct file_system_type myfs_type = {
8      .owner    = THIS_MODULE,
9      .name     = "my_fs",
10     .mount    = myfs_mount,
11     .kill_sb  = my_kill_block_super,
12 };
13
14 static int __init my_init(void) { return register_filesystem(&myfs_type); }
15 static void __exit my_exit(void) { unregister_filesystem(&myfs_type); }
16 module_init(my_init);
17 module_exit(my_exit);

```

Листинг 66. Функции mount, регистрация и deregистрация собственной ФС (сокр.)

```

1  struct myfs_inode {
2      int i_mode;
3      unsigned long i_ino;
4  };
5
6  static struct kmem_cache* inode_cache = NULL;
7  static struct myfs_inode** inode_cached_array = NULL;
8
9  static struct dentry *my_fs_mount(struct file_system_type *type, int flags,
10                                   const char *dev, void *data)
11  {

```

```

12  ...
13  for (i = 0; i < OBJECTS_PER_MOUNT; i++)
14  {
15      struct myfs_inode *inode = kmem_cache_alloc(inode_cache, GFP_KERNEL);
16      if (!inode)
17      {
18          for (int j = 0; j < i; j++)
19              kmem_cache_free(inode_cache, inode_cached_array[j]);
20          return ERR_PTR(-ENOMEM);
21      }
22      ...
23  }
24  ...
25 }
26
27 static int __init myfs_init(void) {
28     ...
29     // Создание slab-кэша в init
30     inode_cache = kmem_cache_create(SLAB_NAME, sizeof(struct myfs_inode), 0, 0, NULL);
31     if (inode_cache == NULL)
32         return -ENOMEM;
33     ...
34 }
35
36 static void __exit myfs_exit(void) {
37     ...
38     // освобождение всех объектов
39     for (i = 0; i < cached_count; i++)
40         kmem_cache_free(inode_cache, inode_cached_array[i]);
41     // удаление slab-кэша
42     kmem_cache_destroy(inode_cache);
43     ...
44 }

```

Листинг 67. Создание и использование своего SLAB-кэша в ЛР

12.14.2 ЛР «Фортунки»: регистрация функций на файле /proc

ЛР о передаче данных между пространством пользователя и ядром; работа с файлом ФС proc ведётся в загружаемом модуле ядра. В модуле 6 точек входа: open, read, write, release, init, exit. В read/write вызываются copy_to_user/copy_from_user, так как ядро работает с физическими адресами, а процессы — с виртуальными.

```

1 static struct proc_ops fops = {
2     .proc_read = fortune_read,
3     .proc_write = fortune_write,

```

```

4  .proc_open = fortune_open,
5  .proc_release = fortune_release,
6  };
7  ...
8  // собственно регистрация при создании вирт. файла:
9  proc_info_file = proc_create(FILENAME, 0666, NULL, &fops);

```

Листинг 68. Регистрация функций, определённых на виртуальном файле

```

1  static char *buffer;
2
3  int fortune_open(struct inode *sp_inode, struct file *sp_file)
4  {
5      printk(KERN_DEBUG "open");
6      return 0;
7  }
8
9  int fortune_release(struct inode *sp_node, struct file *sp_file)
10 {
11     printk(KERN_DEBUG "release");
12     return 0;
13 }
14
15 // buf - адрес источника в пространстве пользователя.
16 static ssize_t fortune_write(struct file *file, const char __user *buf, size_t
    count, loff_t *ppos)
17 {
18     ...
19     if (copy_from_user(&buffer[write_index], buf, count)) {
20         return -EFAULT; // EFAULT - неправильный адрес.
21     }
22     ...
23 }
24
25 static ssize_t fortune_read(struct file *file, char __user *buf, size_t count,
    loff_t *ppos)
26 {
27     ...
28     len = sprintf(tmp, "%s\n", &buffer[read_index]);
29     copy_to_user(buf, tmp, len);
30     ...
31 }
32
33 static int __init fortune_init(void)
34 {
35     // vmalloc - выделить непрерывный блок памяти.

```

```
36  buffer = (char *)vmalloc(buffer_SIZE);
37  ...
38  proc_entry = proc_create("fortune", 0666, NULL, &fops);
39  ...
40 }
41
42 static void __exit fortune_exit(void)
43 {
44  ...
45  remove_proc_entry("fortune", NULL);
46  ...
47 }
```

Листинг 69. Точки входа модуля «фортунки»

13

13 Файловая подсистема: особенности файловой подсистемы Unix/Linux: иерархическая структура файловой подсистемы. Виртуальная файловая система VFS в Linux. Четыре структуры VFS — super_block, inode, dentry, file — их назначение. Адресация файлов большого размера в файловой системе extX и пример, показывающий доступ к файлу /usr/ast/mbox. Кэширование dentry и inode. Slab кэш. Монтирование файловых систем. Команда mount и функции монтирования, пример из лаб раб.

13.1 Файловая система

Файл — поименованная совокупность данных, может быть бессмысленная, хранящаяся во вторичной памяти. Понятие файла связано с долговременным хранением информации.

Файловая система — это порядок, определяющий способ организации хранения, именования и доступа к данным на вторичных носителях информации.

Файловая система используется для управления хранением данных и доступа к ним. ФС должна обеспечить доступ к хранимой в системе на вторичных дисках информации.

13.2 Особенности файловой подсистемы Unix/Linux: иерархическая структура файловой подсистемы

Файловая система — важнейшая составляющая любой вычислительной системы: без долговременного хранения данные нельзя обработать повторно (содержимое ОЗУ при выключении пропадает), поэтому разработчики UNIX начали построение системы именно с неё. Внешние ЗУ — блочные устройства, на них же вытесняются страницы при свопинге (swapping) и пейджинге (paging).

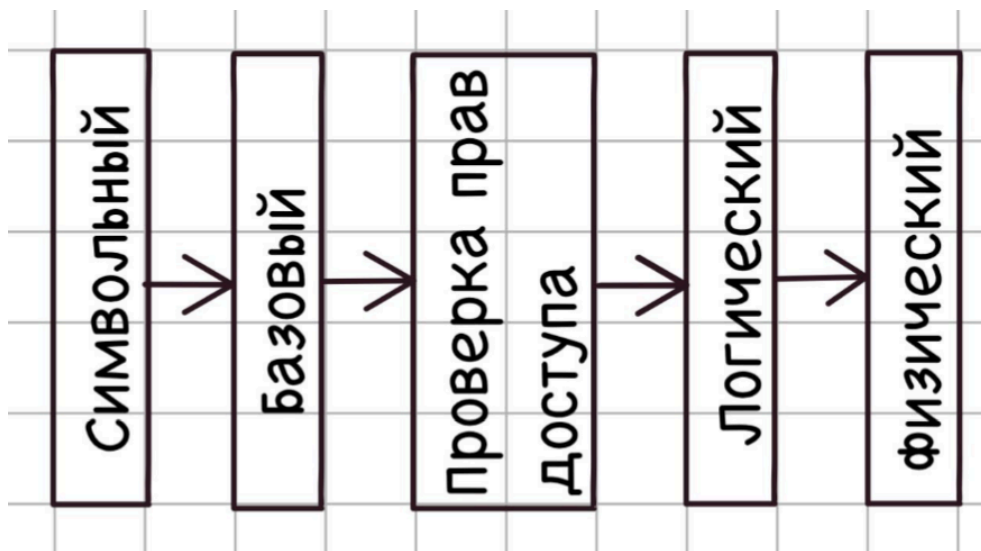
13.3 Задачи файловой системы

- 1) Долговременное хранение данных
- 2) Обеспечение доступа к хранящимся данным

13.4 Уровни иерархии ФС

Для обеспечения доступа к данным нужно предоставить пользователю достаточно удобный способ доступа. Уровни файловой системы:

- 1) Символьный
- 2) Базовый
- 3) Проверка прав доступа
- 4) Логический
- 5) Физический



- **Символьный** уровень — именованье файлов: у файлов **имена** (массив символов), собранные по каталогам; корневой каталог в UNIX/Linux — /. Имя не является идентификатором файла, а лишь позволяет удобно с ним работать; идентификатором файла является inode; определены жёсткие (hard links) и символические (soft links) ссылки.
- **Базовый** уровень — идентификация файла: это inode; идентификатором является номер inode, который описывает физический файл.
- **Проверка прав доступа** — права отдельно для пользователя, группы и остальных (команда chmod).
- **Логический** уровень — процесс открывает файл и выполняет над ним операции: файл считает, что начинается с нулевого смещения (offset), каждая операция чтения/записи смещает указатель; файл байт-ориентированный, размера не имеет. Здесь определяется адресация данных, преобразуемая в физические адреса на диске.
- **Физический** уровень — уровень физической памяти; логический и физический уровни связывает драйвер дискового устройства, переводящий смещения в физические адреса.

13.4.1 Иерархическая структура каталогов

Файловые системы знакомых нам ОС являются иерархическими — это негласное правило, впервые определённое именно в UNIX. Для Unix характерно существование корневого каталога /, от которого ветвится дерево каталогов:

1	/
2	— bin
3	— etc
4	— dev
5	— usr
6	— local

13.5 Виртуальная файловая система VFS в Linux

Для Unix был предложен интерфейс VFS/vnode (virtual file system, virtual node). Linux отказался от vnode, оставив только VFS. VFS — интерфейс, через который ОС получает доступ к большому числу разных ФС, сократив до минимума количество системных вызовов: любая работа с файлом любой ФС идёт через один и тот же интерфейс. Основа такой работы — монтирование: прежде чем ФС станет доступна (станут видны её каталоги и файлы), она должна быть смонтирована.

В основе интерфейса VFS лежат 4 структуры (это ядерные структуры):

- 1) struct super_block;
- 2) struct inode;
- 3) struct dentry;
- 4) struct file.

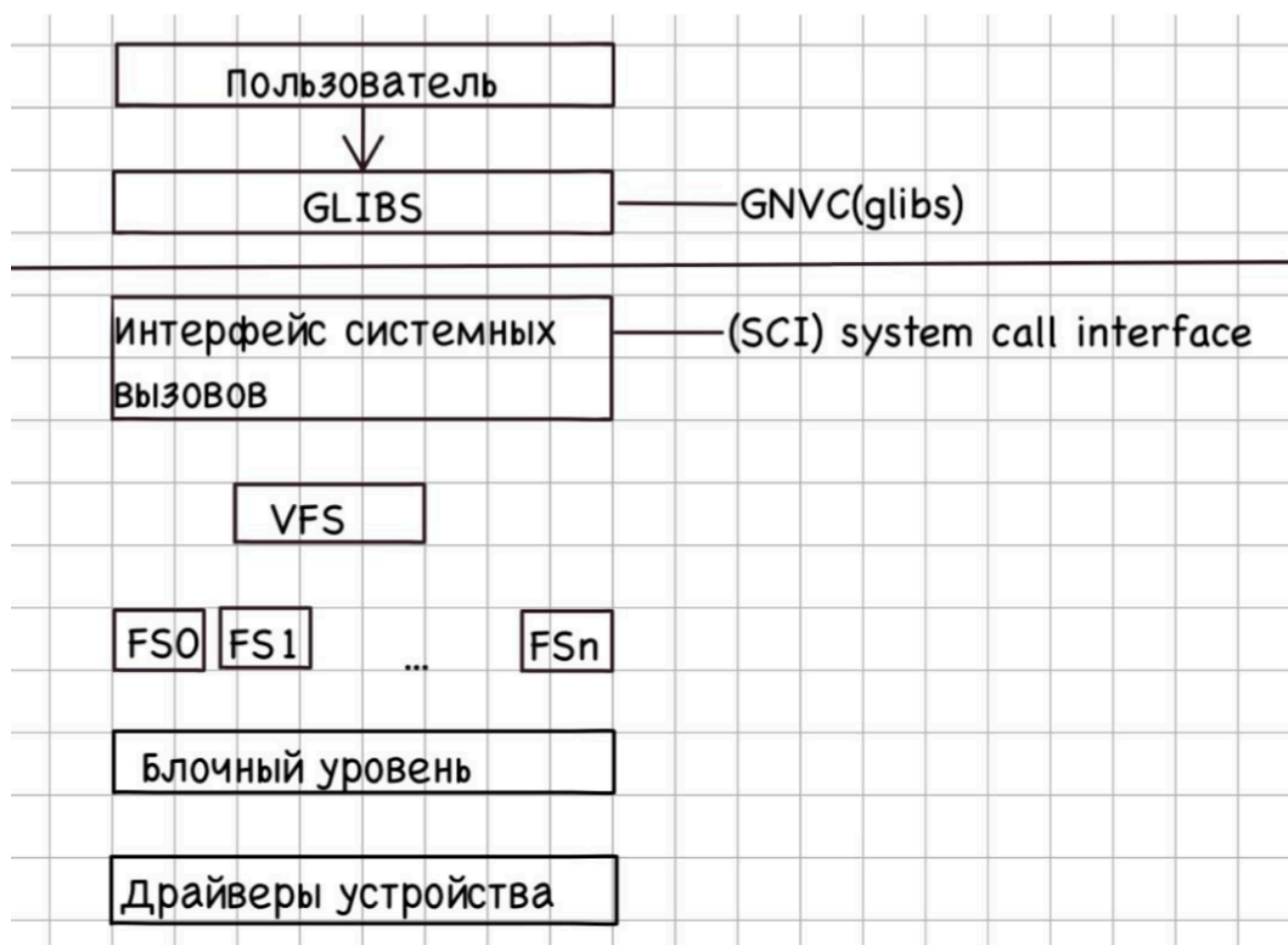


Рис. 40. Virtual File System

13.6 Четыре структуры VFS — `super_block`, `inode`, `dentry`, `file` — их назначение

Все четыре структуры взаимодействуют между собой указателями, и для каждого файла создаются их экземпляры. Кратко их назначение: `super_block` описывает смонтированную ФС, `inode` — физический файл, `dentry` — элемент пути (имя), `file` — открытый файл.

13.6.1 `super_block` — описывает смонтированную ФС

Суперблок представляет в VFS информацию о смонтированной ФС — структура с данными, необходимыми для монтирования и доступа к файлам ФС: метаданные для обращения к файлам, хранящимся «вразброс» (адресация соответствующих участков диска). В каждой ФС ровно один суперблок, он располагается в начале раздела (есть резервная копия). При монтировании суперблок считывается в память ядра и находится там до размонтирования.

```
1 struct super_block {
2     struct list_head      s_list;
3     dev_t                 s_dev;
4     unsigned long         s_blocksize;
5     loff_t                s_maxbytes;    // Max file size
6     struct file_system_type *s_type;
7     const struct super_operations *s_op;
8     unsigned long         s_flags;
9     unsigned long         s_magic;
10    struct dentry          *s_root;      // точка монтирования
11    struct rw_semaphore    s_umount;
12    struct list_head       s_mounts;
13    struct block_device    *s_bdev;
14    struct list_head       s_inodes;     // all inodes
15 };
```

Листинг 70. Структура `super_block` (сокр.)

- `s_blocksize`, `s_maxbytes` — размер блока и максимальный размер файла; `s_magic` — магическое число ФС.
- `s_type`, `s_op` — тип ФС и функции на суперблоке (`super_operations`); `s_root` — точка монтирования (корневой каталог); `s_inodes` — список всех `inode`.
- `s_mounts` — список монтирований данной ФС (одна и та же ФС может быть смонтирована много раз).

⚠ Осторожно

Вместо первого рисунка суперблока, который Рязанова не показывала (показывала), вероятно, хочет видеть второй.

Суперблок:

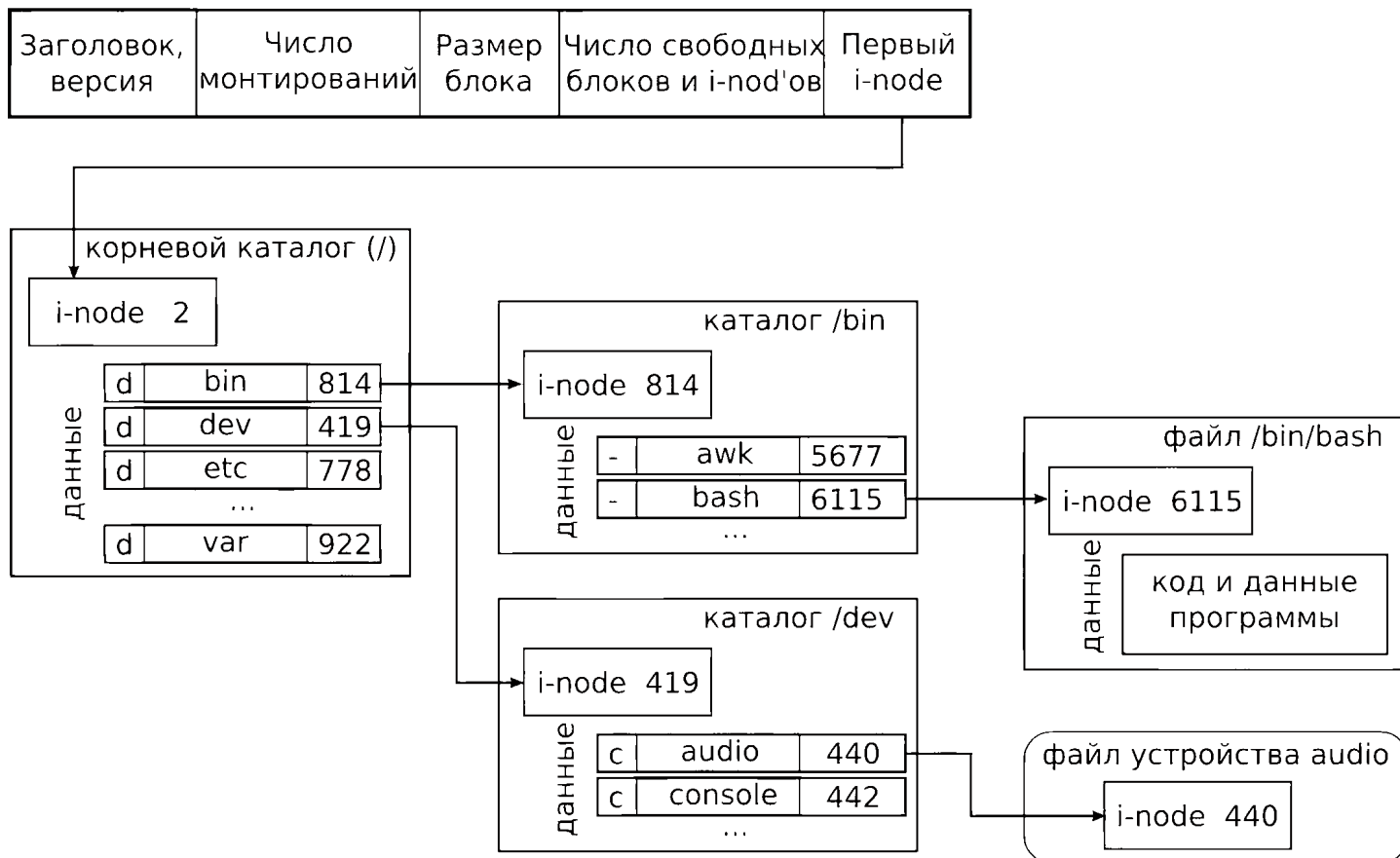


Рис. 41. Суперблок

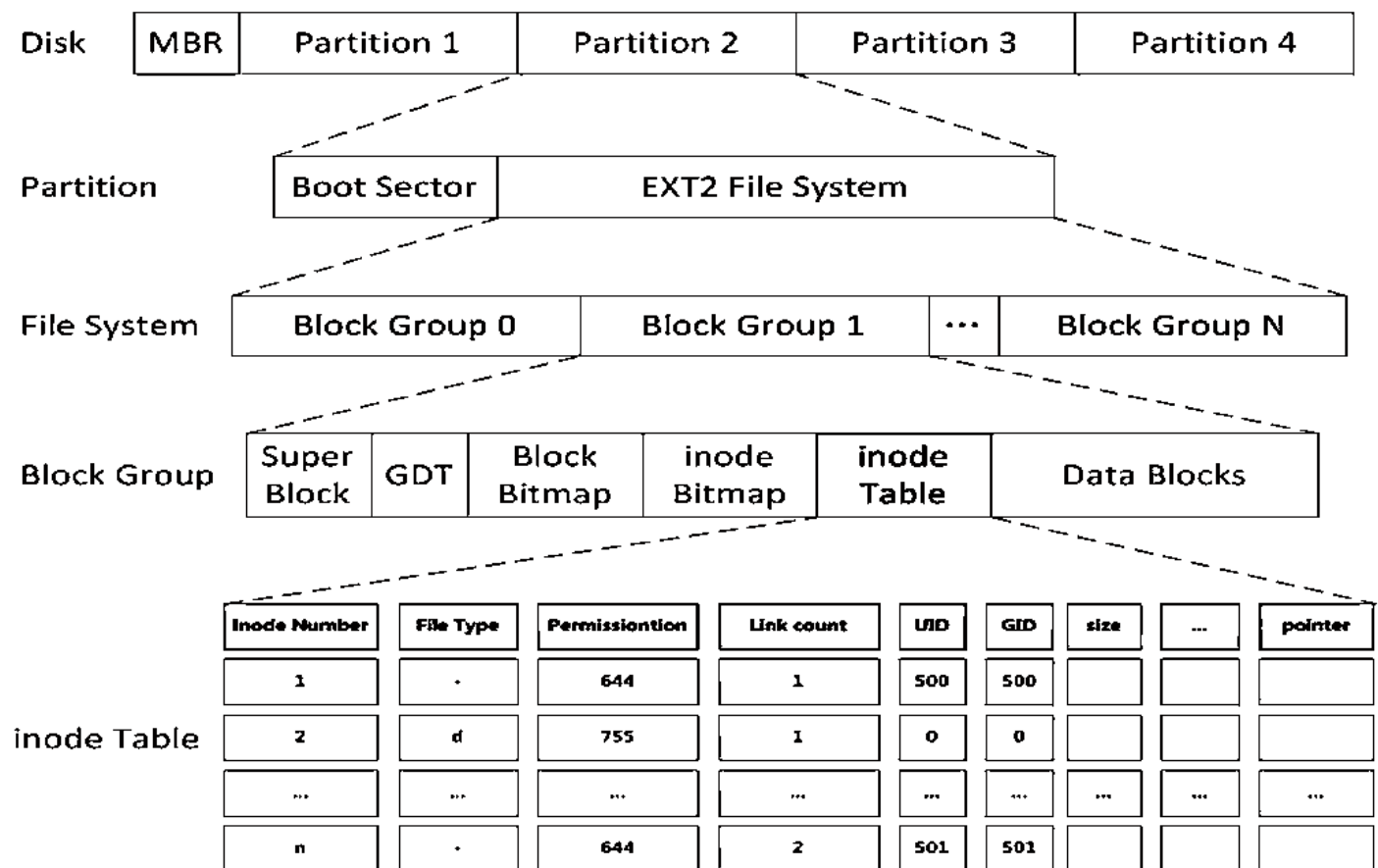


Рис. 42. Суперблок

13.6.2 inode — дескриптор физического файла

inode — Дескриптор файла содержит множество полей, но идентификатором является номер inode (`i_ino`, метаданные). inode описывает физический файл — реально хранящий информацию. Есть 2 вида `struct inode`:

- 1) **дисковый inode** — описывает физический файл, содержит адресную информацию, то есть адреса блоков, в которых хранится информация, записанная в данный файл (на диске адреса и на диске информация);
- 2) **inode в ядре** содержит информацию о файле для ускорения доступа (копия в памяти, дополнительные поля: логический номер устройства, номер индекса, ссылки для хеш-очереди).

```

1 struct inode {
2     umode_t    i_mode;
3     unsigned int i_flags;
4     const struct inode_operations *i_op;
5     struct super_block *i_sb;
6     unsigned long i_ino;
7     unsigned int i_nlink;    // Количество жёстких ссылок
8     loff_t      i_size;
9     u8          i_blkbits;
10    struct rw_semaphore i_rwsem;
11    ...           // списки/хеш: i_lru, i_sb_list, i_hash, i_dentry, ...

```

```

12  const struct file_operations *i_fop;
13  union { struct cdev *i_cdev; char *i_link; ... }; // для спецфайлов
14  };

```

Листинг 71. struct inode (сокр.)

- `i_ino` — номер inode (идентификатор файла); `i_nlink` — количество жёстких ссылок; `i_size` — размер файла.
- `i_op`, `i_fop` — операции на inode и на файле; `i_sb` — указатель на суперблок (физический файл доступен только если ФС смонтирована).
- Флаги (`i_flags`) играют роль ограничений: указывают допустимые операции над файлом и над самим inode.

13.6.3 dentry — элемент пути (directory entry)

В VFS каталоги обрабатываются как файлы и хранятся во вторичной памяти. Элемент пути (от / до следующего /) — это dentry. Например, в `/bin/vi` объекты dentry — `/`, `bin`, `vi` (первые два — каталоги, последний — файл). Объектами dentry являются все элементы пути, включая файлы и точки монтирования; VFS создаёт их «на лету» при операциях с каталогами. Концепция dentry введена потому, что проход по элементам пути — долгое действие, связанное со сравнением строк, а VFS часто выполняет специфичные для каталога операции (поиск элемента пути по имени с проверкой и переходом к следующему).

`struct dentry` не соответствует никакой информации из вторичной памяти: она нигде не хранится и создаётся «на лету» на основе данных директорий на диске. Поэтому у dentry нет флагов изменённости.

```

1  struct dentry {
2      /* RCU lookup touched fields */
3      unsigned int d_flags;           /* protected by d_lock */
4      ...
5      struct hlist_bl_node d_hash;    /* lookup hash list */
6      struct dentry *d_parent;        /* parent directory */
7      struct qstr d_name;
8      struct inode *d_inode;          /* Where the name belongs to - NULL is
   negative */
9      ...
10     const struct dentry_operations *d_op;
11     struct super_block *d_sb;        /* The root of the dentry tree */
12     ...
13     struct list_head d_lru;          /* LRU list */
14     ...
15     struct list_head d_subdirs;      /* our children */
16     ...

```

Листинг 72. struct dentry

- `d_parent` — объект `dentry` родительского каталога, `d_name` — имя, `d_inode` — связанный `inode` (если он `NULL`, элемент пути — `negative`).
- `d_op` — допустимые действия над элементом пути; `d_sb` — суперблок: любой каталог и файл относятся к конкретной смонтированной ФС.

13.6.4 file — дескриптор открытого файла

Различают файлы и открытые файлы: просто файлы представлены деревом каталогов, физический файл описывает `struct inode`. Открытый файл — файл, с которым работает процесс; его описывает `struct file`. Один файл может быть открыт много раз (в том числе разными процессами): сколько открытий, столько дескрипторов открытого файла, каждый со своим смещением.

```

1 struct file {
2     ...
3     spinlock_t f_lock;           // монопольный доступ
4     fmode_t f_mode;             // режим доступа
5     atomic_long_t f_count;       // счётчик ссылок на файл
6     loff_t f_pos;               // логическая позиция в файле
7     ...
8     struct path f_path;         // связь с dentry и ФС
9     struct inode *f_inode;       // физический файл
10    const struct file_operations *f_op; // операции на открытом файле
11    ...
12 }
```

Листинг 73. Структура file (сокр.)

- `f_lock` — монопольный доступ к файлу; `f_mode` — режим доступа; `f_count` — счётчик ссылок на файл.
- `f_pos` — логическая позиция в файле: любой файл считает, что начинается с нулевого смещения, каждая операция чтения/записи изменяет это поле на соответствующее количество байт.
- `f_path` — связь с элементом пути (`dentry`) и смонтированной ФС; `f_inode` — указатель на `inode` (физический файл); `f_op` — операции, определённые на открытом файле.

Связь структур строится относительно процесса (именно процессы открывают файлы): через `task_struct` по файловому дескриптору находится `struct file`; его поле `f_path/f_inode` ведёт к `dentry` и `inode`, а из `inode` (поле `i_sb`) и со стороны монтирования — к `super_block` смонтированной ФС.

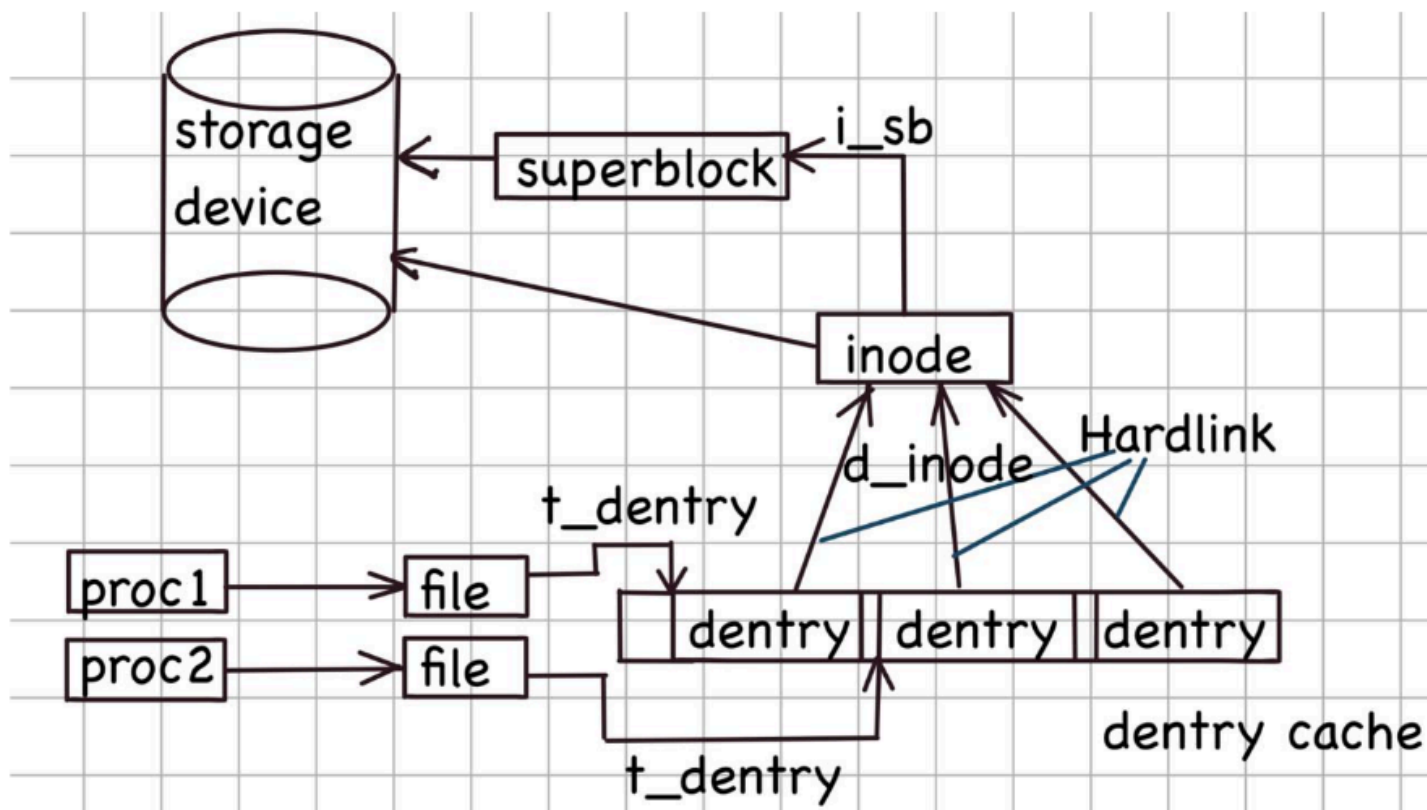


Рис. 43. Связи структур

13.7 Адресация файлов большого размера в файловой системе extX и пример, показывающий доступ к файлу `/usr/ast/mbox`

Адресное пространство диска делится на разделы, один раздел занимает одна ФС.

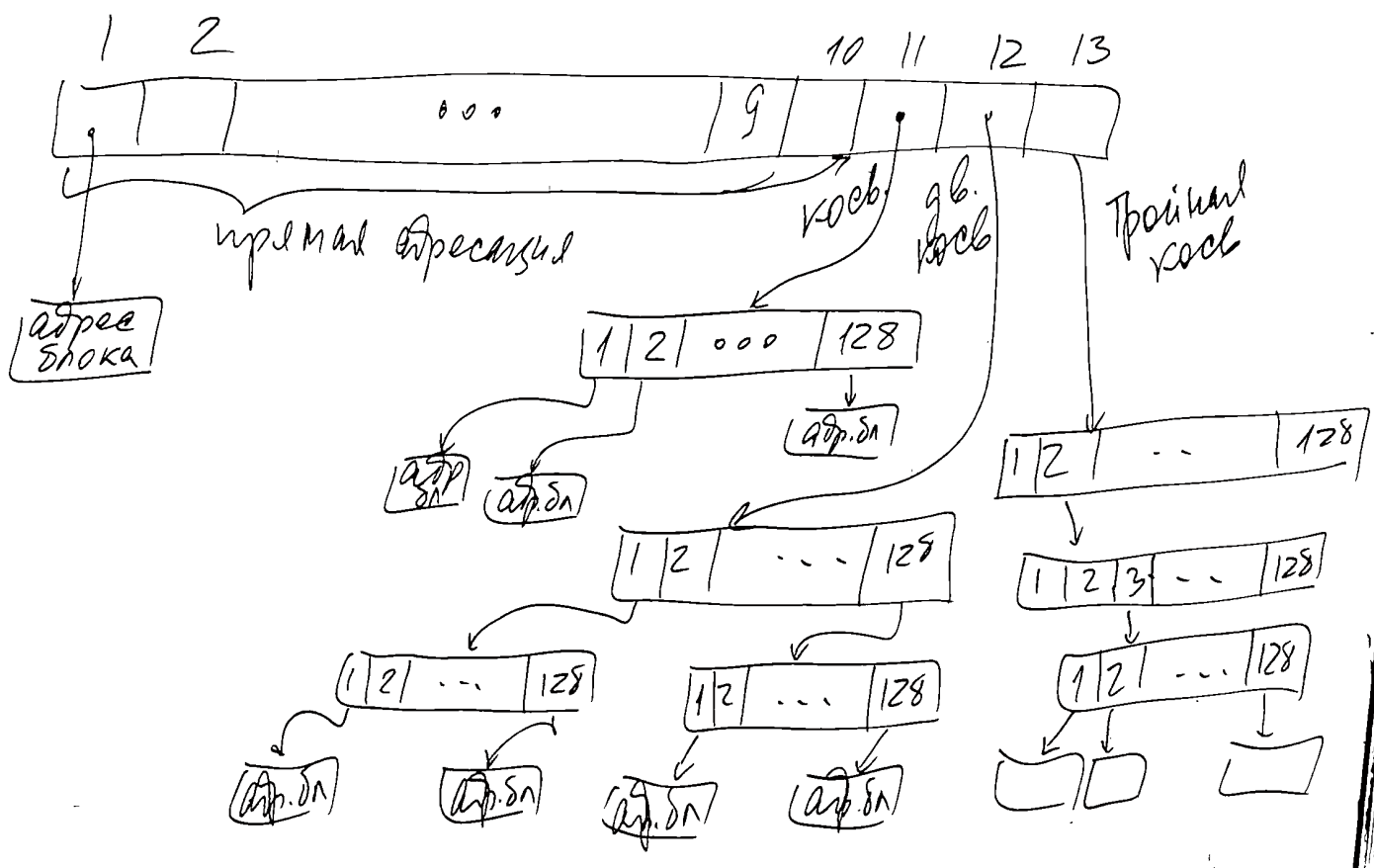


Рис. 45. Адресация файла на жёстком диске

Логический и физический уровни ФС связывает драйвер дискового устройства, переводящий смещения в логическом представлении файла в физические адреса; `struct inode` содержит адреса блоков с данными файла. Адресуемая единица файла — блок.

Каждый блок по 512 байт. С помощью третьего уровня (тройной косвенной адресации) появляется возможность обращаться к $128^3 = 2097152$ блокам. Таким образом, в файловую систему Unix была заложена возможность создавать и работать с очень большими файлами.

Файловая система ext2 (Second Extended File System) была написана так, чтобы обеспечить доступ к хранению очень больших файлов:

- в дисковом `inode` 12 блоков адресуются прямо — `inode` непосредственно содержит адреса 12 блоков данных;
- далее используются косвенная, двойная косвенная и тройная косвенная адресация: блок, содержащий адреса блоков информации, имеет размер 4 Кб (страница), и при размере указателя 4 байта в нём помещается 1024 адреса ($1024 \cdot 4 \text{ байта} = 4 \text{ Кб}$);

- чтобы обратиться к блоку данных через косвенную адресацию, нужно многократно обращаться к диску за получением информации; это не делается аппаратно, а выполняется драйвером, поэтому время кратно увеличивается.

13.7.1 Пример доступа к файлу /usr/ast/mbox

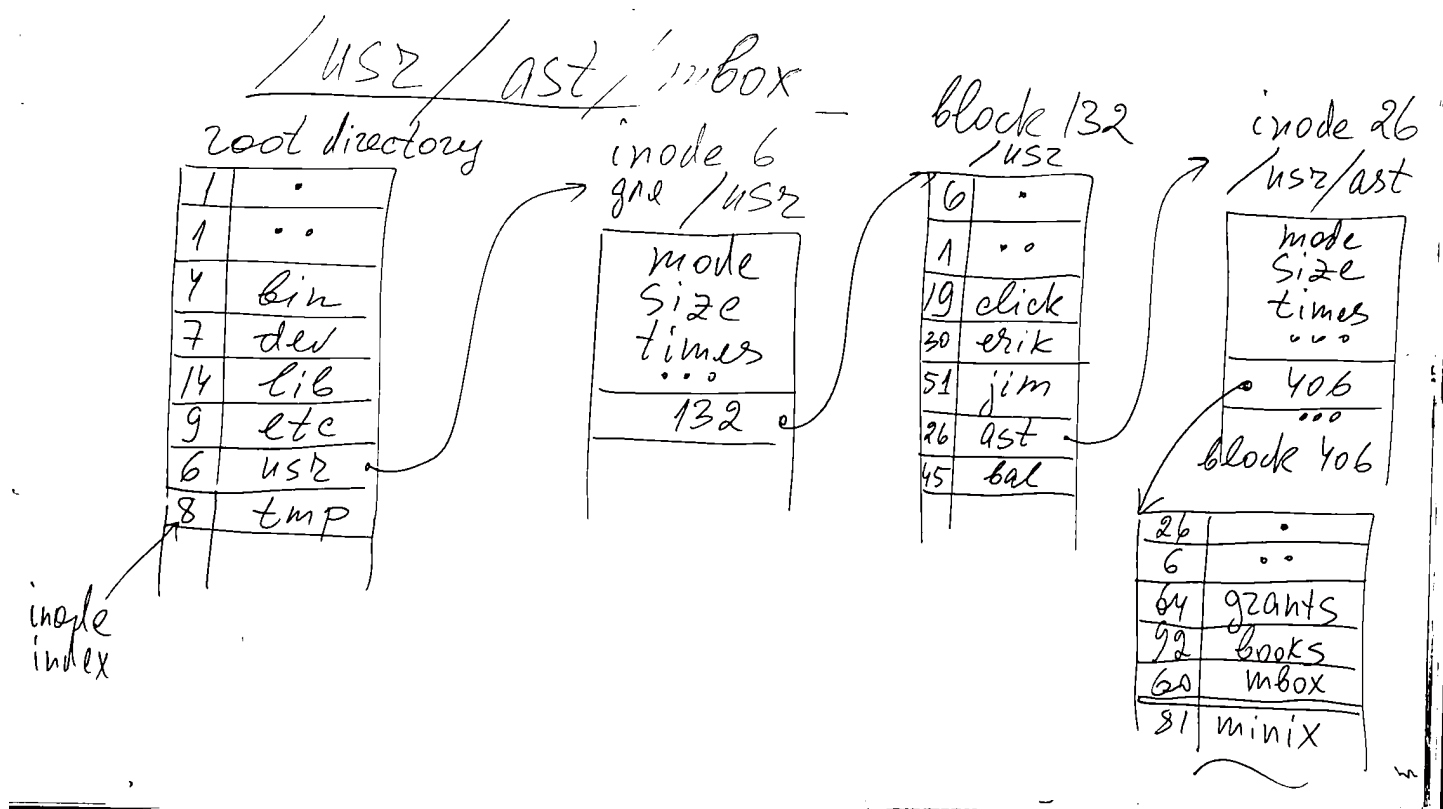
Каталоги хранятся на диске аналогично файлам. Содержимое каталога — таблица, сопоставляющая имена файлов и подкаталогов с номерами их inode:

- В таблице видим имена файлов и папок (folder), видим номера inode.
- number — более правильное слово, потому что index — это всегда смещение.
- Структура inode директории состоит из имён, сопоставленных с inode.

inode number 3470036	
.	3470036
..	3470017
folder1	3470031
file1	3470043

Таблица 8. Структура inode каталога

Поиск файла всегда начинается с корневого каталога (для текущей директории её часть пути уже пройдена).



Доступ к файлу /usr/ast/mbox (в пути 4 объекта dentry: /, usr, ast, mbox):

- 1) Путь начинается с корневого каталога (root directory); его содержимое известно системе.
- 2) В корневом каталоге ищется элемент пути usr — ему сопоставлен inode с номером 6.

- 3) Поля `struct inode` для данного каталога (`mode`, `size`, `times`, ...) позволяют идти дальше: нас интересует адресная часть — в `inode` 6 указано, что информация каталога находится в блоке 132. Обращаемся к диску: блок 132 — это содержимое директории `/usr`.
- 4) В блоке 132 ищется элемент `ast` — ему сопоставлен `inode` с номером 26 (это структура для `/usr/ast`). В этой структуре поля одни и те же, они не меняются в зависимости от того, чей это `inode`.
- 5) Из `inode` 26 берётся адрес блока 406 — это содержимое директории `/usr/ast`.
- 6) В блоке 406 ищется `mbox` — ему сопоставлен `inode` с номером 60; из этого `inode` берутся адреса блоков, в которых хранятся данные файла `mbox`.

Пример показывает, как много действий (обращений к диску) нужно выполнить для обращения к конкретному файлу, поэтому кеширование каталогов и файловых индексов позволяет сократить это время обращения.

13.8 Кеширование `dentry` и `inode`. Slab кэш

13.8.1 Кэш `dentry`

Структуры `dentry` хранятся в кеше. Когда происходит первое обращение к элементу пути, создаётся объект `dentry` и помещается в кеш. При обращении к файлу система проходит по элементам пути и ищет их в `dentry`-кеше; если `dentry` не найден, обращение идёт к диску (см. пример с `mbox` выше).

Действительный объект `dentry` может быть в одном из трёх состояний (`used`, `unused`, `negative`); им соответствуют три части `dentry`-кеша:

- 1) **Список используемых** (`used`): у `dentry` есть `inode` (`d_inode`), счётчик `d_count > 0`; объект не удаляется, пока `d_count` не 0.
- 2) **Список неиспользуемых** (`unused`) — двусвязный LRU-список (`dentry_lru`), сортируется по времени последнего обращения; при нехватке памяти удаляются «хвостовые» объекты.

Состояние `negative` — элемент пути с несуществующим `inode`: кеширует факт отсутствия файла и не удаляется сразу, чтобы повторное обращение (например, повторный `ls` несуществующего файла) не шло к диску.

- 3) **Хеш-таблица** (`dentry_hashtable`) и **функция хеширования** быстро преобразуют путь в `dentry`; хеширование — `d_hash()`, поиск — `d_lookup()`.

Доступ к файлам выполняется «в пространстве»: программы обращаются к разным файлам одного каталога, поэтому кеширование всех элементов пути одного файла сокращает время доступа и к соседним файлам этого каталога. В примере с `mbox` получение корневого индексного дескриптора упрощается: он кеширован уже при

монтировании корневого каталога. Сама dentry не содержит номера индексного дескриптора — он загружается отдельно.

13.8.2 Кэш inode

Кэш inode:

- 1) **Хеш-массив** `inode_hashtable` — inode хешируется по указателю на суперблок и номеру inode.
- 2) **Список** `inode_in_use` — допустимые inode с `i_count > 0` и `i_nlink > 0` (туда `new_inode` добавляет созданный inode).
- 3) **Список** `inode_unused` — допустимые inode с `i_count == 0`.
- 4) **Для каждого суперблока** — **список** `sb->s_dirty` — модифицированные («грязные», `I_DIRTY`) inode с `i_count > 0`; уменьшает накладные расходы на синхронизацию с диском.
- 5) `inode_cached` — slab-кэш.

Кеши взаимодействуют: inode, на которые ссылаются неиспользуемые объекты dentry, не удаляются, пока на них ссылается dentry-кэш.

13.8.3 Slab кэш

Slab — «брусочек». Ядро оперирует ограниченным числом объектов (структур), для каждого нужно выделить память и проинициализировать поля, что занимает время. Если удалять объект после использования, для следующего такого же объекта придётся проделать ту же работу заново.

Идея: не удалять объекты, а сохранять их в slab для повторного использования. Под разные структуры нужны разные кешы — это и реализует Slab-кэш. Каждый кэш хранит объекты одного размера (больше или меньше страницы); есть слабы и для часто используемых размеров буферов (16, 32, 64, ...). Все кешы видны через `cat /proc/slabinfo`.

Важно

Для каждого типа данных создается свой SLAB кэш. Не один на всех, а **для каждого**.

Побочный эффект слабов — сокращение фрагментации памяти.

С версии ядра 2.6.23 SLAB переименован в SLUB (есть ещё SLOB). Это только код ядра: из `userspace` слаб недоступен — память выделяется страницами или слабами уже в ядре.

Имеются структуры `kmem_cache`; каждый кэш представляется такой структурой, а кешы связаны друг с другом в список `cache_chain` (цепочка кешей):

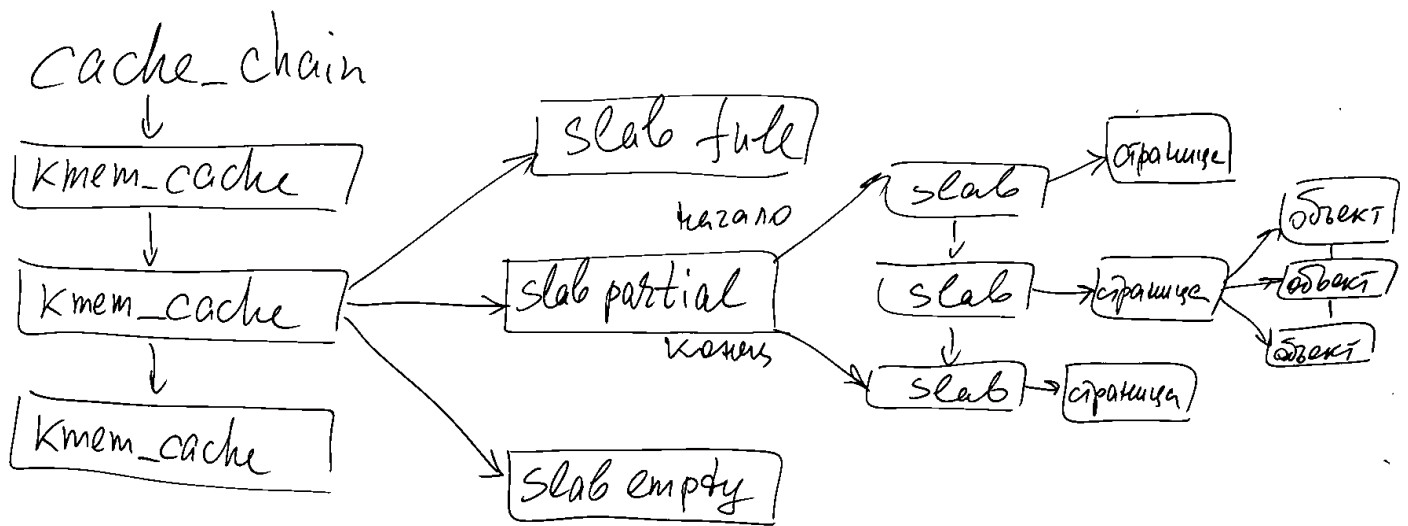


Рис. 47. cache_chain

Каждый slab может находиться в одном из трёх состояний:

- **slab full** — полностью распределённый: нет свободных объектов, все распределены и используются;
- **slab partial** — частично распределённый: есть как выделенные, так и свободные объекты;
- **slab empty** — пустой: нет ни одного распределённого объекта, все объекты свободны для последующего использования.

Выделение: новый объект берётся из частично распределённого слаба, иначе из пустого; если пустых нет, слабы создаются по мере необходимости. Побочный эффект — уменьшение фрагментации памяти.

```

1 /* Создание кэша */
2 struct kmem_cache *kmem_cache_create(const char *name, size_t size, size_t offset,
3                                     unsigned long flags, void (*constructor)
4                                     (void *));
5
6 /* Уничтожение кэша */
7 void kmem_cache_destroy(struct kmem_cache *cache);
8
9 /* Выделить объект из кэша */
10 void *kmem_cache_alloc(struct kmem_cache *cache, gfp_t flags);
11
12 /* Удалить объект из кэша */
13 void kmem_cache_free(struct kmem_cache *cache, void *obj);

```

Листинг 74. Функции для работы с кешем SLAB

Кеш SLAB напрямую используется подсистемой VFS: объекты dentry создаются и уничтожаются вызовами `kmem_cache_alloc()` и `kmem_cache_free()`, а внутри `new_inode` виден аллокатор — `inode` аллоцируется в slab-кеше.

13.9 Монтирование файловых систем. Команда mount и функции монтирования, пример из лаб раб

Дерево файлов и каталогов в Linux формируется из ветвей, соответствующих разным носителям (разным ФС), но это одно большое дерево; некоторые каталоги могут быть точками монтирования.

Монтирование — система действий, делающая ФС (и все её файлы) доступной. Точка монтирования — каталог, к которому подмонтируется ФС: файлы и директории монтируемой ФС становятся файлами и поддиректориями точки монтирования, а её прежние файлы скрываются до размонтирования (поэтому лучше использовать пустые каталоги в качестве точки монтирования). Доступ к файлам ФС возможен только после монтирования. В Unix/Linux всегда есть корневая ФС (root fs); её монтирование — часть инициализации системы.

Когда ФС монтируется, создаётся структура `struct vfsmount`, которая представляет конкретный экземпляр смонтированной ФС:

```
1 struct vfsmount {
2     struct dentry      *mnt_root; /* root of the mounted tree */
3     struct super_block *mnt_sb;   /* pointer to superblock */
4     int                 mnt_flags;
5     struct mnt_idmap    *mnt_idmap;
6 };
```

Листинг 75. Структура `vfsmount`

`mnt_root` — корень подмонтированного дерева, `mnt_sb` — указатель на суперблок, `mnt_flags` — флаги монтирования.

13.9.1 Команда mount

Команда `mount` имеет следующий синтаксис:

```
1 mount <устройство> <точка монтирования>
```

Ключи: `-t <тип ФС>` `-o <опции ФС>`, то есть полная форма:

```
1 mount -t type device dir
```

Точка монтирования — любой каталог; в самой ФС нужно создать корневой каталог (`root`) — именно он монтируется. Ключ `-t` необязателен: система сама может определить тип ФС. После монтирования каталоги и файлы ФС доступны. Обычно монтирование требует прав суперпользователя.

С помощью команды

```
1 mount [-l] [-t type]
```

можно получить список смонтированных файловых систем типа `type`. Размонтируется ФС командой `umount <точка монтирования>`.

13.9.2 Функции монтирования

В ядре определено несколько функций `mount()`; все они имеют модификатор `extern` и возвращают значение типа `struct dentry*`. В версии ядра 6.17.13 определено только две:

```
1 extern struct dentry *mount_bdev(struct file_system_type *fs_type, int flags,  
2                               const char *dev_name, void *data, int (*fill_super)  
3                               (struct super_block *, void *, int));  
4 extern struct dentry *mount_nodev(struct file_system_type *fs_type, int flags,  
5                                  void *data, int (*fill_super)  
6                                  (struct super_block *, void *, int));
```

Параметры: тип файловой системы (`fs_type`), флаги монтирования (`flags`), имя устройства (`dev_name`, только у `mount_bdev`), произвольные данные монтирования (`data`) и указатель на функцию `fill_super`. Внешнее отличие — в параметре `const char *dev_name`, которого нет у `mount_nodev`:

- `mount_bdev` — для монтирования ФС, находящейся на блочном устройстве;
- `mount_nodev` — для монтирования ФС, не связанной ни с каким устройством.

Функциям монтирования передаётся функция `fill_super` — она выполняет основные действия для монтирования: инициализирует поля `struct super_block` (размер блока, смещение, магическое число, `super_operations`). Функции монтирования возвращают объект `dentry`, которым должен быть `root`: для ФС нужно создать корневой каталог, а для этого — `inode`. В системе существует готовая функция `simple_fill_super`, которую находим в коде ядра и используем как основу для собственной `fill_super`.

Для создания `root` надо проинициализировать `inode`, выделив для него память функцией `new_inode()` (внутри неё виден аллокатор — `inode` аллоцируется в `slab`-кеше):

```
1 struct inode *new_inode(struct super_block *sb) {  
2     struct inode *inode = new_inode_pseudo(sb); // -> alloc_inode(sb): выделение  
3     if (inode)                                     из slab  
4         inode_sb_list_add(inode);  
5     return inode;  
6 }
```

Когда файловая система отмонтируется, всё должно быть удалено — этим занимается функция `kill_sb` из `file_system_type`. В ядре для этого определены функции:

```
1 void generic_shutdown_super(struct super_block *sb);
2 void kill_block_super(struct super_block *sb);
3 void kill_anon_super(struct super_block *sb);
4 void kill_litter_super(struct super_block *sb);
```

Общую работу по удалению суперблока выполняет `generic_shutdown_super`.

13.9.3 Пример из лабораторной работы (ЛР VFS)

В ЛР создаётся собственная (виртуальная) ФС в загружаемом модуле ядра; по отношению к VFS выполняются два действия:

- 1) **Регистрация ФС:** объявляем свой `struct file_system_type` (минимально `owner`, `name`, плюс функции `mount` и `kill_sb`); в `init` вызываем `register_filesystem`, в `exit` — `unregister_filesystem`.
- 2) **Монтирование ФС:** в своей функции `mount` вызываем `mount_nodev` (ФС не связана с устройством), передавая ей свою `fill_super` (на основе `simple_fill_super`): она инициализирует поля `struct super_block`, через `new_inode` создаёт `inode` корневого каталога и создаёт `root` (`d_make_root`). Освобождение суперблока выполняет своя функция `kill_sb`.

После загрузки модуля ФС монтируется командой `mount` (не менее 2 раз — одна и та же ФС может быть смонтирована много раз) и показывается двумя способами: в дереве каталогов (`ls -ial`) и через `proc` (`cat /proc/mounts`; зарегистрированный тип ФС — через `cat /proc/filesystems`). Кроме того, в задании создаётся свой SLUB-кеш: объявляем свою структуру, создаём для неё кеш, через `alloc` получаем указатель на `slab` и выделяем объекты.

```
1 static int my_simple_fill_super(struct super_block *s, void *data, int silent)
2 {
3     struct inode *inode;
4     s->s_blocksize = PAGE_SIZE;
5     s->s_magic = MY_MAGIC_NUMBER;
6     s->s_op = &myfs_super_ops;
7
8     inode = new_inode(s);                // inode корневого каталога
9     inode->i_ino = 1;
10    inode->i_mode = S_IFDIR | 0755;
11    inode->i_op = &my_simple_dir_inode_operations;
12    inode->i_fop = &my_simple_dir_operations; // операции на файле
13    set_nlink(inode, 2);
```

```

14  s->s_root = d_make_root(inode);           // создание root
15  return s->s_root ? 0 : -ENOMEM;
16  }

```

Листинг 77. Функция fill_super собственной ФС (сокр.)

```

1  static struct dentry *myfs_mount(struct file_system_type *type, int flags,
2                                  char const *dev, void *data)
3  {
4      return mount_nodev(type, flags, data, my_simple_fill_super);
5  }
6
7  static struct file_system_type myfs_type = {
8      .owner    = THIS_MODULE,
9      .name     = "my_fs",
10     .mount    = myfs_mount,
11     .kill_sb  = my_kill_block_super,
12 };
13
14 static int __init my_init(void) { return register_filesystem(&myfs_type); }
15 static void __exit my_exit(void) { unregister_filesystem(&myfs_type); }
16 module_init(my_init);
17 module_exit(my_exit);

```

Листинг 78. Функции mount, регистрация и deregистрация собственной ФС (сокр.)

```

1  struct myfs_inode {
2      int i_mode;
3      unsigned long i_ino;
4  };
5
6  static struct kmem_cache* inode_cache = NULL;
7  static struct myfs_inode** inode_cached_array = NULL;
8
9  static struct dentry *my_fs_mount(struct file_system_type *type, int flags,
10                                   const char *dev, void *data)
11  {
12      ...
13      for (i = 0; i < OBJECTS_PER_MOUNT; i++)
14      {
15          struct myfs_inode *inode = kmem_cache_alloc(inode_cache, GFP_KERNEL);
16          if (!inode)
17          {
18              for (int j = 0; j < i; j++)
19                  kmem_cache_free(inode_cache, inode_cached_array[j]);
20              return ERR_PTR(-ENOMEM);

```

```

21     }
22     ...
23 }
24 ...
25 }
26
27 static int __init myfs_init(void) {
28     ...
29     // Создание slab-кэша в init
30     inode_cache = kmem_cache_create(SLAB_NAME, sizeof(struct myfs_inode), 0, 0, NULL);
31     if (inode_cache == NULL)
32         return -ENOMEM;
33     ...
34 }
35
36 static void __exit myfs_exit(void) {
37     ...
38     // освобождение всех объектов
39     for (i = 0; i < cached_count; i++)
40         kmem_cache_free(inode_cache, inode_cached_array[i]);
41     // удаление slab-кэша
42     kmem_cache_destroy(inode_cache);
43     ...
44 }

```

Листинг 79. Создание и использование своего SLAB-кэша в ЛР

14

14 Файловая подсистема /proc — назначение, особенности, файлы, поддиректории, ссылка self, информация об окружении, состоянии процесса, прерываниях. Структура proc_dir_entry: функции для работы с элементами /proc. Структура, перечисляющая функции, определенные на файлах. Использование структуры file_operations для регистрации собственных функций работы с файлами. Передача данных из пространства пользователя в пространство ядра и из ядра в пространство пользователя. Обоснование необходимости этих функций. Функция printk() — назначение и особенности. Пример программы «Фортуны» из лаб. работы.

14.1 Файловая подсистема /proc: назначение, особенности, файлы, поддиректории, ссылка self, информация об окружении, состоянии процесса, прерываниях

Виртуальная файловая подсистема /proc (process information pseudo-file-system) позволяет в режиме пользователя получить большой объем информации о ресурсах процесса. Виртуальная — то есть не существующая на самом деле (не хранящаяся во вторичной памяти), а создающаяся “на лету” по мере обращения к данным. По сути интерфейс /proc позволяет получить информацию из структур ядра, не обращаясь к ним напрямую, а потому работая в режиме пользователя. Большинство файлов доступны только для чтения, однако некоторые доступны для записи, что позволяет изменять переменные ядра.

файловая система /proc является немонтируемой. Под этим подразумевается, что она не связана ни с каким внешним носителем (она видна в файловых системах, подключенных к корневому каталогу, но не связана ни с каким внешним устройством. Поэтому она и называется псевдофайловой системой).

Для каждого процесса в /proc определена директория идентификатора процесса (/proc/<pid>/), она будет создана налету. В ней хранятся файлы, директории и символические ссылки, используя которые можно получить развёрнутую информацию о конкретном процессе. Вся информация доступна из usermode:

Элемент	Тип	Содержание
cmdline	Файл	holds the complete command line Содержит командную строку процесса
cwd	Символическая ссылка	Указывает на рабочую директорию процесса
environ	Файл	Содержит список переменных окружения процесса
exe	Символическая ссылка	Указывает на образ процесса (на его файл) Символическая ссылка, содержащая фактический путь к выполняемой команде.

Элемент	Тип	Содержание
comm	Файл	Имя команды, связанное с процессом. Это именно то имя файла, которое приведёт к выполнению нашего кода. Замечание 14.1 Разные потоки в одном и том же процессе могут иметь разные значения, доступные через /proc/<pid>/task/<tid>/comm. task — файл, в котором находятся поддиректории потоков.
stat	Файл	Содержит информацию о процессе (vsize, start_brk и др.)
fd	Директория	Содержит ссылки на открытые процессом файлы Замечание 14.2 В системе различаются файлы и открытые файлы. Открытые файлы описываются структурой struct file. Просто файлы представляются в системе деревом каталогов, а описываются физические файлы struct inode. Один и тот же файл может быть открыт много раз, причём разными процессами. В таблице открытых файлов на каждый такой открытый файл будет дескриптор struct_file
root	soft link	Это символическая ссылка, которая указывает на корень файловой системы процесса. Замечание 14.3 Процессы, которые выполняются на нашем компьютере, могут принадлежать разным файловым системам. Пока мы имя исполняемого файла не указали в командной строке, пока он не начал выполняться, он лежит во вторичной памяти.
maps	Файл	Это адресное пространство того процесса, pid которого мы указали. В зависимости от pid информация будет отличаться, но есть неизменные составляющие:
pagemap	файл	Это 64-х разрядное значение для каждой строки процесса. При этом нас интересует фактически несколько бит: 1) 63 бит — present (in RAM) 2) 62 бит — swap space, область своппинга 3) 61 — shared anonymous page

Элемент	Тип	Содержание
		4) 55 бит — dirty. Он указывается в дескрипторе страницы. С 54 по 0: если страница присутствует в физической памяти, то эти биты являются номером фрейма, то есть номером физической страницы.
task	Директория	Содержит поддиректории потоков
interrupts	Файл	Предоставляет таблицу о количестве прерываний на каждом из процессоров в следующем виде: 1) Первая колонка: номер прерывания 2) Колонки CPUx: счётчики прерываний на каждом из процессоров 3) Следующая колонка: вид прерывания: 3.1) IO-APIC-edge — прерывание по фронту на контроллер I/O APIC. 3.2) IO-APIC-fastedge — прерывание по уровню на контроллер I/O APIC. 3.3) PCI-MSI-edge — MSI прерывание. 3.4) XT-PIC-XT-PIC — прерывание на PIC контроллер. 4) Последняя колонка: устройство, ассоциированное с данным прерыванием.

Таблица 9. Содержание директории /proc/<pid>

14.1.1 Ссылка self

/proc/self — символическая ссылка, существующая для каждого процесса и указывающая на его собственную директорию /proc/<pid>; позволяет процессу обращаться к своим данным, не вызывая getpid().

14.1.2 Информация об окружении

Окружение несёт файл environ (для текущего процесса — /proc/self/environ): в нём исходное окружение, установленное при запуске (execve()), переменные разделены нулевыми символами ('\0'), на строки файл не разбит. Изменения окружения после execve() (например, putenv()) в нём не отражаются.

14.1.3 Информация о состоянии процесса

Информацию о состоянии процесса несёт файл stat — важную, подобранную разработчиками информацию о процессе: его идентификатор pid, состояние state,

имя команды `comm`, идентификатор родителя `ppid`, приоритет `prio`, размер виртуального адресного пространства `vsize`, адрес начала кучи `start_brk` и т.д.

14.1.4 Информация о прерываниях

Файл `/proc/interrupts` несёт информацию о прерываниях на каждом CPU с указанием их источника. Формат записи:

- номер прерывания;
- колонки CPUх — счётчики прерываний на каждом процессоре;
- вид прерывания (IO-APIC-edge/-level, PCI-MSI-edge, XT-PIC и т.д.);
- устройство, ассоциированное с прерыванием.

Кроме `/proc/interrupts`, информацию о системе в целом дают, например, `/proc/cpuinfo`, `/proc/vmstat`, `/proc/softirqs`.

14.2 Структура `proc_dir_entry`: функции для работы с элементами `/proc`

Все файлы и поддиректории файловой системы `proc` описываются структурой `struct proc_dir_entry` — она фактически является дескриптором файла `proc`. Поскольку `proc` — виртуальная ФС, данные не хранятся во вторичной памяти, но `inode` у любого файла, в том числе виртуального, всё равно имеется.

```
1 struct proc_dir_entry {
2     // ...
3     const struct inode_operations *proc_iops;
4     union {
5         const struct proc_ops *proc_ops;
6         const struct file_operations *proc_dir_ops;
7     };
8     union {
9         const struct seq_operations *seq_ops;
10        int (*single_show)(struct seq_file *, void *);
11    };
12    proc_write_t write;
13    void *data;
14    struct proc_dir_entry *parent;
15    char *name;
16    // ...
17 };
```

Листинг 80. Структура `struct proc_dir_entry`

- `proc_iops` — у файла `proc` есть `inode` и определённые на нём `inode_operations`.
- `union (1)`: до 6-й версии ядра была только `file_operations *proc_dir_ops`; рядом введена `proc_ops` с почти теми же функциями (зачем — см. регистрацию).

- union (2): два интерфейса с user-mode — полный seq_operations *seq_ops и упрощённый single_show.
- write — передача из user-mode в kernel-mode (обратно — read); data — приватные данные файла; parent — древовидная структура proc.

Функции для работы с элементами /proc:

- proc_mkdir — создаёт поддиректорию в /proc;
- proc_create_data (и обёртка proc_create) — создаёт виртуальный файл и регистрирует на нём свои функции (подробнее — в подразделе про регистрацию);
- remove_proc_entry — удаляет элемент /proc (вызывается при выгрузке модуля).

14.3 Структура, перечисляющая функции, определённые на файлах

В struct file (дескрипторе открытого файла) поле f_op указывает на структуру file_operations, в которой перечислены функции, определённые на открытом файле:

```

1 struct file_operations {
2     struct module *owner;
3     ...
4     ssize_t (*read)    (struct file *, char __user *, size_t, loff_t *);
5     ssize_t (*write)   (struct file *, const char __user *, size_t, loff_t *);
6     int (*open)        (struct inode *, struct file *);
7     int (*release)     (struct inode *, struct file *);
8     ...
9 };

```

Листинг 81. Структура file_operations

Важно

Здесь перечисляются функции ядра. struct file — структура ядра. Разработчики называют функции ядра аналогично функциям стандартных библиотек, но это **функция ядра**. Поэтому, например, в функции read присутствует указатель __user * на буфер пользователя.

- Для struct file_operations существует поле owner (владелец).
- В перечне функций ядра close отсутствует — закрытию соответствует release.

14.4 Использование структуры file_operations для регистрации собственных функций работы с файлами

Регистрация функций выполняется инициализацией структуры назначенными инициализаторами (синтаксис C99):

```

1 struct file_operations fops = {
2     .read      = device_read,
3     .write     = device_write,
4     .open      = device_open,
5     .release   = device_release
6 };

```

`struct file_operations` переписать нельзя, а её функции вызывают все драйверы. Чтобы её разгрузить, для работы с ФС `proc` введена `struct proc_ops`, функции которой повторяют функции `file_operations`:

```

1 struct proc_ops {
2     ...
3     int      (*proc_open)(struct inode *, struct file *);
4     ssize_t  (*proc_read)(struct file *, char __user *, size_t, loff_t *);
5     ssize_t  (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
6     int      (*proc_release)(struct inode *, struct file *);
7     ...
8 };

```

Листинг 82. Структура `proc_ops`

Свои функции на файле `proc` регистрируют, инициализируя `struct proc_ops` и передавая её функции создания виртуального файла `proc_create_data` (или обёртке `proc_create`, передающей `NULL` в поле приватных данных):

```

1 struct proc_dir_entry *proc_create_data(const char *name, umode_t mode,
2     struct proc_dir_entry *parent, const struct proc_ops *proc_ops, void *data);
3 struct proc_dir_entry *proc_create(const char *name, umode_t mode,
4     struct proc_dir_entry *parent, const struct proc_ops *proc_ops);

```

- `proc_ops` — указатель на проинициализированную структуру; `proc_dir_entry` — дескриптор файла `proc`.
- `parent` задаёт иерархию: для файла в поддиректории её предварительно создают через `proc_mkdir`; при `parent == NULL` файл создаётся в корне `proc`.

14.5 Передача данных из пространства пользователя в пространство ядра и из ядра в пространство пользователя.

Обоснование необходимости этих функций

В своих функциях `read`, `write` нужно вызывать специальные функции ядра для передачи данных из пространства пользователя в пространство ядра и обратно — это `copy_from_user` и `copy_to_user`.

Специальные функции нужны, потому что ядро (и модуль ядра как его часть) оперирует физическими адресами: оно резидентно в физической памяти и не выгружается, а процессы имеют лишь виртуальные адресные пространства и получают физические страницы по мере надобности.

Чтобы передать данные из user в kernel или обратно, ядро должно обратиться к буферу пространства пользователя. Вероятна ситуация, когда буфер (страница), к которому обращается ядро, выгружен — тогда возникнет **страничное прерывание**, которое должно обработать само ядро. Получается, что ядро обратилось, само стало источником прерывания и само же должно его обработать; если не предпринять специальных мер, возникнет паника ядра. Для этого и написаны две функции:

- `copy_to_user` — копирует данные из буфера ядра в буфер пользователя;
- `copy_from_user` — копирует данные из буфера пользователя в буфер ядра.

```
1 static inline __must_check unsigned long
2 _copy_to_user(void __user *to, const void *from, unsigned long n)
3 {
4     might_fault();
5     if (access_ok(to, n))
6         n = raw_copy_to_user(to, from, n);
7     return n;
8 }
```

Листинг 83. Определение функции `copy_to_user`

- Указатель `to` (`void __user *`) — указатель на буфер пользователя, `const from` — указатель на буфер ядра. Для функции `copy_from_user` ситуация обратная.

```
1 static inline __must_check unsigned long
2 _copy_from_user(void *to, const void __user *from, unsigned long n)
3 {
4     ...
5 }
```

Листинг 84. Определение функции `copy_from_user`

14.6 Функция `printk()` — назначение и особенности

`printk` (kernel) — функция ядра, не входящая ни в одну библиотеку (в ядре нельзя использовать стандартную библиотеку Си — там свои функции). Она помещает выходные данные в кольцевой буфер (журнал) ядра; демон `syslog` в пространстве пользователя забирает их и перенаправляет на устройства согласно файлу конфигурации (`/etc/syslog.conf`).

В вызовах `printk` используются макросы уровней протоколирования (определены в `linux/kern_levels.h`):

```
1 #define KERN_EMERG    "<0>" /* system is unusable */
2 #define KERN_ALERT    "<1>" /* action must be taken immediately */
3 #define KERN_CRIT     "<2>" /* critical conditions */
4 #define KERN_ERR      "<3>" /* error conditions */
5 #define KERN_WARNING  "<4>" /* warning conditions */
6 #define KERN_NOTICE   "<5>" /* normal but significant condition */
7 #define KERN_INFO     "<6>" /* informational */
8 #define KERN_DEBUG    "<7>" /* debug-level messages */
```

Листинг 85.

- Чем меньше значение, тем выше приоритет протоколирования.
- Чаще всего используется `KERN_INFO`.

Пример 14.4

```
1 printk(KERN_ERR "root creation failed");
```

Важно

Для нас привычно ставить запятую. `KERN_ERR` — это строковая константа, которая объединена со строкой, и между ними запятая не ставится. Это относится к любому макросу.

Обычно сообщения журналируются в `/var/log/messages`, но могут перенаправляться и на пользовательский порт. Для просмотра кольцевого буфера ядро предоставляет утилиту `dmesg`, выводящую дамп буфера в стандартный поток.

14.7 Пример программы «Фортунки» из лаб. работы

ЛР посвящена передаче данных между пространством пользователя и ядром; работа с файлом ФС `proc` ведётся в загружаемом модуле ядра. В модуле 6 точек входа: `open`, `read`, `write`, `release`, `init`, `exit`. Функции `open/release` вызываются, когда процессу нужно работать с файлом (здесь — процессами `echo` и `cat`, явно мы их не вызываем). В `read/write` вызываются `copy_to_user/copy_from_user`, так как ядро работает с физическими адресами, а процессы — с виртуальными.

```
1 #include <linux/init.h>
2 #include <linux/module.h>
3 #include <linux/kernel.h>
4 #include <linux/proc_fs.h>
```

```

5 #include <linux/uaccess.h>
6 #include <linux/vmalloc.h>
7 #include <linux/string.h>
8 #include <linux/errno.h>
9
10 #define PROC_FILE_NAME "fortune"
11 #define PROC_DIR_NAME "fortune_dir"
12 #define PROC_SYM_NAME "fortune_symlink"
13
14 #define COOKIE_BUF_SIZE PAGE_SIZE
15
16 MODULE_LICENSE("GPL");
17 MODULE_AUTHOR("Zhizhin Nikita");
18
19 static char *cookie_buf;
20 static unsigned int write_index;
21 static unsigned int read_index;
22 static struct proc_dir_entry *proc_file;
23 static struct proc_dir_entry *proc_dir;
24 static struct proc_dir_entry *proc_symlink_entry;
25
26 static ssize_t fortune_write(struct file *file, const char __user *user_buf, size_t
count, loff_t *ppos) {
27     size_t len;
28
29     if (count == 0) return 0;
30     if (count > COOKIE_BUF_SIZE - 1) return -ENOSPC;
31     if (write_index + count + 1 > COOKIE_BUF_SIZE) return -ENOSPC;
32     len = count;
33     if (copy_from_user(cookie_buf + write_index, user_buf, len)) {
34         return -EFAULT;
35     }
36     cookie_buf[write_index + len] = '\0';
37     printk(KERN_INFO "fortune: write %s", cookie_buf + write_index);
38     write_index += len + 1;
39     return len;
40 }
41
42 static ssize_t fortune_read(struct file *file, char __user *user_buf, size_t count,
loff_t *ppos) {
43     size_t len;
44     size_t available;
45
46     if (*ppos > 0) return 0;
47     if (count == 0) return 0;

```

```

48     if (write_index == 0) return 0;
49     if (read_index >= write_index) read_index = 0;
50     available = write_index - read_index;
51     len = strlen(cookie_buf + read_index, available);
52     if (len == 0) {
53         read_index = 0;
54         len = strlen(cookie_buf, write_index);
55         if (len == 0) return 0;
56     }
57     if (len > count) len = count;
58     if (copy_to_user(user_buf, cookie_buf + read_index, len)) {
59         return -EFAULT;
60     }
61     printk(KERN_INFO "fortune: read %s", cookie_buf + read_index);
62     read_index += len + 1;
63     if (read_index >= write_index)
64         read_index = 0;
65     *ppos += len;
66     return len;
67 }
68
69 static int fortune_open(struct inode *inode, struct file *file) {
70     printk(KERN_INFO "fortune: open");
71     return 0;
72 }
73
74 static int fortune_release(struct inode *inode, struct file *file) {
75     printk(KERN_INFO "fortune: release");
76     return 0;
77 }
78
79 static const struct proc_ops fortune_proc_ops = {
80     .proc_open = fortune_open,
81     .proc_read = fortune_read,
82     .proc_write = fortune_write,
83     .proc_release = fortune_release,
84 };
85
86 static int __init fortune_init(void) {
87     cookie_buf = vmalloc(COOKIE_BUF_SIZE);
88     if (!cookie_buf) {
89         printk(KERN_ERR "fortune: cannot allocate cookie buffer");
90         return -ENOMEM;
91     }

```

```

92     memset(cookie_buf, 0, COOKIE_BUF_SIZE);
93     write_index = 0;
94     read_index = 0;
95     proc_dir = proc_mkdir(PROC_DIR_NAME, NULL);
96     if (!proc_dir) {
97         printk(KERN_ERR "fortune: cannot create /proc/%s", PROC_DIR_NAME);
98         vfree(cookie_buf);
99         return -ENOMEM;
100    }
101    proc_file = proc_create(PROC_FILE_NAME, 0666, proc_dir, &fortune_proc_ops);
102    if (!proc_file) {
103        printk(KERN_ERR "fortune: cannot create /proc/%s", PROC_FILE_NAME);
104        proc_remove(proc_dir);
105        vfree(cookie_buf);
106        return -ENOMEM;
107    }
108    proc_symlink_entry = proc_symlink(PROC_SYM_NAME, NULL, "/proc/" PROC_DIR_NAME
"/" PROC_FILE_NAME);
109    if (!proc_symlink_entry) {
110        printk(KERN_ERR "fortune: cannot create /proc/%s", PROC_SYM_NAME);
111        proc_remove(proc_dir);
112        proc_remove(proc_file);
113        vfree(cookie_buf);
114        return -ENOMEM;
115    }
116    printk(KERN_INFO "fortune: module loaded");
117    return 0;
118 }
119
120 static void __exit fortune_exit(void) {
121     proc_remove(proc_symlink_entry);
122     proc_remove(proc_file);
123     proc_remove(proc_dir);
124     vfree(cookie_buf);
125     printk(KERN_INFO "fortune: module unloaded");
126 }
127
128 module_init(fortune_init);
129 module_exit(fortune_exit);

```

15

15 Загружаемые модули ядра. Структура загружаемых модулей. Информация о процессах, доступная в ядре: prio, priority, state, flags. Пример вывода информации о запущенных процессах, символ current (лаб. раб.). Взаимодействие загружаемых модулей в ядре. Экспорт данных. Пример взаимодействия модулей (лаб. раб.). Функция printk() — назначение и особенности. Виртуальная файловая система proc: назначение, примеры директорий и файлов /proc, файлы maps, pagemap, tasks. Регистрация функций работы с файлами proc. Пример проинициализированных полей структуры proc_ops. Передача данных из пространства ядра в пространство пользователя. Примеры из лабораторный работ.

15.1 Загружаемые модули ядра

Загружаемые модули ядра позволяют продвинутым пользователям включать в ядро нужную им функциональность; это механизм, специальные команды и специальная структура модуля.

Загружаемый модуль ядра — объект ядра, который может быть динамически загружен в ядро без необходимости его перекомпиляции. Модуль имеет прямой доступ к структурам ядра.

Чтобы загрузить модуль в ядро, то есть выполнить команду `insmod` (`insert module`), нужны права суперпользователя.

Основные команды для работы с модулем ядра:

- `insmod` — загружает модуль; при этом вызывается функция инициализации модуля;
- `rmmod` — выгружает модуль; при этом вызывается функция завершения модуля;
- `lsmod` — выводит список загруженных модулей.

15.2 Структура загружаемых модулей

Обязательные составляющие модуля — функции `init()` и `exit()`, передаваемые макросам `module_init/module_exit`.

```
1 #include <linux/module.h>
2 #include <linux/init.h>
3
4 static int __init md_init(void)
5 {
6     printk("Module loaded!\n");
7     return 0;
8 }
9
10 static void __exit md_exit(void)
11 {
12     printk("Module unloaded\n");
```

```
13 }  
14  
15 module_init(md_init);  
16 module_exit(md_exit);
```

Листинг 86.

- `__` (два подчёркивания) — знак ядру обратить на действие пристальное внимание; современный синтаксис ядра (в старых версиях модуля их нет).
- Это две точки входа модуля.

Точки входа в ядро ОС:

- 1) все системные вызовы;
 - 2) все исключения;
 - 3) все аппаратные прерывания.
- `module_init` регистрирует функцию инициализации (прототип `int init(void)`): она вызывается при загрузке, запрашивает ресурсы (память и т. д.) и при успехе возвращает 0.
 - `module_exit` регистрирует функцию завершения (прототип `void func(void)`): вызывается при удалении модуля и освобождает ресурсы.
 - `MODULE_LICENSE`, `MODULE_AUTHOR`, `MODULE_DESCRIPTION` описывают модуль (видно командой `modinfo`). `MODULE_LICENSE` обязателен: без `MODULE_LICENSE("GPL")` модуль помечается как испорченный (`tainted`), что влияет на доступ к экспортируемым символам ядра.

Для любой функции ядра требуется обработка ошибок: при ошибке возвращается отрицательное значение (минус добавляет макрос из `linux/errno.h`), а при успешном завершении дополнительная информация не предоставляется и возвращается 0.

15.3 Информация о процессах, доступная в ядре: `prio`, `priority`, `state`, `flags`

В загружаемом модуле можно напрямую получить доступ к полям `struct task_struct` с информацией о процессе:

- `state` (в современных ядрах — `__state`) — состояние процесса; при выводе не переводится в строковую константу, печатается так, как определено в ядре.
- `flags` — флаги процесса (`PF_*`):
 - `PF_NO_SETAFFINITY` — из пользовательского пространства запрещено изменять `cpu_mask`;
 - `PF_FORKNOEXEC` — процесс выполнил `fork`, но не может выполнить `exec`;
 - `PF_NOFREEZE` — поток не может быть заблокирован (заморожен);

- PF_WQ_WORKER — рабочий поток очереди работ, например kworker.
- prio, static_prio, normal_prio, rt_priority — приоритеты (rt_priority — реального времени). Чем меньше prio, тем выше приоритет; prio = 0 — наивысший. Отдельного поля priority в структуре нет.

Рядом — поле policy (политика планирования): 1 — FIFO (без переключения, пока процесс не закончится — выполняется «от начала до конца»), 2 — RR (переключение по кванту), 0 — по умолчанию (самый справедливый: чтобы все процессы получили процессорное время по возможности в равных количествах).

15.4 Пример вывода информации о запущенных процессах, символ current

Информацию о запущенных процессах выводит загружаемый модуль, обходящий кольцевой список процессов. Обход начинается со статической структуры init_task (указатель на первый task_struct в кольцевом списке) и продолжается в цикле do-while через next_task, пока снова не встретится init_task.

Символ current — определённый в ядре указатель на struct task_struct текущего процесса. После обхода ту же информацию выводят для текущего процесса (здесь это insmod, загружающий модуль), поэтому строка с insmod выводится дважды: при обходе списка и в блоке Current.

```
1 static int __init md_init(void)
2 {
3     struct task_struct *task = &init_task;
4     do {
5         printk(KERN_INFO "Task: pid: %d, comm: %s, ppid: %d, state: %ld, flags: %x,
6             prio: %d, policy: %d, root: %s, pwd: %s\n",
7             task->pid, task->comm, task->parent->pid,
8             task->state, task->flags, task->prio, task->policy,
9             task->fs->root.dentry->d_iname, task->fs->pwd.dentry->d_name.name);
10    } while ((task = next_task(task)) != &init_task);
11    printk(KERN_INFO "Current: pid: %d, comm: %s, ppid: %d, state: %ld, flags: %x,
12        prio: %d, policy: %d, root: %s, pwd: %s\n",
13        current->pid, current->comm, current->parent->pid,
14        current->state, current->flags, current->prio, current->policy,
15        current->fs->root.dentry->d_iname, current->fs->pwd.dentry->d_name.name);
16    return 0;
17 }
18 module_init(md_init);
19 module_exit(md_exit);
```

Поле `fs` — информация о файловой системе процесса; в листинге через него получают имя корня файловой системы (`fs->root.dentry`) и имя текущего рабочего каталога (`fs->pwd.dentry`).

15.5 Взаимодействие загружаемых модулей в ядре. Экспорт данных

Взаимодействие модулей — один модуль использует данные (переменные и функции), описанные в другом. Чтобы обратиться к данным другого модуля, их описывают так:

- 1) как `extern`, причём объявление видно (через заголовочный файл) в модуле, который обращается;
- 2) имя передаётся в макрос `EXPORT_SYMBOL`.

Экспортируемые символы — данные и функции, доступные в другом модуле. Имена могут быть экспортированы ранее загруженными модулями, но чаще — ядром; множество экспортируемых имён ядра называют API ядра (пример вызова из него — `printk()`).

Модуль связывается с экспортируемым именем по прямому абсолютному (физическому) адресу: ядро оперирует физическими адресами (загружено в физическую память), а процессы — виртуальными. Два следствия:

- любая пересборка ядра или экспортирующих модулей делает собранный модуль непригодным, поэтому модуль собирается только на месте использования;
- важен порядок загрузки: `md2` берёт абсолютный адрес из `md1`, и если `md1` не загружен, адрес взять неоткуда.

Модуль может использовать только те имена, которые явно экспортированы. Локальное имя (модификатор `static`, например `md1_local()`) непригодно для связывания, а имя, не объявленное как экспортируемое (`md1_noexport()`), также не может быть использовано вне модуля.

После загрузки `lsmod` показывает, что модуль используется другим. Число ссылок на `md1` (со стороны других модулей или объектов ядра) — счётчик ссылок, рядом — список ссылающихся модулей. Пока счётчик не нулевой, модуль не выгрузить, поэтому `sudo rmmod md1` при загруженном `md2` даёт ошибку `ERROR: Module md1 is in use by md2`.

15.6 Пример взаимодействия модулей

Возможность взаимодействия рассматривается (по статье Цилюрика) на примере трёх модулей ядра — `md1`, `md2`, `md3` — и общего заголовочного файла `md.h`. Базовый модуль `md1` экспортирует данные, остальные модули их используют.

В заголовочном файле `md.h` экспортируемые имена объявлены как `extern`:

```
1 extern char *md1_data;
2 extern char *md1_proc(void);
3 extern char *md1_noexport(void);
```

```
1 // md1.c
2 MODULE_LICENSE("GPL");
3
4 char *md1_data = "aaaa";
5 extern char *md1_proc(void) { return md1_data; }
6 static char *md1_local(void) { return md1_data; }      // локальная (static)
7 extern char *md1_noexport(void) { return md1_data; }   // не экспортируется
8
9 EXPORT_SYMBOL(md1_data);
10 EXPORT_SYMBOL(md1_proc);
11
12 // + md_init/md_exit и module_init/module_exit
```

```
1 // md2.c
2 #include "md.h"
3 MODULE_LICENSE("GPL");
4
5 static int __init md_init(void) {
6     printk("+ data string exported from md1 : %s\n", md1_data);
7     printk("+ string returned md1_proc() is : %s\n", md1_proc());
8 // md1_local();      (1) - ошибка компиляции
9 // md1_noexport();  (2) - ошибка компоновки
10     return 0;
11 }
12 // + md_exit и module_init/module_exit
```

Модули `md1` и `md2` собираются и взаимодействуют без ошибок. Ошибочные ситуации:

- строка (1) с `md1_local` — ошибка компиляции (несоответствие `md1_local` в `md1.c` и `md.h`);
- строка (2) с `md1_noexport` — ошибка компоновки (`md1_noexport` не передана в `EXPORT_SYMBOL`, нет абсолютного адреса).

`md3` отличается тем, что функция инициализации возвращает `-1` (ошибку):

```
1 #include "md.h"
2 MODULE_LICENSE("GPL");
3
4 static int __init md_init(void) {
```

```

5     printk("+ module md3 start!\n");
6     // ... использует md1_data / md1_proc()
7     return -1;
8 }
9 module_init(md_init);

```

Такой модуль в ядро загружен не будет — будет выдано соответствующее сообщение об ошибке.

Загрузка md2 раньше md1 (`sudo insmod md2.ko`) завершается ошибкой `insmod: error inserting 'md2.ko': -1 Unknown symbol in module`: модуль ссылается на неизвестные ядру имена (хотя они определены в md1). В журнале (`dmesg`) видны `md2: Unknown symbol md1_data` и `md2: Unknown symbol md1_proc`.

15.7 Функция `printk()` — назначение и особенности

`printk (kernel)` — функция ядра, не входящая ни в одну библиотеку (в ядре нельзя использовать стандартную библиотеку Си — там свои функции). Помещает вывод в кольцевой буфер (журнал) ядра; демон `syslog` в пространстве пользователя забирает эти данные и перенаправляет на устройства по файлу конфигурации (`/etc/syslog.conf`).

В вызовах `printk` используются макросы уровней (в `linux/kern_levels.h`); английские комментарии можно опустить:

```

1 #define KERN_EMERG      "<0>"
2 #define KERN_ALERT     "<1>"
3 #define KERN_CRIT      "<2>"
4 #define KERN_ERR       "<3>"
5 #define KERN_WARNING   "<4>"
6 #define KERN_NOTICE    "<5>"
7 #define KERN_INFO      "<6>"
8 #define KERN_DEBUG     "<7>"

```

- Чем меньше значение, тем выше приоритет протоколирования.
- Чаще всего используется `KERN_INFO`.

```

1 printk(KERN_ERR "root creation failed");

```

Важно

Для нас привычно ставить запятую. `KERN_ERR` — это строковая константа, которая объединена со строкой, и между ними запятая не ставится. Это относится к любому макросу.

Обычно сообщения журналируются в `/var/log/messages`, но могут перенаправляться и на пользовательский порт. Для просмотра кольцевого буфера служит утилита `dmesg`, выводящая дамп буфера в стандартный выходной поток.

15.8 Виртуальная файловая система `proc`: назначение, примеры директорий и файлов `/proc`, файлы `maps`, `pagemap`, `tasks`

Виртуальная файловая подсистема `/proc` (process information pseudo-file-system) позволяет в режиме пользователя получить большой объем информации о ресурсах процесса. Виртуальная — то есть не существующая на самом деле (не хранящаяся во вторичной памяти), а создающаяся “на лету” по мере обращения к данным. По сути интерфейс `/proc` позволяет получить информацию из структур ядра, не обращаясь к ним напрямую, а потому работая в режиме пользователя. Большинство файлов доступны только для чтения, однако некоторые доступны для записи, что позволяет изменять переменные ядра.

файловая система `/proc` является немонтируемой. Под этим подразумевается, что она не связана ни с каким внешним носителем (она видна в файловых системах, подключенных к корневому каталогу, но не связана ни с каким внешним устройством. Поэтому она и называется псевдофайловой системой).

Для каждого процесса в `/proc` определена директория идентификатора процесса (`/proc/<pid>/`), она будет создана налету. В ней хранятся файлы, директории и символические ссылки, используя которые можно получить развёрнутую информацию о конкретном процессе. Вся информация доступна из `usermode`:

Элемент	Тип	Содержание
<code>cmdline</code>	Файл	holds the complete command line Содержит командную строку процесса
<code>cwd</code>	Символическая ссылка	Указывает на рабочую директорию процесса
<code>environ</code>	Файл	Содержит список переменных окружения процесса
<code>exe</code>	Символическая ссылка	Указывает на образ процесса (на его файл) Символическая ссылка, содержащая фактический путь к выполняемой команде.
<code>comm</code>	Файл	Имя команды, связанное с процессом. Это именно то имя файла, которое приведёт к выполнению нашего кода.

Элемент	Тип	Содержание
		Замечание 15.1 Разные потоки в одном и том же процессе могут иметь разные значения, доступные через <code>/proc/<pid>/task/<tid>/comm</code>. <code>task</code> — файл, в котором находятся поддиректории потоков.
stat	Файл	Содержит информацию о процессе (vsize, start_brk и др.)
fd	Директория	Содержит ссылки на открытые процессом файлы Замечание 15.2 В системе различаются файлы и открытые файлы. Открытые файлы описываются структурой <code>struct file</code>. Просто файлы представляются в системе деревом каталогов, а описываются физические файлы <code>struct inode</code>. Один и тот же файл может быть открыт много раз, причём разными процессами. В таблице открытых файлов на каждый такой открытый файл будет дескриптор <code>struct file</code>
root	soft link	Это символическая ссылка, которая указывает на корень файловой системы процесса. Замечание 15.3 Процессы, которые выполняются на нашем компьютере, могут принадлежать разным файловым системам. Пока мы имя исполняемого файла не указали в командной строке, пока он не начал выполняться, он лежит во вторичной памяти.
maps	Файл	Это адресное пространство того процесса, <code>pid</code> которого мы указали. В зависимости от <code>pid</code> информация будет отличаться, но есть неизменные составляющие:
pagemap	файл	Это 64-х разрядное значение для каждой строки процесса. При этом нас интересует фактически несколько бит: 1) 63 бит — present (in RAM) 2) 62 бит — swap space, область своппинга 3) 61 — shared anonymous page 4) 55 бит — dirty. Он указывается в дескрипторе страницы.

Элемент	Тип	Содержание
		С 54 по 0: если страница присутствует в физической памяти, то эти биты являются номером фрейма, то есть номером физической страницы.
task	Директория	Содержит поддиректории потоков
interrupts	Файл	Предоставляет таблицу о количестве прерываний на каждом из процессоров в следующем виде: 1) Первая колонка: номер прерывания 2) Колонки CPUx: счётчики прерываний на каждом из процессоров 3) Следующая колонка: вид прерывания: 3.1) IO-APIC-edge — прерывание по фронту на контроллер I/O APIC. 3.2) IO-APIC-fastestoi — прерывание по уровню на контроллер I/O APIC. 3.3) PCI-MSI-edge — MSI прерывание. 3.4) XT-PIC-XT-PIC — прерывание на PIC контроллер. 4) Последняя колонка: устройство, ассоциированное с данным прерыванием.

Таблица 10. Содержание директории /proc/<pid>

Разные потоки одного процесса могут иметь разные значения, доступные через /proc/<pid>/task/<tid>/comm: директория task содержит поддиректории потоков.

Файл maps — это адресное пространство процесса, pid которого указан; в зависимости от pid информация отличается. Формат файла:

1	address	perms	offset	dev	inode	pathname
2	00400000-00452000	r-xp	00000000	08:02	173521	/usr/bin/dbus-daemon
3	00651000-00652000	r--p	00051000	08:02	173521	/usr/bin/dbus-daemon
4	00652000-00655000	rw-p	00052000	08:02	173521	/usr/bin/dbus-daemon
5	...					
6	00e03000-00e24000	rw-p	00000000	00:00	0	[heap]
7	35b1dac000-35b1fac000	---p	001ac000	08:02	135870	/usr/lib64/libc-2.15.so
8	7ffffb2c0d000-7ffffb2c2e000	rw-p	00000000	00:00	0	[stack]
9	7ffffb2d48000-7ffffb2d49000	r-xp	00000000	00:00	0	[vdso]

Листинг 87. Формат файла /proc/pid/maps (сокращённо)

Различаем регионы по правам доступа perms:

- 1) r-xp — код;
- 2) r--p — константы;

3) `rw-p` — данные.

Диапазон адресов переводим в страницы — виртуальное адресное пространство выделяется страницами. Остальные поля строки:

- `dev` — два числа: старший и младший номера устройства;
- `inode` — номер `inode`;
- `[heap]` — куча (их у процесса может быть несколько);
- `[stack]`, `[vdso]` — стек и `vdso`; для многопоточных процессов появляются анонимные отображения.

Файл `pagemap` содержит 64-разрядное значение для каждой страницы процесса. Интерес представляют отдельные биты:

- 1) бит 63 — `present` (страница в оперативной памяти);
- 2) бит 62 — `swap` (область своппинга);
- 3) бит 61 — `shared anonymous page`;
- 4) бит 55 — `dirty` (указывается в дескрипторе страницы).

Биты с 54 по 0: если страница присутствует в физической памяти, эти биты являются номером фрейма, то есть номером физической страницы.

15.9 Регистрация функций работы с файлами `proc`

Чтобы создать виртуальный файл `proc`, нужно зарегистрировать в ядре свои функции для работы с этой файловой системой (`proc_dir_entry` фактически является дескриптором файла `proc`). Необходимо написать четыре функции, которые вызываются автоматически:

- 1) `open`;
- 2) `read`;
- 3) `write`;
- 4) `release`.

Структуру `struct file_operations` (её функции вызывают все драйверы, а через них — внешние устройства) переписывать нельзя. Чтобы её разгрузить, для ФС `proc` введена структура `struct proc_ops`, функции которой повторяют функции `struct file_operations`:

```
1 struct proc_ops {
2     ...
3     int      (*proc_open)(struct inode *, struct file *);
4     ssize_t  (*proc_read)(struct file *, char __user *, size_t, loff_t *);
5     ssize_t  (*proc_read_iter)(struct kiocb *, struct iov_iter *);
6     ssize_t  (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
7     loff_t   (*proc_lseek)(struct file *, loff_t, int);
8     int      (*proc_release)(struct inode *, struct file *);
```

```
9    ...
10 };
```

Листинг 88. Структура `proc_ops`

Регистрация функций выполняется инициализацией структуры назначенными инициализаторами (синтаксис принят для стандарта C99 начиная с версии GCC 2.5):

```
1 struct file_operations fops = {
2     .read    = device_read,
3     .write   = device_write,
4     .open    = device_open,
5     .release = device_release
6 };
```

Регистрация точек входа происходит при инициализации полей структуры `proc_ops` (до регистрации эти функции должны быть объявлены и описаны), а затем проинициализированная структура передаётся функции создания виртуального файла `proc_create`.

```
1 struct proc_dir_entry *proc_create_data(const char *name, umode_t mode,
2     struct proc_dir_entry *parent, const struct proc_ops *proc_ops, void *data);
3 struct proc_dir_entry *proc_create(const char *name, umode_t mode,
4     struct proc_dir_entry *parent, const struct proc_ops *proc_ops);
```

- `proc_ops` — указатель на проинициализированную структуру.
- `parent` задаёт иерархию `proc`: для файла в поддиректории её создают функцией `proc_mkdir` (до `proc_create_data`); при `parent = NULL` файл создаётся в корневом каталоге `proc`.
- `void *data` — приватные данные файла (например, ключ при авторизации с помощью флешки).

15.10 Пример проинициализированных полей структуры `proc_ops`

В лабораторной работе четыре функции работы с файлом регистрируются в структуре `proc_ops` назначенными инициализаторами:

```
1 static const struct proc_ops fortune_proc_ops = {
2     .proc_open = fortune_open,
3     .proc_read = fortune_read,
4     .proc_write = fortune_write,
5     .proc_release = fortune_release,
6 };
```

15.11 Передача данных из пространства ядра в пространство пользователя

В своих функциях `read` и `write` нужно вызывать специальные функции ядра для передачи данных из пространства пользователя в пространство ядра и обратно — это `copy_from_user` и `copy_to_user`.

Специальные функции нужны потому, что ядро (и загружаемый модуль как его часть) оперирует физическими адресами: ядро загружено в физическую память резидентно (постоянно) и не выгружается, а процессы имеют только виртуальные адресные пространства и получают ресурс «физическая страница» лишь по мере надобности.

Чтобы передать данные из `user` в `kernel` или из `kernel` в `user`, ядро должно обратиться к буферу пространства пользователя. Вероятна ситуация, когда буфер (страница), к которому обращается ядро, выгружен — тогда возникнет страничное прерывание, которое должно обработать само ядро. Получается, что ядро обратилось, само стало источником прерывания и само же должно его обработать; если не предпринять специальных мер, возникнет паника ядра. Для этого написаны две функции:

- `copy_to_user` — копирует данные из буфера ядра в буфер пользователя;
- `copy_from_user` — копирует данные из буфера пользователя в буфер ядра.

```
1 static inline __must_check unsigned long
2 _copy_to_user(void __user *to, const void *from, unsigned long n) {
3     might_fault();
4     if (should_fail_usercopy())
5         return n;
6     if (access_ok(to, n)) {
7         instrument_copy_to_user(to, from, n);
8         n = raw_copy_to_user(to, from, n);
9     }
10    return n;
11 }
```

Листинг 89. Определение функции `copy_to_user`

- Указатель `to` (`void __user *`) — это указатель на буфер пользователя, а `const from` — указатель на буфер ядра. Для функции `copy_from_user` ситуация обратная.

```
1 static inline __must_check unsigned long
2 _copy_from_user(void *to, const void __user *from, unsigned long n) {
3     ...
4 }
```

Листинг 90. Определение функции `copy_from_user`

15.12 Примеры из лабораторных работ

Лабораторная работа «фортунки» посвящена передаче данных из пространства пользователя в пространство ядра и обратно. Работа с файлом ФС `proc` ведётся в загружаемом модуле ядра, поэтому в модуле объединяются все рассмотренные приёмы: регистрация функций на файле `proc`, создание файла и директории, передача данных функциями `copy_to_user/copy_from_user`.

В модуле шесть точек входа: `open`, `read`, `write`, `release`, `init`, `exit`. `open` и `release` вызываются, когда процессу нужно работать с файлом (здесь — процессами `echo` и `cat`). В `read/write` вызываются `copy_to_user/copy_from_user`, так как ядро находится в физической памяти, а процессы имеют виртуальные адресные пространства.

```
1 #include <linux/init.h>
2 #include <linux/module.h>
3 #include <linux/kernel.h>
4 #include <linux/proc_fs.h>
5 #include <linux/uaccess.h>
6 #include <linux/vmalloc.h>
7 #include <linux/string.h>
8 #include <linux/errno.h>
9
10 #define PROC_FILE_NAME "fortune"
11 #define PROC_DIR_NAME  "fortune_dir"
12 #define PROC_SYM_NAME  "fortune_symlink"
13
14 #define COOKIE_BUF_SIZE PAGE_SIZE
15
16 MODULE_LICENSE("GPL");
17 MODULE_AUTHOR("Zhizhin Nikita");
18
19 static char *cookie_buf;
20 static unsigned int write_index;
21 static unsigned int read_index;
22 static struct proc_dir_entry *proc_file;
23 static struct proc_dir_entry *proc_dir;
24 static struct proc_dir_entry *proc_symlink_entry;
25
26 static ssize_t fortune_write(struct file *file, const char __user *user_buf, size_t
count, loff_t *ppos) {
27     size_t len;
28
29     if (count == 0) return 0;
30     if (count > COOKIE_BUF_SIZE - 1) return -ENOSPC;
31     if (write_index + count + 1 > COOKIE_BUF_SIZE) return -ENOSPC;
```

```

32     len = count;
33     if (copy_from_user(cookie_buf + write_index, user_buf, len)) {
34         return -EFAULT;
35     }
36     cookie_buf[write_index + len] = '\0';
37     printk(KERN_INFO "fortune: write %s", cookie_buf + write_index);
38     write_index += len + 1;
39     return len;
40 }
41
42 static ssize_t fortune_read(struct file *file, char __user *user_buf, size_t count,
43     loff_t *ppos) {
44     size_t len;
45     size_t available;
46
47     if (*ppos > 0) return 0;
48     if (count == 0) return 0;
49     if (write_index == 0) return 0;
50     if (read_index >= write_index) read_index = 0;
51     available = write_index - read_index;
52     len = strlen(cookie_buf + read_index, available);
53     if (len == 0) {
54         read_index = 0;
55         len = strlen(cookie_buf, write_index);
56         if (len == 0) return 0;
57     }
58     if (len > count) len = count;
59     if (copy_to_user(user_buf, cookie_buf + read_index, len)) {
60         return -EFAULT;
61     }
62     printk(KERN_INFO "fortune: read %s", cookie_buf + read_index);
63     read_index += len + 1;
64     if (read_index >= write_index)
65         read_index = 0;
66     *ppos += len;
67     return len;
68 }
69
70 static int fortune_open(struct inode *inode, struct file *file) {
71     printk(KERN_INFO "fortune: open");
72     return 0;
73 }
74
75 static int fortune_release(struct inode *inode, struct file *file) {
76     printk(KERN_INFO "fortune: release");

```

```

76     return 0;
77 }
78
79 static const struct proc_ops fortune_proc_ops = {
80     .proc_open = fortune_open,
81     .proc_read = fortune_read,
82     .proc_write = fortune_write,
83     .proc_release = fortune_release,
84 };
85
86 static int __init fortune_init(void) {
87     cookie_buf = vmalloc(COOKIE_BUF_SIZE);
88     if (!cookie_buf) {
89         printk(KERN_ERR "fortune: cannot allocate cookie buffer");
90         return -ENOMEM;
91     }
92     memset(cookie_buf, 0, COOKIE_BUF_SIZE);
93     write_index = 0;
94     read_index = 0;
95     proc_dir = proc_mkdir(PROC_DIR_NAME, NULL);
96     if (!proc_dir) {
97         printk(KERN_ERR "fortune: cannot create /proc/%s", PROC_DIR_NAME);
98         vfree(cookie_buf);
99         return -ENOMEM;
100    }
101    proc_file = proc_create(PROC_FILE_NAME, 0666, proc_dir, &fortune_proc_ops);
102    if (!proc_file) {
103        printk(KERN_ERR "fortune: cannot create /proc/%s", PROC_FILE_NAME);
104        proc_remove(proc_dir);
105        vfree(cookie_buf);
106        return -ENOMEM;
107    }
108    proc_symlink_entry = proc_symlink(PROC_SYM_NAME, NULL, "/proc/" PROC_DIR_NAME
"/" PROC_FILE_NAME);
109    if (!proc_symlink_entry) {
110        printk(KERN_ERR "fortune: cannot create /proc/%s", PROC_SYM_NAME);
111        proc_remove(proc_dir);
112        proc_remove(proc_file);
113        vfree(cookie_buf);
114        return -ENOMEM;
115    }
116    printk(KERN_INFO "fortune: module loaded");
117    return 0;
118 }
119

```

```
120 static void __exit fortune_exit(void) {
121     proc_remove(proc_symlink_entry);
122     proc_remove(proc_file);
123     proc_remove(proc_dir);
124     vfree(cookie_buf);
125     printk(KERN_INFO "fortune: module unloaded");
126 }
127
128 module_init(fortune_init);
129 module_exit(fortune_exit);
```


16

16 Открытые файлы: системный вызов `open()`: `int open(const char* pathname, int flags); int open(const char* pathname, int flags, mode_t mode);` пояснить смысл параметров. Основные флаги. Флаг `CREATE`. Реализация системного вызова `open()` в системе — действия в ядре: `SYSCALL_DEFINE3(open,...) → ksys_open(filename, flags, mode) → do_sys_open() → do_sys_openat2() ...` найти наименьший файловый дескриптор... Действия, если флаг `O_CREATE` установлен?

16.1 Открытые файлы

В системе различаются просто файлы и открытые файлы. Просто файлы — то, что мы видим в каталогах: они «тихо» лежат во вторичной памяти и описываются структурой `struct inode`, которая описывает физический файл. Открытый файл — файл, который открыт в результате выполнения программы, то есть открыт процессом; открытые файлы описываются структурой `struct file` — дескриптором открытого файла.

Любой процесс может открыть много файлов, причём один и тот же файл может быть открыт много раз — одним или несколькими процессами; сколько раз открыт файл, столько в таблице открытых файлов будет дескрипторов открытых файлов.

Каждый процесс имеет собственную таблицу открытых файлов (поле `struct file *fd_array[]` в структуре `files_struct`). Файловый дескриптор — индекс в этой таблице. При порождении процесса для него автоматически выделяются три дескриптора: 0 (`stdin`), 1 (`stdout`), 2 (`stderr`); первый открытый процессом файл получит дескриптор 3. Прежде чем выдать дескриптор, система выполняет поиск наименьшего свободного.

16.2 Системный вызов `open()`: смысл параметров

Чтобы получить возможность прочесть что-то из файла или записать что-то в файл, его нужно открыть системным вызовом `open()`. В пространстве пользователя есть две формы вызова:

```
1 int open(const char *pathname, int flags);
2 int open(const char *pathname, int flags, mode_t mode);
```

- `pathname` — имя открываемого файла;
- `flags` — флаги, определяющие режим доступа и поведение при открытии;
- `mode` — права доступа создаваемого файла.

Третий параметр `mode` необходим, когда `open` вызывается для создания нового файла: тогда нужно указать права доступа для трёх групп пользователей. Если же `open` вызывается для открытия существующего файла, `mode` не нужен. При создании нового файла создаётся новый `inode` (инициализируется структура `inode`),

и, поскольку созданный файл открывается, для него инициализируются поля `struct file`.

Функция возвращает файловый дескриптор — целое неотрицательное число, которое можно использовать в последующих системных вызовах. Это наименьший дескриптор из таблицы открытых процессом файлов, который ещё не занят. Смещение в открытом файле (`f_pos`) устанавливается в начало файла.

16.3 Основные флаги

Флаги (`flags`) в системе играют огромную роль как ограничения: они указывают, что с файлом можно делать, а что нельзя. Параметр `flags` задаёт режим доступа и поведение при открытии файла.

Флаги доступа (один из них указывается обязательно):

- `O_RDONLY` — открыть файл только на чтение;
- `O_WRONLY` — открыть файл только на запись;
- `O_RDWR` — открыть файл на чтение и запись.

Некоторые флаги создания и поведения:

- `O_CREAT` — создать файл, если его не существует (подробно рассмотрен ниже);
- `O_APPEND` — дозапись в существующий файл: перед каждым вызовом `write()` смещение файла переносится на его конец, поэтому ранее записанная информация не теряется;
- `O_TRUNC` — если файл существует и открыт для записи, усекает его до нулевой длины;
- `O_NONBLOCK` — устанавливает режим неблокирования. Эта опция обычно полезна для специальных файлов типа FIFO и устройств типа терминалов: обычно для этих файлов орен блокируется, пока файл не «готов», а если `O_NONBLOCK` установлен, орен возвращается немедленно. Бит также воздействует на чтение и запись: разрешает им возвращаться немедленно с состоянием ошибки, если нет доступного ввода или вывод не может быть записан;
- `O_CLOEXEC` — установить флаг `close-on-exec` на новом файловом дескрипторе;
- `O_PATH` — получить лишь файловый дескриптор (сам файл не будет открыт);
- `O_TMPFILE` — создать неименованный временный обычный файл.

16.4 Флаг CREATE

Флаг `O_CREAT` создаёт новый файл, если его не существует; если файл существует, флаг игнорируется. Создание файла означает создание нового `inode`. При этом идентификаторы владельца и группы файла берутся из действующих идентификаторов пользователя и группы процесса.

Именно с флагом `O_CREAT` связан третий параметр `mode`: он задаёт права доступа создаваемого файла и должен быть указан при использовании `O_CREAT`; во всех остальных случаях `mode` игнорируется.

Для режима предусмотрены символические константы (для владельца / группы / остальных):

- `S_IRWXU` / `S_IRWXG` / `S_IRWXO` — права на чтение, запись и выполнение;
- `S_IRUSR` / `S_IRGRP` / `S_IROTH` — права на чтение;
- `S_IWUSR` / `S_IWGRP` / `S_IWOTH` — права на запись;
- `S_IXUSR` / `S_IXGRP` / `S_IXOTH` — права на выполнение.

Вместе с `O_CREAT` часто используется флаг `O_EXCL`: он заставляет систему проверить, существует ли уже создаваемый файл, чтобы не потерять ранее записанную в него информацию. Если файл с таким именем уже есть, вызов `open` с `O_CREAT | O_EXCL` завершается ошибкой.

Пример:

```
1 open("file.txt", O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR); // права -rw-----
```

Вопросы ЛР

СОДЕРЖАНИЕ

1	Сокеты	6
1.1	Какая связь?	6
1.2	Что возвращает <code>assert</code> ?	6
1.2.1	Зачем <code>assert</code> создаёт копию сокета?	6
1.3	Что делает <code>listen</code> ?	6
1.4	Когда вызывается системный вызов <code>fork</code> ?	6
1.4.1	Зачем вызывается <code>fork</code> ?	6
1.5	Показать сокет сервера в файловой подсистеме с <code>inode</code>	6
1.6	Что обозначается буквой <code>s</code> ?	6
1.7	Что обозначается буквой <code>S</code> ?	6
1.8	В чём отличие второй программы (где сервер отвечает клиенту) от первой?	6
1.8.1	Какой адрес?	6
1.8.2	А зачем мы это делаем?	7
1.9	Когда вызывается <code>pthread_create</code> ? Где создаются потоки?	7
1.10	<code>Epoll</code> какой: многопоточный или однопоточный? Нужны ли средства взаимoisключения?	7
1.11	(мультиплексор) Что происходит в строке <code>if (events[n].data.fd == listenfd)</code> ?	7
1.12	Что означают флаги <code>EPOLLIN</code> , <code>EPOLLET</code> , <code>EPOLL_CTL_ADD</code> ?	7
2	Файловая система	8
2.1	Когда <code>lstat</code> возвращает ошибку?	8
3	Файловая система (<code>proc</code>)	9
3.1	Что будем анализировать?	9
3.2	Где мы видим скрытую фрагментацию?	9
3.3	Чем измеряются адреса?	9
4	NFS	10
5	<code>task_struct</code>	11
5.1	Какой <code>prio</code> ?	11
5.2	Какой <code>policy</code> ?	11
5.3	Что значит <code>policy=0</code> , <code>policy=1</code> , <code>policy=2</code> ?	11
5.4	0, 1, а количество переключений добровольных 3000 — о чём это говорит?	11
5.5	Какой алгоритм используется?	11
5.5.1	Как процесс выполняется по алгоритму FIFO?	11

5.5.2	Что характерно для алгоритма FIFO?	11
5.6	Флаги	11
5.7	Флаг fork по ехес — что он означает? Раскройте смысл.	12
5.7.1	Почему нельзя ехес?	12
5.8	Какой флаг в insmod?	12
5.8.1	Почему у insmod установлен этот флаг?	12
5.9	Что означает флаг PF_RANDOMIZED?	12
5.10	pid 0, pid 1	13
5.10.1	Что за процесс имеет pid 1 и почему у него такой usertime?	13
5.11	Где аргументы, флаги?	13
5.12	Чему равен счётчик добровольных переключений контекста у migration?	13
5.13	Как работает migration?	13
6	Модули ядра	14
6.1	Что нужно модулю для доступа к экспортируемым данным?	14
6.1.1	Почему?	14
6.2	Как должны быть объявлены данные в модуле, если они определены в другом модуле?	14
6.3	Почему важен порядок загрузки модулей ядра?	14
7	Фортуны	15
7.1	Сколько точек входа (в модуль) и какие?	15
7.2	Когда вызываются open, release?	15
7.2.1	Когда это происходит в вашем конкретном случае?	15
7.3	С помощью чего мы реализуем передачу данных из пространства пользователя в пространство ядра и наоборот?	15
7.3.1	Почему нужны специальные функции copy_to_user / copy_from_user?	15
8	VFS	16
8.1	Что мы должны сделать, чтобы получить доступ к ФС и её файлам? ...	16
8.2	Показать ls -ial	16
8.3	Показать, что смонтированы две ФС (две последние строки)	16
8.4	Показать slab (наша первая строка)	16
8.5	Показать, что зарегистрирована ФС (тоже первая строка)	16
8.6	Какие точки входа?	16
8.6.1	Когда вызывается каждая?	16
8.7	Зачем создаём inode?	16
8.8	Какие минимальные действия позволяют зарегистрировать свою ФС (не монтировать, а зарегистрировать, какая функция)?	16

8.8.1	Что передаётся на вход функции <code>register_filesystem</code> ?	17
8.8.2	Какие поля в структуре обязательно указать?	17
8.9	А для монтирования что нужно?	17
8.9.1	Что нужно сделать, чтобы файловую систему можно было смонтировать?	17
8.9.2	Что делает <code>fill_super / fill_sb</code> ?	17
8.10	Чем являются эти симплы?	17
8.11	Как смонтировать и отмонтировать ФС?	17
8.12	Зачем нужен <code>slab/slub cache</code> ?	18
8.12.1	Чему равен цикл, в котором вы вызываете <code>alloc</code> ?	18
9	Прерывания (tasklet, workqueue)	19
9.1	Рассказать, где мы вешаемся на линию <code>irq</code> , и обо всех параметрах этой функции	19
9.2	Как мы освобождаем линию?	19
9.3	Показать вывод	19
9.4	Что делает ваш обработчик аппаратного прерывания?	20
9.5	Разница между <code>tasklet</code> и <code>softirq</code>	20
9.6	Разница между <code>tasklet</code> и <code>workqueue</code>	20
9.6.1	Какая основная особенность очередей работ?	20
9.7	Какие прерывания делятся на аппаратные и отложенные (<code>top half</code> и <code>bottom half</code>)?	20
9.8	Какой демон отвечает за выполнение тасклетов?	20
9.9	Что нужно вызвать в обработчике аппаратного прерывания, чтобы впоследствии выполнился тасклет?	20
9.10	На каком процессоре будет выполняться тасклет?	20
9.11	С помощью чего мы смогли разместить ещё один обработчик прерывания на линию, не убрав старый?	21
9.12	В чём главная особенность тасклетов (сериализуем ли тасклет)?	21
9.13	Для чего вы написали это отложенное действие в виде тасклета?	21
9.14	Для чего вообще создали тасклеты?	21
10	Утилиты (netstat, ss)	22
10.1	В каком состоянии сервер?	22
10.2	Какое преимущество этих утилит?	22
10.3	Какой недостаток этих утилит?	22
11	NAPI	23
11.1	Зачем добавили NAPI и что это вообще такое?	23
12	Может понадобится из прошлого сема	26

12.1 Кэши, их уровни	26
12.2 Схема преобразования виртуального адреса в физический	27
12.3 Что происходит при fork, exec?	27
12.4 Сигналы	29
12.5 Блокирующий ввод-вывод	29

1 Сокеты

1.1 Какая связь?

Дуплексная.

1.2 Что возвращает `accept`?

Копию исходного сокета в состоянии `connected`. Сам исходный сокет остаётся в состоянии `listen`.

1.2.1 Зачем `accept` создаёт копию сокета?

Исходный сокет должен остаться в состоянии `listen`, чтобы принимать новые запросы на соединения.

1.3 Что делает `listen`?

Делает сокет пассивным (принимающим входящие соединения с помощью `accept`) и задаёт размер очереди входящих соединений.

1.4 Когда вызывается системный вызов `fork`?

После установки нового соединения.

1.4.1 Зачем вызывается `fork`?

Чтобы каждое клиентское подключение обрабатывалось в отдельном процессе.

1.5 Показать сокет сервера в файловой подсистеме с `inode`

Написать

```
1 ls -ial
```

и указать на `inode`.

1.6 Что обозначается буквой `s`?

Тип файла `socket`.

1.7 Что обозначается буквой `S`?

Состояние процесса.

1.8 В чём отличие второй программы (где сервер отвечает клиенту) от первой?

Во второй программе клиент вызывает `bind`, чтобы связать сокет с адресом.

1.8.1 Какой адрес?

Адрес сокета — имя файла.

1.8.2 А зачем мы это делаем?

Чтобы сервер мог ответить клиенту через `sendto`.

1.9 Когда вызывается `pthread_create`? Где создаются потоки?

В результате установки соединения (после системного вызова `accept`) — для каждого нового клиентского подключения создаётся отдельный поток.

Осторожно

В наших лекциях/семинарах прямого ответа на этот вопрос нет, он закрыт доп. источником.

1.10 `epoll` какой: многопоточный или однопоточный? Нужны ли средства взаимного исключения?

Однопоточный. Средства взаимного исключения не нужны.

1.11 (мультиплексор) Что происходит в строке `if (events[n].data.fd == listenfd)`?

Проверяется, обработано соединение или нет.

1.12 Что означают флаги `EPOLLIN`, `EPOLLET`, `EPOLL_CTL_ADD`?

Осторожно

В наших лекциях/семинарах разбора этих флагов нет; в доп. источнике их тоже нет.

✱ Claude

- `EPOLLIN` — событие «дескриптор готов к чтению» (на сокете есть данные или входящее соединение).
- `EPOLLET` — режим `edge-triggered` (срабатывание по фронту): событие выдаётся один раз при изменении состояния, а не пока дескриптор готов.
- `EPOLL_CTL_ADD` — операция `epoll_ctl`, добавляющая дескриптор в множество наблюдаемых `epoll`.

2 Файловая система

2.1 Когда `lstat` возвращает ошибку?

Осторожно

В наших лекциях/семинарах и в доп. источнике прямого ответа на этот вопрос нет.

✱ Claude

`lstat` возвращает `-1` и устанавливает `errno`, когда не может получить информацию о файле по пути. Типичные случаи:

- компонента пути не существует (`ENOENT`);
- нет прав на поиск в одном из каталогов пути (`EACCES`);
- компонента пути, используемая как каталог, не является каталогом (`ENOTDIR`);
- слишком длинный путь (`ENAMETOOLONG`);
- недопустимый адрес буфера или пути (`EFAULT`);
- слишком много символических ссылок при разрешении пути (`ELoop`).

В отличие от `stat`, для символической ссылки `lstat` возвращает информацию о самой ссылке, а не о файле, на который она указывает.

3 Файловая система (proc)

Информацию из proc записываем в файл и показываем её.

3.1 Что будем анализировать?

Мы написали 2 сервера: многопоточный и однопоточный. Мы сможем сравнивать для них maps.

3.2 Где мы видим скрытую фрагментацию?

В /proc/<pid>/mem.

3.3 Чем измеряются адреса?

Терабайтами (порядка 16 млн терабайт), а в системе нет такой памяти — значит, это виртуальная память.

4 NFS

- Перекрёстную связь устанавливаем. Обращаемся к буферу другого человека.
- Видим файлы друг друга.
- Лучше показывать в папках.
- Взаимодействуем через NFS.

5 task_struct

5.1 Какой prio?

0, наивысший.

5.2 Какой policy?

1)

5.3 Что значит policy=0, policy=1, policy=2?

Это номер алгоритма планирования:

- policy=0 — SCHED_NORMAL: справедливый алгоритм (вытесняющий, с квантованием времени);
- policy=1 — SCHED_FIFO: процесс поступает в очередь и обслуживается в порядке поступления от начала до конца;
- policy=2 — SCHED_RR: процессы поступают в очередь, но, исчерпав квант времени, ставятся в конец очереди.

5.4 0, 1, а количество переключений добровольных 3000 — о чём это говорит?

Поскольку у него алгоритм FIFO prio 0, вытеснить его никто не может, но он постоянно должен быть в работе на процессоре, а основная задача системы — выполнение приложений. Чтобы не пожирать всё время, он добровольно уходит с выполнения, когда возникает работа.

5.5 Какой алгоритм используется?

FIFO.

5.5.1 Как процесс выполняется по алгоритму FIFO?

Первым пришёл, первым ушёл.

5.5.2 Что характерно для алгоритма FIFO?

Процесс выполняется от начала до конца. Нулевой prio, прервать его нельзя. Приоритетное планирование, никто его вытеснить не сможет.

5.6 Флаги

- PF_NO_SETAFFINITY — из пользовательского пространства запрещено изменять cpu_mask;
- PF_NOFREEZE — поток не может быть заблокирован.

5.7 Флаг `fork no exec` — что он означает? Раскройте смысл.

В результате системного вызова `fork` выполняется код предка. А код предка — это код ядра операционной системы.

То есть он выполняет код ядра и перейти на выполнение другой программы не может.

Системный вызов `exec` переводит процесс на новое адресное пространство программы, путь к которой передаётся в качестве аргумента. Флаг `PF_FORKNOEXEC` запрещает переход на новое адресное пространство (не позволяет внедрить в программу вредоносный код).

5.7.1 Почему нельзя `exec`?

Иначе мы перейдём на выполнение кода, который будет передан `exec` в качестве параметра.

5.8 Какой флаг в `insmod`?

Флаг, говорящий, что процесс воспользовался привилегиями суперпользователя.

5.8.1 Почему у `insmod` установлен этот флаг?

Потому что команда `insmod` загружает модуль в ядро, а для этого (обращения к коду ядра) нужны права суперпользователя.

✱ Claude

Это флаг `PF_SUPERPRIV` — признак того, что процесс воспользовался привилегиями суперпользователя.

5.9 Что означает флаг `PF_RANDOMIZED`?

... может начинаться с любого подходящего адреса.

⚠ Осторожно

В наших лекциях/семинарах развёрнутого определения этого флага нет (в конспекте только обрывочная формулировка выше).

✱ Claude

`PF_RANDOMIZED` — для процесса включена рандомизация размещения адресного пространства (ASLR): расположение сегментов и стека рандомизируется, поэтому

область может начинаться с любого подходящего адреса. Это затрудняет атаки, полагающиеся на фиксированные адреса.

5.10 `pid 0`, `pid 1`

Процесс с `pid 0` — процесс, запустивший систему. В `ps` это будет `init`. Тут он называется `swapper`, но от этого ничего не изменилось.

`pid 1` — процесс, открывший терминал.

5.10.1 Что за процесс имеет `pid 1` и почему у него такой `usertime`?

Это терминал. `usertime` большой, потому что он работает с пользователем.

5.11 Где аргументы, флаги?

В стеке.

5.12 Чему равен счётчик добровольных переключений контекста у `migration`?

Осторожно

В наших материалах и в доп. источнике числового ответа на этот вопрос нет.

5.13 Как работает `migration`?

Демон `migration` совершает набеги на очереди процессов.

6 Модули ядра

6.1 Что нужно модулю для доступа к экспортируемым данным?

Физический адрес экспортируемых данных.

По Цилюрику: модулю нужны абсолютные адреса экспортируемых имён ядра и модулей, по которым происходит связывание имён в компилируемом модуле. Абсолютный адрес — это адрес в физической памяти (Цилюрик называет это абсолютными адресами в пространстве ядра).

6.1.1 Почему?

Потому что ядро оперирует физическими адресами.

6.2 Как должны быть объявлены данные в модуле, если они определены в другом модуле?

С квалификатором `extern`.

6.3 Почему важен порядок загрузки модулей ядра?

[хз верно ли]

— Если загружать в другом порядке будет ошибка `unknown symbols`, что-то такое.

[неверно]

— поскольку обращение идет в физ памяти, то эти данные должны находиться уже там.

✱ Claude

Верный ответ (из доп. источника): модуль не сможет получить физический адрес модуля, который ещё не загружен в ядро, — поэтому модуль с экспортируемыми данными должен быть загружен раньше тех, кто эти данные использует.

7 Фортунки

7.1 Сколько точек входа (в модуль) и какие?

6 — open, read, write, release, init, exit.

Регистрируются точки входа при инициализации полей структуры proc_ops.

7.2 Когда вызываются open, release?

Когда процессу необходимо работать с файлом.

7.2.1 Когда это происходит в вашем конкретном случае?

Вызывают процессы echo и cat. Явно мы это не вызываем.

7.3 С помощью чего мы реализуем передачу данных из пространства пользователя в пространство ядра и наоборот?

copy_to_user / copy_from_user.

7.3.1 Почему нужны специальные функции copy_to_user / copy_from_user?

Ядро находится в физической памяти, а процессы имеют только виртуальные адресные пространства.

Когда ядру необходимо обратиться к пользовательскому буферу, а страница, в которой он находится, выгружена, происходит страничное прерывание, которое ядру необходимо обработать и закончить работу с буфером.

Я бы ответил «ядро оперирует физическими адресами, а процессы имеют виртуальные адресные пространства».

8 VFS

8.1 Что мы должны сделать, чтобы получить доступ к ФС и её файлам?

Смонтировать её.

8.2 Показать `ls -ial`

```
1 ls -ial /mnt
```

8.3 Показать, что смонтированы две ФС (две последние строки)

```
1 sudo cat /proc/mounts
```

8.4 Показать `slab` (наша первая строка)

```
1 sudo cat /proc/slabinfo
```

8.5 Показать, что зарегистрирована ФС (тоже первая строка)

```
1 cat /proc/filesystems
```

8.6 Какие точки входа?

`init`, `exit`, `mount`, `kill` (`superblock`).

8.6.1 Когда вызывается каждая?

- `kill` — когда из командной строки вызывается `umount`;
- `mount` — когда из командной строки вызывается `mount`;
- `init` — при `insmod`;
- `exit` — при `rmmmod`.

8.7 Зачем создаём `inode`?

Точка монтирования.

8.8 Какие минимальные действия позволяют зарегистрировать свою ФС (не монтировать, а зарегистрировать, какая функция)?

`register_filesystem`.

✱ Claude

Из доп. источника (полнее): минимальный набор полей для регистрации — `owner` и `name`; минимальный набор действий — определить структуру `file_system_type`

с этими полями, зарегистрировать ФС вызовом `register_filesystem` и загрузить модуль в ядро.

8.8.1 Что передаётся на вход функции `register_filesystem`?

Указатель на проинициализированную структуру `file_system_type`.

8.8.2 Какие поля в структуре обязательно указать?

⚠ Исходный текст

`this module` и имя

💡 Изменённый вариант

Обязательно указываются поля `owner` (обычно `THIS_MODULE`) и `name`.

8.9 А для монтирования что нужно?

Надо написать свою функцию `mount` и зарегистрировать её через `file_system_type`.

8.9.1 Что нужно сделать, чтобы файловую систему можно было смонтировать?

Нужно инициализировать поле `mount` структуры `file_system_type` своей функцией. В ней вызывается `mount_nodev` и передаётся функция `fill_super`. Основную работу выполняют `myvfs_mount` и `myvfs_fill_super` (последняя инициализирует `inode` и передаёт его в `d_make_root`).

8.9.2 Что делает `fill_super` / `fill_sb`?

- 1) инициализирует поля структуры `super_block`;
- 2) создаёт корневой `dentry`: для этого создаёт `inode`, инициализирует его поля и вызовом `d_make_root` по этому `inode` создаёт `dentry`.

8.10 Чем являются эти симплы?

Предопределёнными функциями.

8.11 Как смонтировать и отмонтировать ФС?

Монтирование:

```
1 sudo mount -o loop -t myvfs ./image ./dir
```

Отмонтирование:

8.12 Зачем нужен slab/slub cache?

- чтобы не выделять всю страницу на структуры;
- на уровне ОС можно выделить только порядок страниц (1, 2, 4 и т.д. страниц, то есть степень двойки);
- слаб-кеш позволяет выделять несколько одинаковых структур в одной странице с учётом выравнивания (например, для структуры в 32 байта делим 1024 на 32, чтобы найти количество структур, используемых в одной странице слаб-кеша);
- слаб НЕ используется напрямую пространством пользователя (нет API для взаимодействия с ним).

Кроме того, слабы нужны для повторного использования проинициализированных объектов; побочный эффект — устранение фрагментации (за счёт того же повторного использования).

8.12.1 Чему равен цикл, в котором вы вызываете alloc?

16 байт 256 раз записали, на 257-й раз создаст новую страницу.

9 Прерывания (тасклет, `workqueue`)

Линия прерывания разделяется всеми обработчиками прерывания (через запятую, три штуки). Для очередей работ показать воркера — имя очереди в суффиксе.

9.1 Рассказать, где мы вешаемся на линию `irq`, и обо всех параметрах этой функции

Вешаемся на линию прерывания функцией `request_irq`. В нашей ЛР это первая линия (IRQ1) — работаем с клавиатурой. Сигнатура функции:

```
1 int request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags, const
  char *name, void *dev)
```

Параметры:

- `irq` — номер прерывания (IRQ0 — прерывание от таймера, IRQ1 — от клавиатуры);
- `handler` — наш обработчик прерывания (функция, вызываемая при возникновении прерывания);
- `flags` — флаги Interrupt ReQuest Flags (IRQF), например `IRQF_SHARED` для разделяемой линии;
- `name` — имя, которое увидим в `/proc/interrupts`;
- `dev` — указатель на данные устройства; на разделяемой линии служит для идентификации обработчика.

`request_irq` — обёртка над базовой функцией `request_threaded_irq`: при её вызове параметр `thread_fn` равен `NULL`.

Осторожно

Параметр `dev` в нашем семинаре в перечне параметров отдельно не расписан (есть только в сигнатуре) — его пояснение добавлено для полноты.

9.2 Как мы освобождаем линию?

Вызываем в командной строке команду `rmmod`.

9.3 Показать вывод

- `cat /proc/interrupts`;
- `ps ajx`, там найти `kworker` для `workqueue`. Поток ядра называется как-то типа `[kworker/R-irq_w]`.

9.4 Что делает ваш обработчик аппаратного прерывания?

Из буфера клавиатуры извлекает код нажатой клавиши и планирует на выполнение тасклет.

9.5 Разница между tasklet и softirq

softirq определяются только статически (определён массив softirq), то есть для добавления своего softirq нужно перекомпилировать ядро. Тасклеты можно определить как статически, так и динамически, и встроить в ядро с помощью модуля. При этом tasklet — разновидность softirq. Тасклет не может работать параллельно, softirq — может.

9.6 Разница между tasklet и workqueue

Работа (именно работа, а не очередь) в workqueue может быть заблокирована, тогда как тасклет не может быть заблокирован. Тасклет выполняется в контексте прерывания (код тасклета неделим), а работа очереди работ выполняется в контексте специального потока ядра и может быть заблокирована.

9.6.1 Какая основная особенность очередей работ?

Работа может быть заблокирована (в отличие от тасклета).

Основная идея очереди работ: однопараллельная множественная работа, в которой работы не должны блокировать друг друга. При этом механизм строится с тем расчётом, чтобы экономить ресурсы: поток, выделенную память, планирование.

9.7 Какие прерывания делятся на аппаратные и отложенные (top half и bottom half)?

Медленные (то есть все, кроме таймера).

9.8 Какой демон отвечает за выполнение тасклетов?

Демон ksoftirqd.

9.9 Что нужно вызвать в обработчике аппаратного прерывания, чтобы впоследствии выполнялся тасклет?

Нужно с помощью tasklet_schedule запланировать его на выполнение.

9.10 На каком процессоре будет выполняться тасклет?

На том, на котором выполнялось аппаратное прерывание.

9.11 С помощью чего мы смогли разместить ещё один обработчик прерывания на линию, не убрав старый?

С помощью флага `IRQF_SHARED`.

9.12 В чём главная особенность тасклетов (сериализуем ли тасклет)?

Один и тот же тип тасклета не может выполняться параллельно. Тасклеты не сериализуются: в конкретный момент времени может выполняться только один конкретный тасклет.

9.13 Для чего вы написали это отложенное действие в виде тасклета?

Для преобразования нажатой клавиши в ASCII-код.

9.14 Для чего вообще создали тасклеты?

Простота — не нужно модифицировать ядро для каждого нового типа обработчика.

10 Утилиты (netstat, ss)

```
1 ss -tulpna
```

```
1 netstat -tulpna
```

10.1 В каком состоянии сервер?

LISTEN.

10.2 Какое преимущество этих утилит?

Эти утилиты работают в пространстве пользователя.

10.3 Какой недостаток этих утилит?

Невозможно выполнять мониторинг: мониторинг можно выполнять только в ядре.

11 NAPI

11.1 Зачем добавили NAPI и что это вообще такое?

Каждый входящий пакет приходит на сетевую карточку (внешнее устройство); пакеты приходят постоянно, и их обработка — это прерывание. Был введён NAPI — замена прерываний опросом (поллингом). В современных системах прямой доступ к памяти — основной способ взаимодействия с внешними устройствами.

Сетевая подсистема Linux построена в виде стека BSD. Передача данных происходит через интерфейс socket. Основная цель NAPI — сократить число прерываний при приёме пакетов. В NAPI-совместимых драйверах прерывание отключается, когда приходит пакет; обработчик вызывает `rx_schedule`, что гарантирует обработку пакета в дальнейшем.

⚠ Осторожно

В наших лекциях/семинарах раздела про NAPI нет, он закрыт доп. источником.

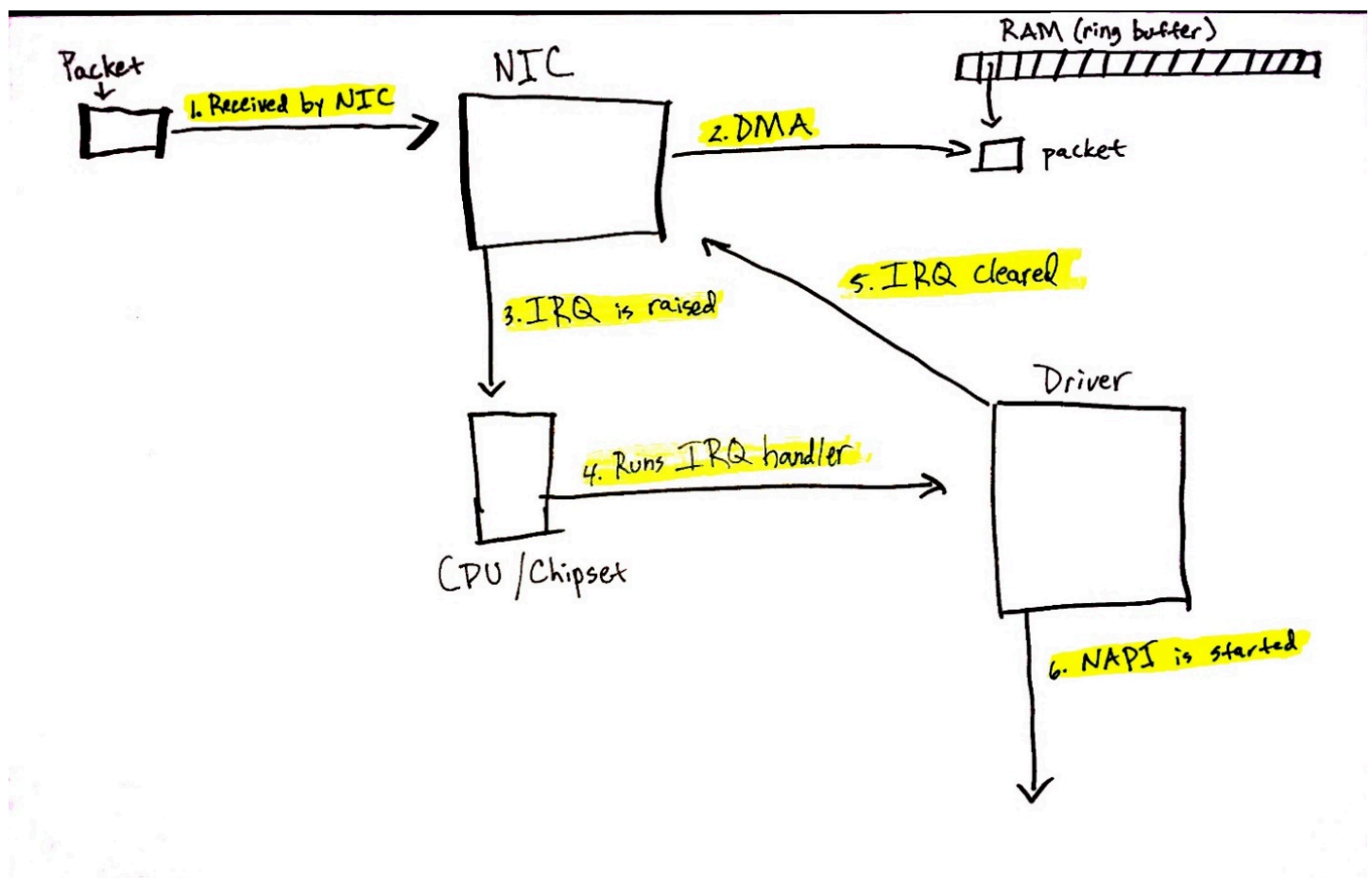


Рис. 1. Приём пакета: от NIC через DMA в кольцевой буфер, аппаратное прерывание и запуск NAPI

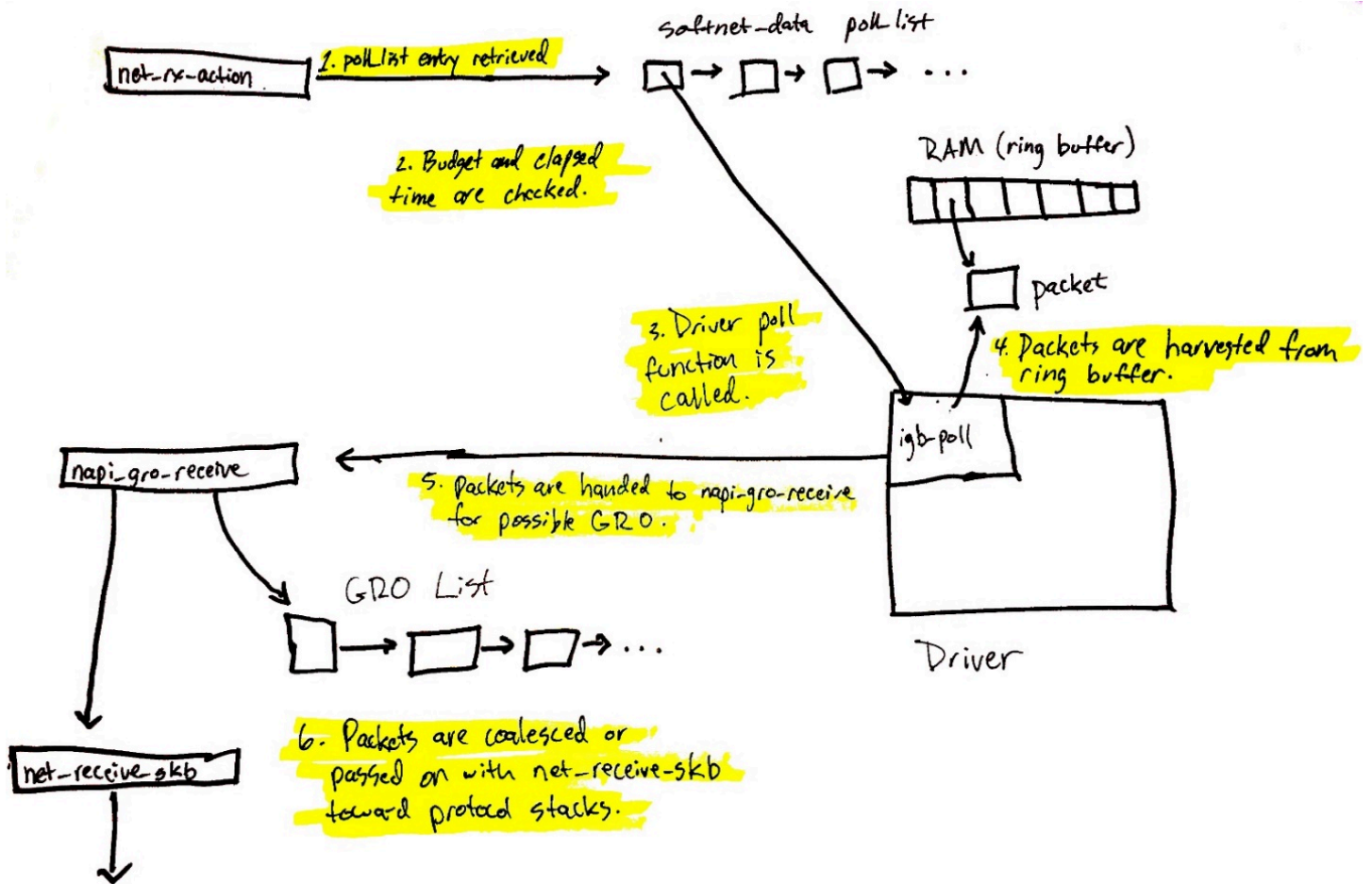


Рис. 2. Опрос (poll): сбор пакетов из кольцевого буфера, GRO и передача в net_receive_skb

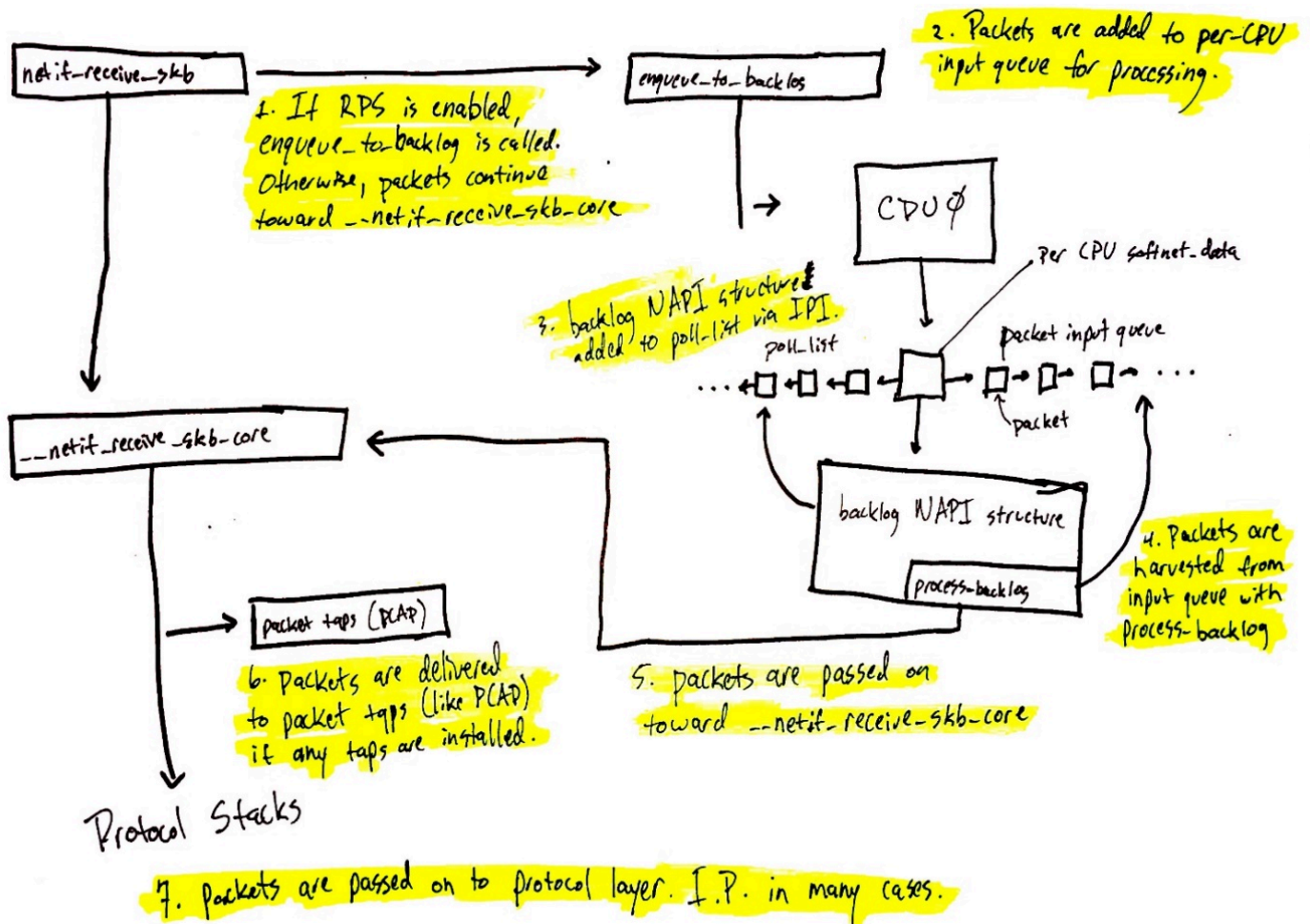


Рис. 3. RPS и передача пакетов вверх по стеку протоколов

12 Может понадобиться из прошлого сема

12.1 Кэши, их уровни

Осторожно

В доп. источнике эта тема только обозначена пунктом без раскрытия; в наших лекциях/семинарах этого семестра отдельного ответа нет (тема из прошлого семестра).

✱ Claude

Кэш — быстрая память между процессором и ОЗУ, хранящая недавно использованные данные и команды, чтобы сократить число обращений к медленной оперативной памяти. Уровни:

- **L1** — самый быстрый и маленький, у каждого ядра свой; обычно разделён на кэш команд (L1i) и кэш данных (L1d);
- **L2** — больше и медленнее L1, как правило, тоже у каждого ядра свой;
- **L3** — ещё больше и медленнее, общий для всех ядер процессора.

Чем выше уровень, тем больше объём и время доступа, но меньше стоимость хранения единицы данных.

12.2 Схема преобразования виртуального адреса в физический

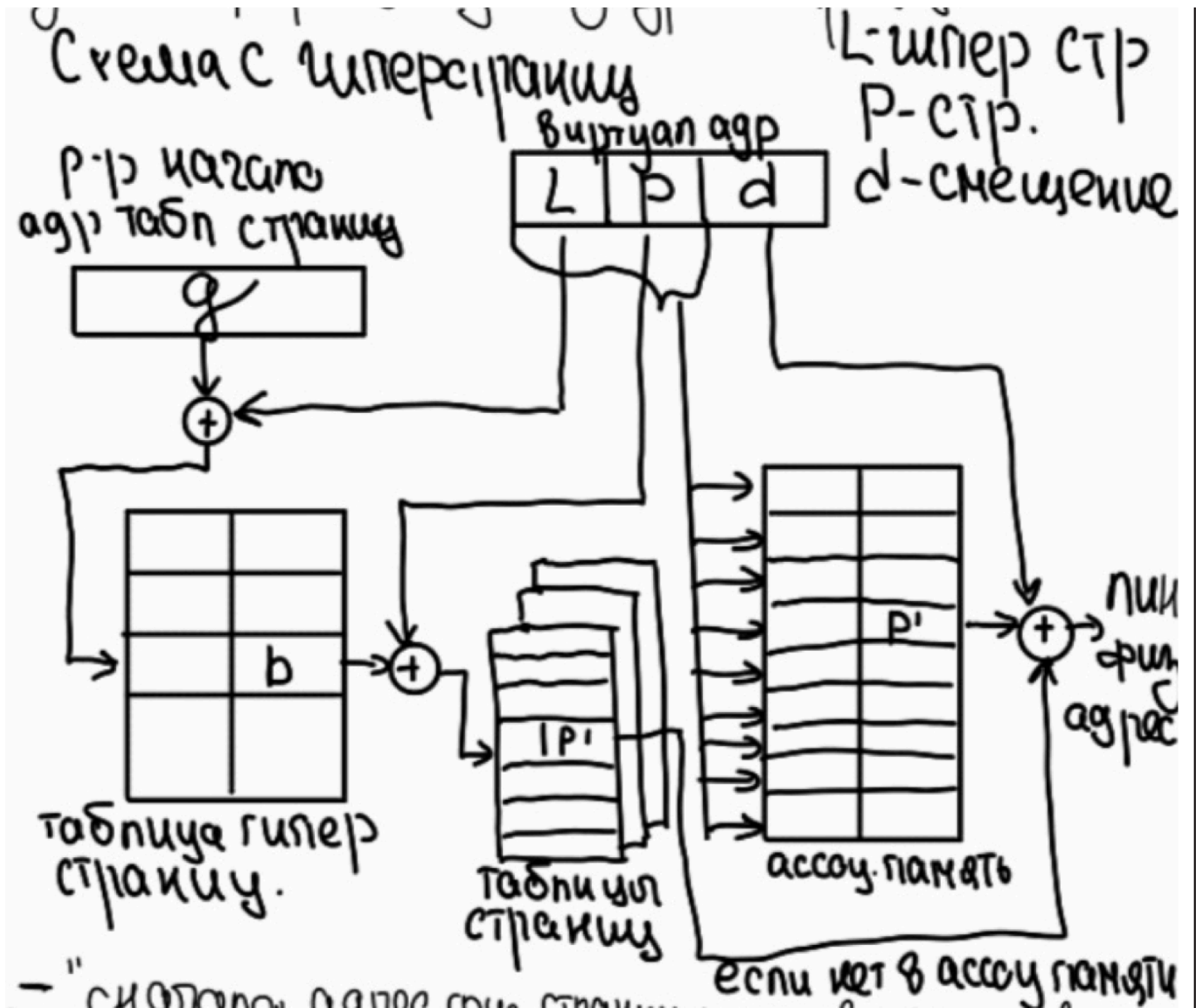


Рис. 4. Преобразование виртуального адреса в физический (схема с гиперстраницами)

✱ Claude

Схема из доп. источника рукописная и плохо читается. Более аккуратные варианты схемы преобразования адреса есть в наших семинарах (Семинары/Виртуальная ФС `procfs.typ`, Семинары/Модули ядра `typ`).

12.3 Что происходит при `fork`, `exec`?

`exec` переводит процесс на новое адресное пространство программы, путь к которой передаётся в качестве аргумента.

Что делается при выполнении вызова `fork`:

- 1) резервируется пространство свопинга для данных и стека процесса-потомка;
- 2) назначается идентификатор процесса PID и структура rproc потомка;
- 3) инициализируется структура rproc потомка. Часть полей копируется от процесса-родителя (идентификаторы пользователя и группы, маски сигналов и группа процессов), часть инициализируется нулём, часть — специфическими для потомка значениями (PID потомка и его родителя, указатель на структуру rproc родителя);
- 4) создаются карты трансляции адресов для процесса-потомка;
- 5) выделяется область u потомка, и в неё копируется область u процесса-предка;
- 6) изменяются ссылки области u на новые карты адресации и пространство свопинга;
- 7) потомок добавляется в набор процессов, которые разделяют область кода программы, выполняемой процессом-родителем;
- 8) постранично дублируются области данных и стека родителя, модифицируются карты адресации потомка;
- 9) потомок получает ссылки на разделяемые ресурсы, которые наследует: открытые файлы (наследует дескрипторы) и текущий рабочий каталог;
- 10) инициализируется аппаратный контекст потомка путём копирования регистров родителя;
- 11) процесс-потомок помещается в очередь готовых процессов;
- 12) возвращается PID в точку возврата из системного вызова в родительском процессе и 0 в процессе-потомке.

Системный вызов `exec` выполняет следующие действия:

- 1) разбирает путь к исполняемому файлу и осуществляет доступ к нему;
- 2) проверяет, имеет ли вызывающий процесс полномочия на выполнение файла;
- 3) читает заголовок и проверяет, что он действительно исполняемый;
- 4) если для файла установлены биты SUID или SGID, то эффективные идентификаторы UID и GID вызывающего процесса изменяются на UID и GID, соответствующие владельцу файла;
- 5) копирует аргументы, передаваемые в `exec`, а также переменные среды в пространство ядра, после чего текущее пользовательское пространство готово к уничтожению;
- 6) выделяет пространство свопинга для областей данных и стека;
- 7) высвобождает старое адресное пространство и связанное с ним пространство свопинга. Если процесс был создан при помощи `vfork`, производится возврат старого адресного пространства родительскому процессу;
- 8) выделяет карты трансляции адресов для нового текста, данных и стека;

- 9) устанавливает новое адресное пространство. Если область текста активна (другой процесс уже выполняет ту же программу), она будет использоваться совместно; в остальных случаях пространство инициализируется из выполняемого файла. Процессы в UNIX обычно разбиты на страницы, поэтому каждая страница считывается в память только по мере необходимости;
- 10) копирует аргументы и переменные среды обратно в новый стек приложения;
- 11) сбрасывает все обработчики сигналов в действия по умолчанию, так как функции обработчиков сигналов не существуют в новой программе; сигналы, проигнорированные или заблокированные перед вызовом `exec`, остаются в тех же состояниях;
- 12) инициализирует аппаратный контекст: большинство регистров сбрасывается в 0, а указатель команд получает значение точки входа программы.

12.4 Сигналы

Базовая идея — сигналы позволяют изменить ход выполнения программы.

✱ Claude

Сигнал — это асинхронное уведомление, посылаемое процессу ядром или другим процессом. При получении сигнала процесс прерывает обычное выполнение и вызывает обработчик сигнала (либо выполняет действие по умолчанию — например, завершение). Обработчик устанавливается системным вызовом `sigaction`. В наших лекциях сигналы рассматриваются в модели ввода-вывода, управляемого сигналами (сигнал `SIGIO`), — см. Лекции/Сокеты.type, «=== Ввод/вывод, управляемый сигналами».

12.5 Блокирующий ввод-вывод

Процесс, выполнивший системный вызов ввода/вывода, будет ожидать получения данных: система вернёт либо код ошибки, либо значение, соответствующее успешному выполнению системного вызова.

Когда приложение запрашивает ввод данных с клавиатуры (на схеме — `recvfrom`), процесс блокируется до тех пор, пока не получит данные, и ядро системы выполняет целую последовательность действий:

- 1) когда введённые пользователем данные поступят в буфер клавиатуры, контроллер клавиатуры отправит сигнал прерывания на контроллер прерываний;
- 2) с контроллера этот сигнал по шине управления поступит на ножку процессора, и процессор перейдёт на выполнение обработчика прерывания;
- 3) в результате процесс, запросивший ввод, будет разблокирован.

Вся последовательность действий выполняется, пока процесс блокирован на запросе ввода данных. Это блокирующий ввод/вывод, в основном мы имеем дело именно с ним.

Блокирующий вв/выв (blocking I/O)

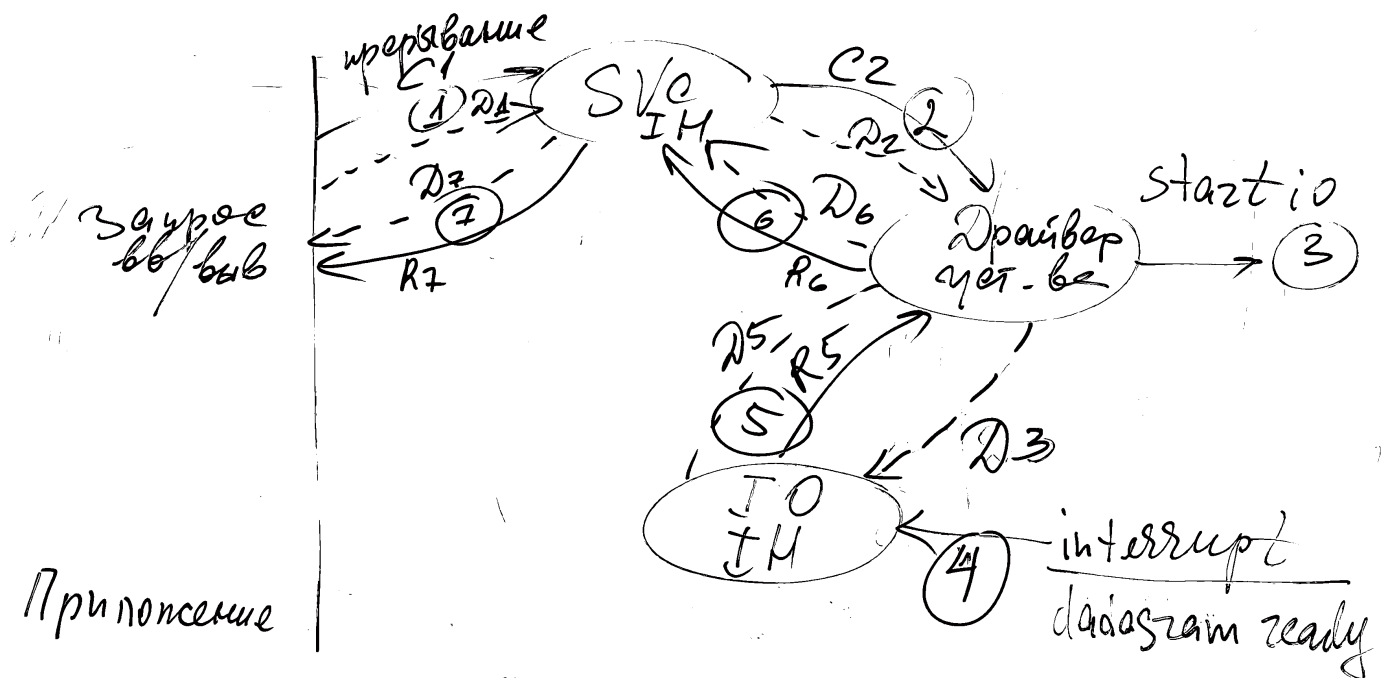


Рис. 5. Блокирующий ввод/вывод. По Шоу